

```
<h1>오늘의 메뉴</h1>
```

```
<ul> <li class="menu">아메리카노</li>
```

```
</ul>
```

```
.menu { color: #2F6B57; font-size: 18px; }
```

```
.card { display: flex; gap: 12px; }
```

```
@media (max-width: 768px) { .card { display: block; } }
```

1 권 · HTML

HTML 과 CSS

HTML 로 글에 이름표를 붙이고, CSS 로 그 모양을 정합니다.

적힌 결과는 전부 진짜 크롬으로 열어서 재어 본 값입니다.

6

단원

31

개념

191

직접 해보기

142

문제

차례

00	개발환경과 Git	11
01	무엇을 깔고 어떻게 확인하나	12
02	Git 이 하는 일	19
03	되돌리기와 .gitignore	26
04	GitHub 에 올리기	33
05	브랜치와 충돌	40
01	HTML 시작하기	53
01	웹페이지와 HTML	54
02	태그의 생김새	63
03	문서의 뼈대	70
04	속성	79
05	주석과 공백 규칙	88
02	텍스트 태그	109
01	제목, h1 부터 h6 까지	110
02	문단과 줄바꿈	120
03	강조와 의미	130
04	span 과 인라인	142
05	특수문자	152
03	목록과 표	173
01	순서없는 목록	174
02	순서있는 목록	183
03	설명 목록	193
04	표 만들기	202
05	칸 합치기	216
04	링크와 이미지	239

01	링크와 a 태그	240
02	경로	249
03	페이지 안 이동	261
04	이미지 img	270
05	그림과 설명 figure	280
05	폼	297
01	폼이란 무엇인가, 그리고 input	298
02	label 로 이름 붙이기, fieldset 으로 묶기	307
03	type 을 바꾸면 칸이 달라집니다	317
04	select 로 고르기, textarea 로 긴 글 받기	328
05	버튼, 그리고 값이 어디로 가는가	340
06	브라우저에게 검사를 시키는 속성들	351
부록 가	연습문제·종합연습 정답	375
부록 나	심화문제 정답	433

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

시작 전 준비 (처음 한 번만)

자바스크립트 자료와 달리 **설치할 프로그램이 거의 없습니다**. HTML 과 CSS 는 브라우저만 있으면 바로 돌아갑니다.

1. 브라우저

크롬(Chrome) 또는 **엣지(Edge)** 를 쓰세요. 이 자료의 설명은 크롬 기준입니다. 이미 깔려 있을 것입니다.

2. VS Code 설치

코드를 쓰는 편집기입니다. <https://code.visualstudio.com> 에서 받아 설치하세요.

설치 후 VS Code 를 열고 **파일 → 폴더 열기**로 이 **HTMLCSS자료** 폴더를 통째로 여세요. 파일 하나만 여는 것보다 폴더째 여는 편이 훨씬 편합니다.

꼭 하세요 — 화면 나누기 HTML/CSS 는 **코드와 결과를 동시에 봐야** 배워집니다. 화면 왼쪽 절반에 VS Code, 오른쪽 절반에 브라우저를 띄워 두세요. 윈도우에서는 창을 잡고 화면 왼쪽·오른쪽 끝으로 밀면 반씩 채워집니다.

3. 파일을 브라우저로 여는 법

.html 파일을 **더블클릭**하면 브라우저가 열립니다. 그게 전부입니다.

더블클릭했는데 VS Code 가 열린다면 — 파일 우클릭 → 연결 프로그램 → 크롬(또는 엣지) → **"항상 이 앱으로 열기"** 체크. VS Code 를 설치하면 자주 이렇게 됩니다.

이것만은 먼저 (제일 많이 하는 실수)

.html 파일은 **저장한다고 화면이 저절로 바뀌지 않습니다**.

VS Code 에서 고친다 → Ctrl + S 로 저장 → 브라우저로 가서 F5 (새로고침)

이 세 단계를 매번 해야 합니다. **"코드는 고쳤는데 화면이 그대로예요"**의 1순위 원인이 **저장을 안 했거나 새로고침을 안 한 것**입니다. 이 자료의  **직접 해보기**는 전부 이 과정을 요구합니다.

파일 읽는 법

개념 파일 (개념01_..., 개념02_...)

브라우저와 VS Code 로 동시에 여세요. 둘을 번갈아 보는 것이 이 자료를 쓰는 방법입니다.

- 브라우저 화면에는 섹션 제목, 짧은 요약, 그리고 실제 결과가 보입니다.
- VS Code 소스에는 그 결과가 어떻게 나왔는지 자세한 설명이 주석으로 들어 있습니다.

파일은 항상 같은 순서로 되어 있습니다.

1. 맨 위 주석 — 이 파일이 무엇을 다루는지, 앞 파일과 어떻게 이어지는지
2. 섹션 N – 제목 · 주제가 바뀌는 지점
3. 점선 상자 — 실제 결과가 그려지는 곳. 소스에서는 `<div class="doc-demo">` 입니다
4.  직접 해보기 · 거의 모든 섹션에 하나씩 있는 짧은 과제. 파란 상자로 화면에도 보입니다. (자주 하는 실수 와 정리 섹션에는 없습니다) 정답은 파일 맨 아래 직접 해보기 정답 주석 블록에 있습니다. 먼저 스스로 해 보세요
5. 자주 하는 실수 · 학생들이 실제로 자주 하는 실수 모음. 빨간 상자(잘못된 예)와 초록 상자(올바른 예)가 나란히 나옵니다
6. — 정리 — · 그 파일의 핵심을 대여섯 줄로 추린 것. 복습할 때 여기만 봐도 됩니다

결과를 적어 둔 주석

주석	뜻	쓰는 곳
<code><!-- 화면: ... --></code>	이 코드가 브라우저에 어떻게 보이는지	HTML 안
<code>/* 화면: ... */</code>	이 규칙이 화면을 어떻게 바꾸는지	<code><style></code> 안

전부 "열어 보면 이렇게 나온다" 는 뜻입니다. 자료에 적힌 결과는 전부 진짜 크롬으로 열어서 재어 본 값입니다. 다르게 나오면 어딘가 잘못 친 것입니다.

파일 맨 아래에 아래는 자료 자동 검사용입니다 라고 적힌 주석 블록이 있습니다. 자료가 스스로 틀리지 않았는지 검사하는 장치입니다. 학습 내용이 아니니 넘어가세요.

연습 파일 (연습문제.html)

TODO가 적힌 빈칸을 채우는 파일입니다. 각 문제는 이렇게 생겼습니다.

```
<!-- —— 문제 1 —— (개념01 섹션2)
```

```
h1 태그로 "우리 동네 빵집" 이라는 제목을 만드세요.
```

```
기대 화면: 큰 굵은 글씨로 "우리 동네 빵집" 이 한 줄 보입니다.
```

```
이렇게 되면 틀린 것: 글씨 크기가 그대로면 h1 이 아니라 p 를 쓴 것입니다.
```

```
-->
```

- 기대 화면과 똑같이 나오면 정답입니다.
- 막히면 연습문제_정답.html 을 보세요. 다만 먼저 10분은 혼자 고민하고 보세요.
- 문제는 기본 → [응용] → [도전] 순서로 어려워집니다. [도전]은 못 풀어도 괜찮습니다.
- 맨 마지막 문제는 [확인] 문항입니다. 일부러 틀리게 써 둔 코드를 주고 "왜 아무 일도 안 일어나는지 찾아서 고치세요"를 시킵니다.

HTML 과 CSS 에는 에러 메시지가 없습니다

이 자료에서 가장 중요한 이야기입니다.

다른 프로그래밍 언어는 틀리면 빨간 글씨로 알려 줍니다. HTML 과 CSS 는 그러지 않습니다. 틀린 부분을 조용히 무시하고, 나머지만 그립니다.

```
태그 이름을 틀림 → 그 태그가 없는 셈 치고 안의 글자만 나옵니다
CSS 속성 이름을 틀림 → 그 한 줄만 버려집니다. 나머지는 멀쩡히 적용됩니다
세미콜론을 빠뜨림 → 그 줄과 다음 줄이 같이 버려지기도 합니다
클래스 이름이 안 맞음 → 아무 일도 일어나지 않습니다
```

그래서 배우는 방법이 다릅니다.

에러를 읽는 게 아니라, 화면이 생각과 다른지 눈으로 알아채야 합니다.

"아무 일도 안 일어난다"가 이 자료에서 가장 자주 만날 증상입니다. 당황하지 말고 아래 순서로 확인하세요.

- 1. 저장했나? (Ctrl + S) → 새로고침했나? (F5)
- 2. 오타는 없나? 특히 클래스 이름, 속성 이름, 콜론과 세미콜론
- 3. F12 를 눌러 개발자도구에서 실제로 어떻게 적용됐는지 본다 (바로 아래)

개발자도구(F12) — 이 자료의 필수 도구

브라우저에서 F12 를 누르면 개발자도구가 열립니다. HTML/CSS 는 이 도구 없이는 배우기 어렵습니다. 10만원부터는 사실상 필수입니다.

하고 싶은 것	방법
이 글자가 어느 태그인지 보기	화면에서 우클릭 → 검사
지금 적용된 CSS 보기	Elements 탭 → 오른쪽 Styles
실제 계산된 값 보기	Elements 탭 → 오른쪽 Computed
박스 모델(여백) 그림 보기	Computed 탭 맨 위의 네모 그림
취소선 그어진 CSS	다른 규칙에 밀려서 안 먹은 것 (08단원 명시도)
휴대폰 크기로 보기	Ctrl + Shift + M (13단원)

F12 를 눌러도 아무 일이 없다면 — 노트북은 Fn + F12 이거나, Ctrl + Shift + I 로도 열립니다. 화면 아무 곳이나 우클릭 → 검사 도 됩니다.

Styles 패널에서는 값을 직접 고쳐 볼 수도 있습니다. 숫자를 클릭하고 위아래 화살표를 누르면 화면이 실시간으로 바뀝니다. 새로고침하면 원래대로 돌아오니 마음껏 만져 보세요.

혼자 공부할 때

1. 브라우저와 VS Code 를 나란히 띄웁니다. 이것부터 하세요.
2. 브라우저에서 결과를 보고, "이건 어느 코드에서 나온 거지?" 를 소스에서 찾습니다.
3. 소스의 주석 설명을 읽습니다. 여기가 진짜 본문입니다.
4.  직접 해보기 는 건너뛰지 마세요. 읽는 것과 직접 고쳐 보는 것은 완전히 다릅니다. 고치고 → 저장하고 → 새로고침하는 습관이 이 자료의 목표 중 하나입니다.
5. 자주 하는 실수 는 빨간 상자와 초록 상자를 비교하면서 읽습니다.
6. 단원이 끝나면 연습문제 를 풀고, 그다음 연습문제_정답 과 비교합니다.
7. 뭔가 이상하면 F12 를 먼저 누르세요. 답은 거의 항상 거기 있습니다.

파일을 망가뜨렸을까 봐 걱정된다면 — 이 폴더를 통째로 한 번 복사해 두고 시작하세요. 마음껏 고칠 수 있어야 빨리 늡니다.

단 원 00

개발환경과 Git

OPEN 00_개발환경과_Git

```
$ git --version
git version 2.55.0.windows.3
$ node --version
v24.18.0
```

01 무엇을 깔고 어떻게 확인하나

02 Git 이 하는 일

03 되돌리기와 .gitignore

04 GitHub 에 올리기

05 브랜치와 충돌

- 연습문제

무엇을 깔고 어떻게 확인하나

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

그리고 터미널을 하나 열어 두세요. 같이 씁니다.

00단원 · 개발환경과 Git

터미널을 하나 열어 두고 읽으세요. 여기 나오는 명령을 직접 쳐 봐야 합니다.

윈도우는 시작 → "terminal" 또는 Windows 터미널, 맥은 Spotlight → "터미널" 입니다.

아직 아무것도 안 깔았어도 괜찮습니다. 섹션 2에서 확인부터 합니다.

섹션 1 — 네 가지를 갑니다

이 과정에서 쓰는 도구는 넷입니다. 전부 무료입니다.

크롬	https://www.google.com/chrome/
VS Code	https://code.visualstudio.com/
Git	https://git-scm.com/downloads
Node.js	https://nodejs.org/ ← LTS 라고 적힌 쪽

Node 는 LTS 를 받으세요. 옆에 있는 숫자 큰 쪽(Current)은 새 기능이 들어간 대신 덜 안정적입니다. 수업에서는 LTS 로 충분합니다.

Git 을 깔 때 물어보는 것이 많습니다. 거의 다 그냥 Next 를 눌러도 됩니다. 딱 하나만 신경 쓰세요 — Adjusting your line ending conventions 화면에서 가운데(Checkout as-is, commit Unix-style) 또는 맨 위 기본값이면 됩니다.

왜 그런지는 개념02 섹션 6에서 봅니다.

✏ 직접 해보기 1 — 네 개를 다 깔고, 각각을 한 번씩 켜 보세요. VS Code 는 켜면 창이 뜨고, Git 은 켜지는 창이 없습니다(당연합니다).

실행하면 나오는 화면 — 무엇을 깔고 어떻게 확인하나

이 단원은 코드를 배우는 단원이 아닙니다. 앞으로 560시간 동안 쓸 도구를 깔고, 제대로 깔렸는지 확인하는 단원입니다.

재미없어 보이지만 여기서 안 잡으면 나중에 두 배로 걸립니다. "제 컴퓨터에서만 안 돼요" 의 절반이 이 단원의 내용입니다.

★ 이 단원이 끝나면 이런 상태가 됩니다.

- 편집기로 파일을 만들고 브라우저로 열 수 있다
- 터미널에서 명령을 칠 수 있다
- 고친 내용을 Git 이 기억하게 할 수 있다
- 그것을 GitHub 에 올려 남에게 보여 줄 수 있다

★★ 이 화면에 적힌 명령 결과는 전부 **진짜로 쳐 보고 옮긴 것**입니다. 여러분 화면과 글자가 조금 다를 수 있습니다. 버전이 다르면 그렇습니다. **모양이 같으면 맞은 것**입니다.

터미널을 하나 열어 두고 읽으세요. 여기 나오는 명령을 직접 쳐 봐야 합니다. 윈도우는 시작 → “terminal” 또는 Windows 터미널, 맥은 Spotlight → “터미널” 입니다. 아직 아무것도 안 깔았어도 괜찮습니다. 섹션 2에서 확인부터 합니다.

네 가지를 깔입니다

섹션 1

이 과정에서 쓰는 도구는 넷입니다. 전부 무료입니다.

무엇을 왜 까는지 먼저 말합니다. 이유를 모르고 깔면 나중에 문제가 생겼을 때 어디를 봐야 하는지 모릅니다.

크롬	만든 화면을 보는 곳. 그리고 F12 로 잘못된 곳을 찾는 곳
VS Code	파일을 쓰는 곳. 메모장으로도 되지만 훨씬 편합니다
Git	고친 내용을 기억해 주는 것. 개념02부터
Node.js	02단원(JS)부터 씁니다. 지금 같이 깔아 둡니다

Node 는 지금 안 써도 됩니다. 그런데 나중에 따로 깔면 “왜 나만 안 되지” 하는 학생이 꼭 나옵니다. 처음에 같이 해 두는 게 낫습니다.

크롬	https://www.google.com/chrome/
VS Code	https://code.visualstudio.com/
Git	https://git-scm.com/downloads
Node.js	https://nodejs.org/ ← LTS 라고 적힌 쪽

Node 는 LTS 를 받으세요. 옆에 있는 숫자 큰 쪽(Current)은 새 기능이 들어간 대신 덜 안정적입니다. 수업에서는 LTS 로 충분합니다.

Git 을 깔 때 물어보는 것이 많습니다. 거의 다 그냥 Next 를 눌러도 됩니다. 딱 하나만 신경 쓰세요 — Adjusting your line ending conventions 화면에서 가운데(Checkout as-is, commit Unix-style) 또는 맨 위 기본값이면 됩니다. 왜 그런지는 개념02 섹션 6에서 봅니다.

▹ 직접 해 보기

1

네 개를 다 깔고, 각각을 한 번씩 켜 보세요. VS Code 는 켜면 창이 뜨고, Git 은 켜지는 창이 없습니다 (당연합니다).

터미널에서 확인합니다

섹션 2

깔았다고 끝이 아닙니다. 깔린 것을 컴퓨터가 찾을 수 있는지가 중요합니다. 확인하는 방법은 언제나 같습니다. --version 을 붙여 물어봅니다.

이 세 줄이 앞으로 계속 나옵니다. “안 된다” 는 학생이 있으면 제일 먼저 이걸 시킵니다.

아래 결과는 이 자료를 만든 컴퓨터에서 실제로 친 것입니다. 숫자는 여러분 것과 다릅니다. 그게 정상입니다.

```
$ git --version
git version 2.55.0.windows.3

$ node --version
v24.18.0

$ npm --version
11.16.0
```

맨 앞의 \$ 는 치는 게 아닙니다. “여기부터 여러분이 치는 명령” 이라는 표시입니다. 윈도우 터미널에는 C:\Users\이름> 처럼 나올 것입니다.

이런 실수를 합니다

잘못된 예

```
'git' is not recognized as an internal or external command
git : 용어 'git'이(가) cmdlet, 함수, 스크립트 파일 ...로 인식되지 않습니다
bash: git: command not found
```

이렇게 나오면 못 찾은 것입니다 글자는 셋 다 다르지만 뜻은 하나입니다. git 이라는 것을 못 찾겠다.

못 찾을 때 할 일은 순서가 정해져 있습니다.

1. 터미널을 껐다 다시 켜다 ← 이걸로 대부분 해결됩니다
2. 컴퓨터를 다시 켜다
3. 그래도 안 되면 다시 껐다

1번이 왜 되냐면 — 프로그램을 깔면 “어디에 있는지” 를 적어 두는 목록(PATH)이 바뀝니다. 그런데 이미 켜져 있던 터미널은 옛 목록을 들고 있습니다. 새로 켜야 새 목록을 읽습니다. ★ 이걸 Git 만의 이야기가 아닙니다. 앞으로 뭘 깔든 똑같습니다. “깔았는데 못 찾는다” 면 일단 터미널을 다시 켜세요.

직접 해보기 2

세 명령을 다 쳐 보고 버전 숫자를 종이에 적어 두세요. 나중에 “버전이 뭐예요?” 라는 질문을 받드시 받습니다.

터미널이 두 가지입니다 (윈도우)

섹션 3

윈도우에는 터미널이 여러 개 있습니다. 이 과정에서는 두 개를 씁니다.

여기서 헷갈리는 학생이 아주 많습니다. “인터넷에서 본 명령이 안 돼요” 의 대부분이 셸이 달라서입니다.

굳이 둘 다 쓰는 이유: · PowerShell 은 윈도우에 이미 있고 npm 같은 것을 돌리기 좋습니다 · Git Bash 는 인터넷의 예제(리눅스·맥 기준)가 그대로 뜹니다

PowerShell	윈도우에 원래 있는 것. npm 명령을 여기서 씁니다
Git Bash	Git 을 깔면 같이 들어옵니다. 리눅스·맥과 같은 명령을 씁니다

같은 일을 하는데 명령이 다릅니다. 몇 개만 보면 이렇습니다.

하려는 일	PowerShell	Git Bash
지금 폴더 보기	ls (또는 dir)	ls
폴더 옮기기	cd 이름	cd 이름
파일 내용 보기	cat 이름	cat 이름
빈 파일 만들기	ni 이름	touch 이름
화면 지우기	cls	clear

겉치는 것이 많아 보이지만 다릅니다. PowerShell 의 ls 는 진짜 ls 가 아니라 별명입니다. 그래서 ls -la 같은 것을 붙이면 안 먹거나 다르게 동작합니다. ★ 인터넷에서 명령을 복사해 왔는데 안 되면 셀부터 의심하세요. 대부분의 예제는 리눅스·맥 기준입니다. 그럴 땐 Git Bash 에서 해 보세요.

VS Code 안에서도 터미널이 열립니다. Ctrl + ` (숫자 1 왼쪽의 키) 를 누르면 아래쪽에 뜹니다. 오른쪽 위 + 옆 화살표로 PowerShell 과 Git Bash 를 골라 열 수 있습니다. ★ 편집기와 터미널을 한 창에서 보는 게 훨씬 편합니다. 이걸 쓰세요.

▸ 직접 해보기 3

VS Code 를 켜고 Ctrl + ` 로 터미널을 여세요. PowerShell 과 Git Bash 를 하나씩 열고 양쪽에서 git -version 을 쳐 보세요.

Git 에게 내가 누구인지 알려 줍니다

섹션 4

Git 은 “누가 이 변경을 했는지” 를 기록으로 남깁니다. 그래서 이름과 메일을 먼저 알려 줘야 합니다. 안 하면 첫 커밋에서 막힙니다.

진짜로 안 알려 주고 커밋해 본 결과입니다. 옮겨 적은 것이 아닙니다.

이런 실수를 합니다

잘못된 예

```
$ git commit -m "시험"
Author identity unknown

*** Please tell me who you are.

Run

git config --global user.email "you@example.com"
```

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address

안 알려 주고 커밋하면 ★ 고치는 법을 오류 메시지가 그대로 알려 줍니다. 그대로 따라 치면 됩니다.
영어라고 겁내지 말고 읽으세요.

그래서 처음에 이렇게 해 둡니다. 컴퓨터마다 한 번만 하면 됩니다.

```
$ git config --global user.name "홍길동"  
$ git config --global user.email "hong@example.com"
```

--global 은 “이 컴퓨터 전체에” 라는 뜻입니다. 붙이지 않으면 지금 폴더에서만 적용됩니다. 메일은 GitHub 에 가입할 때 쓸 것과 같게 하세요. (개념04) 다르면 GitHub 에서 내 기여로 안 세어 줍니다.
그리고 하나 더 해 둡니다. 처음 만드는 가지의 이름입니다.

```
$ git config --global init.defaultBranch main
```

이 설정을 안 하면 어떻게 되는지 실제로 재 봤습니다. 전역 설정을 비운 채 git init 을 한 결과입니다.

안 하면 이런 안내가 매번 나옵니다. \$ git init hint: Using 'master' as the name for the initial branch. This default branch name hint: will change to "main" in Git 3.0. To configure the initial branch name hint: to use in all of your new repositories, which will suppress this warning, hint: call: hint: git config --global init.defaultBranch <name> hint: hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and hint: 'development'.

```
$ git init  
hint: Using 'master' as the name for the initial branch. This default branch name  
hint: will change to "main" in Git 3.0. To configure the initial branch name  
hint: to use in all of your new repositories, which will suppress this warning,  
hint: call:  
hint:  
hint: git config --global init.defaultBranch <name>  
hint:  
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and  
hint: 'development'.
```

▣ 직접 해보기 4

위 세 줄을 치고, git config --global --list 로 제대로 들어갔는지 확인하세요. (나올 때는 q 를 누릅니다)

★ 한글 파일 이름이 깨져 보입니다

섹션 5

한국에서 Git 을 쓰면 반드시 만나는 것입니다. 한글 이름의 파일을 만들고 git status 를 치면 이렇게 나옵니다.

진짜로 재 본 것입니다. 설정.txt 라는 파일을 만들고 친 결과입니다.

이런 실수를 합니다

잘못된 예

```
$ git status --short
M "\354\204\244\354\240\225.txt"
```

기본 상태 설정.txt 인데 알아볼 수가 없습니다.

이렇게 쓰세요

```
$ git config --global core.quotepath false

$ git status --short
?? 한글파일.txt
```

파일이 깨진 게 아닙니다. Git 이 화면에 보여 줄 때만 그렇게 바꾼 것입니다. 내용도 이름도 멀쩡합니다. core.quotepath false 는 "그대로 보여 줘" 라는 뜻입니다. ★ 이 자료는 폴더와 파일 이름이 전부 한글입니다. 이 설정을 꼭 해 두세요.

▸ 직접 해보기

5

아무 폴더에서 한글 이름의 파일을 하나 만들고 설정 전후로 git status --short 를 쳐서 차이를 보세요. (아직 git init 을 안 배웠으면 개념02를 먼저 보고 오세요)

자주 하는 실수

섹션 6

이런 실수를 합니다

깔고 나서 터미널을 안 껐다 켜

섹션 2 그대로입니다. 이게 압도적으로 많습니다. "다시 깔았는데도 안 돼요" 하는 학생의 터미널은 아침에 켜 그 창입니다.

이런 실수를 합니다

\$ 를 같이 치

\$ git --version 을 통째로 복사해 붙입니다. \$ 는 표시일 뿐입니다. git 부터 치세요.

이런 실수를 합니다

명령을 오타 낸 줄 모름

git --varson 처럼 치면 다른 오류가 납니다. 오류 글자를 끝까지 읽으세요. "못 찾겠다" 인지 "그런 옵션 없다" 인지 전혀 다른 이야기입니다.

이런 실수를 합니다

PowerShell 과 Git Bash 를 섞음

인터넷 예제를 PowerShell 에 붙였는데 안 됩니다. 셸이 다릅니다. (섹션 3)

이런 실수를 합니다

--global 을 빼먹음

이름·메일을 설정했는데 다른 폴더에서 또 물어봅니다. --global 없이 하면 그 폴더에서만 적용됩니다.

이런 실수를 합니다

회사 메일로 설정해 두고 개인 저장소에 씀

커밋 기록에 메일이 그대로 남습니다. 지워도 기록에는 남습니다. 공개 저장소에 올릴 것이면 처음부터 공개해도 되는 메일을 쓰세요.

Git 이 하는 일

RUN 이 파일을 브라우저로 열고, 터미널도 같이 여세요.

00

여기 나오는 명령을 직접 쳐 보면서 읽어야 합니다.

00단원 · 개발환경과 Git

터미널에서 같이 따라 하세요. 읽기만 하면 안 남습니다.

바탕화면에 깃연습 이라는 빈 폴더를 하나 만들고 거기서 하세요. 이 자료 폴더 안에서는 하지 마세요. 왜 안 되는지는 섹션 1에 있습니다.

아래 결과는 실제로 쳐 본 것입니다. 커밋 번호(ae4c0c2 같은)만 여러분 것과 다릅니다.

섹션 1 — 기록을 시작합니다

폴더 하나를 정해서 "여기를 기록해라" 라고 말해 줍니다. 딱 한 번만 합니다.

```
$ git init
Initialized empty Git repository in C:/Users/.../깃연습/.git/
```

.git 이라는 숨은 폴더가 하나 생깁니다. 여기에 기록이 전부 들어갑니다. 절대 열어서 고치지 마세요. 이 폴더를 지우면 기록이 통째로 사라집니다.

★ 반대로, "Git 을 그만 쓰고 싶다" 면 .git 을 지우면 됩니다. 파일은 그대로 남습니다.

이 자료 폴더 안에서 하면 안 되는 이유

이 자료는 이미 Git 으로 관리되고 있습니다. 그 안에서 또 git init 을 하면 저장소 안에 저장소가 생겨 아주 헷갈립니다. 빈 폴더에서 연습하세요.

실행하면 나오는 화면 — Git 이 하는 일

파일을 고치다 보면 이런 폴더가 생깁니다.

```
과제.html
과제_수정.html
과제_최종.html
과제_최종_진짜.html
과제_최종_진짜_이게진짜.html
```

농담 같지만 다들 한 번씩 합니다. 그리고 이런 문제가 생깁니다.

- 어느 게 최신인지 모릅니다
- 무엇이 달라졌는지 모릅니다
- 어제 것으로 돌아갈 방법이 없습니다
- 둘이 같이 고치면 하나가 사라집니다

Git 은 이 넷을 전부 해결합니다. 파일을 복사하는 대신 “이 순간의 모습”을 기록해 두는 방식으로요.

★ 이 단원에서 배우는 명령은 딱 넷입니다.

git init	이 폴더를 기록하기 시작해라
git status	지금 무엇이 달라졌나
git add	이번에 기록할 것을 고른다
git commit	골라 둔 것을 기록한다

나머지는 이 넷을 알고 나면 다 걸가지입니다.

터미널에서 같이 따라 하세요. 읽기만 하면 안 남습니다. 바탕화면에 깃연습 이라는 빈 폴더를 하나 만들고 거기서 하세요. 이 자료 폴더 안에서는 하지 마세요. 왜 안 되는지는 섹션 1에 있습니다. 아래 결과는 실제로 쳐 본 것입니다. 커밋 번호(ae4c0c2 같은)만 여러분 것과 다릅니다.

기록을 시작합니다

섹션 1

폴더 하나를 정해서 “여기를 기록해라” 라고 말해 줍니다. 딱 한 번만 합니다.

```
$ git init
Initialized empty Git repository in C:/Users/.../깃연습/.git/
```

.git 이라는 숨은 폴더가 하나 생깁니다. 여기에 기록이 전부 들어갑니다. 절대 열어서 고치지 마세요. 이 폴더를 지우면 기록이 통째로 사라집니다. ★ 반대로, “Git 을 그만 쓰고 싶다” 면 .git 을 지우면 됩니다. 파일은 그대로 남습니다.

이런 실수를 합니다

잘못된 예

이 자료 폴더 안에서 하면 안 되는 이유 이 자료는 이미 Git 으로 관리되고 있습니다. 그 안에서 또 git init 을 하면 저장소 안에 저장소가 생겨 아주 헷갈립니다. 빈 폴더에서 연습하세요.

▫ 직접 해보기 1

바탕화면에 깃연습 폴더를 만들고, 터미널에서 그 폴더로 가서 git init 을 치세요. 그리고 숨은 파일 보기를 켜서 .git 이 생겼는지 보세요.

자리가 세 군데입니다

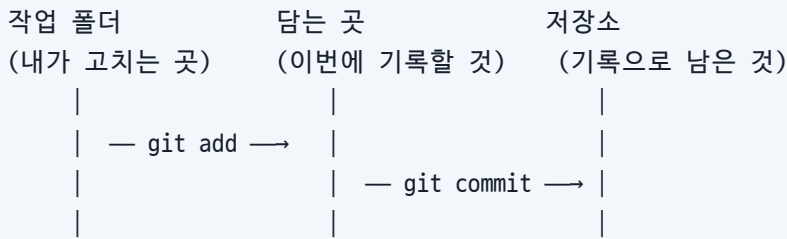
섹션 2

Git 을 헷갈리게 만드는 것은 명령이 아니라 자리입니다. 파일이 있을 수 있는 자리가 셋입니다.

이 그림 하나가 이 단원의 절반입니다. 학생이 “add 를 왜 또 해야 해요?” 라고 물으면 여기로 돌아옵니다.

비유: 작업 폴더 = 책상 위 (지금 쓰는 중) 답는 곳 = 장바구니 (이번에 계산할 것만 담음) 저장소 = 영수증 (계산 끝. 기록으로 남음)

장바구니가 왜 필요하냐 — 고친 파일이 10개인데 그중 3개만 기록하고 싶을 때가 있기 때문입니다.



“add 를 왜 또 해요?” — 고친 파일이 10개인데 그중 3개만 기록하고 싶을 때가 있기 때문입니다.

장바구니에 담는 것과 계산하는 것이 나뉜 것과 같습니다. ★ 지금은 git add . (전부 담기) 로만 써도 됩니다. 나뉘어 있다는 것만 알아 두세요.

만들고 담고 기록합니다

섹션 3

index.html 파일을 하나 만들고 안에 한 줄만 씁니다.

```
<h1>안녕하세요</h1>
```

그리고 git status 를 칩니다.

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  index.html

nothing added to commit but untracked files present (use "git add" to track)
```

Untracked — “이런 파일이 있는데 나는 애플 모른다” 는 뜻입니다. Git 은 폴더 안의 파일을 자동으로 챙기지 않습니다. 내가 알려 줘야 합니다. ★ 그리고 괄호 안에 다음에 뭘 할지 적혀 있습니다. use “git add <file>...”. Git 은 거의 항상 다음 할 일을 알려 줍니다. 영어라고 건너뛰지 말고 읽으세요. 답이 거기 있습니다.

git status 는 말이 길니다. --short 를 붙이면 짧게 나옵니다.

```
$ git status --short
?? index.html
```

담고, 다시 봅니다.

```
$ git add index.html

$ git status --short
A index.html
```

이제 기록합니다.

```
$ git commit -m "첫 페이지를 만들었습니다"
[main (root-commit) ae4c0c2] 첫 페이지를 만들었습니다
1 file changed, 1 insertion(+)
create mode 100644 index.html
```

-m 은 message 입니다. 무엇을 왜 했는지를 적습니다. ae4c0c2 는 이 기록에 붙은 번호입니다. 내용에서 나온 값이라 여러분 것은 다릅니다. root-commit 은 "첫 기록" 이라는 뜻입니다.

```
$ git log --oneline
ae4c0c2 첫 페이지를 만들었습니다
```

◦ 직접 해보기 2

위 순서를 그대로 해 보세요. git add 전후로 git status --short 를 쳐서 ?? 가 A 로 바뀌는 것을 눈으로 보세요.

status 의 기호 읽는 법

섹션 4

--short 로 보면 왼쪽에 글자 두 개가 붙습니다. 이 두 칸이 각각 다른 자리를 가리킵니다.

여기가 이 개념의 핵심입니다. 학생들은 "M" 과 "M " 을 같은 것으로 봅니다. 전혀 다릅니다.

아래 표는 진짜로 재현해 본 것입니다.

첫째 칸 =	답는 곳의 상태	
둘째 칸 =	작업 폴더의 상태	
?? index.html	양쪽 다 모름	→ 새 파일인데 아직 add 안 함
A index.html	담김 (새 파일)	→ add 했다
M index.html	작업 폴더만 고쳐짐	→ 고쳤는데 add 안 함
M index.html	담긴 것도 고쳐짐	→ 고치고 add 했다
MM index.html	담고 나서 또 고쳤다	→ 둘 다 있음
UU index.html	충돌 (개념05)	

M 과 M 은 다릅니다. 앞에 공백이 있으면 "아직 안 담았다", 뒤에 공백이 있으면 "담았다" 입니다. ★ 이걸 모르면 git commit 을 했는데 왜 안 들어갔는지 모릅니다. 담기지 않은 것은 커밋되지 않습니다.

▣ 직접 해보기 3

index.html 에 한 줄을 더 넣고 저장한 뒤 git status --short → git add . → 다시 git status --short 를 쳐서 기호가 어떻게 바뀌는지 보세요.

무엇이 달라졌는지 봅니다

섹션 5

파일에 한 줄을 더 넣었습니다. git diff 로 무엇이 바뀌었는지 봅니다.

```
$ git diff
diff --git a/index.html b/index.html
index 6944b17..87d06c9 100644
--- a/index.html
+++ b/index.html
@@ -1,2 @@
<h1>안녕하세요</h1>
+<p>처음 만든 페이지입니다.</p>
```

앞의 네 줄은 지금 안 봐도 됩니다. 중요한 것은 마지막 두 줄입니다. + 로 시작하면 새로 생긴 줄, - 로 시작하면 없어진 줄, 아무것도 없으면 그대로인 줄입니다.

★ git diff 는 아직 안 담은 것만 보여 줍니다. git add 를 한 뒤에 git diff 를 치면 아무것도 안 나옵니다. 고장이 아닙니다. 담긴 것을 보려면 git diff --staged 입니다. 이것도 섹션 4의 “자리가 둘” 때문입니다.

▣ 직접 해보기 4

파일을 고치고 git diff → git add . → git diff → git diff --staged 순서로 쳐 보세요. 어느 것이 언제 비는지 확인하세요.

커밋 메시지를 어떻게 쓰나

섹션 6

나중에 이 목록만 보고 무슨 일이 있었는지 알아야 합니다.

```
$ git log --oneline
7c6d6e7 인사말 충돌을 풀었습니다
b53be6a 인사말을 환영합니다로
cb46bb8 인사말을 반갑습니다로
73859b6 글자를 파랗게
6d15fca 설명 문단을 넣었습니다
ae4c0c2 첫 페이지를 만들었습니다
```

이런 실수를 합니다

잘못된 예

```
수정
□ L O ㄹ
○○
완료
asdf
update
```

이렇게 쓰면 안 됩니다 일주일 뒤의 나도 무슨 뜻인지 모릅니다.

이렇게 쓰세요

```
첫 페이지를 만들었습니다
설명 문단을 넣었습니다
로그인 버튼이 눌리지 않던 문제를 고쳤습니다
읽지 않는 이미지 파일을 지웠습니다
```

★ 한 번에 하나씩 커밋하세요. “로그인도 고치고 색도 바꾸고 파일도 지웠습니다” 는 메시지를 쓸 수 없습니다. 메시지가 안 써지면 한 번에 너무 많이 한 것입니다.

이 자료의 커밋 메시지도 규칙이 있습니다. 수업에서 만드는 저장소에 AI 도구 이름 같은 것을 넣지 마세요. 기록에 영원히 남고, 나중에 지우기 어렵습니다.

▫ 직접 해보기

5

지금까지 만든 커밋의 메시지를 `git log --oneline` 으로 보세요. 일주일 뒤의 내가 읽어도 알아볼 수 있게 쓰여 있습니까?

자주 하는 실수

섹션 7

이런 실수를 합니다

add 를 하고 commit 을 안 함

담기만 하고 계산을 안 한 것입니다. `git log` 에 아무것도 안 나옵니다. `git status --short` 에 A 나 M 가 남아 있으면 이 경우입니다.

이런 실수를 합니다

고치고 add 를 안 한 채 commit

`nothing added to commit` 이 나옵니다. 고친 것은 있는데 담은 것이 없다는 뜻입니다. 섹션 4를 다시 보세요.

이런 실수를 합니다

-m 을 빼먹음

git commit 만 치면 편집기가 열립니다. 검은 화면에 글자가 잔뜩 나와서 놀라는 학생이 많습니다. 빠져나오는 법: Esc → :q! → Enter ★ 이건 vim 이라는 편집기입니다. 겁먹을 필요 없습니다. 나오는 법만 알면 됩니다.

이런 실수를 합니다

이 자료 폴더에서 git init

섹션 1 그대로입니다. 빈 폴더에서 연습하세요.

이런 실수를 합니다

.git 폴더를 지움

기록이 전부 사라집니다. 되돌릴 방법이 없습니다. 숨은 폴더는 건드리지 마세요.

이런 실수를 합니다

줄 끝 문자 경고

warning: LF will be replaced by CRLF 같은 것이 나올 수 있습니다. 고장이 아닙니다. 윈도우와 리눅스가 줄 바꿈을 다르게 적어서 나는 안내입니다. 수업에서는 그냥 넘어가도 됩니다.

되돌리기와 .gitignore

RUN 이 파일을 브라우저로 열고, 터미널도 같이 여세요.

00단원 · 개발환경과 Git

개념02의 깃연습 폴더에서 이어서 하세요.

이 단원에는 되돌릴 수 없는 명령이 하나 나옵니다 (섹션 4). 연습 폴더에서만 쓰세요.

섹션 1 — 아직 안 담았으면 **restore**

파일을 고쳤는데 마음에 안 듭니다. 아직 **add** 는 안 했습니다.

```
$ git status --short
M index.html

$ git restore index.html

$ git status --short

$ cat index.html
<h1>안녕하세요</h1>
```

`git status` 가 아무것도 안 찍었습니다. 달라진 게 없다는 뜻입니다. 마지막 커밋 상태로 돌아갔습니다.

★ 고친 내용은 완전히 사라집니다

`git restore` 는 휴지통에 안 갑니다. `Ctrl+Z` 로도 못 되돌립니다. 아직 커밋 안 한 것이라 Git 도 모르는 내용이기 때문입니다.

지울 것이 확실할 때만 쓰세요.

실행하면 나오는 화면 — 되돌리기와 .gitignore

Git 을 쓰는 진짜 이유가 이 단원입니다.

"어제 것으로 돌아가고 싶다"

파일을 복사해 두는 방식으로는 이게 잘 안 됩니다. Git 은 기록을 다 갖고 있어서 어느 시점으로든 돌아갈 수 있습니다.

★ 그런데 되돌리는 방법이 **자리마다 다릅니다**. 이게 헛갈립니다.

아직 안 담았다	→ <code>git restore</code>
담았다	→ <code>git restore --staged</code>
커밋했다	→ <code>git revert</code> (기록을 남기며)
	→ <code>git reset</code> (★ 기록을 지우며 - 위험)

개념02 섹션2의 '자리 셋' 을 알고 있으면 그대로 이어집니다. 모르겠으면 개념02를 먼저 보고 오세요.

개념02의 깃연습 폴더에서 이어서 하세요. 이 단원에는 되돌릴 수 없는 명령이 하나 나옵니다 (섹션 4). 연습 폴더에서만 쓰세요.

아직 안 담았으면 restore

섹션 1

파일을 고쳤는데 마음에 안 듭니다. 아직 add 는 안 했습니다.

```
$ git status --short
M index.html

$ git restore index.html

$ git status --short

$ cat index.html
<h1>안녕하세요</h1>
```

git status 가 아무것도 안 찍었습니다. 달라진 게 없다는 뜻입니다. 마지막 커밋 상태로 돌아갔습니다.

이런 실수를 합니다

잘못된 예

★ 고친 내용은 완전히 사라집니다 git restore 는 휴지통에 안 갑니다. Ctrl+Z 로도 못 되돌립니다. 아직 커밋 안 한 것이라 Git 도 모르는 내용이기 때문입니다. 지울 것이 확실할 때만 쓰세요.

직접 해보기 1

index.html 에 아무 줄이나 넣고 저장한 뒤 git restore index.html 로 되돌려 보세요. VS Code 에서 파일이 저절로 바뀌는 것을 보세요.

담은 것을 무르기

섹션 2

git add 를 했는데 “아, 이건 이번에 넣지 말걸” 싶을 때입니다.

```
$ git add .

$ git status --short
M index.html      ← 담겼습니다 (첫째 칸이 M)

$ git restore --staged index.html

$ git status --short
M index.html      ← 담는 곳에서만 빠졌습니다 (둘째 칸이 M)
```

--staged 를 붙이면 담는 곳에서만 뺍니다. 고친 내용은 그대로 남아 있습니다. 기호가 M 에서 M 으로 옮겨 간 것을 보세요. (개념02 섹션4)

★ 완전히 없애고 싶으면 두 번 해야 합니다. git restore --staged 파일 (담는 곳에서 빼고) → git restore 파일 (내용도 되돌리고)

▫ 직접 해보기 2

파일을 고치고 add 한 뒤, --staged 있는 것과 없는 것을 차례로 써 보세요. git status --short 로 기호가 어떻게 움직이는지 보세요.

커밋한 뒤에는 revert

섹션 3

이미 커밋했습니다. 그런데 그 커밋이 잘못됐습니다. 기록은 남겨 두고, 내용만 되돌립니다.

```
$ git log --oneline
7411c6b 잘못된 줄을 넣었습니다
cca186a 설명 문단을 넣었습니다
9480c14 첫 페이지를 만들었습니다

$ git revert --no-edit HEAD
[main 384ffb0] Revert "잘못된 줄을 넣었습니다"
1 file changed, 1 deletion(-)

$ git log --oneline
384ffb0 Revert "잘못된 줄을 넣었습니다"
7411c6b 잘못된 줄을 넣었습니다
cca186a 설명 문단을 넣었습니다
9480c14 첫 페이지를 만들었습니다
```

기록이 줄지 않고 하나 늘었습니다. “그 커밋을 되돌렸다” 는 커밋이 새로 생긴 것입니다. HEAD 는 “지금 있는 자리” 라는 뜻입니다. 제일 최근 커밋을 가리킵니다. --no-edit 은 “메시지를 물어보지 마라” 입니다. 빼면 편집기가 열립니다.

★ 기록을 남기는 게 왜 좋냐 — 남들과 같이 쓸 때 안전하기 때문입니다. 내가 기록을 지워 버리면 남의 컴퓨터에는 그 기록이 남아 있어서 서로 엉킵니다. revert 는 그런 일이 없습니다.

이런 실수를 합니다

잘못된 예

```
$ git revert --no-edit HEAD
error: your local changes would be overwritten by revert.
hint: commit your changes or stash them to proceed.
fatal: revert failed
```

고친 것이 남아 있으면 못 합니다 “고친 것을 먼저 커밋하든지 치워 두라” 는 뜻입니다. 여기서도 hint: 가 답을 알려 줍니다.

▹ 직접 해보기 3

아무 줄이나 넣고 커밋한 뒤 `git revert --no-edit HEAD` 로 되돌려 보세요. `git log --oneline` 이 어떻게 되는지 보세요.

★ reset --hard 는 지웁니다

섹션 4

인터넷에서 제일 많이 보이는 명령입니다. 그리고 제일 위험합니다.

```
$ git reset --hard HEAD~2
HEAD is now at cca186a 설명 문단을 넣었습니다

$ git log --oneline
cca186a 설명 문단을 넣었습니다
9480c14 첫 페이지를 만들었습니다
```

커밋 두 개가 목록에서 사라졌습니다. `HEAD~2` 는 “지금 자리에서 두 칸 뒤” 라는 뜻입니다.

이런 실수를 합니다

잘못된 예

★★ 언제 쓰면 안 되나 이미 GitHub 에 올린 커밋에는 쓰지 마세요. 내 컴퓨터에서만 기록이 사라지고 GitHub 에는 남아 있어서, 다음에 올릴 때 거부당하거나 남의 작업을 지우게 됩니다. 올리기 전, 나 혼자 있는 커밋에만 쓰세요. 남들과 같이 쓰는 것은 revert 입니다. (섹션 3)

그래도 완전히 사라지지는 않습니다. `git reflog` 를 치면 최근에 있었던 자리들이 나옵니다. 거기 적힌 번호로 돌아갈 수 있습니다. ★ 다만 믿고 함부로 쓰라는 뜻이 아닙니다. 커밋 안 한 것은 reflog 에도 없습니다. 그건 진짜로 사라집니다.

▹ 직접 해보기 4

연습 폴더에서 `git reset --hard HEAD~1` 을 해 보고, `git reflog` 로 사라진 커밋 번호를 찾아 보세요. (돌아가려면 `git reset --hard 그번호`)

폴더 안의 모든 파일을 기록해야 하는 것은 아닙니다. 오히려 넣으면 안 되는 것이 있습니다.

node_modules/	남이 만든 것을 받아 온 창고. 수만 개 파일입니다
.env	비밀번호·API 키가 든 파일 ★
dist/ build/	빌드 결과. 소스에서 다시 만들 수 있습니다
.DS_Store	맥이 자동으로 만드는 것
Thumbs.db	윈도우가 자동으로 만드는 것

.gitignore 라는 파일을 만들고 여기에 적으면 Git 이 못 본 척합니다.

```
$ git status --short
?? .env
?? node_modules/

(.gitignore 파일을 만들고 두 줄을 씁니다)
node_modules/
.env

$ git status --short
?? .gitignore
```

.env 와 node_modules/ 가 사라졌습니다. 대신 .gitignore 자신이 나타났습니다. ★ .gitignore 는 같이 올려야 합니다. 팀원 모두가 같은 것을 무시해야 하니까요.

진짜로 무시되는지 확인하려면 — \$ git check-ignore -v .env .gitignore:2:.env .env “몇 번째 줄의 어느 규칙에 걸렸는지” 를 알려 줍니다.

```
$ git check-ignore -v .env
.gitignore:2:.env .env
```

▫ 직접 해보기 5

연습 폴더에 .env 파일을 만들고 git status --short 로 보이는 것을 확인한 뒤, .gitignore 에 적어서 사라지게 해 보세요.

★★ 이미 올린 파일은 .gitignore 가 안 먹습니다

이게 이 단원에서 제일 자주 사고가 나는 곳입니다. 실수로 .env 를 커밋한 뒤에 .gitignore 에 적었습니다.

실제로 재현해 본 것입니다. 설정.txt 를 커밋한 뒤 .gitignore 에 적고 status 를 쳤습니다.

이런 실수를 합니다

잘못된 예

```
(.gitignore 에 설정.txt 를 적었습니다)

$ git status --short
M 설정.txt          ← ★ 그대로 보입니다

$ git check-ignore -v 설정.txt
                    ← ★ 아무것도 안 나옵니다
```

여전히 따라다닙니다

.gitignore 는 “아직 모르는 파일”에만 먹습니다. 이미 알고 있는(추적 중인) 파일은 계속 따라다닙니다. ★ check-ignore 가 아무 말도 안 하는 것이 신호입니다. 규칙이 틀린 게 아니라 이미 추적 중이라 검사 대상이 아닌 것입니다. 확인하려면 --no-index 를 붙입니다. \$ git check-ignore -v --no-index 설정.txt .gitignore:3:설정.txt 설정.txt 규칙은 멀쩡했습니다.

```
$ git check-ignore -v --no-index 설정.txt
.gitignore:3:설정.txt 설정.txt
```

이렇게 쓰세요

```
$ git rm --cached 설정.txt
rm '설정.txt'

$ git status --short
D 설정.txt          ← 저장소에서 지운다는 표시 (D = delete)

$ git check-ignore -v 설정.txt
.gitignore:3:설정.txt 설정.txt    ← 이제 먹습니다
```

이런 실수를 합니다

잘못된 예

★★★ 그런데 비밀번호가 샌 것은 이걸로 안 끝납니다 git rm --cached 를 하고 커밋해도 옛 기록에는 남아 있습니다. 누구든 옛 커밋을 꺼내 보면 그 비밀번호가 보입니다. 키나 비밀번호가 한 번 올라갔으면, 그 키를 폐기하는 것 말고 방법이 없습니다. 지우는 것으로는 해결되지 않습니다. ★ 그래서 .gitignore 를 처음에 만드는 것이 중요합니다.

직접 해보기

6

아무 파일이나 커밋한 뒤 .gitignore 에 적어 보세요. git status 에 계속 보이는 것을 확인하고, git rm --cached 로 빼 보세요.

이런 실수를 합니다

restore 로 지운 것을 되찾으려 함

못 찾습니다. 커밋 안 한 것은 Git 도 모릅니다. (섹션 1)

이런 실수를 합니다

올린 커밋에 **reset --hard**

남들과 엉킵니다. 올린 뒤에는 revert 를 쓰세요. (섹션 3·4)

이런 실수를 합니다

.gitignore 를 나중에 만듦

섹션 6 그대로입니다. 프로젝트를 만들자마자 만드세요. 첫 커밋 전에 만드는 것이 제일 좋습니다.

이런 실수를 합니다

git rm 에 **--cached** 를 빼먹음

진짜 파일이 지워집니다. 항상 **--cached** 를 붙였는지 보세요.

이런 실수를 합니다

.gitignore 에 적었는데 안 먹는다고 규칙을 계속 고침

규칙이 아니라 이미 추적 중이라서입니다. **git check-ignore -v --no-index** 로 규칙이 맞는지부터 확인하세요. (섹션 6)

이런 실수를 합니다

node_modules 를 통째로 올림

파일이 수만 개라 올리는 데 한참 걸리고, 받는 사람도 괴롭습니다. **package.json** 만 있으면 **npm install** 로 다시 만들어집니다.

GitHub 에 올리기

RUN 이 파일을 브라우저로 열고, 터미널과 GitHub 을 같이 여세요.

00

00단원 · 개발환경과 Git

GitHub 계정이 필요합니다. <https://github.com> 에서 가입하세요. 무료입니다.

가입할 때 쓴 메일을 개념01의 `user.email` 과 같게 하세요. 다르면 내가 한 작업으로 안 세어 줍니다.

아이디는 나중에 이력서에 적을 것이라 생각하고 지으세요. hong-gildong 은 괜찮고 xxx-gamer-1234 는 다시 생각해 보세요.

섹션 1 — 인터넷에 빈 저장소를 하나 만듭니다

GitHub 오른쪽 위 + → New repository 를 누릅니다.

```
Repository name  깃연습
Description      (비워도 됩니다)
Public / Private Public 이면 아무나 볼 수 있습니다
                 * 처음에는 Private 으로 하세요

Add a README file   ← * 체크하지 마세요
Add .gitignore      ← * 체크하지 마세요
Choose a license    ← * 고르지 마세요
```

왜 아무것도 체크하지 말라고 하나 — 체크하면 GitHub 이 파일을 먼저 만들어 버립니다. 그러면 내 컴퓨터의 기록과 GitHub 의 기록이 출발점이 달라져서 첫 push 에서 막힙니다. (섹션 5의 그 오류입니다)

★ 빈 저장소로 만들고 내 것을 밀어 넣는 게 제일 쉽습니다.

실행하면 나오는 화면 — GitHub 에 올리기

지금까지는 전부 내 컴퓨터 안의 일이었습니다. 컴퓨터가 고장 나면 기록도 같이 사라집니다.

GitHub 은 그 기록을 인터넷에 하나 더 두는 곳입니다. 그리고 그것 말고도 하는 일이 많습니다.

- 컴퓨터가 죽어도 남습니다
- 다른 컴퓨터에서 이어서 할 수 있습니다
- 남에게 보여 줄 수 있습니다 (포트폴리오)
- 여럿이 같이 고칠 수 있습니다

★ Git 과 GitHub 은 다른 것입니다.

Git	내 컴퓨터에서 기록하는 프로그램
GitHub	그 기록을 올려 두는 웹사이트

Git 없이 GitHub 을 쓸 수 없고, GitHub 없이도 Git 은 잘 돕니다. 비슷한 사이트로 GitLab, Bitbucket 도 있습니다.

★★ 이 파일의 명령 결과는 **진짜로 밀어 보고 옮긴 것입니다**. 다만 남의 계정을 쓸 수 없어서, GitHub 대신 내 컴퓨터 안의 폴더를 원격 삼아 재 봤습니다. 주소가 적힌 줄만 GitHub 주소로 바꿔 적었고, 나머지는 그대로입니다.

GitHub 계정이 필요합니다. <https://github.com> 에서 가입하세요. 무료입니다. 가입할 때 쓴 메일을 개념01의 user.email 과 같게 하세요. 다르면 내가 한 작업으로 안 세어 줍니다. 아이디는 나중에 이력서에 적을 것이라 생각하고 지으세요. hong-gildong 은 괜찮고 xxx-gamer-1234 는 다시 생각해 보세요.

인터넷에 빈 저장소를 하나 만듭니다

섹션 1

GitHub 오른쪽 위 + → New repository 를 누릅니다.

Repository name	깃연습
Description	(비워도 됩니다)
Public / Private	Public 이면 아무나 볼 수 있습니다 ★ 처음에는 Private 으로 하세요
Add a README file	← ★ 체크하지 마세요
Add .gitignore	← ★ 체크하지 마세요
Choose a license	← ★ 고르지 마세요

왜 아무것도 체크하지 말라고 하나 — 체크하면 GitHub 이 파일을 먼저 만들어 버립니다. 그러면 내 컴퓨터의 기록과 GitHub 의 기록이 출발점이 달라져서 첫 push 에서 막힙니다. (섹션 5의 그 오류입니다)
★ 빈 저장소로 만들고 내 것을 밀어 넣는 게 제일 쉽습니다.

Private 으로 시작하세요. 실수로 비밀번호를 올려도 남이 못 봅니다. 포트폴리오로 보여 줄 때 Public 으로 바꾸면 됩니다. (설정 → 맨 아래 Danger Zone → Change visibility)

▢ 직접 해보기 1

깃연습 이라는 이름으로 빈 Private 저장소를 하나 만드세요. 만든 뒤 나오는 안내 화면을 그대로 두세요. 섹션 2에서 씁니다.

내 폴더와 이어 줍니다

섹션 2

내 컴퓨터의 깃연습 폴더에게 “인터넷 쪽 주소는 여기다” 라고 알려 줍니다.

```
$ git remote -v
← 아무것도 없습니다

$ git remote add origin https://github.com/hong/깃연습.git

$ git remote -v
origin https://github.com/hong/깃연습.git (fetch)
origin https://github.com/hong/깃연습.git (push)
```

origin 은 그냥 별명입니다. 관례로 이렇게 씁니다. remote 는 “멀리 있는 것” 이라는 뜻입니다. fetch 는 받아올 때, push 는 보낼 때 쓰는 주소입니다. 보통 같습니다.

주소는 GitHub 화면에 그대로 적혀 있습니다. 저장소를 만들면 나오는 안내에 ...or push an existing repository 밑에 복사할 수 있게 나옵니다. 손으로 치지 말고 복사하세요.

▸ 직접 해보기

2

GitHub 화면의 주소를 복사해 git remote add origin ... 을 치고, git remote -v 로 제대로 들어갔는지 보세요.

밀어 올립니다 (push)

섹션 3

```
$ git branch --show-current
main

$ git push -u origin main
To https://github.com/hong/깃연습.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

-u 는 “앞으로 이 가지는 저기로 보낸다” 고 정해 두는 것입니다. 한 번만 붙이면 됩니다. 다음부터는 git push 만 치면 됩니다.

```
$ git push
To https://github.com/hong/깃연습.git
 9e1ad61..f3bec9d  main -> main
```

9e1ad61..f3bec9d 는 “이 커밋부터 이 커밋까지 보냈다” 는 뜻입니다.

이런 실수를 합니다

잘못된 예

가지 이름이 다르면 막힙니다 내 것이 master 인데 GitHub 은 main 을 씁니다. 개념01 섹션4의 init.defaultBranch main 을 안 했으면 여기서 걸립니다. 고치는 법: git branch -m main (지금 가지의 이름을 바꿉니다)

git push -u origin main 을 치고, GitHub 페이지를 새로고침해서 파일이 올라갔는지 보세요.

★ 비밀번호를 물어보면

섹션 4

처음 push 할 때 로그인 창이 뜹니다. 여기서 많이 막힙니다.

이런 실수를 합니다

잘못된 예

GitHub 비밀번호는 안 통합니다 Support for password authentication was removed. 같은 말이 나옵니다. 2021년부터 비밀번호로는 안 됩니다.

방법이 둘입니다. 둘 중 하나만 하면 됩니다.

이렇게 쓰세요

이렇게 쓰세요

GitHub → 오른쪽 위 프로필 → Settings
→ Developer settings (맨 아래)
→ Personal access tokens → Tokens (classic)
→ Generate new token (classic)

Note	내노트북
Expiration	90 days
Scopes	repo 만 체크

→ Generate token

이런 실수를 합니다

잘못된 예

★★★ 토큰은 비밀번호와 같습니다 코드에 적지 마세요. 카톡으로 보내지 마세요. 화면 공유할 때 조심하세요. 남에게 보였다 싶으면 바로 지우세요. (Settings → Tokens → Delete) 지우면 그 자리에서 못 쓰게 됩니다.

기억시켜 두는 곳을 확인하려면 — 윈도우는 제어판 → 자격 증명 관리자 → Windows 자격 증명 에 git:https://github.com 이 있습니다. 잘못 저장됐으면 여기서 지우고 다시 하세요. ★ “예전 계정으로 로그인돼요” 의 답이 이것입니다.

▹ 직접 해보기 4

push 를 해서 로그인 창을 띄워 보세요. 방법 1로 되면 그걸로 끝입니다. 안 되면 방법 2로 토큰을 만드세요.

받아 오기 (clone · pull)

섹션 5

다른 컴퓨터에서 이어서 하려면 통째로 받아 옵니다.

```
$ git clone https://github.com/hong/깃연습.git 깃연습
Cloning into '깃연습'...
done.
```

clone 은 init + remote add + 내려받기를 한 번에 합니다. git init 을 따로 안 해도 됩니다.

이미 받아 둔 폴더에서 새로 올라온 것만 가져올 때는 pull 입니다.

```
$ git pull
```

이런 실수를 합니다

잘못된 예

```
$ git push
To https://github.com/hong/깃연습.git
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/hong/깃연습.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
```

★★ 남이 먼저 올렸으면 push 가 거부됩니다 “저쪽에 네가 모르는 게 있다. pull 먼저 해라” 는 뜻입니다.

★ 여기서도 hint: 가 답을 그대로 알려 줍니다.

그래서 pull 을 먼저 합니다. 그런데 —

```
$ git pull --no-rebase
From https://github.com/hong/깃연습.git
   f3bec9d..5b9d53a  main      -> origin/main
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

같은 줄을 둘이 고쳤으면 충돌이 납니다. 푸는 법은 개념05 섹션 5에 있습니다. 여기서 그 이야기가 이어집니다.

★ 습관 하나만 들이세요 — 작업 시작 전에 git pull. 그러면 충돌이 훨씬 줄어듭니다.

▹ 직접 해보기 5

GitHub 웹에서 파일을 직접 고쳐 커밋하고(연필 아이콘), 내 컴퓨터에서 git pull 로 받아 보세요.

★ 절대 올리면 안 되는 것

섹션 6

개념03 섹션6에서 말한 것을 다시 짚습니다. 여기서는 진짜로 공개됩니다.

API 키 · 토큰	*** 돈이 나갑니다
비밀번호	.env 파일 통째로
개인정보	주민번호·전화번호·주소가 든 파일
회사 자료	계약서·고객 명단
node_modules/	용량만 잡아먹습니다

이런 실수를 합니다

잘못된 예

Public 저장소는 로봇이 항상 훑고 있습니다 AWS 키를 올리면 몇 분 안에 발견되는 일이 실제로 있습니다. 누군가 그 키로 서버를 잔뜩 켜고, 요금은 올린 사람이 냅니다. ★ 그래서 Private 으로 시작하고, .gitignore 를 처음에 만듭니다. ★★ 지워도 옛 기록에 남습니다. 새어 나갔으면 그 키를 폐기하세요. (개념03 섹션6)

올리기 전에 한 번 보는 습관 — git status 로 무엇이 담겼는지 확인하고 밀으세요. 5초면 됩니다.

▹ 직접 해보기 6

내 저장소를 GitHub 웹에서 열어 파일 목록을 눈으로 훑어보세요. 올리면 안 되는 것이 섞여 있지 않습니까?

자주 하는 실수

섹션 7

이런 실수를 합니다

README 를 체크해서 저장소를 만듦

첫 push 가 거부됩니다. 출발점이 달라서입니다. (섹션 1) 이미 그렸으면 git pull origin main --allow-unrelated-histories 로 붙일 수 있습니다.

이런 실수를 합니다

비밀번호를 넣고 계속 실패

비밀번호는 안 됩니다. 브라우저 로그인이나 토큰을 쓰세요. (섹션 4)

이런 실수를 합니다

토큰을 안 복사하고 창을 닫음

다시 못 봅니다. 새로 만드세요. 흔한 일입니다.

이런 실수를 합니다

git push 만 치고 커밋을 안 함

Everything up-to-date 가 나오는데 GitHub 에는 아무것도 없습니다. 커밋하지 않은 것은 올라가지 않습니다. push 는 커밋을 보내는 것이지 파일을 보내는 게 아닙니다.

이런 실수를 합니다

가지 이름이 master

GitHub 은 main 을 기본으로 씁니다. git branch -m main 으로 바꾸세요. (섹션 3)

이런 실수를 합니다

pull 안 하고 push 만 하다가 거부됨

섹션 5 그대로입니다. 시작 전에 pull 을 습관으로 만드세요.

브랜치와 충돌

RUN 이 파일을 브라우저로 열고, 터미널도 같이 여세요.

00단원 · 개발환경과 Git

연습 폴더에서 같이 따라 하세요. 특히 섹션 4:5는 눈으로 봐야 합니다.

충돌이 나도 아무 일 없습니다. 이 단원에서는 일부러 냅니다.

아래 결과는 실제로 쳐 본 것입니다. 커밋 번호만 다릅니다.

섹션 1 — 가지를 하나 냅니다

```
$ git branch
* main

$ git switch -c 색바꾸기
Switched to a new branch '색바꾸기'
```

-c 는 create 입니다. 만들면서 그리로 옮겨 갑니다. * 는 "지금 여기" 라는 표시입니다.

★ 가지를 내도 파일은 하나도 안 바뀝니다. 폴더를 봐도 똑같습니다. 바뀌는 것은 "지금 어느 줄에 서 있는가" 뿐입니다.

여기서 파일을 고치고 커밋합니다.

```
$ git commit -am "글자를 파랗게"
[색바꾸기 73859b6] 글자를 파랗게
1 file changed, 1 insertion(+)
```

대괄호 안에 main 이 아니라 색바꾸기 입니다. 이 커밋은 그 가지에만 있습니다.

실행하면 나오는 화면 — 브랜치와 충돌

지금까지는 한 줄로만 갔습니다.

첫 페이지 → 설명 문단 → 색 바꾸기 → ...

그런데 이런 일이 생깁니다.

"새 기능을 만들어 보고 싶은데, 잘 안 되면 버리고 싶다"
"둘이 각자 다른 걸 만드는데 서로 안 부딪히게 하고 싶다"

브랜치(가지)가 그것을 합니다. 원래 즐기는 그대로 두고, 옆으로 하나 뻗어서 거기서 일합니다. 잘 되면 합치고, 아니면 버립니다.

★ 이 단원의 진짜 목적은 **충돌을 한 번 내 보는 것**입니다.

충돌은 고장이 아닙니다. 정상입니다. 그런데 처음 보면 파일이 이상해져서 다들 당황합니다. 여기서 일부러 한 번 내고, 풀어 보고 넘어갑니다.

연습 폴더에서 같이 따라 하세요. 특히 섹션 4·5는 눈으로 봐야 합니다. 충돌이 나도 아무 일 없습니다. 이 단원에서는 일부러 냅니다. 아래 결과는 실제로 쳐 본 것입니다. 커밋 번호만 다릅니다.

가지를 하나 냅니다

섹션 1

```
$ git branch
* main

$ git switch -c 색바꾸기
Switched to a new branch '색바꾸기'
```

-c 는 create 입니다. 만들면서 그리로 옮겨 갑니다. * 는 “지금 여기” 라는 표시입니다. ★ 가지를 내도 파일은 하나도 안 바뀝니다. 폴더를 봐도 똑같습니다. 바뀌는 것은 “지금 어느 줄에 서 있는가” 뿐입니다.

여기서 파일을 고치고 커밋합니다.

```
$ git commit -am "글자를 파랑게"
[색바꾸기 73859b6] 글자를 파랑게
1 file changed, 1 insertion(+)
```

대괄호 안이 main 이 아니라 색바꾸기 입니다. 이 커밋은 그 가지에만 있습니다. -am 은 -a(고친 것 전부 담기) + -m(메시지) 입니다. 새로 만든 파일은 -a 로 안 담깁니다. 그건 git add 를 해야 합니다.

원래 줄기로 돌아가 봅시다.

```
$ git switch -
Switched to branch 'main'
```

- 는 “바로 전에 있던 곳” 입니다. git switch main 과 같습니다. ★★ 돌아온 순간 파일 내용이 바뀝니다. VS Code 를 열어 두고 해 보세요. 방금 넣은 파란 줄이 사라져 있습니다. 지워진 게 아니라 그 가지에 있는 것입니다.

▫ 직접 해보기 1

VS Code 에 index.html 을 열어 둔 채로 git switch 로 가지를 오가 보세요. 화면이 저절로 바뀌는 것을 보세요.

어떻게 생겼는지 봅시다

섹션 2

```
$ git log --oneline --all --graph
* 73859b6 글자를 파랑게
* 6d15fca 설명 문단을 넣었습니다
* ae4c0c2 첫 페이지를 만들었습니다
```

--all 은 모든 가지를, --graph 는 그림으로 보여 줍니다. ★ 지금은 한 줄로 나옵니다. main 에서 뺀어 나온 뒤 main 은 가만히 있었기 때문입니다. 갈라진 게 아니라 앞서 나간 것뿐입니다. 섹션 4에서 진짜로 갈라진 모양을 봅니다.

▹ 직접 해보기 2

--all 을 빼고 쳐 보세요. 무엇이 사라집니까? (지금 서 있는 가지에서 보이는 것만 나옵니다)

합칩니다 (merge)

섹션 3

```
$ git merge 색바꾸기
Updating 6d15fca..73859b6
Fast-forward
 index.html | 1 +
 1 file changed, 1 insertion(+)

$ git log --oneline
73859b6 글자를 파랗게
6d15fca 설명 문단을 넣었습니다
ae4c0c2 첫 페이지를 만들었습니다
```

Fast-forward 는 “그냥 따라가면 된다” 는 뜻입니다. main 이 그동안 가만히 있었으니, 표시만 앞으로 옮기면 끝입니다. 새 커밋이 안 생깁니다. ★ 합칠 때 이 말이 나오면 충돌이 날 수가 없습니다. 제일 편한 경우입니다.

다 쓴 가지는 지웁니다. git branch -d 색바꾸기 -d 는 “합쳐진 것만 지워라” 입니다. 안 합쳐진 가지면 거부합니다. 그 거부가 안전장치입니다. (-D 는 무조건 지웁니다. 조심하세요)

▹ 직접 해보기 3

합친 뒤 git branch 로 가지 목록을 보고, -d 로 지워 보세요.

★ 일부러 충돌을 냅니다

섹션 4

같은 파일의 같은 줄을 양쪽에서 다르게 고치면 충돌이 납니다. 만들어 보겠습니다.

```
$ git switch -c 제목바꾸기
Switched to a new branch '제목바꾸기'

(index.html 의 '안녕하세요' 를 '반갑습니다' 로 고칩니다)

$ git commit -am "인사말을 반갑습니다로"
[제목바꾸기 cb46bb8] 인사말을 반갑습니다로
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
$ git switch -
Switched to branch 'main'
```

(같은 줄의 '안녕하세요' 를 '환영합니다' 로 고칩니다)

```
$ git commit -am "인사말을 환영합니다로"
[main b53be6a] 인사말을 환영합니다로
1 file changed, 1 insertion(+), 1 deletion(-)
```

줄 하나가 바뀌었는데 1 insertion(+), 1 deletion(-) 입니다. Git 은 “줄을 고쳤다” 를 “옛 줄을 지우고 새 줄을 넣었다” 로 봅니다. ★ 그래서 개념02의 git diff 에서도 - 와 + 가 짝으로 나옵니다.

이제 합쳐 봅니다.

```
$ git merge 제목바꾸기
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

$ git status --short
UU index.html
```

CONFLICT 가 났습니다. UU 는 개념02 섹션4의 그 기호입니다. 양쪽이 다 고쳤다는 뜻입니다. ★ 고장이 아닙니다. Git 이 “둘 중 뭐가 맞는지 나는 모르겠다, 네가 정해라” 라고 하는 것입니다. 사람이 정할 수밖에 없는 일입니다.

▣ 직접 해보기

4

위 순서를 그대로 해서 충돌을 내 보세요. 무섭지 않다는 것을 손으로 확인하는 게 목적입니다.

충돌을 풉니다

섹션 5

파일을 열어 봅니다. 이렇게 되어 있습니다.

```
<<<<<<< HEAD
<h1>환영합니다</h1>
=====
<h1>반갑습니다</h1>
>>>>>>> 제목바꾸기
<p>처음 만든 페이지입니다.</p>
<p style='color:blue'>파랑</p>
```

읽는 법 <<<<<<< HEAD 여기부터가 ‘지금 내가 서 있는 가지’ 의 것 (내 쪽 내용) ===== 경계선 (상대 쪽 내용) >>>>>>> 제목바꾸기 여기까지가 ‘합치려는 가지’ 의 것 HEAD 는 지금 서 있는 곳입니다. 여기서는 main 입니다.

```
<<<<<<< HEAD          여기부터가 '지금 내가 서 있는 가지' 의 것
(내 쪽 내용)
=====                경계선
(상대 쪽 내용)
>>>>>>> 제목바꾸기     여기까지가 '합치려는 가지' 의 것
```

고치는 방법은 그냥 “파일을 원하는 모양으로 만드는 것”입니다. 특별한 명령이 없습니다. 편집기에서 지우면 됩니다.

(기호 세 줄과 안 쓸 쪽을 지웁니다)

```
<h1>환영합니다</h1>
<p>처음 만든 페이지입니다.</p>
<p style='color:blue'>파랑</p>
```

이런 실수를 합니다

잘못된 예

★ 제일 흔한 실수 — 기호를 안 지움 <<<<<<< 나 ===== 를 남긴 채로 커밋합니다. 그러면 그 기호가 화면에 그대로 보입니다. ★ 커밋하기 전에 파일에서 <<<< 를 검색해 보세요. 하나도 없어야 합니다.

다 고쳤으면 담고 커밋합니다.

```
$ git add index.html

$ git commit -m "인사말 충돌을 풀었습니다"
[main 7c6d6e7] 인사말 충돌을 풀었습니다

$ git log --oneline
7c6d6e7 인사말 충돌을 풀었습니다
b53be6a 인사말을 환영합니다로
cb46bb8 인사말을 반갑습니다로
73859b6 글자를 파랗게
6d15fca 설명 문단을 넣었습니다
ae4c0c2 첫 페이지를 만들었습니다
```

이번에는 1 file changed 같은 줄이 안 나왔습니다. 합치는 커밋은 그 줄을 안 보여 줍니다. 정상입니다. 그리고 cb46bb8(상대 가지의 커밋)이 목록에 들어와 있습니다. 합쳤으니 이제 main 의 역사이기도 합니다.

이렇게 쓰세요

```
$ git merge --abort
```

VS Code 를 쓰면 훨씬 편합니다. 충돌 난 파일을 열면 Accept Current Change / Accept Incoming Change / Accept Both Changes 버튼이 위에 나옵니다. 눌러서 고르면 됩니다. ★ 그래도 기호를 직접

지워 보는 것을 한 번은 해 보세요. 버튼이 뭘 하는지 알고 눌러야 합니다.

▹ 직접 해보기 5

충돌을 손으로 풀어 커밋하고, `git log --oneline` 으로 양쪽 커밋이 다 들어와 있는지 확인하세요.

▹ 직접 해보기 6

충돌을 한 번 더 내고 이번에는 `git merge --abort` 로 빠져나와 보세요. `git status` 가 깨끗해집니까?

자주 하는 실수

섹션 6

이런 실수를 합니다

충돌 기호를 안 지우고 커밋

화면에 <<<<<< HEAD 가 그대로 보입니다. (섹션 5)

이런 실수를 합니다

가지를 안 만들고 main 에서 다 함

틀린 것은 아닙니다. 혼자 연습할 때는 그래도 됩니다. 다만 실험하다 망가지면 되돌리기가 번거롭습니다.

이런 실수를 합니다

고친 것을 안 커밋한 채 switch

error: Your local changes ... would be overwritten by checkout 이 나옵니다. 커밋하거나 되돌린 뒤에 옮기세요.

이런 실수를 합니다

-am 으로 새 파일도 담길 줄 앎

-a 는 이미 알고 있는 파일만 담습니다. 새로 만든 파일은 `git add` 를 해야 합니다. (섹션 1)

이런 실수를 합니다

git branch -D 로 함부로 지움

-d 는 안 합쳐졌으면 거부합니다. -D 는 그냥 지웁니다. 거부당했으면 이유가 있는 것입니다. -D 로 밀어붙이지 마세요.

이런 실수를 합니다

충돌이 나면 뭔가 망가진 줄 앎

정상입니다. 그리고 `git merge --abort` 로 언제든지 되돌아갈 수 있습니다.

개발환경과 Git

푸는 방법

1. 바탕화면에 빈 폴더 하나를 만들고 거기서 하세요.

★ 이 자료 폴더 안에서 하지 마세요. (개념02 섹션1)

2. 문제마다 적힌 대로 터미널에서 명령을 칩니다.

3. '기대 결과'와 모양이 같으면 맞은 것입니다.

커밋 번호(ae4c0c2 같은 것)는 여러분 것과 다릅니다. 정상입니다.

4. 10분 고민해도 안 되면 연습문제_정답.html 을 여세요.

문제 1~10 은 손으로 치는 문제, 11~14 는 답을 적는 문제입니다.

★ 이 단원의 문제는 답을 아는 것보다 **손이 기억하는 것**이 목적입니다.

정답을 보고 나서 반드시 한 번 더 직접 쳐 보세요.

00단원 · 개발환경과 Git

빈 폴더를 하나 만들고 거기서 하세요. 이 자료 폴더 안에서 하면 안 됩니다.

1~10번은 터미널에서 직접 치는 문제입니다. 11~14번은 답을 적는 문제입니다.

커밋 번호는 사람마다 다릅니다. 모양이 같으면 맞은 것입니다.

문제 1 — 깔린 것 확인하기 (개념01 섹션2)

Git 과 Node 의 버전을 확인하는 명령을 각각 치세요.

기대 결과 (숫자는 다릅니다)

```
git version 2.55.0.windows.3
v24.18.0
```

문제 2 — 이름과 메일 확인하기 (개념01 섹션4)

지금 설정된 `user.name` 과 `user.email` 을 화면에 찍어 보세요. 설정하는 게 아니라 확인하는 것입니다.

기대 결과

홍길동

실행하면 나오는 화면 — 개발환경과 Git

빈 폴더를 하나 만들고 거기서 하세요. 이 자료 폴더 안에서 하면 안 됩니다. 1~10번은 터미널에서 직접 치는 문제입니다. 11~14번은 답을 적는 문제입니다. 커밋 번호는 사람마다 다릅니다. 모양이 같으면 맞은 것입니다.

문제 1 — 깔린 것 확인하기 (개념01 섹션2)

Git 과 Node 의 버전을 확인하는 명령을 각각 치세요.

기대 결과 (숫자는 다릅니다)

```
git version 2.55.0.windows.3
v24.18.0
```

문제 2 — 이름과 메일 확인하기 (개념01 섹션4)

지금 설정된 user.name 과 user.email 을 화면에 찍어 보세요. 설정하는 게 아니라 확인하는 것입니다.

기대 결과

```
홍길동
hong@example.com
```

문제 3 — 저장소 만들고 첫 커밋 (개념02 섹션3)

빈 폴더에서 저장소를 만들고, index.html 을 하나 만들어 첫 커밋까지 하세요. 메시지는 첫 페이지를 만들었습니다 로 합니다.

기대 결과 (git log --oneline 을 쳤을 때)

```
ae4c0c2 첫 페이지를 만들었습니다
```

문제 4 — 기호 읽기 (개념02 섹션4)

index.html 에 한 줄을 더 넣고 저장하세요. 그 다음 세 번 git status --short 를 칩니다.

- ① 고치기만 한 직후
- ② git add . 를 한 뒤
- ③ git commit 을 한 뒤

기대 결과

- ① M index.html
- ② M index.html
- ③ (아무것도 안 나옵니다)

★ ①과 ②는 공백의 자리가 다릅니다. 잘 보세요.

문제 5 — 무엇이 바뀌었나 (개념02 섹션5)

파일에 `<p>시험</p>` 한 줄을 더 넣고, 담기 전에 무엇이 바뀌었는지 보는 명령을 치세요.

기대 결과 (마지막 두 줄만 보면 됩니다)

```
<h1>안녕하세요</h1>
+<p>시험</p>
```

문제 6 — 되돌리기 (개념03 섹션1)

문제 5에서 넣은 줄을 커밋하지 말고 원래대로 되돌리세요.

기대 결과 (`git status --short` 를 쳤을 때)

(아무것도 안 나옵니다)

문제 7 — 담은 것 무르기 (개념03 섹션2)

파일을 고치고 `git add .` 까지 한 상태에서, 고친 내용은 남긴 채 담은 곳에서만 빼세요.

기대 결과 (빼기 전 → 뺀 뒤)

```
M index.html
M index.html
```

문제 8 — .gitignore (개념03 섹션5)

폴더에 `.env` 파일을 만들고 `비밀=1234` 라고 씁니다. 그 다음 Git 이 못 본 척하게 만드세요. 그리고 진짜로 무시되는지 확인하는 명령까지 치세요.

기대 결과

```
.gitignore:1:.env .env
```

문제 9 — 가지 만들고 합치기 (개념05 섹션1.3)

인사말이라는 가지를 만들고, 거기서 파일을 고쳐 커밋한 뒤, main 으로 돌아와 합치세요.

기대 결과 (합칠 때)

```
Updating 6d15fca..73859b6
Fast-forward
 index.html | 1 +
 1 file changed, 1 insertion(+)
```

문제 10 — 충돌 내고 풀기 ★ (개념05 섹션4.5)

일부러 충돌을 내고 손으로 푸세요.

- ① 가지 '가' 를 만들고 <h1> 의 글자를 '가나다' 로 고쳐 커밋
- ② main 으로 돌아와 같은 줄을 '라마바' 로 고쳐 커밋
- ③ main 에서 가지 '가' 를 합치기
- ④ 충돌을 풀고 커밋

기대 결과 (③에서)

```
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

★ 다 하고 나면 git merge --abort 로 빠져나오는 것도 한 번 해 보세요.

문제 11 — [손으로] 못 찾는다고 합니다 (개념01)

같이 실습하는 사람이 이렇게 말합니다.

```
"Git 을 분명히 깔았는데 git --version 이 안 돼요.
다시 깔아 봤는데도 똑같아요."
```

(1) 무엇부터 시켜 보겠습니까? (2) 왜 그것이 답이 됩니까? (3) 그래도 안 되면 다음으로 무엇을 보겠습니까?

문제 12 — [손으로] 커밋했는데 안 들어갔습니다 (개념02)

파일을 고치고 git commit -m "고침" 을 쳤는데 이렇게 나옵니다.

```
nothing added to commit but untracked files present
```

(1) 무슨 뜻입니까? (2) 무엇을 빠뜨렸습니까? (3) git status --short 를 치면 무엇이 보이겠습니까?

문제 13 — [손으로] .env 를 올려 버렸습니다 ★ (개념03·04)

API 키가 든 .env 를 실수로 커밋하고 GitHub 에 push 했습니다. 어제 일이고 오늘 알았습니다.

(1) .gitignore 에 .env 를 적었는데도 git status 에 계속 보입니다. 왜입니까? (2) 어떻게 추적에서 뺐니까?
(3) ★ 그렇게 하면 문제가 다 끝납니까? (4) 지금 제일 먼저 해야 할 일은 무엇입니까?

문제 14 — [손으로] push 가 거부됐습니다 (개념04)

git push 를 했더니 이렇게 나옵니다.

```
! [rejected]          main -> main (fetch first)
error: failed to push some refs to '...'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally.
```

(1) 무슨 일이 있었던 것입니까? (2) 무엇을 해야 합니까? (3) 그 다음에 충돌이 났다면 어디를 보고 풀겠습니까? (4) ★ 이런 일이 덜 생기게 하려면 어떤 습관을 들여야 합니까?

단 원 01

HTML 시작하기

OPEN 01_HTML_시작하기

이 글자는 파일 안에 적혀 있던 글자입니다.

김치찌개 8000원

된장찌개 8000원

<!DOCTYPE html>

01 웹페이지와 HTML

02 태그의 생김새

03 문서의 뼈대

04 속성

05 주석과 공백 규칙

- 연습문제

웹페이지와 HTML

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 웹페이지와 HTML

지금 여러분이 보고 있는 이 화면도 웹페이지입니다. 그리고 이 웹페이지는 여러분 컴퓨터 안의 파일 하나입니다. 파일 이름은 개념01_웹페이지와_HTML.html 입니다.

이 자료의 첫 파일이니 큰 그림부터 잡습니다.

내가 쓴 파일 (.html) → 브라우저가 읽음 (크롬) → 화면이 그려짐 (지금 이 화면)

여기서 우리가 하는 일은 맨 왼쪽 하나뿐입니다. 파일에 글자를 적는 일. 그게 전부입니다. 읽고 그리는 일은 브라우저가 알아서 합니다.

그런데 그냥 글자만 적으면 브라우저는 어디가 제목이고 어디가 문단인지 알 수 없습니다. 그래서 글자마다 “이건 문단이야”, “이건 제목이야” 하고 이름표를 붙여 줍니다. 그 이름표를 붙이는 방법이 HTML 입니다.

HTML 은 HyperText Markup Language 의 줄임말입니다. 가운데 Markup(마크업) 이 바로 “표시를 해 둔다” 는 뜻입니다. 프로그래밍 언어가 아닙니다. 계산도 안 하고 조건 판단도 안 합니다. 오직 “이 글자 덩어리가 무엇인지” 를 표시하는 일만 합니다.

이 파일에서는 코드를 많이 쓰지 않습니다. 대신 파일을 만들고, 열고, 고치고, 다시 보는 몸에 밴 습관을 만듭니다. 이 습관이 없으면 앞으로 나오는 모든 내용이 손에 안 잡힙니다.

이 파일은 웹페이지가 무엇인지와 내 손으로 웹페이지 파일을 만들어 여는 방법을 다룹니다. 코드보다 손을 움직이는 내용이 많습니다. 읽기만 하지 말고 꼭 따라 해 보세요. 마지막 섹션에는 01단원에서 앞으로 배울 것의 지도가 있습니다.

웹페이지도 결국 파일입니다

섹션 1

웹페이지는 신기한 무엇이 아니라 글자가 적힌 파일 하나입니다. 브라우저는 그 파일을 읽어서 화면으로 그려 주는 프로그램입니다.

컴퓨터에 있는 파일은 이름 뒤에 점과 몇 글자가 붙어 있습니다.

사진.jpg	그림 파일
보고서.docx	워드 파일
노래.mp3	음악 파일
첫페이지.html	웹페이지 파일

점 뒤에 붙은 이 글자를 확장자라고 부릅니다. 확장자는 “이 파일을 어떤 프로그램으로 열어야 하는지” 를 알려 줍니다.

웹페이지 파일의 확장자는 .html 입니다. 확장자가 .html 이면 더블클릭했을 때 브라우저가 열립니다. 확장자가 .txt 이면 메모장이 열립니다. 내용이 똑같아도 그렇습니다.

지금 보고 있는 이 화면도 마찬가지입니다. 아래 상자 안의 글자는 제가 이 파일에 미리 적어 둔 글자입니다. VS Code 로 이 파일을 열어서 그 글자를 직접 찾아보면 “코드 한 줄 = 화면 한 줄” 이라는 감이 잡힙니다.

화면 점선 상자 안에 이 문장 한 줄이 보입니다

이 글자는 파일 안에 적혀 있던 글자입니다.

브라우저는 크롬, 엣지, 파이어폭스, 사파리 같은 프로그램을 말합니다. 이 자료는 크롬을 기준으로 씁니다. 다른 브라우저를 써도 대부분 같지만, 화면이 조금 다를 수 있습니다.

▸ 직접 해보기

1

VS Code 에서 이 파일을 열고 Ctrl+F 를 눌러 이 글자는 파일 안에 를 검색해 보세요. 코드 안에서 그 줄을 찾으면 “화면의 글자는 파일 어딘가에 반드시 적혀 있다” 를 확인한 것입니다.

HTML 로 하는 일은 딱 하나입니다. 글자 덩어리마다 “이건 무엇이다” 라는 이름표를 붙이는 것입니다.

아래 상자 안에는 이렇게 적혀 있습니다.

```
<p>오늘의 메뉴</p>
<p>김치찌개 8000원</p>
<p>된장찌개 8000원</p>
```

여기서 <p> 와 </p> 가 이름표입니다. p 는 paragraph(문단) 의 줄임말이고, “이 글자 덩어리는 하나의 문단이다” 라는 뜻입니다.

이름표를 붙이면 브라우저가 그 뜻에 맞게 그려 줍니다. 문단이라고 알려 줬으니 문단답게, 즉 줄을 나누고 사이를 띄워서 그립니다. 우리는 “띄워라” 라고 시킨 적이 없습니다. 이름표에 그 뜻이 들어 있는 것입니다.

이름표의 정확한 이름은 태그(tag) 입니다. 태그가 어떻게 생겼는지는 바로 다음 파일인 개념02 에서 뜯어봅니다. 여기서는 “이름표를 붙인다” 는 느낌만 가져가면 충분합니다.

화면 세 문장이 각각 한 줄씩, 모두 세 줄로 나뉘어 보입니다

```
오늘의 메뉴
김치찌개 8000원
된장찌개 8000원
```

▣ 직접 해보기 2

위 상자 안에 <p>순두부찌개 9000원</p> 를 한 줄 더 넣어 보세요. 저장하고(Ctrl+S) 브라우저에서 새로고침(F5) 하면 줄이 하나 늘어납니다.

이름표가 없으면

섹션 3

이름표를 안 붙이고 글자만 쓰면, 아무리 줄을 나눠 써도 브라우저는 전부 한 줄로 이어 붙여 버립니다.

아래 상자 안에는 섹션 2 와 똑같은 글을, 이름표만 빼고 이렇게 적었습니다.

```
오늘의 메뉴
김치찌개 8000원
된장찌개 8000원
```

코드에서는 분명히 세 줄입니다. 그런데 화면에서는 한 줄로 쭉 이어져 나옵니다.

왜 그럴까요? 브라우저는 코드에 있는 줄바꿈을 “그냥 띄어쓰기 한 칸” 으로 취급합니다. 줄을 나누라는 뜻으로 읽지 않습니다. 브라우저 입장에서는 줄을 나눌 이유를 못 들었기 때문입니다.

섹션 2 에서는 왜 세 줄로 나뉘었을까요? p 라는 이름표가 “여기부터 여기까지가 한 문단” 이라고 알려 줬기 때문입니다. 줄을 나눌 것은 우리가 아니라 이름표입니다.

이 규칙은 처음 보면 이상하지만 HTML 전체를 관통하는 성질입니다. 개념05 에서 공백과 줄바꿈만 따로 자세히 다룹니다.

화면	코드에서는 세 줄인데 화면에는 한 줄로 이어져 보입니다
오늘의 메뉴 김치찌개 8000원 된장찌개 8000원	

같은 글자인데 화면이 다릅니다. 달라진 것은 이름표를 붙였는지 여부뿐입니다. 이것이 HTML 을 배우는 이유입니다.

직접 해보기

3

위 상자 안의 글을 브라우저에서 마우스로 드래그해 보세요. 한 줄로 이어진 글자 전체가 끊긴 데 없이 한 덩어리로 칠해집니다. 그다음 섹션 2 의 상자를 드래그해서 어떻게 다른지 비교해 보세요.

내 손으로 .html 파일 만들기

섹션 4

읽기만 해서는 늘지 않습니다. 지금 빈 파일 하나를 직접 만들어 브라우저로 열어 봅니다.

순서는 이렇습니다. 그대로 따라 하세요.

- 1 바탕화면에 새 폴더를 하나 만들고 이름을 html연습 이라고 짓습니다.
- 2 VS Code 를 켵니다. 메뉴에서 파일 → 폴더 열기 를 눌러 html연습 을 엽니다.
- 3 왼쪽 목록 빈 곳에서 마우스 오른쪽 → 새 파일 을 누릅니다.
- 4 파일 이름을 first.html 이라고 칩니다. 확장자 .html 까지 꼭 칩니다.
- 5 아래 코드를 그대로 칩니다. 복사하지 말고 손으로 치세요.
- 6 Ctrl+S 로 저장합니다.
- 7 탐색기에서 first.html 을 더블클릭합니다. 브라우저가 열립니다.

VS Code 가 없다면 메모장으로도 됩니다. 단 메모장은 저장할 때 파일 형식을 모든 파일 로 바꾸고 이름을 first.html 이라고 쳐야 합니다. 그러지 않으면 first.html.txt 가 됩니다. (섹션 7 의 첫 번째 실수가 바로 이것입니다)

아래 코드에서 지금 이해해야 하는 줄은 딱 하나, <p> 로 감싼 줄입니다. 나머지 줄은 모든 HTML 파일에 똑같이 들어가는 뼈대입니다. 뼈대가 왜 이렇게 생겼는지는 개념03 에서 한 줄씩 뜯어봅니다. 지금은 “이건 늘 이렇게 쓰는 것” 이라고 생각하고 그대로 따라 쓰세요.

```
<!DOCTYPE html>
<html lang="ko"> <head> <meta charset="UTF-8" /> <title>내 첫 웹페이지
</title> </head> <body> <p>안녕하세요. 제 첫 웹페이지입니다.</p> </body>
</html>
```

위 코드를 저장하고 브라우저로 열면 아래 상자와 똑같은 화면이 나옵니다. 글자 하나, 줄 하나가 그대로 나옵니다. 브라우저 창 맨 위 탭에는 내 첫 웹페이지 라고 적힙니다. 그 글자는 title 에 적은 글자입니다. 이것도 개념03 에서 다룹니다.

화면 이 문장 한 줄만 보입니다. 다른 것은 아무것도 없습니다

안녕하세요. 제 첫 웹페이지입니다.

브라우저 주소창을 보면 file:///C:/Users/... 로 시작합니다. 인터넷에서 받아 온 것이 아니라 내 컴퓨터의 파일을 그대로 연 것이라는 뜻입니다. 인터넷이 끊겨 있어도 잘 보입니다.

▹ 직접 해보기

4

위 코드로 first.html 을 만들어 브라우저로 열어 보세요. 그다음 <p> 로 감싼 줄의 글자를 여러분 이름으로 바꿔 보세요.

저장하고 새로고침하기

섹션 5

코드를 고쳤는데 화면이 그대로일 때, 원인의 거의 전부는 이 두 가지입니다. 저장을 안 했거나, 새로고침을 안 했거나.

브라우저는 파일을 한 번 읽고 나면 그 내용을 화면에 붙들고 있습니다. 파일이 바뀌었는지 계속 지켜보지 않습니다.

VS Code 에서 코드를 고침

↓

Ctrl+S 로 저장

← 이걸 안 하면 파일은 옛날 그대로입니다

↓

브라우저에서 F5

← 이걸 안 하면 화면은 옛날 그대로입니다

↓

바뀐 화면이 나옴

VS Code 는 저장하지 않은 파일의 탭 이름 옆에 흰 동그라미를 하나 띄웁니다. 그 동그라미가 보이면 아직 저장이 안 된 것입니다. Ctrl+S 를 누르면 사라집니다.

F5 대신 Ctrl+R 을 눌러도 같습니다. 그래도 안 바뀌면 Ctrl+F5 를 누르세요. 브라우저가 기억해 둔 옛날 내용까지 버리고 파일을 처음부터 다시 읽습니다.

이 두 동작은 앞으로 하루에 수백 번 하게 됩니다. 고칠 때마다 Ctrl+S, F5. 손이 저절로 움직일 때까지 몸에 붙이세요.

화면 "이 문장을 고쳐 보세요." 라는 문장 한 줄이 보입니다

이 문장을 고쳐 보세요.

고쳤는데 안 바뀐다고 코드를 다시 뜯어보기 전에, Ctrl+S 를 눌렀는지부터 확인하세요. 초보자가 시간을 가장 많이 버리는 지점입니다.

▹ 직접 해보기 5

위 문장을 VS Code 에서 다른 문장으로 바꾸되, 저장하지 말고 먼저 F5 를 눌러 보세요. 화면이 안 바뀝니다. 그다음 Ctrl+S 로 저장하고 다시 F5 를 눌러 보세요. 이제 바뀝니다.

01단원 지도

섹션 6

01단원은 HTML 이라는 언어의 문법 자체만 다룹니다. 예쁘게 꾸미는 일(CSS)은 07단원부터입니다. 지금은 꾸미지 않습니다.

01단원 파일 다섯 개는 이렇게 이어집니다.

개념01	웹페이지와 HTML	← 지금 이 파일. 파일을 만들고 여는 법
↓		
개념02	태그의 생김새	이름표는 정확히 어떻게 생겼나
↓		
개념03	문서의 뼈대	모든 HTML 파일에 공통으로 들어가는 틀
↓		
개념04	속성	태그에 정보를 덧붙이는 법
↓		
개념05	주석과 공백 규칙	사람만 보는 메모, 그리고 공백이 사라지는 규칙
↓		
연습문제		직접 손으로 풀어 보기

01단원을 마치면 여러분은 "HTML 파일을 하나 만들어서 글을 몇 줄 띄워 보여 줄 수 있는" 상태가 됩니다. 아직 제목도, 목록도, 링크도, 그림도 못 넣습니다. 그건 02~04단원에서 합니다. 조금해하지 말고 이 다섯 개를 확실히 붙잡으세요.

화면 여섯 줄이 위에서 아래로 차례대로 보입니다

개념01 - 웹페이지와 HTML (지금 이 파일)

개념02 - 태그의 생김새

개념03 - 문서의 뼈대

개념04 - 속성

개념05 - 주석과 공백 규칙

연습문제 · 연습문제 정답

▹ 직접 해보기 6

탐색기에서 이 파일이 들어 있는 폴더를 열고, 위 여섯 줄에 해당하는 파일이 실제로 있는지 눈으로 확인해 보세요.

자주 하는 실수

섹션 7

여기 나오는 것들은 에러 메시지가 뜨지 않습니다. 그냥 원하는 화면이 안 나올 뿐입니다. 그래서 스스로 알아채야 합니다.

이런 실수를 합니다

확장자가 .txt 로 붙어 버림

이런 실수를 합니다

실수 1 — 파일 이름이 first.html.txt 가 됨

```
first.html.txt
```

메모장에서 파일 형식을 텍스트 문서 그대로 두고 저장하면 이렇게 됩니다. 더블클릭하면 브라우저가 아니라 메모장이 다시 열립니다. 코드가 화면에 글자 그대로 보이면 십중팔구 이 경우입니다.

이렇게 하세요

```
first.html
```

이런 실수를 합니다

확장자를 잘못 칩

이런 실수를 합니다

실수 2 — 확장자를 .html 로 잘못 칩

```
first.html
```

html 의 글자 순서가 바뀌었습니다. 컴퓨터는 이런 확장자를 모르니 어떤 프로그램으로 열어야 할지 물어보는 창이 뜨거나, 엉뚱한 프로그램이 열립니다. 이름을 고치면 바로 해결됩니다.

이런 실수를 합니다

저장을 안 하고 새로고침

이런 실수를 합니다

실수 3 — 저장하지 않고 새로고침

코드를 고쳤는데 화면이 하나도 안 바뀝니다. 에러도 안 납니다. VS Code 탭 이름 옆에 흰 동그라미가 있으면 아직 저장 전입니다. Ctrl+S 를 누르고 다시 F5 를 누르세요.

이런 실수를 합니다

새로고침을 안 함

이런 실수를 합니다

실수 4 — 저장은 했는데 새로고침을 안 함

이 경우도 화면이 안 바뀝니다. 브라우저는 한 번 읽은 화면을 계속 들고 있습니다. F5 를 눌러야 파일을 다시 읽습니다.

이런 실수를 합니다

워드나 한글로 작성

이런 실수를 합니다

실수 5 — 워드나 한글 프로그램으로 코드를 씀

워드나 한글은 글자 말고도 글꼴·색·표 같은 정보를 파일에 함께 저장합니다. 그래서 브라우저가 읽으면 이상한 글자가 잔뜩 섞여 나오거나 아예 안 열립니다. 코드는 반드시 VS Code 나 메모장처럼 글자만 저장하는 프로그램으로 씁니다.

정리

▫ 직접 해보기

7

정답

1) VS Code 에서 Ctrl+F 를 누르고 이 글자는 파일 안에 를 칩니다. 섹션 1 의 doc-demo 상자 안 줄이 노랗게 표시됩니다.

```
<p id="file-line">이 글자는 파일 안에 적혀 있던 글자입니다.</p>
```

→ 화면은 아무것도 안 바뀝니다. 코드에서 위치를 찾는 것이 목적입니다.

2) 섹션 2 의 doc-demo 상자 안, 마지막 p 아래에 이렇게 넣습니다.

<p>순두부찌개 9000원</p>

→ 상자 안의 줄이 세 줄에서 네 줄로 늘어납니다.

줄 사이 간격도 앞의 줄들과 똑같이 벌어집니다.

3) 섹션 3의 상자를 드래그하면 파란 칠이 한 줄로 죽 이어집니다. 끊긴 데가 없습니다. 코드에서는 세 줄이지만 화면에서는 한 줄이기 때문입니다. 섹션 2의 상자를 드래그하면 파란 칠이 문단마다 끊겨 세 도막으로 보입니다. 문단 사이 여백에는 칠이 안 들어가기 때문입니다.

→ 화면은 바뀌지 않습니다. 브라우저가 두 상자를 다르게 취급한다는 것을

눈으로 확인하는 것이 목적입니다.

4) 섹션 4의 코드를 그대로 쳐서 first.html로 저장하고 더블클릭합니다. 이름을 바꾸려면 p로 감싼 줄만 고칩니다.

<p>안녕하세요. 저는 김민준입니다.</p>

→ 브라우저에 그 한 줄만 보입니다. 탭 이름은 내 첫 웹페이지 그대로입니다.

탭 이름은 title에 적힌 글자라서 p를 고쳐도 안 바뀝니다.

5) 저장하지 않고 F5를 누르면 화면이 그대로입니다. Ctrl+S를 누른 뒤 F5를 누르면 그제서야 바뀝니다.

→ 브라우저가 보는 것은 VS Code 화면이 아니라 저장된 파일이라는 뜻입니다.

6) 폴더 안에 아래 일곱 개 파일이 있으면 맞습니다.

```
개념01_웹페이지와_HTML.html
개념02_태그의_생김새.html
개념03_문서의_뼈대.html
개념04_속성.html
개념05_주석과_공백_규칙.html
연습문제.html
연습문제_정답.html
```

→ 화면은 안 바뀝니다. 앞으로 열게 될 파일의 위치를 익히는 것이 목적입니다.

태그의 생김새

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

01

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 태그의 생김새

앞 파일에서 “HTML 은 글에 이름표를 붙이는 일” 이라고 했습니다. 그 이름표를 태그(tag) 라고 부릅니다. 태그는 항상 이렇게 생겼습니다.

<p>안녕하세요</p>

└──┬──┬──┐

| | └─┬─ 달는 태그 : 이름 앞에 빗금(/) 이 붙습니다

| └─┬─┬─ 내용 : 화면에 실제로 보이는 글자

└─┬─┬─ 여는 태그 : 꺾쇠 < > 로 이름을 감쌉니다

여는 태그부터 닫는 태그까지 통째로 묶어서 요소(element) 라고 부릅니다. 즉 위의 한 덩어리가 “p 요소” 하나입니다.

이 두 단어는 앞으로 계속 나옵니다. 지금 구분해 두세요.

태그 = 꺾쇠로 감싼 이름표 하나 → <p> 또는 </p>
 요소 = 여는 태그 + 내용 + 닫는 태그 덩어리 전체

이 파일은 태그가 어떻게 생겼는지 하나만 다룹니다. 화면에 보이는 결과와, VS Code 에서 본 코드를 계속 번갈아 보세요. “이 글자는 어느 코드에서 나온 걸까?” 를 찾는 연습이 오늘 할 일의 전부입니다.

여는 태그와 닫는 태그

섹션 1

태그는 여는 것과 닫는 것이 짝입니다. 그 사이에 넣은 글자가 화면에 보입니다.

아래 상자 안에 이렇게 썼습니다.

```
<p>저는 첫 번째 문단입니다.</p>
<p>저는 두 번째 문단입니다.</p>
```

p 는 paragraph(문단) 의 줄임말입니다. p 요소는 “이 글자 덩어리는 하나의 문단이다” 라고 알려 줍니다.

브라우저는 문단과 문단 사이를 저절로 띄웁니다. 여러분이 띄우라고 시킨 적이 없는데도 그렇습니다. 태그에 그런 뜻이 이미 들어 있기 때문입니다. 이것이 “글에 이름표를 붙인다” 의 실제 효과입니다.

화면 두 줄이 보이고, 두 줄 사이가 약간 벌어져 있습니다

```
저는 첫 번째 문단입니다.
저는 두 번째 문단입니다.
```

▣ 직접 해보기 1

위 상자 안에 <p>세 번째 문단입니다.</p> 를 한 줄 더 넣어 보세요. 저장하고(Ctrl+S) 브라우저에서 새로고침(F5) 하면 줄이 하나 늘어납니다.

닫는 태그를 빠뜨리면

섹션 2

여기서 HTML 의 가장 중요한 성질이 나옵니다. HTML 은 틀려도 에러가 나지 않습니다. 조용히 이상하게 그려집니다.

프로그래밍 언어를 배워 본 사람은 “틀리면 빨간 글씨가 뜬다” 고 생각합니다. HTML 은 그렇지 않습니다. 브라우저는 어떻게든 화면을 그려 내려고 합니다. 틀린 부분은 자기 나름대로 고쳐서 그립니다.

그래서 HTML 을 배울 때 제일 어려운 것은 에러를 읽는 일이 아니라 “내가 생각한 모양과 화면이 다르다” 를 알아채는 일입니다.

아래 상자 안에는 이렇게 썼습니다. 닫는 태그가 한쪽에 없습니다.

```
<p>닫는 태그를 안 썼습니다.  
<p>이건 그다음 문단입니다.</p>
```

결과를 보면 두 줄이 멀쩡히 나옵니다. 브라우저가 "다음 p 가 시작됐으니 앞의 p 는 여기서 끝난 거겠지" 하고 빠진 닫는 태그를 알아서 채워 넣었기 때문입니다.

운이 좋아서 맞은 것입니다. 다른 태그에서는 이렇게 안 됩니다. 섹션 5 에서 진짜로 망가지는 경우를 봅니다.

화면 그래도 두 줄이 정상으로 보입니다. 에러 메시지는 어디에도 없습니다

```
닫는 태그를 안 썼습니다.  
이건 그다음 문단입니다.
```

브라우저가 고쳐 주니까 안 닫아도 된다는 뜻이 아닙니다. 브라우저마다 고치는 방법이 다를 수 있고, 나중에 CSS 를 붙이면 그때부터 어긋납니다. 항상 닫으세요.

⇒ 직접 해보기 2

섹션 1 의 첫 번째 `<p>` 를 지웠다가 새로고침해 보고, 다시 되돌려 놓으세요. 화면이 안 바뀌는 것을 눈으로 확인하는 것이 목적입니다.

닫지 않는 태그도 있다

섹션 3

내용을 감쌀 필요가 없는 태그는 여는 태그 하나로 끝납니다.

태그는 보통 무언가를 감쌉니다. 그런데 감쌀 내용이 없는 태그가 있습니다.

```
<br />      줄을 바꿉니다  
<hr />      가로줄을 하나 긋습니다
```

"줄바꿈" 은 그 자리에서 일어나는 사건이지, 무언가를 감싸는 일이 아닙니다. 그래서 닫는 태그가 없습니다. 이런 것을 빈 태그라고 부릅니다.

빈 태그는 셋 다 똑같이 동작합니다.

```
<br> <br/> <br />
```

이 자료에서는 마지막 모양인 `<br />` 로 통일해 씁니다. 빈 태그라는 것이 눈에 잘 띄기 때문입니다.

화면 "줄을" 과 "여기서 바꿨습니다." 가 두 줄로 나뉘고,

그 아래에 회색 가로줄이 하나 그어집니다

```
줄을여기서 바꿨습니다.
```

```
가로줄 아래입니다.
```


 를 줄 간격 벌리는 용도로 여러 개 쓰지 마세요. 간격은 09~10단원에서 CSS 로 조절합니다.
 는 주소나 시(詩)처럼 진짜로 그 자리에서 줄이 바뀌어야 하는 글에만 씁니다.

▫ 직접 해보기

3

위 상자에 <hr /> 를 하나 더 넣어 가로줄을 두 개로 만들어 보세요.

태그 안에 태그 넣기 (중첩)

섹션 4

태그 안에 다른 태그를 넣을 수 있습니다. 이것을 중첩이라고 합니다.

문단 안의 한 단어만 굵게 만들고 싶다고 합시다. p 요소의 내용 안에 strong 요소를 넣으면 됩니다.

```
<p>오늘은 <strong>화요일</strong>입니다.</p>
```

여기서 관계를 부르는 이름이 생깁니다.

p 는 strong 의 부모(parent)
strong 은 p 의 자식(child)

부모·자식이라는 말은 앞으로 CSS 를 배울 때 계속 나옵니다. 지금은 “감싼 쪽이 부모, 감싸인 쪽이 자식” 만 기억하면 됩니다.

중첩에는 지켜야 하는 규칙이 하나 있습니다. 나중에 연 태그를 먼저 닫습니다. 상자를 포개는 것과 같습니다.

맞음 <p>오늘은 화요일입니다.</p>
 ↳ p 를 열고 → strong 을 열고 → strong 을 닫고 → p 를 닫음

틀림 <p>오늘은 화요일</p>
 ↳ strong 을 열어 놓고 p 를 먼저 닫아 버렸습니다

틀린 쪽도 에러는 안 납니다. 섹션 5 에서 실제 결과를 봅니다.

화면 "화요일" 세 글자만 굵게 보이고 나머지는 보통 굵기입니다

오늘은 화요일입니다.

코드를 쓸 때 자식은 한 단계 안으로 들여씁니다(스페이스 2칸). 브라우저는 들여쓰기를 보지 않지만, 사람이 부모·자식 관계를 눈으로 찾을 수 있게 됩니다.

▫ 직접 해보기

4

위 문장에서 “오늘은” 도 굵게 만들어 보세요. 을 하나 더 쓰면 됩니다.

자주 하는 실수

섹션 5

다시 한 번 강조합니다. 아래 실수들은 전부 에러가 나지 않습니다. 화면이 이상해질 뿐입니다. 그래서 눈으로 알아채는 연습이 필요합니다.

이런 실수를 합니다

빗금을 빠뜨림

이런 실수를 합니다

실수 1 — 닫는 태그에 빗금(/) 을 안 씀

```
<p>안녕하세요<p>
```

닫으려고 쓴 <p> 가 사실은 새 문단을 여는 태그입니다. 그래서 빈 문단이 하나 더 생깁니다. 글자는 그대로 보이니 알아채기 어렵습니다.

이렇게 쓰세요

```
<p>안녕하세요</p>
```

이런 실수를 합니다

대문자로 닫기 — 이걸 사실 괜찮습니다

이런 실수를 합니다

실수 2 — 태그 이름을 대문자로 씀

```
<P>안녕하세요</P>
```

이건 실제로 잘 동작합니다. 태그 이름은 대소문자를 가리지 않습니다. 그런데도 실수로 치는 이유는, 요즘 코드는 전부 소문자로 쓰기로 약속되어 있어서 대문자가 섞이면 남이 읽을 때 걸리기 때문입니다. 항상 소문자로 쓰세요.

이런 실수를 합니다

중첩 순서를 어김

이런 실수를 합니다

실수 3 — 나중에 연 태그를 나중에 닫음

```
<p>오늘은 <strong>화요일</p></strong>
```

브라우저가 알아서 순서를 바로잡아 줍니다. 이 예에서는 운이 좋아서 제대로 쓴 것과 완전히 똑같은 결과가 나옵니다. 그래서 화면만 보서는 틀린 줄을 영영 모릅니다. 순서를 어겨서 쓴 실제 결과 오늘은 화요일 문제는 겹치는 태그가 늘어날 때입니다. 그때는 브라우저가 요소를 둘로 쪼개거나 바깥으로 밀어내서 내가 쓴 것과 다른 구조를 만듭니다. 08단원에서 CSS로 요소를 골라 꾸밀 때 그 어긋난 구조 때문에 원하는 대로 안 먹습니다. 운에 기대지 말고 나중에 연 것을 먼저 닫으세요.

이런 실수를 합니다

꺾쇠를 빠뜨림

이런 실수를 합니다

실수 4 — 꺾쇠를 빠뜨림

```
p>안녕하세요</p>
```

여는 꺾쇠 < 가 없으니 태그로 인식되지 않습니다. 화면에 p>안녕하세요 라는 글자가 그대로 보입니다. "코드가 화면에 글자로 보인다" 면 십중팔구 꺾쇠나 빗금이 빠진 것입니다. p>안녕하세요

이런 실수를 합니다

한글 자판 상태에서 침

이런 실수를 합니다

실수 5 — 한글 자판으로 태그를 침

```
<ㄱ>안녕하세요</ㄱ>
```

p를 치려다 한영 전환을 안 해서 ㄱ가 들어갔습니다. 초보자에게 가장 많이 나는 실수입니다. 그런데 결과가 좀 이상합니다. 안녕하세요 안녕하세요" 까지만 보입니다. 뒤쪽 닫는 태그는 사라졌습니다 → 앞의 <ㄱ>는 글자로 그대로 보이고, 뒤의 </ㄱ>는 흔적도 없이 사라집니다. 브라우저는 꺾쇠 다음에 영어 글자가 와야만 태그로 봅니다. ㄱ는 영어가 아니니 앞쪽은 그냥 글자 취급을 받고(실수 4와 같은 상황), 뒤쪽 </로 시작한 것은 주석으로 삼켜져 화면에서 없어집니다. 그래서 증상이 "글자가 반쪽만 이상하게 보인다"로 나타납니다. 화면에 < 나 >가 보이면 한영 전환부터 확인하세요.

정리

▫ 직접 해보기 5

정답

1) 섹션 1 의 doc-demo 상자 안, 두 번째 p 아래에 이렇게 넣습니다.

```
<p>세 번째 문단입니다.</p>
```

→ 상자 안의 줄이 두 줄에서 세 줄로 늘어납니다.

줄 사이 간격도 똑같이 벌어집니다.

2) 섹션 1 의 첫 줄을 이렇게 바꿨다가

```
<p>저는 첫 번째 문단입니다.
```

새로고침하면 화면이 그대로입니다. 브라우저가 빠진 닫는 태그를 알아서 채웠기 때문입니다. 확인했으면 </p> 를 다시 붙여 두세요.

3) 섹션 3 의 상자 안, 마지막 p 아래에 이렇게 넣습니다.

```
<hr />
```

→ 가로줄이 두 개가 됩니다. 빈 태그라서 닫는 태그가 없다는 점을 확인하세요.

4) 섹션 4 의 문장을 이렇게 바꿉니다.

```
<p id="nest-demo"><strong>오늘은</strong> <strong>화요일</strong>입니다.</p>
```

→ “오늘은” 과 “화요일” 이 둘 다 굵어지고, 가운데 “입니다” 만 보통 굵기로 남습니다.

strong 요소가 두 개가 되었고, 둘 다 p 의 자식입니다.

문서의 뼈대

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 문서의 뼈대

개념02 에서 태그 하나가 어떻게 생겼는지 봤습니다. 그런데 태그를 아무 데나 풀어 놓는다고 웹페이지가 되지는 않습니다. HTML 파일에는 정해진 틀이 있습니다. 그것을 뼈대라고 부릅니다.

모든 HTML 파일은 예외 없이 이 모양입니다.

```
<!DOCTYPE html>
<html lang="ko"> <head>
  ... 화면에 안 보이는 정보 ...
</head> <body>
  ... 화면에 보이는 내용 ...
</body>
</html>
```

네 부분으로 나뉩니다.

```

<!DOCTYPE html>  이 파일은 HTML 입니다 라는 알림
<html>           전체를 감싸는 가장 바깥 상자
<head>           화면에 안 보이는 정보를 담는 곳
<body>           화면에 보이는 내용을 담는 곳

```

핵심은 head 와 body 의 차이 하나입니다.

```

head 에 넣은 것 → 화면에 안 보입니다
body 에 넣은 것 → 화면에 보입니다

```

개념01 에서 만든 first.html 도 이 뼈대였습니다. 그때는 “그냥 따라 쓰세요” 라고 했습니다. 이제 한 줄씩 뜯어봅니다.

이 파일은 모든 HTML 파일에 똑같이 들어가는 네 부분을 다룹니다. 가장 중요한 것은 head 는 안 보이고 body 는 보인다는 차이입니다. 마지막에는 한글이 깨져 보이는 사고의 원인을 직접 만들어 봅니다.

뼈대 전체 그림

섹션 1

HTML 파일은 상자 안에 상자가 들어 있는 구조입니다. 먼저 전체 모양을 눈에 익힙니다.

아래 코드가 HTML 파일 한 벌의 최소 형태입니다. 이보다 더 줄일 수는 있지만, 실무에서는 언제나 이 모양으로 씁니다.

상자가 어떻게 포개져 있는지 보세요.

```

html 상자
├ head 상자 → 안 보이는 정보
└ body 상자 → 보이는 내용

```

html 이 head 와 body 를 감싸고 있습니다. 개념02 에서 배운 말로 하면 html 은 부모, head 와 body 는 자식입니다. 그리고 head 와 body 는 서로 형제(sibling) 입니다. 형제란 같은 부모 아래 나란히 있는 요소들을 말합니다.

<!DOCTYPE html> 만 상자 밖에 혼자 있습니다. 이유는 섹션 2 에서 설명합니다.

들여쓰기를 보세요. 감싸인 쪽이 두 칸씩 안으로 들어가 있습니다. 브라우저는 들여쓰기를 신경 쓰지 않지만, 사람은 이걸 보고 어느 상자가 어느 상자 안에 있는지 한눈에 알아봅니다.

```

<!DOCTYPE html>
<html lang="ko"> <head> <meta charset="UTF-8" /> <title>가게 안내
</title> </head> <body> <p>오늘도 문을 엽니다.</p> </body>
</html>

```

위 코드를 저장해서 브라우저로 열면 아래 상자와 같은 화면이 나옵니다. 코드는 열 줄인데 화면에 보이는 것은 한 줄뿐입니다. 나머지 아홉 줄은 화면에 안 나오는 뼈대이기 때문입니다.

화면 이 한 줄만 보입니다. head 에 적은 글자는 하나도 안 보입니다

오늘도 문을 엽니다.

▫ 직접 해보기 1

VS Code 에서 지금 이 파일의 맨 위로 올라가 `<html lang="ko">`, `<head>`, `<body>` 가 어디에 있는지 찾아보세요. 각각의 닫는 태그도 같이 찾아보세요.

DOCTYPE

섹션 2

파일의 맨 첫 줄에 늘 이것을 씁니다. 태그처럼 생겼지만 태그가 아니라 알림입니다.

`<!DOCTYPE html>` 을 소리 내어 읽으면 “독타입 에이치티엠엘” 입니다. DOCTYPE 은 document type, 즉 문서 종류라는 뜻입니다. 브라우저에게 “이 파일은 요즘 방식의 HTML 입니다” 라고 알려 줍니다.

태그가 아닌 이유는 세 가지로 눈에 보입니다.

1 닫는 태그가 없습니다. `<!/DOCTYPE>` 같은 것은 없습니다.

2 이름 앞에 느낌표가 붙어 있습니다.

3 감싸는 내용이 없습니다. 그냥 선언 한 줄로 끝입니다.

DOCTYPE 은 대소문자를 가리지 않아서 `<!doctype html>` 이라고 써도 똑같이 동작합니다. 그래도 이 자료는 관례대로 대문자 `<!DOCTYPE html>` 로 씁니다.

빠면 어떻게 될까요? 여기서도 에러는 안 납니다. 화면도 대충 비슷하게 나옵니다. 그래서 빼먹고도 한참 모릅니다. 문제는 브라우저가 아주 옛날 규칙으로 화면을 그리기 시작한다는 점입니다. 지금은 차이가 잘 안 보이지만, 10단원에서 상자 크기를 다루기 시작하면 같은 코드인데 크기가 몇 픽셀씩 어긋나는 원인이 됩니다.

그러니 이유를 따지지 말고 첫 줄에 무조건 쓰세요. 고민할 거리가 아닌 몇 안 되는 규칙입니다.

화면 이 문장 한 줄이 보입니다. DOCTYPE 자체는 화면에 안 나옵니다

이 파일도 첫 줄에 DOCTYPE 이 적혀 있습니다.

`<!DOCTYPE html>` 위에는 아무것도 쓰지 않습니다. 빈 줄이나 주석이 위에 있으면 브라우저가 첫 줄을 못 찾을 수 있습니다.

▣ 직접 해보기 2

개념01에서 만든 first.html을 열어 첫 줄 `<!DOCTYPE html>`을 지우고 저장한 뒤 새로고침해 보세요. 화면이 거의 그대로인 것을 확인하고, 다시 되돌려 놓으세요.

html과 lang

섹션 3

html 요소는 문서 전체를 감싸는 하나뿐인 바깥 상자입니다. DOCTYPE을 뺀 모든 것이 이 안에 들어갑니다.

html 태그는 파일에 딱 한 번만 나옵니다.

```
<html lang="ko">
  ... 여기 안에 head와 body가 들어갑니다 ...
</html>
```

여기 붙은 `lang="ko"`는 속성(attribute)입니다. 속성은 태그에 정보를 덧붙이는 장치입니다. 생김새와 규칙은 바로 다음 파일인 개념04에서 제대로 배웁니다. 지금은 `lang="ko"`라는 덩어리를 통째로 외워 쓰면 됩니다.

`lang`은 language(언어)의 줄임말이고 `ko`는 Korean(한국어)입니다. 즉 “이 문서는 한국어로 쓰였습니다”라고 알려 주는 것입니다.

빠면 화면은 하나도 안 바뀝니다. 여기서도 에러는 없습니다. 대신 이런 일이 생깁니다.

화면 낭독기가 한국어 글자를 영어 발음으로 읽으려고 합니다.
화면 낭독기는 눈이 불편한 분이 쓰는, 화면의 글자를 소리로 읽어 주는 프로그램입니다. 앞으로 여러 단원에서 계속 나옵니다.

브라우저의 번역 기능이 무슨 언어인지 몰라 엉뚱하게 동작합니다.

화면에 안 보인다고 쓸모없는 것이 아닙니다. 이렇게 사람 눈에는 안 보이지만 기계가 읽는 정보가 HTML에는 꽤 많습니다.

화면 이 문장 한 줄이 보입니다. `lang="ko"`는 화면에 흔적을 남기지 않습니다

이 문서는 한국어로 쓰였다고 선언되어 있습니다.

▣ 직접 해보기 3

first.html의 `<html lang="ko">`를 `<html lang="en">`으로 바꾸고 저장한 뒤 새로고침해 보세요. 화면이 하나도 안 바뀌는 것을 확인하고 `ko`로 되돌리세요.

head 안에 넣은 것은 화면에 나오지 않습니다. 대신 브라우저가 이 문서를 어떻게 다룰지 알아내는 데 쓰입니다.

head 는 사람 머리가 아니라 “맨 앞부분” 이라는 뜻입니다. 여기에는 문서에 대한 설명을 넣습니다. 문서의 내용이 아니라 문서에 대한 정보입니다.

01단원에서 쓰는 head 안 식구는 둘뿐입니다.

```
<meta charset="UTF-8" />   글자를 어떤 방식으로 읽을지 (섹션 6)
<title>...</title>         문서의 제목
```

title 은 화면 본문에는 안 나옵니다. 그 대신 이런 곳에 나옵니다.

브라우저 창 맨 위의 탭 이름
즐거찾기에 저장했을 때 저장되는 이름
검색 결과에 제목으로 뜨는 글자

지금 브라우저 탭을 한번 보세요. 01단원 개념03 - 문서의 뼈대 라고 적혀 있을 것입니다. 그 글자는 이 파일 head 안 title 에 적힌 글자와 똑같습니다. VS Code 에서 이 파일 5번째 줄을 열어 확인해 보세요.

title 을 아예 안 쓰면 탭에 파일 이름이 그대로 나옵니다. 보기에 좋지 않으니 언제나 씁니다.

참고로 이 자료 파일들은 head 에 한 줄이 더 있습니다.

```
<link rel="stylesheet" href="../공통.css" />
```

자료의 겉모양을 만드는 줄입니다. 07단원에서 배웁니다. 지금은 신경 쓰지 마세요.

화면 이 문장 한 줄이 보입니다. title 글자는 본문 어디에도 안 나옵니다

이 상자 위쪽 탭에 적힌 글자가 title 입니다.

head 안에 <p>안녕하세요</p> 같은 것을 넣으면 어떻게 될까요? 브라우저가 “이건 본문이네” 하고 몰래 body 로 옮겨서 그립니다. 에러는 안 납니다. 여기서도 조용히 고쳐 버립니다.

▹ 직접 해보기

4

first.html 의 <title> 안 글자를 내 연습장 으로 바꾸고 저장한 뒤 새로고침해서 브라우저 탭 이름이 바뀌는지 확인해 보세요.

body

화면에 보이고 싶은 것은 전부 body 안에 씁니다. 앞으로 배울 태그의 거의 전부가 여기 들어갑니다.

body 는 몸통이라는 뜻입니다. 문서의 알맹이가 들어가는 곳입니다.

```
<body> <p>오늘도 문을 엽니다.</p> <p>영업시간은 아침 9시부터입니다.</p>
</body>
```

지금 화면에 보이는 이 글자들도 전부 이 파일 body 안에 적혀 있습니다. 제목 줄도, 섹션 상자도, 이 문장도 그렇습니다.

head 와 body 를 다시 붙여서 비교하면 이렇습니다.

```
head 안의 title → 탭 이름으로만 나옴. 본문에는 안 나옴
body 안의 p      → 본문에 그대로 나옴
```

같은 글자를 head 에 쓰느냐 body 에 쓰느냐로 결과가 완전히 달라집니다. “안 보이는 정보는 head, 보이는 내용은 body” 이 한 줄이 이 파일의 핵심입니다.

body 는 파일에 딱 하나만 있습니다. 두 개를 쓰면 브라우저가 뒤에 나온 것을 무시하고 하나로 합쳐 버립니다. 이때도 에러는 안 납니다.

화면 두 줄이 보이고, 두 줄 사이가 약간 벌어져 있습니다

오늘도 문을 엽니다.
영업시간은 아침 9시부터입니다.

▸ 직접 해보기 5

first.html 의 <body> 안에 <p> 문단을 두 개 더 넣어 보세요. 그다음 그중 한 줄을 잘라내 <head> 안으로 옮기고 새로고침해서 어떻게 되는지 보세요.

charset 과 한글 깨짐

섹션 6

한글이 알아볼 수 없는 글자로 보이는 사고는 거의 전부 이 한 줄 때문입니다. 원인을 알면 3초 만에 고칩니다.

컴퓨터는 글자를 그대로 저장하지 않습니다. 숫자로 바꿔서 저장합니다. 글자를 숫자로 바꾸는 방식은 여러 가지가 있습니다.

```
UTF-8   전 세계 글자를 다 담는 요즘 표준 방식
EUC-KR  한국에서 옛날에 쓰던 방식
그 밖에 나라마다 여러 가지
```

문제는 여기서 생깁니다. 파일에는 숫자만 남아 있어서, 파일만 봐서는 어떤 방식으로 저장했는지 알 수 없습니다. 저장할 때 쓴 방식과 읽을 때 쓴 방식이 다르면 글자가 엉망이 됩니다.

```
UTF-8 로 저장한 안녕하세요 를 EUC-KR 로 읽으면
아래 상자에 있는 것 같은 글자가 나옵니다.
```

그래서 파일 안에 이렇게 적어 둡니다.

```
<meta charset="UTF-8" />
```

charset 은 character set, 즉 글자 집합이라는 뜻입니다. “이 파일은 UTF-8 로 저장했으니 UTF-8 로 읽어 주세요” 라는 쪽지입니다.

meta 는 닫는 태그가 없는 빈 태그입니다. 개념02 섹션 3 에서 배운
 와 같은 종류입니다. 그리고 반드시 head 안에, 그중에서도 되도록 맨 위에 씁니다. 브라우저가 글자를 읽기 시작하기 전에 이 쪽지를 먼저 봐야 하기 때문입니다.

그럼 charset 을 아예 안 쓰면 어떻게 될까요? 브라우저가 추측합니다. 요즘 크롬은 이 추측을 꽤 잘해서, 안 써도 멀쩡히 보일 때가 많습니다. 하지만 추측은 추측입니다. 아래 상황에서는 빗나갑니다.

파일에 영어가 대부분이고 한글은 몇 글자뿐일 때
다른 브라우저나 다른 나라 컴퓨터에서 열었을 때
인터넷에 올렸는데 서버가 다른 방식이라고 알려 줄 때

추측이 빗나가면 아래 상자처럼 됩니다. 그러니 언제나 씁니다.

화면 알아볼 수 없는 글자와 검은 마름모 안 물음표가 섞여서 두 줄로 보입니다

뎡꿔쌔꺠꺠
꺠꺠- 꺠꺠꺠 꺠꺠꺠꺠

위 두 줄은 원래 안녕하세요 와 우리 동네 빵집 이었습니다. 글자가 사라진 것이 아니라 잘못 읽힌 것입니다. 읽는 방식만 바로잡으면 원래 글자가 그대로 돌아옵니다.

반대 방향도 있습니다. 메모장에서 파일 형식을 ANSI 로 저장해 놓고 charset="UTF-8" 이라고 써 두면, 한글 자리에 검은 마름모가 줄줄이 나옵니다. 저장한 방식과 적어 둔 방식을 맞추는 것이 전부입니다. VS Code 는 기본이 UTF-8 이라 그냥 쓰면 맞습니다.

▸ 직접 해보기

6

first.html 의 <meta charset="UTF-8" /> 를 <meta charset="EUC-KR" /> 로 바꾸고 저장한 뒤 새로고침해 보세요. 한글이 깨지는 것을 직접 보고, 다시 UTF-8 로 되돌리세요.

자주 하는 실수

섹션 7

아래 다섯 가지도 전부 에러가 나지 않습니다. 브라우저가 조용히 자기 방식으로 처리해 버립니다.

이런 실수를 합니다

보이라고 만든 것을 head 에 씀

이런 실수를 합니다

실수 1 — 화면에 보이길 원하는 글을 head 에 씀

```
<head>
<title>가게 안내</title>
<p>오늘도 문을 엽니다.</p>
</head>
```

브라우저가 <p> 를 보는 순간 “여기부터는 본문이구나” 하고 멋대로 body 를 시작해 버립니다. 결과가 우연히 맞아 보일 때도 있지만, 내가 쓴 구조와 브라우저가 만든 구조가 달라져서 나중에 CSS 가 안 먹는 원인이 됩니다.

이렇게 쓰세요

```
<head>
<title>가게 안내</title>
</head>
<body>
<p>오늘도 문을 엽니다.</p>
</body>
```

이런 실수를 합니다

charset 을 body 에 씀

이런 실수를 합니다

실수 2 — charset 을 body 안이나 파일 아래쪽에 씀

```
<body>
<meta charset="UTF-8" />
<p>안녕하세요</p>
</body>
```

브라우저는 이미 글자를 다 읽은 뒤에 이 쪽지를 보게 됩니다. 늦게 본 쪽지는 소용이 없어서 그냥 무시됩니다. 한글이 깨질 수 있는데도 코드에는 charset 이 분명히 적혀 있어서 원인을 찾기 어렵습니다. charset 은 head 안 맨 위에 씁니다.

이런 실수를 합니다

DOCTYPE 위에 뭔가를 씀

이런 실수를 합니다

실수 3 — DOCTYPE 위에 빈 줄이나 다른 글자를 둠

```
내가 만든 페이지
<!DOCTYPE html>
<html lang="ko">
```

맨 위에 있던 내가 만든 페이지 라는 글자가 화면 맨 위에 그대로 튀어나옵니다. 그리고 DOCTYPE 이 첫 줄이 아니게 되어 브라우저가 옛날 규칙으로 그리기 시작합니다. <!DOCTYPE html> 은 언제나 파일의 첫 줄입니다.

이런 실수를 합니다

head 나 body 를 안 닫음

이런 실수를 합니다

실수 4 — head 를 닫지 않고 body 를 시작함

```
<head>
<title>가게 안내</title>
<body>
<p>오늘도 문을 엽니다.</p>
</body>
```

닫는 </head> 가 없습니다. 그런데도 화면은 멀쩡하게 나옵니다. 브라우저가 알아서 닫아 주기 때문입니다. 운이 좋아 맞은 것이니 습관으로 삼지 마세요. 여는 것마다 반드시 닫으세요.

이런 실수를 합니다

title 을 두 개 씀

이런 실수를 합니다

실수 5 — title 을 두 개 씀

```
<title>가게 안내</title>
<title>영업시간</title>
```

탭에는 앞의 것 하나만 나오고 뒤의 것은 조용히 버려집니다. "탭 이름을 바꿨는데 왜 안 바뀌지?" 하고 헤매게 됩니다. 위쪽에 title 이 이미 있는지 먼저 확인하세요.

속성

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

01

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

마우스를 올려 보세요.

달는 태그에 쓴 속성은 그냥 무시됩니다.

속성 세 개를 붙인 문단입니다.

공백이 있는 값에서 따옴표를 뺐습니다.

따옴표로 감싸면 값이 통째로 들어갑니다.

김치찌개 8000원

된장찌개 8000원

실행하면 나오는 화면 — 속성

개념03 에서 이런 것을 봤습니다.

```
<html lang="ko">
<meta charset="UTF-8" />
```

태그 이름 뒤에 뭔가가 하나 더 붙어 있습니다. 그때는 “통째로 외워 쓰세요” 하고 넘어갔습니다. 이제 정체를 밝힙니다.

이렇게 태그에 덧붙이는 정보를 속성(attribute) 이라고 부릅니다.

```
<p title="가게 안내">오늘도 문을 엽니다.</p>
|   |   |
|   |   └─ 값(value) : 어떤 정보인지. 따옴표로 감쌉니다
|   └─ 이름(name) : 무엇을 덧붙이는지
└─ 태그 이름
```

태그가 “이건 문단이다” 라고 종류를 말한다면, 속성은 “그런데 이 문단은 이러이러한 문단이다” 라고 사정을 덧붙입니다.

태그 = 무엇인가 → p 요소다
속성 = 그것에 대한 추가 정보 → 이 p 의 설명은 가게 안내 다

모양은 언제나 이 하나뿐입니다.

이름="값"

이름과 값 사이에 등호, 값 양쪽에 따옴표. 이게 전부입니다. 이 파일에서는 그 규칙과, 규칙을 어겼을 때 조용히 벌어지는 일을 봅니다.

이 파일은 속성의 문법 이름="값" 하나를 다룹니다. 속성은 대부분 화면 모양을 바꾸지 않습니다. 그래서 잘못 써도 티가 안 납니다. “붙였는데 화면이 그대로다” 를 이상하게 여기지 않는 것이 이 파일의 목표입니다.

속성의 생김새

섹션 1

속성은 여는 태그 안, 태그 이름 뒤에 한 칸 띄우고 씁니다. 모양은 언제나 이름="값" 입니다.

아래 상자 안에는 이렇게 썼습니다.

<p title="이 문단은 인사말입니다">마우스를 올려 보세요.</p>

하나씩 뜯어봅니다.

p	태그 이름
(한 칸 띄움)	태그 이름과 속성 사이에는 반드시 공백이 있습니다
title	속성 이름
=	등호. 앞뒤로 공백 없이 붙여 씁니다
"이 문단은 인사말입니다"	속성 값. 큰따옴표로 감쌉니다

여기서 title 은 개념03 에서 본 <title> 태그와 이름만 같고 완전히 다른 것입니다. 태그 title 은 브라우저 탭 이름이고, 속성 title 은 그 요소에 붙이는 짧은 설명입니다. HTML 에는 이렇게 이름이 겹치는 것이 가끔 있습니다. 자리를 보고 구분하세요.

화면 결과를 보세요. 글자가 굵어지지도, 색이 바뀌지도 않았습니. 속성은 대부분 이렇습니다. 화면에 흔적을 남기지 않습니다. 그래서 초보자는 “안 먹었나?” 하고 헛갈립니다. 속성은 브라우저에게 정보를 주는 것이지 모양을 바꾸는 것이 아닙니다. 모양을 바꾸는 일은 07단원부터 CSS 가 맡습니다.

화면 "마우스를 올려 보세요." 한 줄이 보입니다. 글자 모양은 하나도 안 달라졌습니다

마우스를 올려 보세요.

위 문장 위에 마우스 포인터를 올린 채 1초쯤 가만히 두면, 포인터 옆에 작은 상자가 뜨면서 이 문단은 인사말입니다 가 나타납니다. title 속성에 적은 값을 브라우저가 그렇게 보여 주는 것입니다.

▹ 직접 해보기 1

위 문단의 title 값을 여러분이 하고 싶은 말로 바꾸고 저장(Ctrl+S) 후 새로고침(F5) 해서 마우스를 올려 보세요.

속성은 여는 태그에만

섹션 2

닫는 태그에는 절대 속성을 쓰지 않습니다. 써도 에러는 안 나지만, 브라우저가 통째로 버립니다.

규칙은 짧습니다.

맞음 `<p title="가게 안내">오늘도 문을 엽니다.</p>`
틀림 `<p>오늘도 문을 엽니다.</p title="가게 안내">`

왜 여는 태그에만 쓸까요? 속성은 “이 요소는 이런 요소다” 라는 정보입니다. 요소가 시작되는 자리에서 한번 알려 주면 충분합니다. 닫는 태그는 “여기서 끝” 이라는 표시일 뿐이라 정보를 담을 자리가 아닙니다.

아래 상자에는 진짜로 닫는 태그에 속성을 써 봤습니다.

```
<p id="close-attr">닫는 태그에 쓴 속성은 그냥 무시됩니다.</p id="close-attr">
```

결과는 멀쩡합니다. 글자도 그대로 나옵니다. 브라우저가 닫는 태그의 속성을 읽지도 않고 버렸기 때문입니다.

여기가 무서운 지점입니다. 틀리게 썼는데 화면이 멀쩡하면, 틀린 줄을 영영 모릅니다. 나중에 “왜 이 속성이 안 먹지?” 하고 몇 시간을 헤매게 됩니다.

화면 이 문장 한 줄이 아무 문제 없이 보입니다. 경고도 표시도 없습니다

닫는 태그에 쓴 속성은 그냥 무시됩니다.

빈 태그는 여는 태그 하나뿐이라 고민할 것이 없습니다. `<meta charset="UTF-8" />` 처럼 그 하나에 속성을 씁니다.

▹ 직접 해보기 2

위 상자의 닫는 태그에서 `id="close-attr"` 를 지우고 새로고침해 보세요. 화면이 하나도 안 바뀌는 것을 확인하는 것이 목적입니다.

속성 여러 개 쓰기

섹션 3

한 태그에 속성을 여러 개 붙일 수 있습니다. 사이를 공백으로 띄우기만 하면 됩니다.

아래 상자 안에는 이렇게 썼습니다.

```
<p id="multi-attr" lang="ko" title="속성 세 개를 붙였습니다">
  속성 세 개를 붙인 문단입니다.
</p>
```

규칙은 두 가지뿐입니다.

- 1 속성과 속성 사이는 공백으로 띄웁니다. 심표를 찍지 않습니다.
- 2 순서는 아무래도 상관없습니다. lang 을 먼저 써도 결과가 같습니다.

속성이 많아져서 줄이 길어지면 이렇게 줄을 나눠도 됩니다.

```
<p id="multi-attr" lang="ko" title="속성 세 개를 붙였습니다"
>
```

브라우저는 태그 안의 줄바꿈을 공백 하나로 봅니다. 그래서 결과가 똑같습니다. 다만 01단원에서는 한 줄로 쓰는 편이 읽기 쉽습니다.

같은 속성을 두 번 쓰면 어떻게 될까요?

```
<p id="a" id="b">
```

앞의 것만 살고 뒤의 것은 조용히 버려집니다. 여기서도 예외는 없습니다.

화면 이 문장 한 줄이 보입니다. 속성을 세 개 붙였지만 모양은 그대로입니다

속성 세 개를 붙인 문단입니다.

⇒ 직접 해보기

3

위 문단에 lang="en" 을 하나 더 붙여 같은 속성이 두 개가 되게 만들어 보세요. 화면과 마우스 올렸을 때의 설명이 그대로인지 확인하세요.

따옴표 규칙

섹션 4

값은 큰따옴표로 감쌉니다. 안 감싸도 대충 넘어가 주지만, 값에 공백이 있으면 그때부터 조용히 망가집니다.

먼저 되는 것들부터 정리합니다.

```
<p title="가게 안내"> 큰따옴표. 이 자료는 언제나 이걸 씁니다
<p title='가게 안내'> 작은따옴표. 동작은 같습니다
<p title=안내> 따옴표 없음. 값에 공백이 없으면 동작은 합니다
```

셋 다 동작합니다. 그런데도 큰따옴표를 쓰는 이유가 있습니다. 아래 상자를 보세요. 이렇게 썼습니다.

```
<p id="quote-space" title=주의 사항>공백이 있는 값에서 따옴표를 뺏습니다.</p>
```

title 값으로 주의 사항을 넣으려고 했습니다. 그런데 브라우저는 공백을 만나는 순간 “여기서 값이 끝났구나” 하고 자릅니다.

title 의 값 = 주의 사항	← 사항 은 잘려 나갔습니다
	← 이걸 이름이 사항 인 다른 속성이 되어 버립니다

마우스를 올려 보면 주의 만 뜹니다. 사항은 사라졌습니다. 화면 글자는 멀쩡하니 알아채기가 정말 어렵습니다.

따옴표는 “여기부터 여기까지가 한 덩어리입니다” 라고 묶어 주는 끈입니다. 끈이 없으면 공백에서 끊깁니다. 그러니 값이 짧은 길든 언제나 감싸세요.

값 안에 큰따옴표를 또 쓰면 어떻게 될까요?

```
<p title="그는 "안녕" 이라고 했다">
```

두 번째 큰따옴표에서 값이 끝나 버립니다. 그래서 값은 그는 까지만 들어갑니다. 이런 경우에는 바깥을 작은따옴표로 바꿔서 감싸면 됩니다.

```
<p title='그는 "안녕" 이라고 했다'>
```

화면 이 문장 한 줄이 멀쩡히 보입니다. 잘려 나간 흔적은 화면에 없습니다

공백이 있는 값에서 따옴표를 뺏습니다.

화면 이 문장 한 줄이 보입니다. 위 상자와 화면상으로는 구별이 안 됩니다

따옴표로 감싸면 값이 통째로 들어갑니다.

두 상자를 마우스로 올려 보면 차이가 드러납니다. 위 상자는 주의 만, 아래 상자는 주의 사항 이 뜹니다. 화면만 봐서는 절대 알 수 없습니다.

▫ 직접 해보기 4

위 두 문단에 차례로 마우스를 올려 뜨는 글자를 비교해 보세요. 그다음 첫 번째 문단의 값을 title="주의 사항" 으로 고쳐 두 상자가 같아지는지 확인하세요.

대표 속성 맛보기

섹션 5

이름만 익혀 둡니다. 지금은 화면이 안 바뀌는 것이 정상입니다.

01단원에서 이름이라도 알아 둘 속성은 네 개입니다.

lang="ko" 이 부분은 한국어로 쓰였다. (개념03 에서 봤습니다)
 title="설명" 이 요소에 대한 짧은 설명. 마우스를 올리면 뜹니다
 id="이름" 이 요소 하나에만 붙이는 고유한 이름표
 class="이름" 여러 요소에 같이 붙일 수 있는 분류 이름표

id 와 class 를 조금 더 봅니다. 지금은 뜻만 알아 두면 됩니다.

id 한 파일 안에서 겹치면 안 되는 이름입니다.
 주민등록번호처럼 그 요소 하나를 꼭 집어 가리킵니다.
`<p id="mainTitle">` 처럼 씁니다.

class 겹쳐도 됩니다. 오히려 겹치라고 만든 것입니다.
 같은 성격의 요소들을 한 무리로 묶는 이름표입니다.
`<p class="menu-item">` 를 여러 문단에 똑같이 붙일 수 있습니다.

이름 짓는 약속도 지금 정해 둡니다.

id 는 카멜 표기법 mainTitle, userName
 class 는 케밥 표기법 menu-item, price-tag

아래 상자의 두 문단에는 id 와 class 를 붙여 봤습니다. 그런데 화면은 아무것도 안 바뀝니다. 정상입니다. id 와 class 는 그 자체로는 아무 일도 하지 않는, 그냥 이름표이기 때문입니다.

그럼 왜 붙일까요? 08단원에서 CSS 선택자를 배우면, 이 이름표를 불러서 "menu-item 이라는 이름표가 붙은 것들만 빨강게" 같은 일을 하게 됩니다. 그때 진가가 나옵니다. 지금은 이름표를 붙이는 문법만 익혀 두세요.

참고로 이 자료 파일에 계속 나오는 class="doc-demo", class="doc-try" 도 전부 같은 class 속성입니다. 자료의 걸모양을 만드는 데 쓰고 있습니다.

화면 두 줄이 보입니다. 이름표를 붙이기 전과 화면이 완전히 똑같습니다

김치찌개 8000원
 된장찌개 8000원

id 와 class 는 08단원에서 CSS 로 요소를 골라 쓸 때 진가가 나옵니다. 지금은 붙여도 화면이 안 바뀐다는 사실만 확인하면 충분합니다.

▹ 직접 해보기 5

위 상자의 두 번째 문단에 id="secondMenu" 를 붙이고 새로고침해 보세요. 화면이 그대로인 것을 확인하면 됩니다.

속성 관련 실수는 화면에 흔적이 거의 안 남습니다. 그래서 이 섹션이 특히 중요합니다.

이런 실수를 합니다

따옴표를 한쪽만 닫음

이런 실수를 합니다

실수 1 — 따옴표를 한쪽만 닫음

```
<p title="가게 안내">오늘도 문을 엽니다.</p>
<p>그다음 문단입니다.</p>
```

닫는 따옴표가 없으니 브라우저는 다음 큰따옴표가 나올 때까지 전부 값으로 삼습니다. 뒤에 큰따옴표가 아예 없으면 파일 끝까지 삼킵니다. 그러면 화면이 통째로 하얗게 됩니다. 에러 메시지는 없습니다. 화면이 갑자기 비면 따옴표부터 세어 보세요.

이렇게 쓰세요

```
<p title="가게 안내">오늘도 문을 엽니다.</p>
<p>그다음 문단입니다.</p>
```

이런 실수를 합니다

등호를 빠뜨림

이런 실수를 합니다

실수 2 — 등호를 빠뜨림

```
<p title "가게 안내">오늘도 문을 엽니다.</p>
```

이름과 값이 등호로 이어져 있지 않으니 하나의 속성으로 안 묶입니다. title 은 값이 없는 속성이 되고, 남은 글자는 공백에서 또 잘려 “가게 와 안내” 라는 이름이 이상한 속성 두 개가 됩니다. 글자는 멀쩡히 보이고 설명만 안 뜹니다.

이런 실수를 합니다

속성 이름에 오타

이런 실수를 합니다

실수 3 — 속성 이름에 오타

```
<p titel="가게 안내">오타가 난 속성입니다.</p>
```

title 이 titel 이 되었습니다. 그런 이름의 속성은 없으니 브라우저는 알아듣지 못하고 그냥 무시합니다. 아래 상자가 실제로 그렇게 쓴 결과입니다. 마우스를 올려도 아무것도 안 뜹니다. 오타 난 속성을 쓴 결과 오타가 난 속성입니다.

이런 실수를 합니다

닫는 태그에 속성

이런 실수를 합니다

실수 4 — 닫는 태그에 속성을 씀

```
<p>오늘도 문을 엽니다.</p title="가게 안내">
```

섹션 2 에서 본 그대로입니다. 브라우저가 읽지도 않고 버립니다. 글자는 멀쩡하고 속성만 없던 일이 됩니다. 속성은 언제나 여는 태그 안에 씁니다.

이런 실수를 합니다

한글 자판으로 등호나 따옴표를 칩

이런 실수를 합니다

실수 5 — 등근 따옴표가 섞여 들어감

```
<p title="가게 안내">오늘도 문을 엽니다.</p>
```

위드나 한글 프로그램에서 쓴 글을 복사해 붙이면 곧은 따옴표가 등근 따옴표로 자동으로 바뀌어 따라옵니다. 이건 코드에서 쓰는 따옴표가 아닙니다. 브라우저는 이것을 그냥 글자로 보고, 값을 감싸는 끈이 없으니 섹션 4 에서 본 것처럼 공백에서 값을 잘라 버립니다. 그래서 title 의 값은 "가게 까지만 들어갑니다. 모양이 비슷해서 눈으로 찾기가 아주 어렵습니다. 코드는 반드시 VS Code 같은 코드 편집기에서 치세요.

정리

▣ 직접 해보기

6

정답

1) 섹션 1 의 문단을 예를 들어 이렇게 바꿉니다.

```
<p id="attr-basic" title="제가 처음 붙여 본 속성입니다">마우스를 올려 보세요.</p>
```

→ 화면 글자는 그대로입니다. 마우스를 올렸을 때 뜨는 작은 상자의 글자만 바뀝니다.

2) 섹션 2 의 줄을 이렇게 바꿉니다.

```
<p id="close-attr">닫는 태그에 쓴 속성은 그냥 무시됩니다.</p>
```

→ 화면이 하나도 안 바뀝니다.

애초에 브라우저가 그 속성을 읽지도 않았기 때문입니다.

3) 섹션 3 의 문단을 이렇게 바꿉니다.

```
<p id="multi-attr" lang="ko" lang="en" title="속성 세 개를 붙였습니다">
```

→ 화면도 그대로, 마우스를 올렸을 때 뜨는 설명도 그대로입니다.

같은 속성이 두 번 나오면 앞의 것만 살고 뒤의 것은 버려집니다.

4) 첫 번째 문단에 마우스를 올리면 주의 만 뜨고, 두 번째 문단에 올리면 주의 사항 이 뜹니다. 첫 번째를 이렇게 고칩니다.

```
<p id="quote-space" title="주의 사항">공백이 있는 값에서 따옴표를 뺐습니다.</p>
```

→ 이제 두 상자 모두 주의 사항 이 뜹니다. 화면 글자는 처음부터 끝까지 그대로였습니다.

5) 섹션 5 의 두 번째 문단을 이렇게 바꿉니다.

```
<p id="secondMenu" class="menu-item">된장찌개 8000원</p>
```

→ 화면이 하나도 안 바뀝니다.

id 는 이름표일 뿐이라 그 자체로는 아무 일도 하지 않습니다.

08단원에서 CSS 로 이 이름을 부르게 되면 그때부터 쓸모가 생깁니다.

주석과 공백 규칙

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 주석과 공백 규칙

01단원의 마지막 파일입니다. 서로 달라 보이는 두 가지를 함께 다룹니다.

주석 코드에 남기는 메모. 화면에 안 나옵니다
공백 규칙 스페이스와 줄바꿈을 브라우저가 어떻게 다루는가

두 가지를 같이 두는 이유가 있습니다. 둘 다 “내가 파일에 분명히 썼는데 화면에는 안 나오는 것” 이기 때문입니다.

개념01 섹션 3 에서 이런 것을 봤습니다.

오늘의 메뉴
김치찌개 8000원
된장찌개 8000원

코드에서는 세 줄인데 화면에서는 한 줄로 이어져 나왔습니다. 그때는 “개념05 에서 설명합니다” 하고 넘어갔습니다. 이제 그 약속을 지킵니다.

규칙을 미리 한 줄로 말하면 이렇습니다.

연달아 있는 공백과 줄바꿈은 전부 합쳐져서 '공백 한 칸' 이 됩니다.

그래서 코드를 아무리 예쁘게 정리해도 화면은 안 바뀝니다. 반대로 말하면, 코드를 마음껏 정리해도 화면이 안 망가진다는 뜻이기도 합니다. 이 규칙을 알고 나면 들여쓰기를 왜 하는지도 저절로 이해됩니다.

이 파일은 화면에 안 나오는 것들을 다룹니다. 주석, 그리고 여러 개 써도 하나로 줄어드는 공백과 무시되는 줄바꿈입니다. 개념01 에서 미뤄 뒀던 “왜 세 줄이 한 줄이 되나” 의 답이 여기 있습니다.

주석 쓰는 법

섹션 1

주석은 사람이 읽으라고 코드에 남기는 메모입니다. 브라우저는 주석을 만나면 통째로 건너뛵니다.

주석은 이렇게 생겼습니다. 아래 코드 상자에 글자 그대로 적어 뒀습니다.

여는 쪽은 꺾쇠, 느낌표, 하이픈 두 개. 닫는 쪽은 하이픈 두 개, 꺾쇠. 그 사이에 무엇을 쓰든 화면에 안 나옵니다.

주석은 세 가지 일에 씁니다.

1) 왜 이렇게 썼는지 이유를 적어 둡니다.

코드는 무엇을 하는지 보여 주지만 왜 그런지는 안 알려 줍니다.

2) 긴 파일에서 구역을 나눕니다.
이 자료의 섹션 표시가 전부 주석입니다.

3) 코드를 지우지 않고 잠시 꺼 둡니다.
지우면 되살리기 번거로우니 주석으로 감싸 둡니다.

VS Code에서는 Ctrl+/ 를 누르면 커서가 있는 줄이 주석으로 바뀝니다. 한 번 더 누르면 원래대로 돌아옵니다. 자주 쓰게 되니 지금 눌러 보세요.

지금 여러분이 읽고 있는 이 글도 주석입니다. 브라우저 화면에는 한 글자도 안 나옵니다.

```
<p>영업시간</p> <!-- 아래 문단은 나중에 가격을 넣을 자리입니다 -->
<p>오전 9시부터 오후 6시까지</p>
```

아래 문단은 나중에 가격을 넣을 자리입니다

화면 두 줄만 보입니다. 가운데 주석 줄은 흔적도 없습니다

영업시간

오전 9시부터 오후 6시까지

▣ 직접 해보기 1

위 상자 안 <p>영업시간</p> 줄에 커서를 두고 VS Code 에서 Ctrl+/ 를 눌러 보세요. 저장하고 새로고침하면 그 줄이 화면에서 사라집니다. 한 번 더 눌러 되돌리세요.

주석은 소스보기로 다 보입니다

섹션 2

주석은 화면에 안 나올 뿐, 파일 안에는 그대로 들어 있습니다. 누구나 3초 만에 꺼내 볼 수 있습니다.

지금 이 페이지에서 Ctrl+U 를 눌러 보세요. 새 탭이 열리면서 이 파일의 코드가 통째로 나옵니다. 이것을 소스 보기라고 합니다.

거기에는 주석도 전부 그대로 들어 있습니다. 지금 읽고 계신 이 문장도 거기서 읽힙니다.

마우스 오른쪽 버튼을 누르고 페이지 소스 보기 를 골라도 같습니다.

그래서 절대로 주석에 적으면 안 되는 것이 있습니다.

비밀번호

개인 연락처, 주소

남에게 보이면 곤란한 메모

“화면에 안 나오니까 괜찮겠지” 는 완전히 틀린 생각입니다. 웹페이지의 코드는 그것을 여는 사람 모두에게 열려 있습니다. 인터넷에 올린 페이지라면 전 세계 누구나 볼 수 있습니다.

아래 상자 안에는 주석을 한 줄 숨겨 놔습니다. Ctrl+U 를 누르고 Ctrl+F 로 소스보기로만 이라고 검색해 보세요.

이 줄은 화면에 안 나오지만 소스보기로만 하면 바로 찾을 수 있습니다

화면 이 문장 한 줄만 보입니다. 숨긴 주석은 화면 어디에도 없습니다

이 상자 안에는 주석이 한 줄 숨어 있습니다.

주석은 감추는 장치가 아니라 정리하는 장치입니다. 진짜로 감춰야 하는 정보는 애초에 HTML 파일에 쓰지 않습니다.

▣ 직접 해보기 2

이 페이지에서 Ctrl+U 를 눌러 소스 보기를 열고, Ctrl+F 로 소스보기로만 을 검색해 숨겨 둔 주석을 찾아보세요.

공백 여러 개는 하나로

섹션 3

스페이스를 열 번 눌러도 화면에는 한 칸만 나옵니다. HTML 의 가장 당황스러운 규칙입니다.

아래 상자에는 이렇게 썼습니다. 위아래 줄의 차이는 공백 개수뿐입니다.

```
<span id="space-many">사과      바나나</span>
<span id="space-one">사과 바나나</span>
```

위 줄에는 공백을 여섯 칸, 아래 줄에는 한 칸 넣었습니다. 그런데 화면에서는 두 줄이 완전히 같습니다. 똑까지 똑같습니다.

왜 이럴까요? HTML 은 글의 구조를 표시하는 언어입니다. 글자를 어디에 놓을지 정하는 언어가 아닙니다. 공백을 몇 칸 넣었는지는 글의 구조와 아무 상관이 없습니다. 그래서 브라우저는 연달아 있는 공백을 전부 한 칸으로 합쳐 버립니다.

이것을 공백 합치기라고 부릅니다. 규칙은 딱 한 줄입니다.

공백이 몇 개든, 몇 줄이든, 탭이든 → 전부 합쳐서 공백 한 칸

그래서 스페이스로 글자 위치를 맞추려는 시도는 전부 실패합니다. 위치를 맞추는 일은 09~10단원에서 CSS 가 맡습니다.

정말로 공백을 여러 칸 넣어야 하는 경우도 아주 가끔 있습니다. 그때 쓰는 특별한 표기가 따로 있는데, 02단원에서 배웁니다. 지금은 “여러 칸은 안 된다” 만 알아 두세요.

화면 "사과 바나나" 로 보입니다. 공백은 한 칸뿐입니다

사과 바나나

화면 위 상자와 글자도 폭도 완전히 똑같습니다

사과 바나나

▫ 직접 해보기 3

첫 번째 상자의 공백을 스무 칸으로 늘리고 저장한 뒤 새로고침해 보세요. 화면이 하나도 안 바뀌는 것을 확인하는 것이 목적입니다.

줄바꿈도 무시됩니다

섹션 4

엔터를 눌러 줄을 나눠도 화면에서는 한 줄로 이어집니다. 줄바꿈도 공백 한 칸으로 취급되기 때문입니다.

아래 상자 안에는 이렇게 썼습니다. 코드에서는 분명히 세 줄입니다.

```
<p id="newline-demo">첫 줄입니다.  
두 번째 줄입니다.  
세 번째 줄입니다.</p>
```

그런데 화면에서는 한 줄로 이어져 나옵니다. 섹션 3의 규칙에 줄바꿈도 포함되기 때문입니다.

공백 여러 개	→ 공백 한 칸
줄바꿈	→ 공백 한 칸
탭	→ 공백 한 칸

이제 개념01 섹션 3의 수수께끼가 풀립니다. 세 줄로 쓴 메뉴가 한 줄로 나왔던 이유가 바로 이것입니다.

브라우저가 줄을 나누는 때는 딱 두 가지뿐입니다.

1 글이 상자 폭보다 길어서 저절로 넘어갈 때

2 우리가 줄을 나누라고 시켰을 때 (섹션 5)

코드의 엔터는 여기 없습니다. 브라우저에게 엔터는 그냥 공백입니다.

이 성질에는 좋은 면도 있습니다. 코드가 너무 길어지면 아무 데서나 줄을 나눠 정리해도 화면이 안 망가집니다.

화면 세 문장이 한 줄로 죽 이어져서 보입니다

```
첫 줄입니다.  
두 번째 줄입니다.  
세 번째 줄입니다.
```

▹ 직접 해보기 **4**

위 상자에 네 번째 줄을 엔터로 나눠서 한 줄 더 넣어 보세요. 저장하고 새로고침하면 여전히 한 줄로 이어지는 것을 확인하세요.

그래서 br 이 필요합니다

섹션 5

코드의 엔터가 무시되니, 줄을 나누라고 따로 시켜야 합니다. 그 시키는 방법이 `<br />` 입니다.

개념02 섹션 3에서 `<br />` 를 봤습니다. 그때는 “줄을 바꾸는 빈 태그” 라고만 했습니다. 이제 왜 필요한지가 설명됩니다.

아래 상자에는 섹션 4와 똑같은 문장을 이렇게 썼습니다.

```
<p id="br-demo">첫 줄입니다.<br />두 번째 줄입니다.<br />세 번째 줄입니다.</p>
```


코드에서는 한 줄인데 화면에서는 세 줄로 나옵니다. 섹션 4 와 정확히 반대입니다. 나란히 놓고 보면 이렇습니다.

코드에서 엔터를 눌렀다 → 화면은 한 줄 (섹션 4)
코드는 한 줄인데 br 을 썼다 → 화면은 세 줄 (지금)

코드의 생김새와 화면의 생김새는 아무 관계가 없습니다. 화면을 정하는 것은 오직 태그입니다. 이것이 01단원 전체를 관통하는 이야기입니다.

 는 닫는 태그가 없는 빈 태그입니다. 감쌀 내용이 없기 때문입니다. 줄바꿈은 그 자리에서 일어나는 사건입니다.

주의할 점 하나.
 는 줄 간격을 벌리는 용도가 아닙니다.

 처럼 여러 개를 쓰면 간격이 벌어지기는 합니다. 하지만 그건 CSS 가 할 일을 태그로 억지로 하는 것입니다. 09~10단원에서 제대로 배웁니다.
 는 주소나 시(詩)처럼 진짜로 그 자리에서 줄이 바뀌어야 하는 글에만 씁니다.

화면 세 문장이 각각 한 줄씩, 세 줄로 나뉘어 보입니다

첫 줄입니다.두 번째 줄입니다.세 번째 줄입니다.

섹션 4 상자와 이 상자를 나란히 보세요. 글자는 완전히 같습니다. 다른 것은
 이 있느냐 없느냐 하나뿐입니다.

▣ 직접 해보기 5

위 상자에서
 두 개를 지우고 새로고침해 보세요. 섹션 4 상자과 똑같아집니다. 확인했으면 되돌리세요.

들여쓰기는 사람을 위한 것

섹션 6

브라우저는 들여쓰기를 전혀 신경 쓰지 않습니다. 그런데도 들여쓰기를 하는 이유가 있습니다.

아래 예에 <div> 가 나옵니다. 처음 보는 태그이니 짧게 짚고 갑니다.

<div> 는 아무 뜻이 없고, 여러 태그를 하나로 '묶기만' 하는 상자입니다.
화면 모양은 바꾸지 않습니다.
왜 그런 것이 필요한지, 언제 쓰는지는 06단원에서 제대로 배웁니다.
지금은 "묶는 상자" 라고만 알아 두면 충분합니다.

아래 두 상자는 화면 결과가 완전히 같습니다. 높이까지 같습니다. 코드만 다릅니다.

줄여 쓴 코드

```
<div><p>가게 이름</p><p>영업시간</p></div>
```

들여 쓴 코드

```
<div> <p>가게 이름</p> <p>영업시간</p> </div>
```

들여쓰기는 공백입니다. 그리고 공백은 합쳐집니다. 그러니 브라우저에게는 두 코드가 똑같이 보입니다.

그럼 왜 들여쓸까요? 사람이 읽기 위해서입니다. 들여쓰기를 보면 어느 태그가 어느 태그 안에 들어 있는지 한눈에 보입니다. 개념02 에서 배운 부모와 자식 관계가 눈에 그려집니다.

div	← 부모
p	← 자식
p	← 자식

태그가 열 겹, 스무 겹으로 겹쳐지는 06단원쯤 가면 들여쓰기 없이는 자기가 쓴 코드도 못 읽습니다.

이 자료의 약속은 이렇습니다.

한 단계 안으로 들어갈 때마다 스페이스 두 칸
탭 대신 스페이스를 씁니다

VS Code 에서 Shift+Alt+F 를 누르면 들여쓰기를 자동으로 정리해 줍니다. 코드가 엉망이 되었을 때 눌러 보세요.

화면 두 줄이 보이고, 두 줄 사이가 약간 벌어져 있습니다

가게 이름영업시간

화면 위 상자와 완전히 똑같습니다. 줄 수도 간격도 같습니다

가게 이름
영업시간

▫ 직접 해보기 6

두 번째 상자의 들여쓰기를 스페이스 열 칸으로 늘리고 새로고침해 보세요. 화면이 안 바뀌는 것을 확인한 뒤 두 칸으로 되돌리세요.

자주 하는 실수

섹션 7

주석 관련 실수는 글이 통째로 사라지는 형태로 나타납니다. 에러 메시지는 여기서도 없습니다.

이런 실수를 합니다

주석을 닫지 않음

이런 실수를 합니다

실수 1 — 주석을 닫지 않음

```
<!-- 여기부터 정리해야 함
<p>오늘도 문을 엽니다.</p>
<p>영업시간은 아침 9시부터입니다.</p>
```

닫는 표시가 없으니 브라우저는 파일이 끝날 때까지 전부 주석으로 봅니다. 그 아래 있던 글이 통째로 사라집니다. 화면이 갑자기 하얗게 비면 이것부터 의심하세요. VS Code 는 주석 부분을 흐린 초록색으로 보여 줍니다. 초록색이 파일 끝까지 이어져 있으면 닫는 표시를 빠뜨린 것입니다.

이렇게 쓰세요

```
<!-- 여기부터 정리해야 함 -->
<p>오늘도 문을 엽니다.</p>
<p>영업시간은 아침 9시부터입니다.</p>
```

이런 실수를 합니다

주석 안에서 닫는 표시를 실수로 만들

이런 실수를 합니다

실수 2 — 주석 안에서 닫는 표시가 저절로 만들어짐

```
<!-- 설명을 그림으로 그려 봅니다
이름 --> 값 이 들어갑니다
이 줄부터는 화면에 그대로 새어 나옵니다 -->
```

화살표를 그리려고 하이픈 두 개와 꺾쇠를 이어 쓴 순간 주석을 닫는 표시가 저절로 만들어졌습니다. 주석은 거기서 끝나 버리고, 그 뒤에 적어 둔 글이 화면에 글자로 튀어나옵니다. 코드에서는 두 줄인데 화면에서는 한 줄로 이어져 나옵니다. 섹션 4 에서 배운 대로 코드의 줄바꿈이 공백 한 칸이 되기 때문입니다. 맨 끝의 → 도 갈 곳을 잃고 글자로 같이 보입니다. 주석 안에서 화살표를 그릴 때는 하이픈 대신 → 같은 글자를 쓰면 이 사고를 피할 수 있습니다.

이런 실수를 합니다

주석에 비밀을 적음

이런 실수를 합니다

실수 3 — 주석에 비밀번호나 개인정보를 적음

```
<!-- 관리자 비밀번호: 1234 -->
```

화면에는 안 나오지만 Ctrl+U 한 번이면 누구에게나 그대로 보입니다. 아무 일도 안 일어난 것처럼 보여서 위험한 줄도 모릅니다. 주석은 감추는 장치가 아닙니다.

이런 실수를 합니다

스페이스로 글자 위치를 맞춤

이런 실수를 합니다

실수 4 — 스페이스를 여러 개 넣어 위치를 맞추려 함

```
<p>김치찌개      8000원</p>
```

가격을 오른쪽에 나란히 맞추려고 공백을 잔뜩 넣었습니다. 하지만 공백은 한 칸으로 줄어들어 아무 일도 일어나지 않습니다. 아래가 실제 결과입니다. 공백을 열 칸 넣은 결과 김치찌개 8000원

이런 실수를 합니다

br 을 여러 개 써서 간격을 벌림

이런 실수를 합니다

실수 5 — br 을 여러 개 써서 간격을 벌림

```
<p>가게 이름</p>
<br /><br /><br />
<p>영업시간</p>
```

이건 실제로 간격이 벌어집니다. 그래서 더 나쁩니다. 되니까 계속 쓰게 됩니다. 하지만 나중에 간격을 조금 줄이려면
 를 하나씩 지웠다 넣었다 해야 합니다. 간격은 09~10단원에서 CSS 로 숫자를 적어 조절합니다. 그때가 훨씬 편합니다.

정리

▸ 직접 해보기 7

정답

1) 섹션 1 의 doc-demo 상자 안 <p>영업시간</p> 줄에 커서를 두고 Ctrl+/ 를 누르면, 그 줄 앞뒤에 주석을 여는 표시와 닫는 표시가 저절로 붙습니다. VS Code 에서 그 줄이 흐린 초록색으로 바뀌면 성공한 것입니다.

→ 화면에서 영업시간 줄이 사라지고 아래 한 줄만 남습니다.

Ctrl+/ 를 한 번 더 누르면 표시가 떨어져 나가고 그대로 돌아옵니다.

2) Ctrl+U 를 누르면 새 탭에 이 파일의 코드가 전부 나옵니다. Ctrl+F 로 소스보기로만 을 검색하면 섹션 2 상자 안에 숨겨 둔 주석 줄이 찾아집니다.

→ 화면은 안 바뀝니다.

"화면에 없다" 와 "파일에 없다" 가 완전히 다른 말이라는 것을 확인하는 것이 목적입니다.

3) 섹션 3 첫 상자를 이렇게 바꿉니다. 공백을 스무 칸으로 늘립니다.

```
<span id="space-many">사과                바나나</span>
```

→ 화면이 하나도 안 바뀝니다. 여전히 사과 바나나 로 보입니다.

연달아 있는 공백은 몇 개든 한 칸으로 합쳐지기 때문입니다.

4) 섹션 4 상자를 이렇게 바꿉니다.

```
<p id="newline-demo">첫 줄입니다.  
  두 번째 줄입니다.  
  세 번째 줄입니다.  
  네 번째 줄입니다.</p>
```

→ 네 문장이 여전히 한 줄로 이어져 나옵니다. 줄 수가 늘어나지 않습니다.

5) 섹션 5 상자를 이렇게 바꿉니다.

```
<p id="br-demo">첫 줄입니다.두 번째 줄입니다.세 번째 줄입니다.</p>
```

→ 세 줄이 한 줄로 합쳐집니다.

화면을 나눈 것은 코드의 엔터가 아니라 br 이었다는 뜻입니다.
섹션 4 상자와 거의 같아지는데, 문장 사이에 공백 한 칸이 없다는 점만 다릅니다.
섹션 4 는 코드의 줄바꿈이 공백 한 칸으로 바뀌어 들어가 있기 때문입니다.
확인했으면 br 두 개를 다시 넣으세요.

6) 섹션 6 두 번째 상자를 이렇게 바꿉니다.

```
<div id="indent-nice"> <p>가게 이름</p> <p>영업시간</p>  
</div>
```

→ 화면이 하나도 안 바뀝니다.

들여쓰기는 공백이고 공백은 합쳐지기 때문입니다.
읽기 좋은 두 칸으로 되돌려 놓으세요.

HTML 시작하기

푸는 방법

1. 이 파일을 VS Code 와 브라우저에서 동시에 엽니다.
2. 각 문제 상자 안에 코드를 씁니다.
3. 저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 합니다.
4. 문제마다 적힌 기대 화면 과 똑같이 나오면 정답입니다.
5. 10분 고민해도 안 되면 연습문제_정답.html 을 여세요.

문제 1~10 은 기본, 11~13 은 응용, 14~15 는 도전입니다.

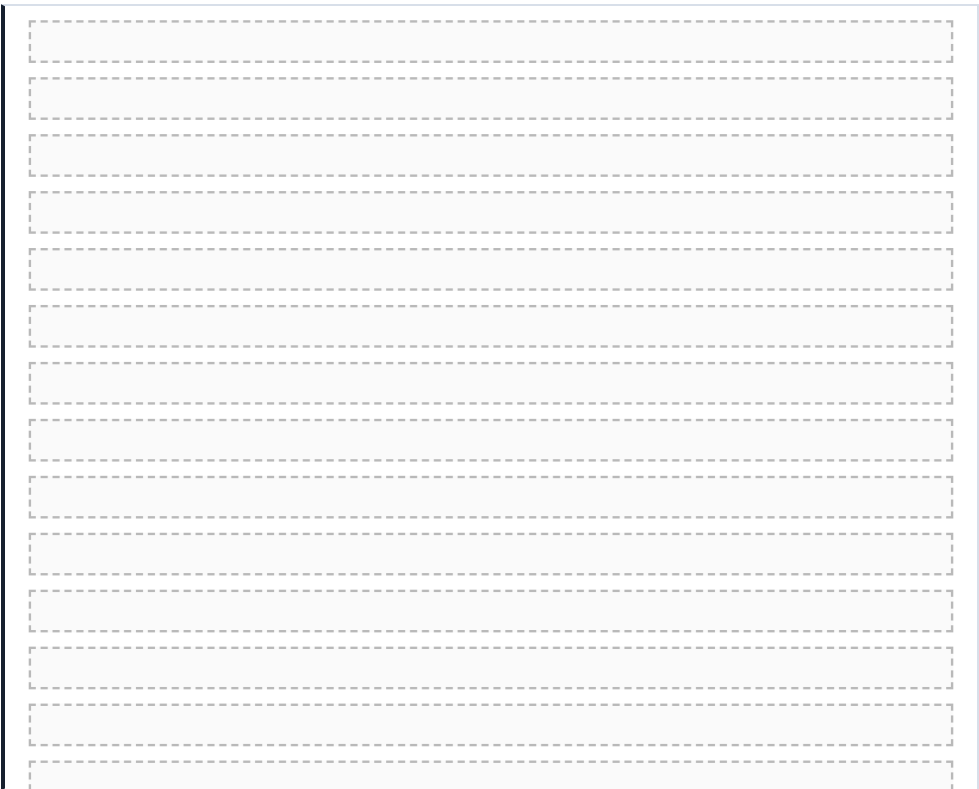
마지막 16번은 고장난 코드를 고치는 문제라 맨 뒤에 있습니다.

중요한 마음가짐

HTML 은 틀려도 에러 메시지가 뜨지 않습니다.

빨간 글씨를 기다리지 마세요. 화면을 눈으로 보고 판단해야 합니다.

“아무 일도 안 일어났다” 도 훌륭한 단서입니다.



실행하면 나오는 화면 — HTML 시작하기

각 문제의 점선 상자 안에 코드를 쓰고 저장(Ctrl+S) → 새로그침(F5) 하세요. 상자 아래 회색 글씨가 다 맞았을 때 나와야 하는 화면입니다. 더 자세한 설명과 오답 진단은 VS Code 로 이 파일을 열면 각 문제 위 주석에 있습니다.

문제 1 (개념02 섹션1)

p 태그로 우리 동네 빵집 이라는 문단을 하나 만드세요.

기대 출력 상자 안에 "우리 동네 빵집" 이 한 줄로 보입니다.
이렇게 되면 틀린 것: 상자가 그대로 비어 있으면 코드를 안 쓴 것입니다.
꺾쇠가 글자로 보이면 여는 꺾쇠를 빠뜨린 것입니다.

문제 1 — 문단 하나 만들기

p 태그로 우리 동네 빵집 이라는 문단을 하나 만드세요. (개념02 섹션 1)

여기에 코드를 쓰세요

완성하면 상자 안에 우리 동네 빵집 이 한 줄로 보입니다.

문제 2 (개념02 섹션1)

아래 두 문장을 각각 문단으로 만드세요. 매일 아침 7시에 문을 엽니다. 월요일은 쉽니다.

기대 출력 두 줄이 보이고, 두 줄 사이가 약간 벌어져 있습니다.
이렇게 되면 틀린 것: 두 문장이 한 줄로 붙어 나오면 p 로 감싸지 않고
엔터만 친 것입니다. 코드의 엔터는 화면에서 무시됩니다.

문제 2 — 문단 두 개 만들기

매일 아침 7시에 문을 엽니다. 와 월요일은 쉽니다. 를 각각 문단으로 만드세요. (개념02 섹션 1)

여기에 코드를 쓰세요

완성하면 두 줄이 보이고 두 줄 사이가 약간 벌어져 있습니다.

문제 3 (개념02 섹션3)

문단 하나 안에서 주소를 두 줄로 나누세요. 문단은 반드시 하나여야 합니다. 서울시 마포구 행복로 12 1층
우리 동네 빵집

기대 출력 두 줄로 나뉘어 보입니다.
 문제 2 의 두 문단보다 줄 사이가 눈에 띄게 좁습니다.
 이렇게 되면 틀린 것: 한 줄로 이어져 나오면 줄바꿈 태그를 안 넣은
 것입니다.
 코드에서 엔터를 쳐도 화면은 안 나뉩니다.

문제 3 — 문단 하나 안에서 줄 바꾸기

p 하나 안에서 `<br />` 로 주소를 두 줄로 나누세요. 첫 줄은 서울시 마포구 행복로 12, 둘째 줄은 1층 우리 동네 빵집 입니다. (개념02 섹션 3)

여기에 코드를 쓰세요

완성하면 두 줄로 나뉘고, 문제 2 의 두 문단보다 줄 사이가 좁습니다.

문제 4 (개념02 섹션3)

문단 두 개 사이에 가로줄을 하나 그으세요. 오늘의 빵 (가로줄) 내일의 빵

기대 출력 글자 한 줄, 회색 가로줄, 글자 한 줄 순서로 보입니다.
 이렇게 되면 틀린 것: 가로줄이 안 보이면 빈 태그를 잘못 쓴 것입니다.
 hr 은 닫는 태그가 없습니다.

문제 4 — 가로줄 긋기

오늘의 빵 문단과 내일의 빵 문단 사이에 `<hr />` 로 가로줄을 하나 그으세요. (개념02 섹션 3)

여기에 코드를 쓰세요

완성하면 글자, 회색 가로줄, 글자 순서로 보입니다.

문제 5 (개념02 섹션4)

휴무일은 월요일입니다. 라는 문단을 만들고, 월요일 세 글자만 굵게 만드세요.

기대 출력 한 줄인데 월요일 세 글자만 굵게 보입니다.
 이렇게 되면 틀린 것: 문장 전체가 굵어졌다면 strong 이 p 안의 글자 전체를
 감싼 것입니다. 굵게 할 글자만 감싸야 합니다.

문제 5 — 한 단어만 굵게 (중첩)

휴무일은 월요일입니다. 라는 문단을 만들고, strong 으로 월요일 세 글자만 굵게 만드세요. (개념02 섹션 4)

여기에 코드를 쓰세요

완성하면 한 줄인데 월요일 세 글자만 굵게 보입니다.

문제 6 (개념05 섹션1)

문단 하나를 만들고, 그 위에 주석으로 메모를 한 줄 남기세요. 문단 글자 소금빵 2500원 메모 글자 가격은 매달 확인할 것

기대 출력 상자 안에 "소금빵 2500원" 한 줄만 보입니다. 메모는 안 보입니다.
이렇게 되면 틀린 것: 메모 글자가 화면에 보이면 주석 표시를 잘못 쓴 것입니다.
메모 아래 문단까지 사라졌다면 닫는 표시를 빠뜨린 것입니다.

문제 6 — 주석으로 메모 남기기

소금빵 2500원 문단을 만들고, 그 위 줄에 주석으로 가격은 매달 확인할 것 이라고 메모를 남기세요. (개념05 섹션 1)

여기에 코드를 쓰세요

완성하면 소금빵 2500원 한 줄만 보이고 메모는 안 보입니다.

문제 7 (개념04 섹션1)

단팥빵 2000원 이라는 문단에 title 속성으로 오늘 구운 빵입니다 를 붙이세요.

기대 출력 "단팥빵 2000원" 한 줄이 보입니다. 글자 모양은 안 바뀝니다.
그 글자 위에 마우스를 올리고 1초쯤 기다리면 작은 상자가 뜨면서 오늘 구운 빵입니다 가 나타납니다.
이렇게 되면 틀린 것: 화면에 title="..." 글자가 그대로 보이면 속성을 여는 태그 밖에 쓴 것입니다.

문제 7 — 속성 붙이기

단팥빵 2000원 문단에 title 속성으로 오늘 구운 빵입니다 를 붙이세요. (개념04 섹션 1)

여기에 코드를 쓰세요

완성하면 글자 모양은 그대로이고, 마우스를 올리면 작은 상자가 뜹니다.

문제 8 (개념04 섹션5)

우리 동네 빵집 문단에 id 속성으로 shopName 을 붙이세요.

기대 출력 "우리 동네 빵집" 한 줄이 보입니다.
id 를 붙이기 전과 화면이 완전히 똑같습니다.
이렇게 되면 틀린 것: 화면이 달라졌다면 id 말고 다른 것을 같이 고친
것입니다.
id 는 그 자체로는 화면을 절대 바꾸지 않습니다.

문제 8 — id 붙이기

우리 동네 빵집 문단에 id 속성으로 shopName 을 붙이세요. (개념04 섹션 5)

여기에 코드를 쓰세요

완성해도 화면은 한 줄 그대로입니다. 안 바뀌는 것이 정답입니다.

문제 9 (개념05 섹션3)

단팥빵 과 2000원 사이에 스페이스를 열 칸 넣은 문단을 만드세요. 그리고 화면이 어떻게 되는지
확인하세요.

기대 출력 "단팥빵 2000원" 으로, 사이가 한 칸만 벌어져 보입니다.
이렇게 되면 틀린 것: 화면에 넓은 빈 자리가 그대로 보인다면
스페이스가 아닌 다른 방법을 쓴 것입니다.

문제 9 — 공백은 몇 칸이 될까

단팥빵 과 2000원 사이에 스페이스를 열 칸 넣은 문단을 만들고 화면을 확인하세요. (개념05 섹션 3)

여기에 코드를 쓰세요

완성하면 단팔빵 2000원 으로, 사이가 한 칸만 벌어져 보입니다.

문제 10 (개념05 섹션4)

문단 하나 안에 아래 세 문장을 엔터로 줄을 나눠서 쓰세요. 줄바꿈 태그는 쓰지 마세요. 아침 7시에 엽니다. 오후 8시에 닫습니다. 월요일은 쉽니다.

기대 출력 세 문장이 한 줄로 죽 이어져서 보입니다.
이렇게 되면 틀린 것: 세 줄로 나뉘어 보이면 br 을 넣은 것입니다.
이 문제는 br 없이 쓰는 것이 목적입니다.

문제 10 — 엔터를 쳐도 줄이 안 나뉜다

문단 하나 안에 아침 7시에 엽니다. 오후 8시에 닫습니다. 월요일은 쉽니다. 를 엔터로 줄을 나눠서 쓰세요. 줄바꿈 태그는 쓰지 않습니다. (개념05 섹션 4)

여기에 코드를 쓰세요

완성하면 세 문장이 한 줄로 죽 이어져서 보입니다.

문제 11 [응용] (개념05 섹션5)

문제 10 의 코드를 그대로 가져와서, 이번에는 진짜로 세 줄이 되게 고치세요. 문단은 여전히 하나여야 합니다.

기대 출력 세 문장이 각각 한 줄씩, 세 줄로 나뉘어 보입니다.
이렇게 되면 틀린 것: 줄 사이가 문제 2 처럼 넓게 벌어졌다면
문단을 세 개로 나눈 것입니다. 문단은 하나여야 합니다.

문제 11 — [응용] 진짜로 세 줄 만들기

문제 10 의 코드를 가져와
 로 세 줄이 되게 고치세요. 문단은 여전히 하나여야 합니다. (개념05 섹션 5)

여기에 코드를 쓰세요

완성하면 세 문장이 각각 한 줄씩, 세 줄로 보입니다.

문제 12 [응용] (개념04 섹션3)

문단 하나에 속성을 두 개 붙이세요.

글자	오전 7시부터 오후 8시까지
id	openHours
title	영업 시간 안내

기대 출력 "오전 7시부터 오후 8시까지" 한 줄이 보입니다.
 마우스를 올리면 작은 상자에 영업 시간 안내 가 통째로 뜹니다.
 이렇게 되면 틀린 것: 마우스를 올렸을 때 영업 만 뜨면 값을 따옴표로
 감싸지 않은 것입니다. 공백에서 값이 잘렸습니다.

문제 12 — [응용] 속성 두 개 붙이기

오전 7시부터 오후 8시까지 문단에 id="openHours" 와 title="영업 시간 안내" 를 함께 붙이세요.
 (개념04 섹션 3)

여기에 코드를 쓰세요

완성하면 마우스를 올렸을 때 영업 시간 안내 가 통째로 뜹니다.

문제 13 [응용] (개념02 섹션4 · 개념05 섹션6)

div 하나 안에 문단 두 개를 넣고, 들여쓰기를 두 칸씩 맞춰서 쓰세요. 첫째 문단 오늘의 추천 (문장 전체를 굵게) 둘째 문단 소금빵 2500원 (2500원 만 굵게)

기대 출력 두 줄이 보입니다.
 첫 줄은 전체가 굵고, 둘째 줄은 2500원 만 굵습니다.
 이렇게 되면 틀린 것: 둘째 줄 전체가 굵으면 strong 이 문장 전체를 감싼
 것입니다.
 한 줄만 보이면 문단 하나에 다 넣은 것입니다.

문제 13 — [응용] 두 겹으로 감싸기

div 하나 안에 문단 두 개를 넣으세요. 첫째 문단은 오늘의 추천 전체를 굵게, 둘째 문단은 소금빵 2500원 중 2500원 만 굵게 합니다. 들여쓰기를 두 칸씩 맞추세요. (개념02 섹션 4 · 개념05 섹션 6)

여기에 코드를 쓰세요

완성하면 두 줄이 보이고, 첫 줄은 전체가 둘째 줄은 2500원 만 굵습니다.

문제 14 [도전] (개념03 섹션 1~6)

이 문제는 이 파일이 아니라 여러분이 만든 새 파일에서 풀니다.

VS Code 로 bakery.html 이라는 새 파일을 만들고, 아래 빈칸을 채워 HTML 파일 한 벌을 완성하세요. 그리고 브라우저로 열어 확인하세요. body 안에는 문단을 두 개 넣습니다.

기대 출력 브라우저 탭에 우리 동네 빵집 이라고 적히고,
본문에는 문단 두 개가 두 줄로 보입니다. 한글이 안 깨집니다.
이렇게 되면 틀린 것: 한글이 알아볼 수 없는 글자로 나오면 charset 줄이
빠졌거나 값이 틀린 것입니다.
탭에 bakery.html 이 그대로 뜨면 title 이 빠진 것입니다.

문제 14 — [도전] 새 파일에 뼈대 세우기

VS Code 로 bakery.html 을 만들고 아래 빈칸을 채워 완성하세요. 브라우저 탭 이름은 우리 동네 빵집 이어야 하고, body 안에는 문단을 두 개 넣습니다. (개념03 섹션 1~6)

```
-----
<html ____="ko"> <____> <meta _____="UTF-8" /> <____>우리 동네 빵집
</____> </____> <____>
    (여기에 문단 두 개)
    </____>
</html>
```

이 문제는 새 파일에서 풁니다. 완성하면 탭에 우리 동네 빵집 이 적히고 본문에 두 줄이 보이며, 한글이 안 깨집니다.

문제 15 [도전] (개념02 · 개념04 · 개념05 종합)

- 1 코드 맨 위에 주석으로 안내문 시작 이라고 메모를 남깁니다.
- 2 첫 문단 우리 동네 빵집 입니다. → 우리 동네 빵집 만 굵게
- 3 가로줄 하나
- 4 둘째 문단 주소를 두 줄로. 문단은 하나여야 합니다.

서울시 마포구 행복로 12
1층 우리 동네 빵집

5) 셋째 문단 전화번호는 안내문 아래를 보세요.

이 문단에 title="문의는 전화로 부탁드립니다" 를 붙입니다.

기대 출력 위에서부터

- 1) "우리 동네 빵집 입니다." 한 줄. 앞부분 우리 동네 빵집 만 굵습니다.
- 2) 회색 가로줄 하나
- 3) 주소 두 줄. 한 문단 안의 두 줄이라 줄 사이가 좁습니다.
- 4) "전화번호는 안내문 아래를 보세요." 한 줄.

마우스를 올리면 작은 상자가 뜹니다.

글자는 모두 합쳐 네 줄이고 가로줄이 하나입니다. 메모는 안 보입니다.

이렇게 되면 틀린 것: 메모가 화면에 보이면 주석 표시가 잘못된 것입니다.

주소가 한 줄로 이어지면 br 을 안 넣은 것입니다.

문제 15 — [도전] 가게 안내문 만들기

지금까지 배운 것을 한 번에 씁니다. 조건은 다섯 가지입니다. 주석 메모 한 줄, 우리 동네 빵집 만 굵은 첫 문단, 가로줄 하나,
 로 두 줄인 주소 문단 하나, title 속성이 붙은 마지막 문단. (개념02 · 개념04 · 개념05 종합)

여기에 코드를 쓰세요

완성하면 글자 네 줄과 회색 가로줄 하나가 보입니다. 메모는 안 보입니다.

문제 16 [확인] (개념02 · 개념04 · 개념05)

아래 첫 번째 상자는 일부러 고장 난 코드를 그대로 그린 결과입니다. 에러 메시지는 어디에도 안 뜹니다. 그런데 세 군데가 틀렸습디다. 무엇이 이상한지 찾아서, 두 번째 상자에 고친 코드를 쓰세요.

고장 난 코드는 이렇습디다.

```
p>우리 동네 빵집</p>
<p>매일 아침 7시에 문을 엽니다.
오후 8시에 닫습디다.</p>
<p title=영업 안내>궁금한 점은 전화 주세요.</p>
```

- 1 첫 줄이 문단으로 안 보이고 코드가 글자로 그대로 보입니다.
- 2 두 문장을 두 줄로 쓰려고 했는데 한 줄로 이어져 나옵니다.
- 3 마지막 문단에 마우스를 올리면 설명이 영업 까지만 뜹니다.

기대 출력 고치면 네 줄이 보입니다.
 "우리 동네 빵집"
 "매일 아침 7시에 문을 엽니다."
 "오후 8시에 닫습니다."
 "궁금한 점은 전화 주세요."
 둘째와 셋째 줄은 한 문단 안의 두 줄이라 사이가 좁습니다.
 마지막 줄에 마우스를 올리면 영업 안내 가 통째로 뜹니다.
 이렇게 되면 틀린 것: 화면에 p> 같은 글자가 아직 보이면 1번을 못 고친
 것입니다.

문제 16 — [확인] 고장난 코드 고치기

아래 첫 상자는 일부러 고장 낸 코드를 그대로 그린 결과입니다. 여러 메시지는 어디에도 안 뜹니다. 그런데 세 군데가 틀렸습니다. 무엇이 이상한지 찾아 두 번째 상자에 고쳐 쓰세요. (개념02 · 개념04 · 개념05)

p>우리 동네 빵집
 매일 아침 7시에 문을 엽니다.
 오후 8시에 닫습니다.
 궁금한 점은 전화 주세요.

힌트 1 — 첫 줄은 왜 코드가 글자로 보일까요? 개념02 섹션 5 를 보세요. 힌트 2 — 두 문장이 왜 한 줄이 되었을까요? 개념05 섹션 4 를 보세요. 힌트 3 — 마지막 문단에 마우스를 올리면 설명이 중간에 잘립니다. 개념04 섹션 4 를 보세요.

여기에 코드를 쓰세요

고치면 네 줄이 보입니다. 둘째와 셋째 줄은 한 문단 안의 두 줄이라 사이가 좁습니다.

01단원 마무리 HTML 시작하기

자주 넘어지는 자리

- 1 저장(Ctrl+S) → 새로고침(F5) 세 단계. 이걸 안 하면 이후 전 단원이 막힙니다
- 2 HTML 은 틀려도 **에러가 안 난다**는 것. 닫는 태그를 지워 보이세요

텍스트 태그

OPEN 02_텍스트_태그

h1 제목입니다

h2 제목입니다

h3 제목입니다

h4 제목입니다

01 제목, h1 부터 h6 까지

02 문단과 줄바꿈

03 강조와 의미

04 span 과 인라인

05 특수문자

- 연습문제

제목, h1 부터 h6 까지

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 제목, h1 부터 h6 까지

01단원에서 “HTML 은 글에 이름표를 붙이는 일” 이라고 했습니다. 02단원부터는 그 이름표를 하나씩 배웁니다. 첫 번째가 제목입니다.

종이책을 한번 떠올려 보세요. 책에는 이런 것들이 있습니다.

책 제목	요리 백과사전
큰 목차	1장 국과 찌개
작은 목차	김치찌개 만들기
더 작은 것	재료 준비

안으로 들어갈수록 더 작은 덩어리입니다. 이렇게 “무엇이 무엇에 속하는가” 를 나타내는 구조를 계층이라고 부릅니다.

HTML 에는 이 계층을 표시하는 태그가 여섯 개 있습니다.

h1 h2 h3 h4 h5 h6

h 는 heading(제목) 의 첫 글자이고, 뒤의 숫자가 단계입니다. 숫자가 작을수록 큰 단계입니다. h1 이 가장 큰 단계, h6 이 가장 작은 단계입니다.

여기서 이 파일에서 가장 중요한 이야기가 나옵니다. 제목 태그를 처음 보면 누구나 이렇게 생각합니다.

"h1 은 글씨가 크고 h6 은 글씨가 작구나. 크기 조절 태그구나."

그렇지 않습니다. h 태그는 크기 태그가 아니라 목차 태그입니다. 글씨 크기는 그저 브라우저가 덤으로 얹어 준 것일 뿐입니다. 섹션 5 에서 이 생각이 왜 틀렸는지를 눈으로 확인합니다.

이 파일은 제목 태그 여섯 개를 다룹니다. 크기가 어떻게 달라지는지 보는 것이 목적이 아니라, 제목이 글의 목차를 만든다는 것을 이해하는 것이 목적입니다. 화면과 VS Code 를 계속 번갈아 보세요.

제목 태그 여섯 개

섹션 1

h1 부터 h6 까지 여섯 개가 전부입니다. h7 은 없습니다.

제목 태그는 다른 태그와 똑같이 생겼습니다. 여는 태그와 닫는 태그가 짝입니다.

```
<h1>가장 큰 제목</h1>
<h2>그다음 제목</h2>
```

아래 상자 안에는 여섯 개를 전부 한 번씩 썼습니다. 브라우저가 붙여 주는 기본 크기를 크롬에서 재 보면 이렇습니다.

h1	32px	
h2	24px	
h3	18.72px	
h4	16px	← 보통 문단(p) 과 똑같은 크기입니다
h5	13.28px	← 문단보다 작습니다
h6	10.72px	← 문단보다 훨씬 작습니다

h4 부터는 글씨가 커지지 않습니다. 오히려 작아집니다. "제목 = 큰 글씨" 가 아니라는 첫 번째 증거입니다. 섹션 5 에서 다시 봅니다.

여섯 개 모두 굵게 나옵니다. 굵기 값은 700 으로 전부 같습니다.

알림 · 이 상자 안의 h1 은 크기를 보여 주려고 넣은 시범용입니다.

원래 한 페이지에는 h1 이 하나뿐이어야 합니다.
그 이야기는 섹션 3 에서 합니다.

화면 여섯 줄이 위에서 아래로 차례로 놓이고, 글자가 점점 작아집니다.

여섯 줄 모두 굵은 글씨입니다. 각 줄은 상자 가로폭을 통째로 차지합니다

h1 제목입니다
h2 제목입니다
h3 제목입니다
h4 제목입니다
h5 제목입니다
h6 제목입니다

제목 태그는 한 줄을 통째로 차지합니다. 옆에 다른 글자를 붙일 수 없고, 다음 내용은 반드시 아랫줄로 내려갑니다. 이런 성질을 가진 요소를 블록 요소라고 부릅니다. 블록 요소는 개념04 에서 제대로 다룹니다.

▣ 직접 해보기 1

위 상자의 `<h3>h3 제목입니다</h3>` 안의 글자를 여러분 이름으로 바꿔 보세요. 저장하고(Ctrl+S) 브라우저에서 새로고침(F5) 하면 바뀝니다.

제목은 목차를 만듭니다

섹션 2

제목 태그의 진짜 일은 꾸미기가 아니라 글을 덩어리로 나누는 것입니다.

요리 페이지를 하나 쓴다고 합시다. 내용을 이렇게 나눌 수 있습니다.

김치찌개 만들기	← 이 덩어리 전체의 제목
재료 준비	← 그 안의 작은 덩어리
만드는 순서	← 그 안의 작은 덩어리

“재료 준비”와 “만드는 순서”는 “김치찌개 만들기”에 속합니다. 속한 쪽이 한 단계 아래이므로 h3을 씁니다.

```
<h2>김치찌개 만들기</h2> <h3>재료 준비</h3> <p>돼지고기와 김치를 준비합니다.</p> <h3>만드는 순서</h3> <p>냄비에 김치를 볶습니다.</p>
```

코드에서 h3을 한 칸 들여쓴 것을 보세요. 브라우저는 들여쓰기를 보지 않습니다. 사람이 계층을 눈으로 찾으라고 넣은 것입니다.

이렇게 써 두면 무슨 이득이 있을까요. 세 가지입니다.

1 사람이 훑어 읽을 때 어디가 어디인지 바로 보입니다.

2 눈이 불편한 사람이 쓰는 화면 낭독기는

“제목만 건너뛰며 듣기”를 할 수 있습니다. 목차가 있어야 가능한 일입니다.
3. 검색 사이트가 이 페이지가 무엇에 대한 글인지 파악할 때 제목을 봅니다.

p 로만 쓴 글에는 이 셋 중 아무것도 없습니다. 글자만 똑같이 보일 뿐입니다.

아래 상자는 이 페이지(자료 파일) 안에 들어 있는 조각이라 h2 부터 시작합니다. 진짜 요리 페이지였다면 "김치찌개 만들기" 자리에 h1 을 썼을 것입니다.

화면 "김치찌개 만들기" 가 가장 크게, "재료 준비" 와 "만드는 순서" 가

그보다 조금 작게 보입니다. 두 h3 사이에 보통 크기의 설명 문단이 한 줄씩 끼어 있습니다

김치찌개 만들기
재료 준비
돼지고기와 김치를 준비합니다.
만드는 순서
냄비에 김치를 볶습니다.

▹ 직접 해보기 2

위 상자 맨 아래에 <h3>보관 방법</h3> 을 한 줄 넣어 보세요. "만드는 순서" 와 똑같은 크기로 세 번째 소제목이 생깁니다.

h1 은 페이지에 하나

섹션 3

h1 은 "이 페이지 전체가 무엇에 대한 글인가" 를 적는 자리입니다. 그런 문장은 페이지마다 하나뿐입니다.

신문 1면을 생각해 보세요. 가장 큰 머리기사 제목은 하나입니다. 머리기사가 세 개면 그건 머리기사가 아니라 그냥 기사 세 개입니다.

h1 도 똑같습니다. h1 이 세 개면 "이 페이지의 주제" 가 세 개라는 뜻이 되어, 읽는 사람도 기계도 무엇이 주제인지 알 수 없게 됩니다.

규칙은 간단합니다.

- h1 → 페이지에 하나. 페이지 전체의 제목.
- h2 → 큰 덩어리마다 하나씩. 여러 개 써도 됩니다.
- h3 → h2 안의 작은 덩어리. 여러 개 써도 됩니다.

h2 부터는 몇 개를 써도 괜찮습니다. 제한이 있는 것은 h1 하나뿐입니다.

— 이 파일에 대한 솔직한 이야기

지금 이 파일에도 h1 이 두 개 있습니다. 하나는 맨 위의 "개념 01 — 제목, h1 부터 h6 까지" 이고, 다른 하나는 섹션 1 상자 안의 시범용 "h1 제목입니다" 입니다. 크기를 보여 주려고 일부러 규칙을 어긴 것입니다. 여러분이 만드는 페이지에서는 h1 을 하나만 쓰세요.

아래 상자는 h1 이 이미 있는 페이지 안의 조각이라 h2 부터 시작합니다.

"인기 있는 빵" 만 그보다 한 단계 작게 보입니다

우리 동네 빵집
아침 7시에 문을 엽니다.
인기 있는 빵
소보로빵과 단팥빵이 잘 나갑니다.
찾아오는 길
시장 정문 바로 앞입니다.

이런 실수를 합니다

잘못된 예

```
<h1>우리 동네 빵집</h1>
<h1>인기 있는 빵</h1>
<h1>찾아오는 길</h1>
```

셋 다 h1 이라 어느 것이 페이지의 주제인지 알 수 없습니다. 화면에는 큰 글씨 세 줄이 멀쩡하게 나옵니다. 보기에는 아무 문제가 없어서 더 알아채기 어렵습니다.

이렇게 쓰세요

```
<h1>우리 동네 빵집</h1>
<h2>인기 있는 빵</h2>
<h2>찾아오는 길</h2>
```

▣ 직접 해보기 3

VS Code 에서 Ctrl+F 를 누르고 <h1 을 찾아보세요. 몇 군데가 찾아지는지 세고, 그중 진짜로 화면에 그려지는 h1 은 몇 개인지도 세어 보세요.

단계를 건너뛰지 않기

섹션 4

h2 다음에 갑자기 h4 를 쓰면 안 됩니다. 한 단계씩 내려갑니다.

왜 안 될까요. 계단을 떠올려 보세요. 2층에서 3층으로 올라가는 계단이 없고 4층으로 바로 뛰는 계단만 있다면, 3층은 존재하지 않는 층이 됩니다.

제목도 같습니다. h2 다음에 h4 가 오면 "h3 짜리 덩어리가 하나 있는데 제목이 비어 있다" 는 뜻이 됩니다. 읽는 사람은 없는 층을 찾느라 헤매게 됩니다.

틀림	<code><h2>김치찌개 만들기</h2></code> <code><h4>재료 준비</h4></code>	← h3 을 건너뛰었습니다
맞음	<code><h2>김치찌개 만들기</h2></code> <code><h3>재료 준비</h3></code>	

여기서 사람들이 왜 h3 을 건너뛸까요. 거의 항상 이유가 하나입니다.

"h3 은 글씨가 너무 커서요. h4 가 크기가 딱 맞아서 h4 를 썼어요."

크기 때문에 단계를 고른 것입니다. 이것이 이 단원에서 가장 흔한 실수이고, 섹션 5 의 주제입니다.

아래 상자 위쪽이 건너뛴 예, 아래쪽이 제대로 쓴 예입니다. 두 경우의 화면 차이는 글씨 크기뿐입니다. 에러는 나지 않습니다.

화면 "김치찌개 만들기" 는 크게 보이고, "재료 준비" 는 아래 설명 문단과

똑같은 크기라서 제목처럼 보이지 않습니다. 굵기만 다릅니다

김치찌개 만들기
재료 준비
돼지고기와 김치를 준비합니다.

화면 "재료 준비" 가 아래 설명 문단보다 확실히 커서 제목으로 보입니다

김치찌개 만들기
재료 준비
돼지고기와 김치를 준비합니다.

올라올 때는 건너뛰어도 됩니다. h3 으로 작은 덩어리를 끝내고 다음 h2 로 올라가는 것은 정상입니다. 내려갈 때만 한 단계씩 내려가면 됩니다.

▫ 직접 해보기 4

첫 번째 상자의 `<h4>` 와 `</h4>` 를 `<h3>` 와 `</h3>` 로 고쳐 보세요. 두 상자가 똑같아집니다.

크기 때문에 h 태그를 고르면 안 되는 이유

섹션 5

이 파일에서 가장 중요한 섹션입니다. 제목 태그는 크기 조절 도구가 아닙니다.

초보자가 가장 많이 하는 일이 이것입니다.

"이 줄을 좀 크게 하고 싶은데... h1 로 감싸면 커지네? h1 쓰자."

화면만 보면 성공한 것 같습니다. 글씨가 실제로 커집니다. 그래서 이 실수는 몇 달씩 들이지 않고 굴러갑니다.

무엇이 잘못되는지 세 가지입니다.

- [1] 목차가 거짓말이 됩니다.
"오늘의 메뉴는 김치찌개입니다" 를 h1 으로 감싸면
기체는 이 페이지의 주제가 그 문장이라고 읽습니다.
크게 보이려고 한 것뿐인데 글의 뼈대가 통째로 어긋납니다.
- [2] 화면 낭독기가 그 줄을 제목으로 읽어 줍니다.
눈이 불편한 사람에게는 "제목: 오늘의 메뉴는 김치찌개입니다" 로 들립니다.
본문 한 줄이 갑자기 목차 항목이 되어 버립니다.
- [3] 애초에 목적을 이루지도 못합니다.
제목 태그로 낼 수 있는 크기는 여섯 가지뿐입니다.
"조금만 더 크게" 는 h 태그로 할 수 없습니다.

그러면 글씨 크기는 무엇으로 바꿀까요. 09단원에서 배우는 CSS 의 font-size 로 바꿉니다. 지금은 "크기는 CSS 가 하는 일" 이라는 것만 알아 두세요.

아래 상자에서 결정적인 증거를 봅시다.

h4 로 쓴 제목	→ 16px	
그냥 p 로 쓴 문단	→ 16px	← 크기가 똑같습니다
h6 로 쓴 제목	→ 10.72px	← 문단보다 오히려 작습니다

"제목 태그를 쓰면 커진다" 가 사실이라면 이런 일이 생길 수 없습니다. h 태그가 정하는 것은 크기가 아니라 단계이기 때문에 생기는 결과입니다.

화면 위 두 줄은 글자 크기가 똑같고 첫 줄만 굵습니다.

세 번째 줄은 굵지만 앞의 두 줄보다 눈에 띄게 작습니다

h4 로 쓴 제목
그냥 p 로 쓴 문단
h6 로 쓴 제목

이런 실수를 합니다

잘못된 예

```
<h1>오늘의 메뉴는 김치찌개입니다</h1>
```

글씨를 크게 하려고 h1 을 썼습니다. 화면에서는 실제로 커집니다. 그래서 잘못된 줄 모르고 넘어갑니다.
하지만 이 페이지의 주제가 "오늘의 메뉴는 김치찌개입니다" 로 바뀌어 버렸습니다.

이렇게 쓰세요

```
<p>오늘의 메뉴는 김치찌개입니다</p>
```

▣ 직접 해보기 5

위 상자에서 `<h6 id="tiny-h6">` 를 `<h2 id="tiny-h6">` 로, `</h6>` 를 `</h2>` 로 바꿔 보세요. 같은 글자가 얼마나 커지는지 확인한 뒤 원래대로 되돌리세요.

02

자주 하는 실수

섹션 6

01단원에서 배운 것을 다시 기억하세요. 아래 실수들은 전부 에러가 나지 않습니다. 화면이 조용히 이상해질 뿐입니다.

이런 실수를 합니다

크기 때문에 단계를 고름

이런 실수를 합니다

실수 1 — 크기가 마음에 든다는 이유로 단계를 고름

```
<h2>김치찌개 만들기</h2>
```

```
<h5>재료 준비</h5>
```

h3 은 커 보이고 h5 는 딱 좋아 보여서 h5 를 골랐습니다. 화면은 원하는 대로 나옵니다. 그래서 아무도 안 알려 줍니다. 대신 목차에 h3, h4 두 층이 비어 있는 상태가 됩니다.

이런 실수를 합니다

h7

이런 실수를 합니다

실수 2 — h7 을 씀

```
<h7>더 작은 제목</h7>
```

h7 이라는 태그는 없습니다. 브라우저는 모르는 태그를 없는 셈 치고 안의 글자만 그립니다. 굵어지지도 커지지도 않고 그냥 보통 글자로 나옵니다. 아래 상자가 실제 결과입니다. h7 을 쓴 실제 결과 더 작은 제목

이런 실수를 합니다

닫는 태그의 숫자를 다르게 씀

이런 실수를 합니다

실수 3 — 여는 쪽과 닫는 쪽의 숫자가 다름

```
<h2>우리 동네 빵집</h3>
```

h2 로 열고 h3 으로 닫았습니다. 브라우저는 짝이 안 맞는 </h3> 를 그냥 버리고 h2 를 알아서 닫습니다. 결과는 h2 로 보이지만, 코드를 나중에 읽는 사람은 h3 인 줄 알고 헷갈립니다.

이런 실수를 합니다

제목 안에 문단을 넣음

이런 실수를 합니다

실수 4 — 제목 안에 문단을 넣음

```
<h2>빵집 소개
<p>아침 7시에 엽니다</p>
</h2>
```

제목은 한 줄짜리 이름표입니다. 그 안에 문단이 들어갈 자리가 없습니다. 그런데 브라우저는 이것도 에러로 보지 않고 p 를 h2 안에 그대로 넣어 둡니다. 그러면 안쪽 p 가 h2 의 큰 글씨와 굵기를 그대로 물려받습니다. 본문으로 쓴 줄이 제목처럼 크고 굵게 나오는 것입니다. 아래가 실제 결과입니다. 제목 안에 문단을 넣은 실제 결과 빵집 소개아침 7시에 엽니다 이게 원래 본문 크기입니다

이런 실수를 합니다

제목만 잔뜩

이런 실수를 합니다

실수 5 — 본문 없이 제목만 이어 씀

```
<h2>메뉴</h2>
<h3>아메리카노</h3>
<h3>카페라떼</h3>
<h3>녹차라떼</h3>
```

제목은 “이 아래에 내용이 있다” 는 표시입니다. 내용이 없으면 표시할 것이 없습니다. 메뉴 이름을 늘어놓는 것은 목록이 할 일이고, 목록 태그는 03단원에서 배웁니다. 지금은 p 로 쓰면 됩니다.

정리

▫ 직접 해보기

6

정답

1) 섹션 1 상자의 h3 줄을 이렇게 바꿉니다.

```
<h3 id="step-h3">김민준</h3>
```

→ 세 번째 줄 글자만 바꿉니다. 크기와 굵기는 그대로입니다.

태그를 그대로 두고 내용만 바꿨기 때문입니다.

2) 섹션 2 상자의 마지막 p 아래에 이렇게 넣습니다.

```
<h3>보관 방법</h3>
```

→ “만드는 순서”와 똑같은 크기의 소제목이 한 줄 더 생깁니다.

같은 h3 이므로 같은 단계, 같은 모양입니다.

3) VS Code 에서 Ctrl+F 로 <h1> 을 찾으면 네 군데에 표시가 됩니다. 그런데 그중 진짜 h1 요소는 두 개뿐입니다.

```
맨 위 제목 줄 (class 가 doc-head 인 것) ← 진짜 h1
섹션 1 상자 안의 시범용 (id 가 step-h1) ← 진짜 h1
```

나머지 두 군데는 주석 안에 적어 둔 글자입니다. 하나는 섹션 1의 설명 주석이고, 하나는 지금 읽고 있는 이 정답 주석입니다. 주석에 적은 글자는 화면에 그려지지 않습니다(01단원 개념05).

→ 섹션 3에서 설명한 그대로입니다. 진짜 h1 은 두 개입니다.

여러분의 페이지에서는 이 개수가 하나여야 합니다.

4) 섹션 4 첫 상자를 이렇게 바꿉니다.

```
<h3 id="skip-h4">재료 준비</h3>
```

→ “재료 준비”가 커져서 두 상자의 화면이 똑같아집니다.

h4 는 문단과 같은 16px, h3 은 18.72px 이기 때문입니다.

5) 섹션 5 상자의 h6 줄을 이렇게 바꿉니다.

```
<h2 id="tiny-h6">h6 로 쓴 제목</h2>
```

→ 10.72px 이던 글자가 24px 로 커집니다. 같은 글자인데 두 배 넘게 커집니다.

확인했으면 다시 h6 으로 되돌려 놓으세요.

문단과 줄바꿈

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 문단과 줄바꿈

앞 파일에서 제목을 배웠습니다. 이제 제목 아래에 들어갈 본문 차례입니다. 본문을 담는 태그가 p 입니다. paragraph(문단) 의 줄임말입니다.

이 파일에서 넘어야 할 고비가 하나 있습니다. 01단원 개념05 에서 배운 규칙입니다.

소스에 아무리 엔터를 치고 공백을 넣어도
브라우저는 그것을 전부 공백 한 칸으로 줄여 버립니다.

워드나 한글 문서를 쓰던 사람에게 이걸 아주 이상한 일입니다. 엔터를 세 번 쳤는데 화면에서는 아무 일도 일어나지 않으니까요. 그래서 초보자는 이렇게 생각합니다.

"그러면 줄바꿈은 어떻게 하지? 아, br 이라는 게 있구나!"
"간격을 벌리고 싶으면 br 을 여러 개 쓰면 되겠네!"

앞의 생각은 맞고 뒤의 생각은 틀렸습니다. 이 파일은 그 둘을 갈라 놓는 것이 목표입니다. 배울 태그는 셋입니다.

p	문단 하나를 감쌉니다.	닫는 태그가 있습니다.
br	그 자리에서 줄을 바꿉니다.	빈 태그라 닫지 않습니다.
hr	가로줄을 하나 긋습니다.	빈 태그라 닫지 않습니다.

이 파일은 p, br, hr 세 태그를 다룹니다. 특히 왜 소스에서 엔터를 쳐도 줄이 안 바뀌는지, 그리고 왜 br 을 여러 개 쓰면 안 되는지를 확실히 하고 넘어갑니다.

p — 문단

섹션 1

글 한 덩어리를 <p> 와 </p> 로 감싸면 문단이 됩니다.

문단이란 “여기서부터 여기까지가 한 이야기 덩어리다” 라는 뜻입니다. 종이 위에서 한 칸 띄우고 새로 시작하는 그 덩어리와 같습니다.

```
<p>저는 첫 번째 문단입니다.</p>
<p>저는 두 번째 문단입니다.</p>
```

브라우저는 문단마다 이렇게 해 줍니다.

- 1
- 문단은 한 줄을 통째로 차지합니다. 옆에 다른 것이 올 수 없습니다.
- 2
- 문단과 문단 사이를 저절로 띄웁니다.

크롬에서 재 보면 문단 하나의 높이는 27px 정도이고, 문단 위아래에 16px 씩 여백이 붙습니다. 여러분이 띄우라고 시킨 적이 없는데도 그렇습니다. 태그에 그 뜻이 들어 있기 때문입니다.

개념01 에서 이름만 본 블록 요소가 바로 이것입니다. p 도 h2 와 같은 부류로, 한 줄을 통째로 차지합니다. 개념04 에서 제대로 다룹니다. 지금은 이름만 기억하세요.

화면

두 줄이 위아래로 놓고, 두 줄 사이가 눈에 띄게 벌어져 있습니다.

각 줄은 점선 상자의 가로폭을 통째로 차지합니다

```
저는 첫 번째 문단입니다.
저는 두 번째 문단입니다.
```

문단 사이 간격은 브라우저가 정한 기본값입니다. 이 간격을 여러분이 원하는 만큼 바꾸는 방법은 10단원 (여백)에서 배웁니다. 지금은 간격을 손대려 하지 말고 기본값 그대로 쓰세요.

직접 해보기 1

위 상자 안에 <p>저는 세 번째 문단입니다.</p> 를 한 줄 더 넣어 보세요. 저장하고(Ctrl+S) 새로고침 (F5) 하면 줄이 하나 늘어납니다.

01단원 개념05 에서 본 규칙을 여기서 다시 만납니다. 스페이스를 몇 번 치든, 엔터를 몇 번 치든 공백 한 칸이 됩니다.

아래 상자 안에는 정확히 이렇게 썼습니다. 소스를 직접 열어서 확인해 보세요.

```
<p id="ws-demo">공백을      많이      넣고
  줄도
  여러 번 바꿨습니다</p>
```

스페이스를 다섯 칸, 여섯 칸 넣었고 엔터도 두 번 쳤습니다. 그런데 화면에는 한 줄로 붙어서 나옵니다.

왜 이렇게 만들었을까요. 이유가 있습니다. HTML 소스는 사람이 읽기 좋게 들여쓰기를 합니다. 만약 그 들여쓰기 공백이 전부 화면에 나온다면 코드를 보기 좋게 정리할 때마다 화면이 망가질 것입니다.

그래서 브라우저는 이렇게 약속했습니다.

연달아 붙은 공백과 줄바꿈은 → 공백 한 칸으로 취급한다

이것을 공백이 접힌다고 표현합니다. 영어로는 whitespace collapsing 이라고 하는데, 지금은 우리말로만 기억해도 됩니다.

그래서 화면에서 줄을 바꾸려면 엔터가 아니라 태그를 써야 합니다. 섹션 3 의 br 이 그 태그입니다.

화면 "공백을 많이 넣고 줄도 여러 번 바꿨습니다" 가 한 줄로 붙어서 나옵니다.

넉넉히 넣은 공백은 전부 한 칸으로 줄었습니다

```
공백을      많이      넣고
  줄도
  여러 번 바꿨습니다
```

공백을 일부러 남기고 싶을 때도 방법이 있습니다. 라는 특수문자를 씁니다. 개념05 에서 배웁니다.

▫ 직접 해보기

2

위 상자의 문단 안에 엔터를 세 번 더 치고 스페이스도 잔뜩 넣어 보세요. 저장하고 새로고침해도 화면이 전혀 바뀌지 않는 것을 확인하는 것이 목적입니다.

br — 줄바꿈

 는 빈 태그입니다. 감쌀 내용이 없어서 닫지 않습니다.

01단원에서 빈 태그를 배웠습니다. 줄바꿈은 그 자리에서 일어나는 사건이지 무언가를 감싸는 일이 아니라서 닫는 태그가 없습니다.

셋 다 똑같이 동작하지만 이 자료에서는 마지막 모양으로 통일합니다.

```
<br> <br/> <br />
```

아래 상자 안에는 이렇게 썼습니다.

```
<p id="br-demo">서울시 강남구 테헤란로 123<br />4층 402호</p>
```

주소는 원래 두 줄로 적는 것이 자연스럽습니다. 그런데 주소 두 줄은 서로 다른 문단이 아닙니다. 한 덩어리 안의 두 줄입니다. 이럴 때 br 을 씁니다.

br 을 써도 되는 경우는 사실상 이 정도입니다.

주소	서울시 강남구 테헤란로 123 4층 402호
시(詩)	한 줄 한 줄이 의미가 있는 글
가사	노래 가사

공통점이 보이나요. 전부 “그 자리에서 줄이 바뀌는 것 자체가 내용의 일부” 인 글입니다. 단순히 보기 좋으라고 줄을 바꾸는 것은 여기에 해당하지 않습니다.

화면	한 문단 안에서 "서울시 강남구 테헤란로 123" 과 "4층 402호" 가
----	---

두 줄로 나뉘어 보입니다. 두 줄 사이는 벌어지지 않고 바로 붙어 있습니다

```
서울시 강남구 테헤란로 1234층 402호
```

섹션 1 의 두 문단은 사이가 벌어졌는데 여기 두 줄은 붙어 있습니다. 문단이 두 개인 것과, 한 문단 안에서 줄만 바뀐 것은 다릅니다. 화면에서 간격을 보고 구분할 수 있습니다.

▸ 직접 해보기 3

위 상자에
 를 하나 더 넣어 주소를 세 줄로 만들어 보세요.

br 을 남용하면 안 되는 이유

섹션 4

 를 간격 벌리는 도구로 쓰지 마세요. 화면에서는 되는 것처럼 보이지만 되는 것이 아닙니다.

간격을 조금 벌리고 싶습니다. br 을 하나 넣어 봅니다. 조금 벌어집니다. 더 벌리고 싶습니다. 두 개 넣습니다. 더 벌어집니다. 이렇게 br 이 세 개, 네 개로 늘어나는 것이 초보자 코드의 전형적인 모습입니다.

아래 첫 번째 상자가 br 을 세 개 쓴 결과입니다. 높이를 재 보면 109px 입니다. 한 줄이 27px 이니까 네 줄짜리 자리를 차지한 셈입니다.

무엇이 문제일까요. 세 가지입니다.

- [1] 간격이 글자 크기에 묶입니다.
br 한 개가 만드는 간격은 그 글자의 한 줄 높이입니다.
09단원에서 글자 크기를 바꾸는 순간 간격도 같이 흔들립니다.
"여기를 30px 띄우고 싶다" 를 br 로는 표현할 수 없습니다.
- [2] 뜻이 없습니다.
br 세 개는 "여기서 줄을 세 번 바꿔라" 라는 뜻입니다.
여러분이 하려던 말은 "여기를 좀 띄워라" 였습니다. 둘은 다른 말입니다.
화면 낭독기는 이 자리에서 아무 의미도 읽어 내지 못합니다.
- [3] 나중에 고치기가 어렵습니다.
간격을 조금 줄이고 싶으면 br 개수를 세어 가며 지워야 합니다.
CSS 라면 숫자 하나만 고치면 됩니다.

그러면 간격은 무엇으로 벌릴까요. 10단원에서 배우는 CSS 의 margin(바깥 여백) 으로 벌립니다. 지금은 "간격은 CSS 가 하는 일" 이라는 것만 알아 두세요.

두 번째 상자는 문단 맨 끝에 br 을 붙인 경우입니다. 높이를 재 보면 27px, 즉 br 이 없을 때와 똑같습니다. 문단 끝에서는 어차피 줄이 바뀌기 때문에 br 이 아무 일도 하지 않습니다.

화면 "위쪽 글자" 와 "아래쪽 글자" 사이에 빈 줄 두 개가 생겨

네 줄짜리 자리를 차지합니다

위쪽 글자아래쪽 글자

화면 한 줄만 보입니다. br 을 붙이기 전과 아무 차이가 없습니다

문단 끝에 br 을 붙였습니다.

이런 실수를 합니다

잘못된 예

```
<p>첫 번째 이야기</p>
<br /><br /><br />
<p>두 번째 이야기</p>
```

문단 사이를 벌리려고 br 을 넣었습니다. 벌어지기는 합니다. 뜻은 없습니다.

이렇게 쓰세요

```
<p>첫 번째 이야기</p>
<p>두 번째 이야기</p>
```


▣ 직접 해보기 4

첫 번째 상자의 `<br />` 세 개 중 두 개를 지워 보세요. 간격이 어떻게 줄어드는지 확인한 뒤 되돌려 놓으세요.

hr — 가로줄

섹션 5

02

`<hr />` 도 빈 태그입니다. 가로줄을 하나 긁습니다.

hr 은 horizontal rule(가로줄) 의 줄임말입니다.

그런데 hr 의 진짜 뜻은 “선을 그어라” 가 아닙니다. “여기서 이야기가 바뀐다” 입니다. 선은 그 뜻을 눈에 보이게 한 결과일 뿐입니다.

소설책에서 장면이 바뀔 때 가운데에 짧은 줄이나 별표를 넣는 것을 본 적이 있을 겁니다. 그것과 같은 자리에 씁니다.

```
<p>오전에는 회의가 있었습니다.</p>
<hr />
<p>점심 이후 이야기입니다.</p>
```

크롬에서 hr 을 재 보면 높이 2px, 색은 회색(rgb(128, 128, 128)) 이고 상자 가로폭을 통째로 차지합니다. 위아래로 8px 씩 여백이 붙습니다.

선의 색이나 두께를 바꾸고 싶을 수 있습니다. 그건 10단원의 border(테두리) 로 합니다. 지금은 기본 모양 그대로 씁니다.

주의 · 칸을 나누는 장식 선으로 hr 을 남발하지 마세요.

“여기서 이야기가 바뀐다” 가 아닌 자리에 쓰면 br 남용과 똑같은 잘못입니다.

화면 두 문단 사이에 상자 가로폭만큼 긴 회색 가로줄이 하나 그어집니다.

아주 얇은 선입니다

오전에는 회의가 있었습니다.

점심 이후 이야기입니다.

▣ 직접 해보기 5

위 상자 맨 아래에 `<hr />` 와 `<p>저녁 이야기입니다.</p>` 를 차례로 넣어 가로줄을 두 개로 만들어 보세요.

다시 강조합니다. 아래 실수들은 전부 에러가 나지 않습니다. 화면이 조용히 이상해질 뿐입니다.

이런 실수를 합니다

문단 안에 문단

이런 실수를 합니다

실수 1 — p 안에 p 를 넣음

```
<p>바깥 문단입니다.
<p>안쪽 문단입니다.</p>
</p>
```

문단 안에 문단이 들어갈 자리는 없습니다. 브라우저는 안쪽 p 를 만나는 순간 바깥 p 를 강제로 닫아버립니다. 그래서 부모와 자식으로 만들려던 두 문단이 나란한 형제가 됩니다. 거기에 짝을 잃은 </p> 때문에 빈 문단이 하나 더 생깁니다. 아래가 실제 결과입니다. p 안에 p 를 넣은 실제 결과 바깥 문단입니다. 안쪽 문단입니다.

이런 실수를 합니다

빈 문단으로 간격 벌리기

이런 실수를 합니다

실수 2 — 빈 문단으로 간격을 벌리려 함

```
<p></p>
```

내용이 없는 문단은 높이가 0px 입니다. 아무 일도 일어나지 않습니다. “빈 줄을 넣으면 띄워지겠지” 하고 넣었다가 화면이 그대로라서 당황하게 됩니다. 아래 상자에는 두 문단 사이에 빈 문단이 하나 들어 있습니다. 빈 문단을 끼워 넣은 실제 결과 위쪽 문단입니다. 아래쪽 문단입니다.

이런 실수를 합니다

br 을 닫으려 함

이런 실수를 합니다

실수 3 — br 을 닫는 태그로 닫으려 함

```
앞<br />뒤</br>더
```

br 은 빈 태그라서 닫는 태그가 없습니다. 그런데 브라우저는 </br> 를 버리지 않고 또 하나의 줄바꿈으로 취급합니다. 그래서 두 줄이 나올 자리에 세 줄이 나옵니다. 아래가 실제 결과입니다. br 을 닫으려 한 실제 결과 앞뒤더

이런 실수를 합니다

hr 로 무언가를 감싸려 함

이런 실수를 합니다

실수 4 — hr 로 글자를 감싸려 함

```
<hr>가로줄 안에 글자</hr>
```

hr 도 빈 태그입니다. 감쌀 내용이 없습니다. 그런데 에러는 나지 않습니다. 브라우저는 가로줄을 긋고, 글자는 hr 밖으로 밀어냅니다. 결과만 보면 “선 아래에 글자가 있네” 라서 잘못을 알아채기 어렵습니다. hr 로 글자를 감싸려 한 실제 결과 가로줄 안에 글자

이런 실수를 합니다

엔터로 줄을 바꾸려 함

이런 실수를 합니다

실수 5 — 소스에서 엔터를 쳐서 줄을 바꾸려 함

```
<p>첫 줄  
두 번째 줄</p>
```

섹션 2 의 규칙 그대로입니다. 엔터는 공백 한 칸이 되어 “첫 줄 두 번째 줄” 이 한 줄로 붙어 나옵니다. 워드를 쓰던 사람이 거의 예외 없이 한 번씩 겪는 일입니다. 진짜로 줄을 바꾸려면
 를 쓰세요.

이런 실수를 합니다

br 로 문단을 대신함

이런 실수를 합니다

실수 6 — p 를 안 쓰고 **br** 로만 글을 씀

```
첫 번째 이야기<br />  
두 번째 이야기<br />  
세 번째 이야기
```

화면에는 세 줄이 보기 좋게 나옵니다. 그런데 이 글에는 문단이 하나도 없습니다. 기계에게는 줄만 바뀌는 긴 글자 한 덩어리로 보입니다. 문단이 세 개면 p 를 세 개 쓰세요. br 은 문단을 대신할 수 없습니다.

정답

1) 섹션 1 상자의 두 번째 p 아래에 이렇게 넣습니다.

```
<p>저는 세 번째 문단입니다.</p>
```

→ 상자 안의 줄이 두 줄에서 세 줄로 늘어납니다.

새 줄 위에도 똑같은 간격이 붙습니다. p 가 원래 그런 태그이기 때문입니다.

2) 섹션 2 상자의 문단을 예를 들어 이렇게 바꿉니다.

```
<p id="ws-demo">공백을      많이  
  
      넣고  
  
      줄도      여러 번 바꿨습니다</p>
```

→ 화면은 전혀 바뀌지 않습니다. 한 줄 그대로입니다.

연달아 붙은 공백과 엔터가 전부 공백 한 칸으로 접혔기 때문입니다.

3) 섹션 3 상자를 이렇게 바꿉니다.

```
<p id="br-demo">서울시 강남구 테헤란로 123<br />4층 402호<br />왼쪽 문</p>
```

→ 주소가 세 줄이 됩니다. 줄 사이는 여전히 벌어지지 않고 붙어 있습니다.

문단이 늘어난 것이 아니라 한 문단 안에서 줄만 늘어났기 때문입니다.

4) 섹션 4 첫 상자를 이렇게 바꿉니다.

```
<p id="br-many">위쪽 글자<br />아래쪽 글자</p>
```

→ 네 줄이던 자리가 두 줄로 줄어듭니다.

br 하나가 만드는 간격이 정확히 한 줄이라는 것을 확인할 수 있습니다.
확인했으면 br 을 다시 세 개로 되돌려 놓으세요.

5) 섹션 5 상자의 마지막 p 아래에 이렇게 넣습니다.

```
<hr />  
<p>저녁 이야기입니다.</p>
```

→ 회색 가로줄이 두 개가 되고, 문단이 세 덩어리로 나뉩니다.

hr 은 빈 태그라 닫는 태그를 쓰지 않았다는 점도 확인하세요.

강조와 의미

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 강조와 의미

지금까지 배운 태그는 글을 덩어리로 나누는 태그였습니다. 이번에는 문단 안의 한두 단어에 이름표를 붙이는 태그를 배웁니다.

오늘 회의는 3시입니다.
↳ "3시" 만 굵게

여기서 이 단원 전체에서 가장 중요한 이야기가 나옵니다.

HTML 에는 화면 결과가 똑같은 태그 쌍이 있습니다.

strong 과 b → 둘 다 굵게 보입니다
em 과 i → 둘 다 기울어 보입니다

화면이 똑같은데 왜 두 개씩 있을까요. 모양은 같고 뜻이 다르기 때문입니다.

strong	"이 부분이 중요하다"	← 뜻
b	"이 부분을 굵게 그려라"	← 모양
em	"이 부분을 힘주어 읽어라"	← 뜻
i	"이 부분을 기울여 그려라"	← 모양

뜻을 적는 태그와 모양을 적는 태그가 따로 있는 것입니다. 그리고 HTML에서는 거의 언제나 뜻을 적는 쪽을 씁니다.

왜 그런지는 섹션 3에서 셋으로 나누어 설명합니다. 지금은 이 한 줄만 기억하세요.

태그는 뜻으로 고르고, 모양은 CSS로 바꿉니다.

이 파일은 문단 안의 단어에 뜻을 붙이는 태그들을 다룹니다. strong b em i mark small del ins sub sup q blockquote 입니다. 개수가 많아 보이지만 모양이 아니라 뜻으로 고른다는 원칙 하나로 전부 통합니다.

strong 과 b

섹션 1

둘 다 굵게 보입니다. strong은 중요하다는 뜻이고, b는 굵게 그려라는 모양 지시입니다.

strong은 strong importance(강한 중요도)의 줄임말입니다. b는 bold(굵은)의 첫 글자입니다.

이름부터 다릅니다. 하나는 중요도 이야기이고 하나는 굵기 이야기입니다.

```
<p>내일은 <strong>휴무</strong>입니다.</p>
    ↳ 이 정보를 놓치면 안 된다는 뜻

<p>주인공의 이름은 <b>홍길동</b>입니다.</p>
    ↳ 중요하다기보다 눈에 띄게 두고 싶은 이름
```

크롬에서 재 보면 둘 다 굵기 값이 700으로 똑같습니다. 보통 글자의 굵기 값은 400입니다. 그러니까 화면상으로는 완전히 같습니다.

그러면 초보자는 어느 쪽을 써야 할까요. 판단 기준은 아주 간단합니다.

```
소리 내어 읽을 때 그 부분을 힘주어 읽게 되는가?
그렇다 → strong
아니다 → b (그리고 사실 이런 경우는 드뭅니다)
```

실무에서 쓰는 굵은 글씨의 대부분은 strong입니다. b를 써야 하는 경우는 상품 이름, 사람 이름처럼 "중요해서가 아니라 관례상 굵게 쓰는" 자리 정도입니다. 헛갈리면 strong을 쓰세요.

화면 두 줄 모두 가운데 두세 글자만 굵게 보이고 나머지는 보통 굵기입니다.

두 줄의 굵기는 서로 완전히 똑같아 보입니다

내일은 휴무입니다.
주인공의 이름은 홍길동입니다.

▫ 직접 해보기 1

위 상자에 <p>준비물은 신분증입니다.</p> 를 한 줄 더 넣어 보세요. 저장하고 (Ctrl+S) 새로고침(F5) 하면 굵은 단어가 하나 늘어납니다.

em 과 i

섹션 2

둘 다 기울여 보입니다. em 은 힘주어 읽으라는 뜻이고, i 는 기울여 그려라는 모양 지시입니다.

em 은 emphasis(강조) 의 줄임말입니다. i 는 italic(기울임꼴) 의 첫 글자입니다.

em 의 강조는 strong 의 강조와 종류가 다릅니다.

strong	중요도가 높다	→ 놓치면 안 되는 정보
em	말의 억양이 바뀐다	→ 이 단어에 힘을 주어 읽는다

예를 하나 보면 확실합니다. 같은 문장인데 힘주는 곳이 다르면 뜻이 달라집니다.

나는 오늘 간다.	→ 내일이 아니라 오늘 간다
나는 오늘 간다.	→ 다른 사람 말고 내가 간다

문장을 소리 내어 읽어 보면 차이가 들립니다. 그게 em 입니다.

i 는 배 이름, 학명, 외국어처럼 “관례상 기울여 쓰는” 자리에 씁니다. 힘주어 읽지는 않는데 기울여 적는 것이 관습인 경우입니다.

— 한글에서 주의할 점

크롬에서 재 보면 em 과 i 모두 font-style 값이 italic 으로 같습니다. 그런데 맑은 고딕 같은 한글 글꼴에는 진짜 기울임꼴이 들어 있지 않습니다. 그래서 브라우저가 글자를 억지로 비스듬히 눕혀서 그립니다. 이것을 가짜 기울임이라고 부릅니다. 영어보다 어색해 보이고 읽기도 조금 불편합니다. 그래서 한글 페이지에서는 em 을 아껴 쓰는 편이 좋습니다.

화면 두 줄 모두 가운데 몇 글자가 오른쪽으로 비스듬히 기울어 보입니다.

굽어지지는 않았습니니다. 두 줄의 기울기는 서로 똑같아 보입니다

나는 오늘 간다.
배 이름은 이순신함입니다.

strong 과 em 은 겹쳐 쓸 수 있습니다. 글자 처럼 쓰면 굵으면서 기울어집니다. 두 태그의 효과가 둘 다 적용되기 때문입니다.

▣ 직접 해보기 2

위 상자의 첫 줄을 <p>나는 오늘 간다.</p> 로 바꿔 보세요. 기울어지는 단어가 옮겨 가고, 문장을 소리 내어 읽었을 때 뜻이 달라지는지 확인해 보세요.

같이 보여도 뜻이 다릅니다

섹션 3

이 파일에서 가장 중요한 섹션입니다. 아래 상자에서 어느 쪽이 strong 이고 어느 쪽이 b 인지 맞춰 보세요.

아래 상자에는 같은 글자가 두 번 들어 있습니다. 하나는 strong 으로, 하나는 b 로 썼습니다. 순서는 알려드리지 않습니다.

화면만 보고 구분할 수 있나요. 못 합니다. 크롬에서 재 보면 둘 다 굵기 값이 정확히 700 입니다. em 과 i 도 마찬가지로 둘 다 italic 입니다.

이것이 요점입니다. 차이는 화면에 없습니다. 화면 밖에 있습니다. 어디에 있는지 셋으로 나눠 보겠습니다.

[1] 화면 낭독기

눈이 불편한 사람은 화면을 소리로 바꿔 주는 프로그램으로 웹을 읽습니다.
이 프로그램은 strong 을 만나면 목소리를 바꾸거나 힘주어 읽습니다.
b 는 그냥 읽습니다. 굵은 것은 소리로 옮길 수 없으니까요.
strong 으로 써 둔 글만 강조가 전달됩니다.

[2] 검색 사이트

검색 프로그램은 페이지에서 무엇이 중요한지 찾습니다.
strong 은 "글쓴이가 중요하다고 표시한 부분" 이라는 힌트가 됩니다.
b 는 그냥 굵은 글씨라 힌트가 되지 않습니다.

[3] 나중에 CSS 로 꾸밀 때

이게 가장 피부에 와닿는 이유입니다.
08단원에서 배우는 선택자로 "중요한 부분만 빨갭게" 를 하려고 하면 이렇게 씁니다.

```
strong { color: red; }
```

strong 으로 써 둔 곳만 빨갭습니다. b 로 써 둔 곳은 그대로입니다.
뜻을 기준으로 태그를 골라 두지 않으면 나중에 골라낼 방법이 없습니다.
(위 CSS 는 07단원부터 배웁니다. 지금은 이렇게 생겼다는 것만 보세요)

그래서 결론은 하나입니다.

굵게 보이고 싶어서 태그를 고르지 말고,
중요해서 굵어지도록 태그를 고르세요.

모양이 마음에 안 들면 09단원에서 CSS 로 바꾸면 됩니다. 뜻은 나중에 되돌릴 수 없지만 모양은 언제든지 바꿀 수 있습니다.

화면 첫 줄의 굵은 두 덩어리가 서로 완전히 똑같아 보입니다.

둘째 줄의 기울어진 두 덩어리도 서로 완전히 똑같아 보입니다. 어느 쪽이 어느 태그인지 화면만으로는 구분할 수 없습니다

중요합니다
그리고 중요합니다
기울입니다 그리고 기울입니다

정답은 앞쪽이 strong 과 em, 뒤쪽이 b 와 i 입니다. VS Code 에서 소스를 보면 바로 알 수 있습니다. 소스에는 차이가 있고 화면에는 차이가 없다는 것이 이 섹션의 전부입니다.

▫ 직접 해보기

3

위 상자의 와 를 과 로 바꿔 보세요. 화면이 하나도 안 바뀌는 것을 눈으로 확인하는 것이 목적입니다.

mark 와 small

섹션 4

mark 는 형광펜으로 칠한 부분, small 은 작게 덧붙이는 말입니다.

— mark

책을 읽다가 형광펜으로 칠하는 그 동작입니다. 중요해서가 아니라 “지금 이 맥락에서 눈여겨볼 곳” 이라는 뜻입니다.

strong 과 헷갈리기 쉬우니 이렇게 갈라 두세요.

strong 글쓰기가 원래부터 중요하다고 여기는 부분
mark 지금 읽는 사람 사정 때문에 눈여겨볼 부분

검색 결과에서 찾던 단어가 노랗게 칠해져 나오는 것이 mark 의 대표적인 쓰임입니다. 글이 중요해서가 아니라 그 사람이 그 단어를 찾았기 때문에 칠해진 것입니다.

크롬에서 재 보면 mark 는 배경색이 노랑 rgb(255, 255, 0), 글자색이 검정 rgb(0, 0, 0) 으로 바뀝니다. 이 자료의 보통 글자색은 rgb(34, 34, 34) 이라 mark 안의 글자만 더 새까맣습니다.

— small

글자를 작게 그립니다. 뜻은 “결다리로 덧붙이는 말” 입니다. 저작권 표시, 단서 조항, 주석 같은 것에 씁니다.

크롬에서 재 보면 13.3333px 입니다. 보통 글자가 16px 이니 조금 작습니다.

여기서도 같은 함정이 있습니다. “글씨를 작게 하고 싶어서” small 을 쓰면 안 됩니다. 크기는 09단원의 CSS 로 바꿉니다. small 은 결다리 말일 때만 씁니다.

화면 첫 줄의 “김치찌개” 네 글자 뒤에 노란 형광펜 칠이 되어 있습니다.

둘째 줄의 “(부가세 포함)” 은 앞의 글자보다 눈에 띄게 작습니다

검색어 김치찌개 가 들어간 글입니다.
가격 12,000원 (부가세 포함)

⇒ 직접 해보기 4

위 상자 첫 줄의 <mark> 를 으로 바꿔 보세요. 노란 칠이 사라지고 굵어집니다. 확인했으면 다시 <mark> 로 되돌리세요.

del 과 ins

섹션 5

del 은 지운 내용, ins 는 새로 넣은 내용입니다.

del 은 delete(지우다), ins 는 insert(넣다) 의 줄임말입니다.

문서를 고칠 때 “무엇을 지웠고 무엇을 새로 넣었는지” 를 남겨 두는 태그입니다. 가격이 바뀐 상품, 수정된 공지, 규정 개정문 같은 데 씁니다.

원래 가격 30,000원 할인가 <ins>19,000원</ins>

크롬에서 재 보면

del → text-decoration-line 값이 line-through (글자 가운데 줄)
ins → text-decoration-line 값이 underline (밑줄)

입니다.

주의 · del 을 그냥 취소선 도구로 쓰지 마세요.

지운 적이 없는 글에 del 을 붙이면 “이건 지워진 내용” 이라는 거짓말이 됩니다.
줄만 긋고 싶으면 09단원의 text-decoration 으로 그으면 됩니다.

주의 · ins 의 밑줄은 링크의 밑줄과 똑같이 생겼습니다.

04단원에서 배우는 링크(a) 도 기본이 밑줄입니다.

읽는 사람이 눌러 보려다 안 눌러서 당황할 수 있으니 ins 는 아껴 씁니다.

화면 "30,000원" 글자 가운데를 가로지르는 줄이 그어져 있고,

"19,000원" 아래에는 밑줄이 그어져 있습니다

원래 가격 30,000원

할인가 19,000원

▫ 직접 해보기 5

위 상자에 <p>모임 시간 오후 2시 <ins>오후 4시</ins></p> 를 한 줄 더 넣어 보세요.

sub 와 sup

섹션 6

sub 는 아래첨자, sup 는 위첨자입니다. 화학식과 수식에 씁니다.

sub 는 subscript(아래첨자), sup 는 superscript(위첨자) 의 줄임말입니다.

물 $H_{2}O$
2의 10제곱 2^{10}
제곱미터 m^{2}

크롬에서 재 보면 둘 다 글자 크기가 13.3333px 로 작아지고, vertical-align 이라는 값이 각각 sub 와 super 로 바뀝니다. vertical-align 은 "글자를 세로로 어디에 놓을지" 를 정하는 값입니다. 브라우저가 sub 와 sup 에 알아서 넣어 주는 값이라 우리가 쓸 일은 없습니다. 이 자료에서도 직접 쓰지는 않습니다. 이름만 보고 넘어가세요.

첨자가 들어간 문단은 줄 높이가 조금 커집니다. 위아래로 빠져나온 글자를 담아야 하기 때문입니다. 재 보면 보통 문단이 27px 인데 첨자가 있는 문단은 33px 입니다.

주의 · 첨자는 뜻이 분명한 자리에만 씁니다.

글자를 살짝 위로 올리고 싶다는 이유로 sup 를 쓰면 안 됩니다.

화면 낭독기가 "위첨자" 라고 읽어 버립니다.

화면 "H2O" 의 2 가 글자 아래쪽에 작게 붙어 있고,

"2 10" 의 10 이 글자 위쪽에 작게 붙어 있습니다

물은 H2O 이고,

210 은 1024 입니다.

▣ 직접 해보기 6

위 상자에 <p>넓이는 30m² 입니다.</p> 를 한 줄 더 넣어 제곱미터를 만들어 보세요.

q 와 blockquote

섹션 7

남의 말을 옮길 때 씁니다. q 는 문장 안의 짧은 인용, blockquote 는 덩어리로 떨어진 긴 인용입니다.

q 는 quotation(인용) 의 첫 글자입니다.

```
<p>그는 <q>배고파요</q> 라고 했습니다.</p>
```

여기서 아주 재미있는 일이 일어납니다. 소스에는 따옴표를 하나도 쓰지 않았는데 화면에는 따옴표가 나옵니다. 브라우저가 “이건 인용이구나” 하고 알아서 그려 준 것입니다.

더 재미있는 것은 그 따옴표가 글자가 아니라는 점입니다. 화면에는 보이지만 실제 글자로는 존재하지 않습니다. 마우스로 클릭해서 복사해 보면 따옴표가 안 떨어져 오는 것을 확인할 수 있습니다.

이게 개념03 전체가 하려던 이야기의 증거입니다. 태그는 화면을 꾸미는 물감이 아니라 뜻을 적는 이름표이고, 브라우저는 그 뜻을 읽고 자기 나름대로 행동합니다. 따옴표를 그려 준 것은 그 행동 중 하나입니다.

— blockquote

긴 인용은 문단 통째로 떨어져 나옵니다. blockquote 는 block(덩어리) + quote(인용) 입니다. 한 줄을 통째로 차지합니다. 크롬에서 재 보면 왼쪽에 40px, 오른쪽에도 40px 만큼 여백이 붙습니다. 글이 짧으면 왼쪽만 들어간 것처럼 보이지만, 글이 길어져 줄이 넘어가면 오른쪽도 40px 안으로 들어와 있는 것이 눈에 보입니다.

주의 · 이 들여쓰기 때문에 blockquote 를 “들여쓰기 도구” 로 쓰는 사람이 많습니다.

인용이 아닌 글을 blockquote 로 감싸면 “이건 남의 말” 이라는 거짓말이 됩니다.
들여쓰기는 10단원의 margin 으로 합니다.

화면 첫 줄의 “배고파요” 양옆에 곡선 큰따옴표가 붙어 있습니다.

아래 인용문은 한 줄을 통째로 차지하면서 왼쪽이 안쪽으로 들여져 있습니다

그는 배고파요 라고 했습니다.
인생은 가까이서 보면 비극이지만 멀리서 보면 희극이다.

q 안의 따옴표는 브라우저가 그린 것이라 글자로는 없습니다. 이 자료의 자동 검사기로 q 안의 글자를 읽어 보면 따옴표 없이 배고파요 네 글자만 나옵니다.

위 상자의 인용문을 마우스로 긁어서 복사한 뒤 메모장에 붙여넣어 보세요. 그리고 첫 줄의 배고파요 도 따옴표까지 긁어서 붙여넣어 보세요. 어느 쪽 따옴표가 따라오는지 확인해 보세요.

자주 하는 실수

섹션 8

여기 실수들은 화면이 원하는 대로 나오기 때문에 특히 안 들립니다. 에러도 물론 나지 않습니다.

이런 실수를 합니다

굵게 만들려고 태그를 고름

이런 실수를 합니다

실수 1 — 굵게 보이고 싶어서 strong 을 씀

```
<p><strong>메뉴 안내</strong></p>
```

제목을 굵게 만들고 싶어서 strong 을 썼습니다. 굵어지기는 합니다. 하지만 이건 제목이므로 <h2> 를 써야 합니다. strong 으로 쓴 제목은 목차에 잡히지 않습니다. 개념01 섹션 5 에서 본 것과 똑같은 종류의 잘못입니다.

이런 실수를 합니다

문단 전체를 strong

이런 실수를 합니다

실수 2 — 문단 전체를 strong 으로 감쌌

```
<p><strong>모든 문장이 다 중요합니다.  
안 중요한 문장이 없습니다.</strong></p>
```

전부 강조하면 아무것도 강조되지 않습니다. 화면 낭독기에게도 문단 전체가 강조로 들려서 오히려 알아듣기 어려워집니다. 강조는 한 문단에 한두 군데면 충분합니다. 아래가 실제 결과입니다. 문단 전체를 감싼 실제 결과 모든 문장이 다 중요합니다. 안 중요한 문장이 없습니다.

이런 실수를 합니다

blockquote 를 들여쓰기 도구로

이런 실수를 합니다

실수 3 — 들여쓰려고 `blockquote` 를 씀

```
<blockquote>여기는 그냥 제 설명입니다</blockquote>
```

왼쪽으로 40px 들여져서 보기에는 좋습니다. 하지만 뜻은 “이 문장은 남이 한 말을 옮긴 것” 이 됩니다. 내 말인데 남의 말이라고 표시한 셈입니다. 아래가 실제 결과입니다. 들여쓰기 용도로 쓴 실제 결과 보통 문단입니다. 여기는 그냥 제 설명입니다.

이런 실수를 합니다

`del` 을 취소선 도구로

이런 실수를 합니다

실수 4 — 줄만 갖고 싶어서 `del` 을 씀

```
<del>오늘의 할 일</del>
```

지운 적이 없는 글에 `del` 을 붙이면 “이 내용은 삭제되었다” 는 뜻이 붙습니다. 화면 낭독기는 “삭제됨” 이라고 알려 줍니다. 장식으로 줄을 갖고 싶다면 09단원의 `text-decoration` 을 쓰세요.

이런 실수를 합니다

태그 이름을 헛갈림

이런 실수를 합니다

실수 5 — `strong` 을 `string` 으로 씀

```
<string>중요합니다</string>
```

`string` 이라는 태그는 없습니다. 오타인데도 에러가 나지 않습니다. 브라우저는 모르는 태그를 없는 셈 치고 안의 글자만 그립니다. 그래서 굵어지지 않은 보통 글자가 나옵니다. “왜 안 굵어지지?” 싶으면 태그 이름 철자부터 확인하세요. 철자를 틀린 실제 결과 중요합니다

정리

▹ 직접 해보기 8

정답

1) 섹션 1 상자의 마지막 p 아래에 이렇게 넣습니다.

```
<p>준비물은 <strong>신분증</strong>입니다.</p>
```

→ 굵은 단어가 하나 늘어납니다. 굵기는 앞의 두 줄과 똑같습니다.

```
strong 이든 b 든 굵기 값이 700 으로 같기 때문입니다.
```

2) 섹션 2 상자의 첫 줄을 이렇게 바꿉니다.

```
<p><em id="s-em">나는</em> 오늘 간다.</p>
```

→ 기울어지는 단어가 “오늘” 에서 “나는” 으로 옮겨 갑니다.

소리 내어 읽으면 "다른 사람 말고 내가" 라는 뜻으로 들립니다.

3) 섹션 3 상자의 b 를 strong 으로 바꿉니다.

```
<span id="cmp-b-wrap"><strong id="cmp-b">중요합니다</strong></span>
```

→ 화면이 하나도 바뀌지 않습니다.

두 태그의 굵기 값이 똑같아서 눈으로는 구분할 방법이 없기 때문입니다.
확인했으면 b 로 되돌려 놓으세요.

4) 섹션 4 상자의 첫 줄을 이렇게 바꿉니다.

```
<p>검색어 <strong id="s-mark">김치찌개</strong> 가 들어간 글입니다.</p>
```

→ 노란 형광펜 칠이 사라지고 글자가 굵어집니다.

mark 는 배경색을 바꾸고 strong 은 굵기를 바꾸기 때문입니다.
확인했으면 mark 로 되돌려 놓으세요.

5) 섹션 5 상자의 마지막 p 아래에 이렇게 넣습니다.

```
<p>모임 시간 <del>오후 2시</del> <ins>오후 4시</ins></p>
```

→ “오후 2시” 에 가운데 줄이, “오후 4시” 에 밑줄이 그어집니다.

시간이 바뀌었다는 사실이 화면에도 뜻에도 남습니다.

6) 섹션 6 상자의 마지막 p 아래에 이렇게 넣습니다.

```
<p>넓이는 30m<sup>2</sup> 입니다.</p>
```

→ m 뒤의 2 가 위쪽에 작게 붙어 제곱미터로 보입니다.

7) 인용문을 굵어서 붙여넣으면 글자만 따라옵니다. “배고파요” 를 따옴표까지 굵어도 붙여넣기 결과에는 따옴표가 없습니다.

→ 따옴표는 브라우저가 그려 준 것이라 실제 글자가 아니기 때문입니다.

태그의 뜻을 브라우저가 읽고 행동한 결과입니다.

span 과 인라인

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — span 과 인라인

지금까지 태그를 열 개 넘게 배웠습니다. 그런데 눈치채셨을지 모르겠지만 태그들이 두 무리로 나뉘어 행동했습니다.

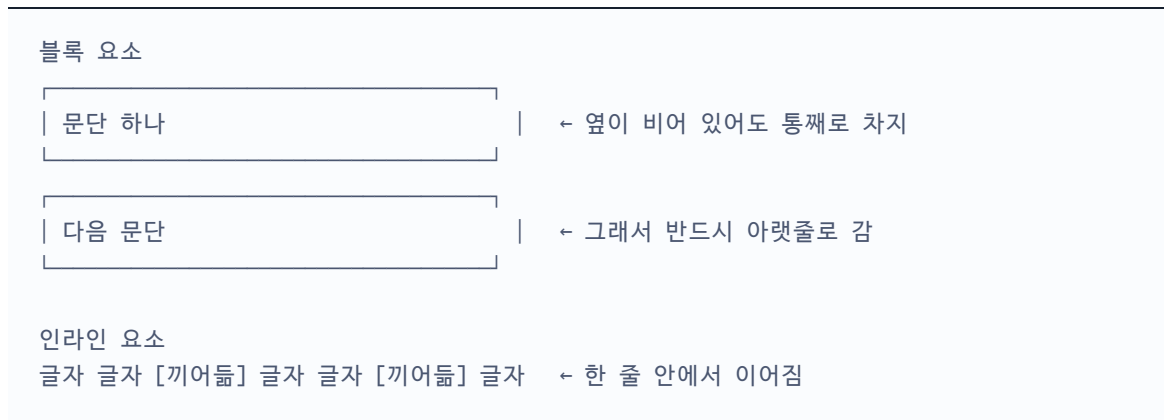
h1 p hr blockquote	→ 항상 한 줄을 통째로 차지했습니다
strong em mark del sub	→ 글자 사이에 쏙 끼어 들어갔습니다

이 두 무리에 이름이 있습니다.

블록(block) 요소	한 줄을 통째로 차지합니다
인라인(inline) 요소	글자 흐름 안에 깎입니다

이것이 HTML 을 배우면서 만나는 가장 중요한 구분 중 하나입니다. 이걸 모르면 07단원부터 시작하는 CSS 에서 계속 막힙니다. “왜 이걸 옆에 안 붙이지?” “왜 이걸 폭을 줘도 안 먹지?” 의 답이 전부 여기 있습니다.

그림으로 그리면 이렇습니다.



그리고 이 파일에서 태그를 하나 더 배웁니다. span 입니다. span 은 아무 뜻도 없는 인라인 요소입니다. 뜻이 없다는 것이 무슨 소린지, 그런 게 왜 필요한지 섹션 1 에서 봅니다.

예고 · 요소가 블록인지 인라인인지는 display 라는 값이 정합니다.

이 값을 여러분이 직접 바꾸는 것은 11단원에서 배웁니다.
이 파일에서는 값을 바꾸지 않고 읽기만 합니다.

이 파일은 span 태그 하나와, 블록 요소와 인라인 요소라는 큰 구분을 다룹니다. 뒤에 나오는 CSS 단위 전체가 이 구분 위에 세워집니다. 데모 상자에서 무엇이 줄을 차지하고 무엇이 끼어드는지를 눈으로 익히세요.

span

섹션 1

span 은 감싸기만 하고 아무 일도 하지 않습니다. 화면도 전혀 바뀌지 않습니다.

지금까지 배운 태그는 전부 뜻이 있었습니다. p 는 문단, strong 은 중요, mark 는 형광펜이었습니다.

span 은 다릅니다. 뜻이 하나도 없습니다.

```
<p>가격은 <span>12,000원</span> 입니다.</p>
```

span 을 넣기 전과 후의 화면이 완전히 같습니다. 굵어지지도, 색이 바뀌지도, 줄이 바뀌지도 않습니다.

그러면 왜 쓸까요. 이유는 하나입니다.

나중에 CSS 로 그 부분만 골라 꾸미려고 붙잡아 두는 손잡이

예를 들어 “가격만 빨강게” 하고 싶다고 합시다. 빨강게 할 부분을 브라우저에게 가리켜 줘야 하는데, 글자에는 손잡이가 없습니다. 그래서 span 으로 감싸서 손잡이를 만듭니다. 08단원에서 이렇게 씁니다.

```
<p>가격은 <span class="price">12,000원</span> 입니다.</p>
```

```
.price { color: red; }
```

여기서 판단 기준이 하나 생깁니다.

뜻이 있으면 → 뜻에 맞는 태그 (strong, mark, del ...)
뜻이 정말 없으면 → span

“빨강게 하고 싶으니까 무조건 span” 이 아닙니다. 중요해서 빨강게 하는 것이면 strong 이 맞습니다. 개념03 의 원칙이 여기서도 그대로 적용됩니다.

span 은 인라인 요소입니다. 그래서 감싼 글자가 원래 있던 줄에 그대로 남습니다. 크롬에서 재 보면 display 값이 inline 입니다.

화면 두 줄이 완전히 똑같아 보입니다. 위 줄에만 span 이 들어 있는데

화면에서는 아무 차이도 찾을 수 없습니다

```
가격은 12,000원 입니다.  
가격은 12,000원 입니다.
```

span 은 이 자료에서 아직 화면을 바꾸지 못합니다. 07단원에서 CSS 를 배우기 전까지는 손잡이일 뿐입니다. 지금은 “이런 태그가 있고 인라인이다” 만 알아 두세요.

▫ 직접 해보기

1

위 상자 첫 줄의 과 를 지워 보세요. 저장하고(Ctrl+S) 새로고침(F5) 해도 화면이 하나도 안 바뀌는 것을 확인한 뒤 되돌려 놓으세요.

블록 요소

섹션 2

p, h1~h6, hr, blockquote 가 블록 요소입니다.

블록 요소의 성질은 두 가지입니다.

- [1] 가로로 한 줄을 전부 차지합니다.
글자가 짧아도 그렇습니다. 오른쪽이 비어 있어도 그 자리는 이미 임자가 있습니다.
- [2] 그래서 다음 요소는 반드시 아랫줄로 내려갑니다.
옆에 붙일 방법이 없습니다.

아래 상자에 짧은 문단 세 개를 나란히 썼습니다. 글자는 다섯 글자뿐인데 세 줄이 나옵니다.

크롬에서 재 보면 문단 하나의 가로폭이 824px 입니다. 점선 상자 안쪽 폭이 딱 그만큼입니다. 즉 글자 길이와 상관없이 딱 채웁니다. 이것이 "한 줄을 통째로 차지한다" 의 정확한 의미입니다.

그림으로 보면 이렇습니다.

```

| 블록 하나 (오른쪽은 비었지만 자리는 차지) |
|
| 블록 둘 |
|

```

여기서 초보자가 가장 많이 하는 질문이 나옵니다. "그럼 문단 두 개를 옆에 나란히 못 놓나요?" 지금은 못 놓습니다. 11단원과 12단원에서 CSS 로 놓는 법을 배웁니다. HTML 만으로는 안 됩니다.

화면 세 줄이 위에서 아래로 차례로 놓입니다. 글자는 짧은데도

옆에 나란히 붙지 않고 각자 한 줄씩 차지합니다

```

블록 하나
블록 둘
블록 셋

```

▣ 직접 해보기 2

위 상자의 세 문단을 한 줄에 이어서 써 보세요. <p>블록 하나</p><p>블록 둘</p> 처럼 소스에서 붙여 써도 화면에서는 여전히 줄이 나뉩니다.

인라인 요소

섹션 3

span, strong, em, mark, del, ins, sub, sup, q, code 가 인라인 요소입니다.

인라인 요소의 성질도 두 가지입니다.

- [1] 딱 글자만큼만 차지합니다. 남은 자리를 차지하지 않습니다.
- [2] 그래서 앞뒤에 다른 것이 계속 이어 붙습니다. 줄이 안 바뀝니다.

아래 상자에 span 세 개를 붙여 썼습니다. 세 개가 한 줄에 나란히 나옵니다. 블록 세 개는 세 줄이었는데 인라인 세 개는 한 줄입니다. 이 차이가 전부입니다.

크롬에서 재 보면 이렇습니다.

```

span "인라인 하나" → 가로 86px (글자 길이만큼)
span "인라인 둘"   → 가로 70px (글자 길이만큼)

```

길이가 서로 다릅니다. 글자가 다르니 폭도 다른 것입니다. 블록은 글자가 뭐든 824px 로 똑같았습니다. 정확히 반대입니다.

— 왜 흐름이라고 부르나

글자는 왼쪽에서 오른쪽으로 흘러가다가 오른쪽 끝에 닿으면 아랫줄로 넘어갑니다. 이 흐름을 문서 흐름이라고 부릅니다. 인라인 요소는 이 흐름 안에 글자처럼 끼어듭니다. 흐름을 끊지 않습니다. 블록 요소는 흐름을 끊고 새 줄부터 시작합니다.

— 목록의 code 는 무엇인가

위 목록에 code 가 하나 끼어 있습니다. 이 자료에서 코드 글자를 보여 줄 때 쓰는 태그인데, 수업에서 따로 배우지는 않습니다. 지금은 “이런 태그도 인라인이구나” 만 보고 넘어가세요.

주의 · 인라인 요소에는 가로폭을 줄 수 없습니다.

10단원에서 width 를 배운 다음 span 에 폭을 줘 봐도 안 먹습니다.
"인라인이라서 안 된다" 가 그 답인데, 왜 그런지와 어떻게 푸는지는
11단원에서 제대로 다룹니다.
지금 이 구분을 익혀 두면 그때 해매지 않습니다.

화면 세 덩어리가 한 줄에 빈틈없이 이어 붙어 "인라인 하나인라인 둘인라인 셋"으로

보입니다. 줄이 나뉘지 않습니다

인라인 하나인라인 둘인라인 셋

위 결과에 띄어쓰기가 없는 것은 소스에서 `</span><span>` 를 붙여 썼기 때문입니다. 인라인 요소는 글자와 똑같이 취급됩니다. 띄우고 싶으면 사이에 공백 한 칸을 넣으면 됩니다.

▫ 직접 해보기

3

위 상자에서 `</span>` 와 `<span>` 사이에 스페이스를 한 칸씩 넣어 보세요. 세 덩어리가 띄어져 보이지만 여전히 한 줄입니다.

나란히 놓고 비교하기

섹션 4

글자를 똑같이 두고 감싸는 태그만 바꾸면 차지하는 폭이 달라집니다.

아래 상자에는 “짧은 글자” 라는 똑같은 다섯 글자가 두 번 들어 있습니다.

```
<p id="cmp-block">짧은 글자</p>          ← p 는 블록
<p><span id="cmp-inline">짧은 글자</span></p> ← span 은 인라인
```

화면만 보면 둘 다 그냥 한 줄에 "짧은 글자" 입니다. 똑같아 보입니다. 그런데 크롬에서 가로폭을 재 보면 완전히 다릅니다.

p → 824px (상자 안쪽을 꽉 채움)
span → 70px (글자 길이만큼만)

눈에는 안 보이는데 실제로는 열 배 넘게 차이가 납니다. p 의 오른쪽 754px 은 비어 보이지만 이미 p 가 차지한 자리입니다. 그래서 그 자리에 다른 것을 붙일 수 없습니다.

08단원에서 CSS 로 배경색을 칠해 보면 이 차이가 눈에 보이게 됩니다. 지금은 자로 잰 숫자로만 확인합니다. 이 파일 맨 아래 자동 검사 목록에 그 숫자가 적혀 있습니다.

화면 두 줄 모두 "짧은 글자" 로 똑같아 보입니다.

화면만으로는 차지한 폭의 차이를 알 수 없습니다

짧은 글자
짧은 글자

지금까지 배운 태그를 두 무리로 나누면 이렇습니다. 블록 — h1~h6, p, hr, blockquote 인라인 — span, strong, b, em, i, mark, small, del, ins, sub, sup, q, code, br

▣ 직접 해보기 4

개념03 파일을 열고 <mark> 데모를 보세요. 노란 칠이 글자 뒤에만 있고 줄 끝까지 이어지지 않습니다. mark 가 인라인이라 글자만큼만 차지하기 때문입니다.

인라인 안에 블록을 넣으면

섹션 5

인라인 요소 안에는 블록 요소를 넣지 않습니다. 넣어도 에러는 안 납니다. 대신 줄이 쪼개집니다.

규칙은 이렇습니다.

블록 안에 인라인 → 됩니다. <p>글자 강조</p>
블록 안에 블록 → 됩니다. (06단원에서 div 로 많이 하게 됩니다)
인라인 안에 인라인 → 됩니다. 글자 강조
인라인 안에 블록 → 안 됩니다.

마지막 것만 안 됩니다. 이유는 성질이 서로 부딪히기 때문입니다. 인라인은 "줄을 안 끊는다" 는 약속인데 그 안에 "줄을 끊는" 블록이 들어오면 약속을 지킬 수가 없습니다.

그래서 브라우저는 어떻게든 그려 내려고 이렇게 합니다.

span 은 그대로 인라인으로 두고,
안의 p 는 블록이니까 줄을 끊어서 그립니다.

결과가 아래 상자입니다. 한 줄이던 것이 세 줄로 쪼개집니다.

앞글자
안에 넣은 문단
뒷글자

span 하나 안에 든 글자인데 세 줄로 흩어졌습니다. 크롬에서 재 보면 span 의 display 값은 여전히 inline 입니다. 인라인인데 높이가 107px 로 세 줄만큼 커져 버린 이상한 상태입니다.

나중에 CSS 로 이 span 에 배경색이나 테두리를 주면 모양이 크게 어긋납니다. 그때 원인을 찾기가 아주 어렵습니다. 지금 규칙으로 익혀 두세요.

인라인 안에는 인라인만 넣습니다.

화면 "앞글자", "안에 넣은 문단", "뒷글자" 가 세 줄로 쪼개져 보입니다.

한 줄로 이어질 것 같은데 그렇지 않습니다

앞글자 안에 넣은 문단 뒷글자

이렇게 쓰세요

<p>앞글자 가운데 글자 뒷글자</p>

▹ 직접 해보기 5

위 첫 상자의 <p id="p-in-span"> 를 로, </p> 를 로 바꿔 보세요.
세 줄이 한 줄로 합쳐집니다. 확인했으면 되돌려 놓으세요.

자주 하는 실수

섹션 6

블록과 인라인을 헛갈리면 화면이 미묘하게 어긋납니다. 에러는 나지 않으니 눈으로 알아채야 합니다.

이런 실수를 합니다

span 으로 줄을 바꾸려 함

이런 실수를 합니다

실수 1 — span 을 나누면 줄이 바뀔 줄 알았음

```
<span>첫 줄</span>
<span>둘째 줄</span>
```

소스에서 줄을 나눠 썼으니 화면에서도 나뉠 것 같습니다. 안 나뉩니다. "첫 줄 둘째 줄" 이 한 줄로 붙어 나옵니다. span 이 인라인이라 줄을 끊지 않고, 소스의 엔터는 개념02 에서 배운 대로 공백 한 칸으로 접하기 때문입니다. span 을 줄 나눠 쓴 실제 결과 첫 줄 둘째 줄

이런 실수를 합니다

블록을 옆에 붙이려 함

이런 실수를 합니다

실수 2 — 문단 두 개를 옆에 나란히 놓으려 함

```
<p>왼쪽</p><p>오른쪽</p>
```

소스에서 아무리 붙여 써도 두 줄로 나옵니다. p 가 블록이라 각자 한 줄씩 통째로 차지하기 때문입니다. 옆에 나란히 놓는 방법은 11단원과 12단원에서 CSS 로 배웁니다. HTML 만으로는 방법이 없습니다.

이런 실수를 합니다

뜻이 있는데 span 을 씀

이런 실수를 합니다

실수 3 — 뜻이 있는데도 span 으로 감쌌

```
<p>마감은
<span class="important">오늘</span>입니다</p>
```

중요하다는 뜻이 있으니 이 맞습니다. span 으로 감싸고 CSS 로 굵게 만들면 화면은 똑같이 나옵니다. 하지만 화면 낭독기에게는 아무 강조도 전달되지 않습니다. 개념03 섹션 3 에서 본 것과 똑같은 문제입니다.

이런 실수를 합니다

모든 글자를 span 으로

이런 실수를 합니다

실수 4 — 모든 글자를 span 으로 감쌌

```
<span>안</span><span>녕</span>
```

화면은 안녕 그대로입니다. 아무 문제 없어 보입니다. 하지만 손잡이가 필요 없는 곳에 손잡이를 단 것이라 소스만 길어지고 읽기 어려워집니다. span 은 꾸밀 계획이 있는 곳에만 붙이세요.

이런 실수를 합니다

인라인에 폭을 주려 함

이런 실수를 합니다

실수 5 — 인라인 요소에 가로폭을 주려 함

```
span { width: 300px; }
```

10단원에서 width 를 배우고 나면 꼭 한 번 해 보게 되는 일입니다. 아무 일도 일어나지 않습니다. 에러도 안 납니다. 인라인 요소는 글자만큼만 차지하기로 되어 있어서 폭 지시를 무시합니다. 해결법은 11단원의 display 에서 배웁니다. (위 코드는 CSS 입니다. 07단원부터 배우니 지금은 눈으로만 보세요)

정리

▹ 직접 해보기 6

정답

1) 섹션 1 상자의 첫 줄을 이렇게 바꿉니다.

```
<p>가격은 12,000원 입니다.</p>
```

→ 화면이 하나도 바뀌지 않습니다. 두 줄이 여전히 똑같습니다.

span 은 화면을 바꾸는 태그가 아니기 때문입니다.
확인했으면 span 을 다시 붙여 놓으세요.

2) 섹션 2 상자를 이렇게 한 줄로 붙여 씁니다.

```
<p id="blk-a">블록 하나</p><p id="blk-b">블록 둘</p><p id="blk-c">블록 셋</p>
```

→ 화면은 그대로 세 줄입니다.

소스에서 붙여 쓰는 것과 화면에서 붙는 것은 아무 상관이 없습니다.
블록 요소는 한 줄을 통째로 차지하기 때문입니다.

3) 섹션 3 상자를 이렇게 바꿉니다.

```
<p id="inline-row"><span id="inl-a">인라인 하나</span> <span id="inl-b">인라인 둘</span> <span id="inl-c">인라인 셋</span></p>
```

→ “인라인 하나 인라인 둘 인라인 셋” 처럼 사이가 한 칸씩 띄어집니다.

줄은 여전히 하나입니다. 인라인은 줄을 끊지 않기 때문입니다.

4) 개념03 파일 섹션 4 의 mark 데모를 보면 노란 칠이 “김치찌개” 네 글자 뒤에서 끝납니다.

→ mark 가 인라인이라 글자 길이만큼만 차지하기 때문입니다.

만약 mark 가 블록이었다면 노란 칠이 줄 끝까지 이어졌을 것입니다.

5) 섹션 5 첫 상자를 이렇게 바꿉니다.

```
<span id="span-with-p">앞글자 <span id="p-in-span">안에 넣은 문단</span> 뒷글자</span>
```

→ 세 줄이던 것이 “앞글자 안에 넣은 문단 뒷글자” 한 줄로 합쳐집니다.

인라인 안에 인라인만 들어가서 흐름이 끊기지 않기 때문입니다.
확인했으면 되돌려 놓으세요.

특수문자

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 특수문자

01단원 개념02 의 마지막에 이런 실수를 봤습니다.

```
p>안녕하세요</p>
```

여는 꺾쇠를 빠뜨렸더니 태그가 안 되고 글자가 그대로 화면에 나왔습니다. “코드가 화면에 글자로 보이면 꺾쇠가 빠진 것” 이라고 배웠습니다.

이번에는 정반대 사고를 다룹니다.

글자로 쓰고 싶었는데 태그로 먹혀 버리는 경우

예를 들어 HTML 을 설명하는 글을 쓴다고 합시다. “p 태그는 <p>처럼 생겼습니다” 라고 쓰고 싶습니다. 그대로 쓰면 브라우저는 가운데 <p> 를 진짜 태그로 읽어 버립니다. 글자로 보여 주려던 것이 명령으로 실행되는 것입니다.

브라우저는 꺾쇠를 보면 무조건 “태그가 시작되나 보다” 하고 생각합니다. 꺾쇠에게 그런 일을 시켜 놓았으니 어쩔 수 없습니다.

그래서 HTML 에는 이런 글자들을 위한 대체 표기가 준비되어 있습니다.

```
&amplt   → 화면에 < 로 보입니다
&ampgt    → 화면에 > 로 보입니다
&amp; → 화면에 & 로 보입니다
&quot; → 화면에 " 로 보입니다
```

이것을 특수문자 표기라고 부릅니다. 정확한 이름은 문자 참조(character reference) 인데, “글자를 직접 쓰지 않고 이름으로 가리킨다” 는 뜻입니다.

생김새는 셋으로 정해져 있습니다.

```
& amp ;
| |   ↳ 세미콜론으로 끝냅니다
|   ↳ 글자의 이름
↳ 앰퍼샌드로 시작합니다
```

이 파일에서는 왜 필요한지, 무엇을 알아야 하는지, 그리고 공백을 남기는 ` ` 와 저작권 기호 같은 것들을 배웁니다.

이 파일은 글자로 쓰고 싶은데 태그로 먹혀 버리는 문제를 해결합니다. `<` `>` `&` `"` ` ` `©` 를 배우고, 이모지를 넣는 법도 함께 봅니다.

왜 필요한가

섹션 1

브라우저는 `<` 를 보는 순간 태그가 시작된다고 판단합니다. 글자로 쓴 것인지 아닌지는 구분해 주지 않습니다.

아래 첫 번째 상자 안에는 이렇게 썼습니다. 꺾쇠를 그대로 쳤습니다.

```
<p id="raw-demo">p 태그는 <p> 처럼 생겼습니다</p>
```

의도는 “p 태그는 `<p>` 처럼 생겼습니다” 라는 한 문장이었습니다. 하지만 브라우저는 가운데 `<p>` 를 진짜 태그로 읽습니다.

개념02 에서 배웠듯이 p 안에 p 가 들어오면 바깥 p 가 강제로 닫힙니다. 그래서 실제로는 이런 일이 벌어집니다.

```
문단 1  "p 태그는"
문단 2  "처럼 생겼습니다"
```

한 문장이 두 문단으로 쪼개졌고, 보여 주려던 `<p>` 는 사라졌습니다. 에러는 물론 나지 않습니다.

두 번째 상자에는 꺾쇠를 특수문자로 바꿔서 이렇게 썼습니다.

```
<p id="esc-demo">p 태그는 &lt;p&gt; 처럼 생겼습니다</p>
```

이제 한 문단으로 제대로 나오고 <p> 도 글자로 보입니다.

중요 · 이 자료의 검은 코드 상자들이 전부 이 방법으로 만들어져 있습니다.

VS Code 에서 이 파일의 doc-code 를 열어 보면
모든 꺾쇠가 < 와 > 로 적혀 있는 것을 볼 수 있습니다.

화면 "p 태그는" 과 "처럼 생겼습니다" 가 두 줄로 쪼개져 보입니다.

보여 주려던 꺾쇠 표기는 어디에도 없습니다

p 태그는 처럼 생겼습니다

화면 "p 태그는 <p> 처럼 생겼습니다" 가 한 줄로 제대로 보입니다

p 태그는 <p> 처럼 생겼습니다

01단원 개념02 의 실수 4 는 꺾쇠를 빠뜨려서 코드가 글자로 보인 경우였고, 여기는 꺾쇠를 그대로 써서 글자가 코드로 먹힌 경우입니다. 같은 꺾쇠 때문에 생기는 정반대 사고입니다.

▫ 직접 해보기

1

두 번째 상자의 <p> 를 으로 바꿔 보세요. 저장하고(Ctrl+S) 새로고침(F5) 하면
화면에 다른 태그 이름이 글자로 보입니다.

꼭 알아야 할 네 가지

섹션 2

수십 개가 있지만 이 넷만 외우면 됩니다. 나머지는 필요할 때 찾아 쓰세요.

표기와 결과를 나란히 놓으면 이렇습니다.

표기	화면	이름의 유래	언제 쓰나
<	<	less than (작다)	태그를 글자로 보여 줄 때
>	>	greater than (크다)	위와 같음
&	&	ampersand (앰퍼샌드)	회사 이름, 조건식 등
"	"	quotation mark (따옴표)	속성 값 안에 따옴표를 쓸 때

— < 와 >

이 둘은 짝으로 외우세요. l 은 left(왼쪽), g 는 great 로 기억해도 됩니다. 실무에서는 < 를 훨씬 자주 씁니다. 여는 꺾쇠만 조심하면 되기 때문입니다.

— &

앰퍼샌드는 특수문자 표기의 시작 신호입니다. 그러니 앰퍼샌드 자신을 글자로 쓰려면 표기가 필요합니다.

& 는 “여기 앰퍼샌드는 시작 신호가 아니라 그냥 글자다” 라는 표시입니다.

여기서 재미있는 일이 하나 있습니다. 화면에 < 라는 글자를 그대로 보여 주고 싶으면 어떻게 할까요.

&lt; → 화면에 < 로 보입니다

& 가 & 가 되고, 뒤의 lt; 는 그냥 글자라서 그대로 남습니다. 이 자료가 “이렇게 씁니다” 하고 표기 자체를 보여 줄 때 쓰는 방법입니다.

— "

속성 값을 감싸는 큰따옴표 안에 큰따옴표를 또 쓰면 값이 거기서 끊깁니다. 그럴 때 " 를 씁니다.

속성은 03단원의 colspan, 04단원의 href 처럼 뒤로 갈수록 자주 쓰게 됩니다. 지금은 문장 안의 따옴표를 안전하게 쓰는 용도만 익혀 두세요.

— 알아 두면 편한 것

브라우저는 앰퍼샌드가 혼자 있으면 대체로 그냥 글자로 봐 줍니다. “Tom & Jerry” 도 그대로 나옵니다. 하지만 항상 그렇지는 않습니다. 섹션 5 의 실수 1 에서 실제로 깨지는 경우를 봅니다. 그래서 앰퍼샌드는 습관적으로 & 로 쓰는 편이 안전합니다.

화면 네 줄이 차례로 보입니다.

- 1 “조건은 1 < 2 이고 3 > 2 입니다.”
- 2 “Tom & Jerry”
- 3 그는 “안녕” 이라고 했습니다
- 4 “< 라고 쓰면 꺾쇠가 글자로 보입니다”

조건은 1 < 2 이고 3 > 2 입니다.
Tom & Jerry
그는 “안녕” 이라고 했습니다
< 라고 쓰면 꺾쇠가 글자로 보입니다

⇒ 직접 해보기 2

위 상자에 <h1>제목</h1> 을 담은 문단을 한 줄 넣어 보세요. 화면에 h1 태그가 글자 그대로 보이면 성공입니다.

nbsp 는 non-breaking space 의 줄임말입니다. 우리말로 옮기면 “끊어지지 않는 공백” 입니다.

성질이 두 가지입니다.

- [1] 여러 개를 써도 접하지 않습니다.
보통 스페이스는 네 칸을 쳐도 한 칸이 되지만
 는 네 개를 쓰면 네 칸이 그대로 남습니다.
- [2] 그 자리에서 줄이 바뀌지 않습니다.
"홍 길동" 이라고 쓰면 오른쪽 끝에 닿아도
"홍" 과 "길동" 이 위아래로 갈라지지 않습니다.
이름이나 "10 kg" 같은 단위에서 쓸모가 있습니다.

가 나
가 나

 네 개 → 23px
스페이스 네 칸 → 6px

네 배 가까이 차이가 납니다. 아래쪽은 한 칸으로 접혀 버린 것입니다.

이 성질 때문에 를 들여쓰기나 간격 조절에 쓰는 사람이 아주 많습니다. 개념02의 br 남용과 정확히 같은 잘못입니다.

이렇게 쓰면 들여쓰 것처럼 보이기는 합니다. 하지만 “여기 공백이 네 칸 있다” 라는 뜻일 뿐 들여쓰기라는 뜻이 아닙니다. 들여쓰기는 10단원의 padding 이나 margin 으로 합니다.

홍 길동	이름이 갈라지면 안 될 때
10 kg	숫자와 단위가 갈라지면 안 될 때
2026. 8. 11	날짜가 갈라지면 안 될 때

화면 위 줄은 "가" 와 "나" 사이가 눈에 띄게 벌어져 있고,

아래 줄은 한 칸만 띄어져 있습니다

가	나
가	나

이런 실수를 합니다

잘못된 예

[illegible]

들여쓰기처럼 보이지만 뜻은 “공백 네 칸” 입니다. 들여쓰기라는 뜻이 아닙니다.

이렇게 쓰세요

<p>홍 길동 님, 10 kg 짐이 도착했습니다</p>

➤ 직접 해보기 3

위 상자 아래 줄의 스페이스 네 칸을 네 개로 바꿔 보세요. 두 줄의 간격이 같아집니다. 확인했으면 되돌려 놓으세요.

기호와 이모지

섹션 4

저작권 기호, 화살표, 수학 기호처럼 자판에 없는 글자도 같은 방법으로 씁니다.

자주 쓰는 것만 모으면 이 정도입니다.

표기	화면	쓰는 곳
©	©	저작권 표시
®	®	등록 상표
™	™	상표
·	·	가운뎃점
…	...	말줄임표
→	→	오른쪽 화살표
←	←	왼쪽 화살표
☆	☆	별표
×	×	곱하기
÷	÷	나누기
°	°	도(온도, 각도)
±	±	플러스마이너스
“	“	여는 곡선 큰따옴표
”	”	닫는 곡선 큰따옴표
—	—	긴 줄표
•	•	가운데 점

이름을 전부 외울 필요는 없습니다. 필요할 때 찾아 쓰면 됩니다. 다만 © 정도는 페이지 맨 아래 저작권 줄에 늘 쓰게 되니 외워 두면 편합니다.

— 숫자로 쓰는 방법

이름 대신 번호로도 쓸 수 있습니다.

```
&copy; 와 &#169; 는 똑같습니다
&hearts; 와 &#9829; 는 똑같습니다
```

모든 글자에는 세계 공통 번호가 붙어 있습니다. 이름이 없는 글자는 번호로 씁니다. 이름이 있으면 이름 쪽이 읽기 좋습니다.

— 이모지는 그냥 붙여넣으세요

이모지도 특수문자 표기가 필요할 것 같지만 그렇지 않습니다.

```
<p>🍏 사과를 좋아합니다</p>
```

이렇게 그냥 붙여넣으면 그대로 나옵니다. 01단원에서 배운 이 한 줄 덕분입니다.

```
<meta charset="UTF-8" />
```

charset 은 “이 파일의 글자를 어떤 규칙으로 읽어라” 를 알려 주는 표시입니다. UTF-8 은 세상의 거의 모든 글자를 담는 규칙이라 한글도 이모지도 다 들어 있습니다. 이 줄이 없으면 한글이 깨지고 이모지도 깨집니다.

주의 · 이모지를 뜻 전달에 기대지 마세요.

화면 낭독기는 이모지를 이름으로 읽습니다.
🍏 는 "붉은 사과" 라고 읽힙니다. 장식으로 여러 개 늘어놓으면
그 이름이 전부 소리로 나와서 듣기 괴로워집니다.

화면 다섯 줄이 차례로 보입니다.

- 1 저작권 기호, 등록상표, 상표, 오른쪽 화살표, 별, 가운데점, 말줄임표
- 2 곱하기, 나누기, 플러스마이너스, 도 기호, 그리고 30도씨
- 3 "© 와 ® 는 같은 글자입니다"
- 4 사과와 달과 별 이모지가 그림으로 보입니다
- 5 페이지 맨 아래에 흔히 넣는 저작권 한 줄

© ® ™ → ☆ · …
× ÷ ± ° 30°C
© 와 ® 는 같은 글자입니다
🍏 🌙 ☆ 이모지는 그냥 붙여넣으면 됩니다
© 2026 우리 동네 빵집 · 모든 권리 보유

▫ 직접 해보기 4
위 상자에 여러분 이름으로 저작권 줄을 한 줄 만들어 보세요. © 와 · 를 써 보세요.

자주 하는 실수 섹션 5

특수문자 실수는 글자가 엉뚱하게 보이는 것으로 나타납니다. 여기서도 예러는 나지 않습니다.

이런 실수를 합니다
앰퍼샌드를 날것으로 씀

이런 실수를 합니다
실수 1 — 앰퍼샌드를 그대로 씀

<p>AT© 라는 회사</p>

회사 이름을 그대로 쳤을 뿐인데 브라우저는 © 를 저작권 기호로 읽어 버립니다. 화면에 “AT© 라는 회사” 가 나옵니다. 아래가 실제 결과입니다. 앰퍼샌드 뒤에 우연히 특수문자 이름이 오면 이런 일이 생깁니다. 앰퍼샌드를 그대로 쓴 실제 결과 AT© 라는 회사 해결은 간단합니다. 앰퍼샌드를 & 로 쓰면 됩니다.

이런 실수를 합니다

세미콜론을 빠뜨림

이런 실수를 합니다

실수 2 — 세미콜론을 빠뜨림

 < ©

세미콜론 ; 은 “이름이 여기서 끝난다” 는 표시입니다. 빠뜨리면 브라우저가 어디까지가 이름인지 몰라서 뒤에 오는 글자까지 이름으로 삼켜 버릴 수 있습니다. 실수 1 의 AT© 라는 이 정확히 그 경우입니다. 세미콜론은 항상 붙이세요.

이런 실수를 합니다

철자를 틀림

이런 실수를 합니다

실수 3 — 이름의 철자를 틀림

 ps; &NBSP;

nbsp 를 nbps 로 치거나 대문자로 치는 실수입니다. 그런 이름은 없으니 브라우저는 쓴 그대로 화면에 보여줍니다. 아래가 실제 결과입니다. 이름은 대소문자를 가립니다. “이상한 글자가 화면에 보인다” 면 철자부터 확인하세요. 철자를 틀린 실제 결과 ps; 철자를 틀렸습니다 &NBSP; 대문자로 썼습니다

이런 실수를 합니다

nbsp로 여백을 만듦

이런 실수를 합니다

실수 4 — 를 늘어놓아 여백을 만듦

<p>제 목 &
 &내 용</p>

간격이 벌어지기는 합니다. 하지만 뜻은 “공백 여섯 칸” 일 뿐입니다. 글자 크기를 바꾸면 간격도 같이 흔들립니다. 개념02 의
 남용과 똑같은 문제입니다. 간격은 10단원의 CSS 로 조절하세요.

이런 실수를 합니다

이미 바꾼 것을 또 바꿈

이런 실수를 합니다

실수 5 — 이미 바꾼 것을 한 번 더 바꿈

```
&lt;p&gt;
```

< 라고 쓰려다가 앰퍼샌드까지 한 번 더 바꿔 버린 경우입니다. 화면에는 <p> 라는 글자가 그대로 나옵니다. 꺾쇠가 아니라 표기 자체가 보이는 것입니다. 아래가 실제 결과입니다. 이 자료가 표기를 설명할 때 일부를 쓰는 방법이기도 합니다. 두 번 바꾼 실제 결과 <p>

이런 실수를 합니다

charset 을 빠뜨림

이런 실수를 합니다

실수 6 — meta charset 을 빠뜨리고 이모지를 씀

```
<head>
<title>내 페이지</title>
</head>
```

<meta charset="UTF-8" /> 가 없습니다. 이러면 한글과 이모지가 알아볼 수 없는 글자로 깨져서 나올 수 있습니다. 01단원에서 이 줄을 왜 넣는지 배웠습니다. 여기서 그 이유가 눈에 보입니다. 이 파일에도 맨 위에 그 줄이 들어 있습니다.

정리

▢ 직접 해보기 5

정답

1) 섹션 1 의 두 번째 상자를 이렇게 바꿉니다.

```
<p id="esc-demo">p 태그는 &lt;strong&gt; 처럼 생겼습니다</p>
```

→ 화면에 “p 태그는 처럼 생겼습니다” 가 한 줄로 보입니다.

꺾여지지 않습니다. 태그가 아니라 글자이기 때문입니다.

2) 섹션 2 상자의 마지막 p 아래에 이렇게 넣습니다.

```
<p>&lt;h1&gt;제목&lt;/h1&gt;</p>
```

→ 화면에 “<h1>제목</h1>” 이 글자 그대로 보입니다.

글자가 커지지 않는 것으로 태그가 아님을 확인할 수 있습니다.

3) 섹션 3 상자의 아래 줄을 이렇게 바꿉니다.

[illegible]

→ 아래 줄의 간격이 위 줄과 똑같이 벌어집니다.

폭을 재면 6px 이던 것이 23px 가 됩니다.
확인했으면 스페이스 네 칸으로 되돌려 놓으세요.

4) 섹션 4 상자의 마지막 p 아래에 이렇게 넣습니다.

<p>© 2026 김민준 · 개인 연습용</p>

→ "© 2026 김민준 · 개인 연습용" 이 한 줄 보입니다.

저작권 기호와 가운뎃점이 자판에 없는데도 나옵니다.

텍스트 태그

푸는 방법

1. 이 파일을 VS Code 와 브라우저에 동시에 열어 두세요.
2. 각 문제의 점선 상자 안에 있는 TODO 자리에 코드를 씁니다.
3. 저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 합니다.
4. 소스에 적힌 기대 화면과 같은지 눈으로 확인합니다.

실행하면 나오는 화면 — 텍스트 태그

문제 1~11 은 기본, 12~14 는 응용, 15 는 도전, 16 은 확인 문항입니다. 쓸 수 있는 태그는 01단원과 02단원에서 배운 것뿐입니다. 목록(`ul`, `ol`), 표(`table`), 링크(`a`), 이미지(`img`) 는 아직 배우지 않았습니다.

답이 막히면 개념 파일의 해당 섹션을 다시 열어 보세요. 문제마다 어느 개념 파일 어느 섹션에서 나온 것인지 적어 두었습니다.

각 문제의 점선 상자 안에 코드를 쓰세요. 상자 안에 적어 둔 TODO 자리가 여러분이 쓸 곳입니다. 정답을 확인하려면 같은 폴더의 연습문제_정답.html 을 여세요.

문제 1 (개념01 섹션1)

문제 1

제목 태그 h2 로 우리 동네 빵집 이라는 제목을 만드세요.

기대 출력 굵고 큰 글씨로 "우리 동네 빵집" 이 한 줄 보입니다.
상자 안의 다른 글자보다 확실히 큼니다.
이렇게 되면 틀린 것: 글씨 크기가 그대로면 h2 가 아니라 p 를 쓴 것입니다.

여기에 코드를 쓰세요

문제 2 (개념01 섹션2)

문제 2

h2 로 메뉴 를 쓰고, 그 아래에 h3 로 빵 과 음료 를 차례로 쓰세요.

기대 출력 세 줄이 보입니다. "메뉴" 가 가장 크고,
"빵" 과 "음료" 는 서로 같은 크기이면서 "메뉴" 보다 작습니다.
이렇게 되면 틀린 것: 세 줄 크기가 모두 같으면 세 줄에 같은 태그를 쓴
것입니다.
"빵" 과 "음료" 크기가 서로 다르면 h3 과 h4 를 섞어 쓴 것입니다.

여기에 코드를 쓰세요

문제 3 (개념02 섹션1)

문제 3

문단 태그 `p` 로 두 문단을 만드세요. 내용은 아침 7시에 문을 엽니다. 와 저녁 9시에 문을 닫습니다. 입니다.

기대 출력 두 줄이 보이고, 두 줄 사이가 눈에 띄게 벌어져 있습니다.
이렇게 되면 틀린 것: 두 줄이 딱 붙어 있으면 `p` 를 두 개 쓴 것이 아니라
한 문단 안에서 `br` 로 줄만 바꾼 것입니다.

여기에 코드를 쓰세요

문제 4 (개념02 섹션3)

문제 4

문단 하나 안에서 주소를 두 줄로 나누세요. 내용은 서울시 강남구 테헤란로 123 과 2층 201호 입니다.

기대 출력 두 줄로 보이는데 두 줄 사이가 벌어지지 않고 바로 붙어 있습니다.
이렇게 되면 틀린 것: 두 줄 사이가 벌어지면 `p` 를 두 개 쓴 것입니다.
한 줄로 이어져 나오면 소스에서 엔터만 친 것입니다.
엔터는 공백 한 칸으로 접힙니다.

여기에 코드를 쓰세요

문제 5 (개념02 섹션5)

문제 5

문단 오전 이야기입니다. 와 오후 이야기입니다. 를 쓰고, 그 사이에 가로줄을 하나 그으세요.

기대 출력 두 문단 사이에 상자 가로폭만큼 긴 회색 가로줄이 하나 보입니다.
이렇게 되면 틀린 것: 줄이 안 보이면 hr 철자나 꺾쇠를 확인하세요.
hr 은 빈 태그라 닫는 태그를 쓰지 않습니다.

여기에 코드를 쓰세요

문제 6 (개념03 섹션1)

문제 6

모임은 3시입니다. 라는 문단을 만들고, 3시 두 글자만 중요하다고 표시하세요.

기대 출력 한 줄이 보이고 그중 "3시" 두 글자만 굵습니다.
이렇게 되면 틀린 것: 문장 전체가 굵으면 strong 이 문장 전체를 감싼
것입니다.
굵어지지 않으면 태그 이름 철자를 확인하세요.

여기에 코드를 쓰세요

문제 7 (개념03 섹션2)

문제 7

나는 오늘 간다. 라는 문단을 만들고, 오늘 두 글자를 힘주어 읽으라고 표시하세요.

기대 출력 한 줄이 보이고 그중 "오늘" 두 글자만 오른쪽으로 비스듬히 기울어집니다.
 굵어지지 않습니다.
 이렇게 되면 틀린 것: 굵어졌으면 em 이 아니라 strong 을 쓴 것입니다.

여기에 코드를 쓰세요

문제 8 (개념03 섹션4)

문제 8

오늘의 메뉴는 김치찌개 입니다 (1인분 기준) 라는 문단을 만들되, 김치찌개 에는 형광펜을, (1인분 기준) 에는 결다리 말 표시를 하세요.

기대 출력 한 줄이 보입니다. "김치찌개" 네 글자 뒤에만 노란 칠이 되어 있고,
 "(1인분 기준)" 은 나머지 글자보다 작습니다.
 이렇게 되면 틀린 것: 노란 칠이 줄 끝까지 이어지면 mark 가 너무 넓게 감싼
 것입니다.
 "(1인분 기준)" 이 안 작아지면 small 을 안 쓴 것입니다.

여기에 코드를 쓰세요

문제 9 (개념03 섹션5)

문제 9

가격이 30,000원 에서 19,000원 으로 바뀌었습니다. 지운 값과 새로 넣은 값을 각각 표시해 한 문단으로 만드세요.

기대 출력 한 줄이 보입니다. "30,000원" 글자 가운데를 가로지르는 줄이 그어져 있고, "19,000원" 아래에는 밑줄이 그어져 있습니다.
이렇게 되면 틀린 것: 두 값 모두 가운데 줄이면 ins 자리에 del 을 쓴 것입니다.

여기에 코드를 쓰세요

문제 10 (개념04 섹션1)

문제 10

가격은 12,000원 입니다. 라는 문단을 만들고, 12,000원 만 span 으로 감싸세요. 나중에 CSS 로 그 부분만 꾸미려고 손잡이를 다는 것입니다.

기대 출력 한 줄이 보입니다. span 을 넣기 전과 후의 화면이 완전히 같습니다.
굵어지지도 색이 바뀌지도 줄이 바뀌지도 않습니다.
이렇게 되면 틀린 것: 줄이 두 줄로 나뉘면 span 이 아니라 p 나 div 를 쓴 것입니다.
span 은 인라인이라 줄을 끊지 않습니다.

여기에 코드를 쓰세요

문제 11 (개념05 섹션1, 섹션2)

문제 11

화면에 p 태그는 <p> 라고 씁니다 라는 한 줄이 보이도록 문단을 만드세요. 꺾쇠가 글자로 보여야 합니다.

기대 출력 한 줄로 "p 태그는 <p> 라고 씁니다" 가 보입니다.
 꺾쇠 안의 p 가 글자로 그대로 보입니다.
 이렇게 되면 틀린 것: 줄이 두 개로 쪼개지고 꺾쇠가 사라지면
 꺾쇠를 그대로 쓴 것입니다. 브라우저가 태그로 읽어 버립니다.

여기에 코드를 쓰세요

문제 12 [응용] (개념03 섹션6)

문제 12 [응용]

물은 H₂O 이고, 3제곱수 3² 는 9입니다. 라는 문단을 만들되, H₂O 의 2 는 아래첨자로, 3² 의 뒤쪽 2 는 위첨자로 만드세요. 화면에서 물 분자식과 거듭제곱이 제대로 보이면 됩니다.

기대 출력 한 줄이 보입니다.
 "H₂O" 의 2 가 H 와 O 사이 아래쪽에 작게 붙어 있고,
 "3²" 의 뒤쪽 2 는 3 의 오른쪽 위에 작게 붙어 있습니다.
 두 숫자 모두 나머지 글자보다 작습니다.
 이렇게 되면 틀린 것: 두 숫자가 다른 글자와 같은 높이에 있으면
 sub 와 sup 을 안 쓴 것입니다.
 위아래가 반대면 sub 와 sup 을 바꿔 쓴 것입니다.

여기에 코드를 쓰세요

문제 13 [응용] (개념05 섹션3, 섹션4)

문제 13 [응용]

페이지 맨 아래에 넣을 저작권 한 줄을 만드세요. 화면에 © 2026 우리 동네 빵집 · 대표 홍길동 이 보이면 됩니다. 홍 과 길동 사이는 줄이 갈라지지 않는 공백으로 쓰세요.

기대 출력

한 줄로 "© 2026 우리 동네 빵집 · 대표 홍길동" 이 보입니다.
저작권 기호와 가운뎃점이 자판에 없는데도 제대로 나옵니다.
이렇게 되면 틀린 것: 화면에 © 처럼 앰퍼샌드부터 세미콜론까지 글자로 보이면
이름의 철자가 틀린 것입니다. 이름은 대소문자도 가립니다.

여기에 코드를 쓰세요

문제 14 [응용] (개념01 섹션2, 개념02 섹션1, 개념03 섹션1)

문제 14 [응용]

간단한 요리법 한 덩어리를 만드세요. 필요한 것은 이렇습니다. h2 로 김치찌개 만들기, 그 아래 h3 로 재료 와 순서, 각 h3 아래에 설명 문단 하나씩, 그리고 재료 설명 안의 김치 한 컵 을 중요 표시하세요.

기대 출력

다섯 줄이 보입니다. (제목 3줄 + 문단 2줄)
"김치찌개 만들기" 가 가장 크고,
"재료" 와 "순서" 가 같은 크기로 그보다 작습니다.
각 소제목 아래에 보통 크기 문단이 한 줄씩 있고,
첫 문단 안의 "김치 한 컵" 만 굵습니다.
이렇게 되면 틀린 것: "재료" 와 "순서" 가 "김치찌개 만들기" 와 같은 크기이면
둘 다 h2 로 쓴 것입니다. 단계를 한 칸 내려야 합니다.

여기에 코드를 쓰세요

문제 15 [도전] (개념01~05 종합)

문제 15 [도전]

상품 소개 한 덩어리를 만드세요. 아래 여섯 가지를 모두 넣어야 합니다.

- 1) h2 로 상품명 수제 단팥빵 · 2) 설명 문단 하나, 그 안에 당일 생산 을 중요 표시 · 3) 값이 3,000원 에서 2,500원 으로 바뀐 표시 · 4) (3개 이상 구매 시) 를 겹다리 말로 · 5) 가로줄 하나 · 6) 마지막 줄에 © 2026 우리 동네 빵집 을 겹다리 말로

기대 출력

위에서부터
"수제 단팥빵" 이 크고 굵게 한 줄,
설명 문단 한 줄에서 "당일 생산" 만 굵고,
가격 줄에서 "3,000원" 에 가운데 줄, "2,500원" 에 밑줄,
"(3개 이상 구매 시)" 가 작게,
회색 가로줄 하나,
맨 아래 "© 2026 우리 동네 빵집" 이 작게 보입니다.

글자는 모두 네 줄이고 가로줄이 하나입니다.
"(3개 이상 구매 시)" 는 새 줄이 아니라 가격과 같은 줄에 이어집니다.

이렇게 되면 틀린 것: 상품명이 다른 글자와 크기가 같으면 h2 가 아니라 p 나

strong 을 쓴 것입니다. 제목은 h 태그로 씁니다.

여기에 코드를 쓰세요

문제 16 [확인] (개념02 섹션6, 개념03 섹션8, 개념05 섹션5)

문제 16 [확인]

아래 상자의 코드는 에러가 하나도 안 납니다. 그런데 만든 사람이 원한 화면과 다르게 나옵니다. 잘못된 곳 세 군데를 찾아 고치세요.

만든 사람이 원한 화면은 이랬습니다. 1) 오후 3시 가 굵게 보인다 · 2) 회사 이름이 AT© 라는 회사 그대로 보인다 · 3) 마지막 줄이 한 줄로 이어져 보인다

아래 상자 안의 코드는 이렇게 되어 있습니다.

```
<p>모임은 <strng>오후 3시</strng>입니다.</p>
<p>회사 이름은 AT&copy 라는 회사 입니다.</p>
<p>자세한 내용은 <p>공지 게시판</p> 을 보세요.</p>
```

세 줄에 각각 잘못이 하나씩 있습니다. 찾을 때는 “무슨 일이 일어났는가” 를 먼저 말로 설명해 보세요.

첫째 줄: 굵어져야 할 글자가 왜 안 굵어졌나?
둘째 줄: 회사 이름 가운데 글자가 왜 사라지고 기호가 생겼나?
셋째 줄: 한 줄이어야 할 것이 왜 세 줄로 쪼개졌나?

기대 출력 고치고 나면 세 줄이 보입니다.
첫 줄의 "오후 3시" 만 굵고,
둘째 줄에 "AT© 라는 회사" 가 글자 그대로 보이고,
셋째 줄이 "자세한 내용은 공지 게시판 을 보세요." 한 줄로 이어집니다.
이렇게 되면 틀린 것: 둘째 줄에 저작권 기호가 여전히 보이면 앰퍼샌드를
아직 안 고친 것입니다.

모임은 오후 3시입니다.
회사 이름은 AT© 라는 회사 입니다.
자세한 내용은 공지 게시판 을 보세요.

02단원 마무리 텍스트 태그

자주 넘어지는 자리

1 h4 는 문단과 크기가 같고 h6 은 더 작다 — “제목 = 큰 글씨” 오해를 여기서 깨세요

목록과 표

OPEN 03_목록과_표

정상 항목입니다
저는 p 입니다
또 정상 항목입니다
사과 한 개 1,200원

01 순서없는 목록

02 순서있는 목록

03 설명 목록

04 표 만들기

05 칸 합치기

- 연습문제

순서없는 목록



실행하면 나오는 화면 — 순서없는 목록

03단원 · 개념 01 — 순서 없는 목록

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

02단원에서 여러 줄을 쓰는 법을 배웠습니다. p 를 여러 개 쓰면 됐습니다.

```
<p>사과</p>
<p>바나나</p>
<p>포도</p>
```

화면에는 세 줄이 나옵니다. 원하는 모양은 나왔습니다. 그런데 p 는 “문단” 이라는 뜻입니다. “이 셋은 한 묶음이다” 라는 뜻이 어디에도 없습니다. 사람 눈에는 목록으로 보이지만, 브라우저에게는 그냥 문단 세 개입니다.

01단원에서 “HTML 은 글에 이름표를 붙이는 일” 이라고 했습니다. 목록에는 목록용 이름표가 따로 있습니다. 그것이 이번 파일에서 배울 두 태그입니다.

ul	목록 전체를 감싸는 상자	(unordered list, 순서 없는 목록)
li	항목 하나	(list item, 목록 항목)

두 태그는 항상 짝으로 씁니다. 혼자서는 쓰지 않습니다.

```
<ul>
  <li>사과</li>
  <li>바나나</li>
</ul>
```

| |
| ↳ 항목 하나가 li 하나입니다. 항목이 세 개면 li 도 세 개입니다.
↳ 항목들을 담는 바깥 상자입니다. 목록 전체에 하나만 있습니다.

“순서 없는” 이 무슨 뜻인지 먼저 짚고 갑니다. 장바구니에 담을 물건을 적는다고 해 봅시다.

사과 · 바나나 · 포도

이 순서를 바꿔서 “포도 · 사과 · 바나나” 라고 써도 뜻이 하나도 안 달라집니다. 이렇게 순서가 뜻을 갖지 않는 목록이 순서 없는 목록입니다. 순서가 뜻을 갖는 목록(요리 순서, 순위)은 다음 파일에서 배우는 ol 을 씁니다.

이 파일에서는 점(·) 이 앞에 붙는 목록을 만듭니다. 태그는 ul 과 li 딱 두 개뿐입니다. 대신 어느 태그 안에 무엇이 들어갈 수 있는지가 이 파일의 진짜 주제입니다. 화면과 코드를 계속 번갈아 보면서, 점이 붙는 줄과 안 붙는 줄을 찾아보세요.

ul 과 li 로 목록 만들기

섹션 1

ul 로 감싸고, 그 안에 항목마다 li 를 하나씩 씁니다.

아래 상자 안에 이렇게 씁니다.

```
<ul id="fruit-list"> <li>사과</li> <li>바나나</li> <li>포도</li>
</ul>
```

li 는 ul 의 자식이므로 스페이스 2칸만큼 안으로 들여썼습니다. 01단원에서 배운 대로, 브라우저는 들여쓰기를 보지 않습니다. 사람이 부모·자식 관계를 눈으로 찾기 쉽게 하려고 들여쓰는 것입니다.

결과를 보면 두 가지가 저절로 일어납니다.

1 항목마다 앞에 까만 점(•) 이 붙습니다.

2 목록 전체가 왼쪽에서 조금 안으로 들어갑니다.

이 두 가지는 여러분이 시킨 적이 없습니다. ul 태그에 그 뜻이 들어 있어서 브라우저가 알아서 그렇게 그린 것입니다. 얼마나 들어가는지 재 보면 왼쪽 여백이 40px 입니다. 이 숫자는 10단원에서 바꾸는 법을 배웁니다.

점의 모양도 바꿀 수 있지만 그건 CSS 일입니다. 지금은 손대지 않습니다.

화면 까만 점(•) 이 앞에 붙은 세 줄이 왼쪽에서 한 칸 들어간 자리에 보입니다

사과
바나나
포도

ul 은 unordered list, li 는 list item 의 줄임말입니다. 영어 뜻을 알아 두면 ol(ordered list) 과 헷갈리지 않습니다.

▫ 직접 해보기 1

위 목록에 딸기 를 한 줄 더 넣어 보세요. 저장하고(Ctrl+S) 브라우저에서 새로고침(F5) 하면 점이 붙은 줄이 하나 늘어납니다.

ul 의 자식은 li 뿐

섹션 2

이 단원에서 제일 중요한 규칙입니다. ul 바로 안에는 li 말고 다른 것을 넣지 않습니다.

01단원에서 “감싼 쪽이 부모, 감싸인 쪽이 자식” 이라고 배웠습니다. 여기에 한 단어를 더합니다. 부모 바로 아래 한 단계에 있는 자식을 자식이라 하고, 더 깊이 들어간 것까지 전부 묶어 부를 때는 자손이라고 합니다.

```
<ul>           ← 부모
  <li>사과</li>  ← 자식
  <li>바나나    ← 자식
    <strong>맛있음</strong> ← ul 의 자식은 아니고 자손입니다
</li>
</ul>
```

규칙은 자식 자리에 대한 것입니다. ul 의 자식 자리에는 li 만 옵니다. p 도, 글자도 거기 오면 안 됩니다.

그런데 넣으면 어떻게 될까요? 01단원에서 배운 대로 에러는 안 납니다. 아래 상자 안에 일부러 이렇게 써 보았습니다.

```
<ul id="mixed-list"> <li>정상 항목입니다</li> <p>저는 p 입니다</p> <li>또 정상
항목입니다</li>
</ul>
```

결과를 보세요. 가운데 줄만 점이 없습니다. 브라우저가 “p 는 목록 항목이 아니니 점을 붙이지 않겠다” 고 판단한 것입니다. 글자는 멀쩡히 보이는데 점만 없으니, 코드를 안 보면 왜 그런지 알 수 없습니다. 이런 종류의 조용한 어긋남이 HTML 에서 제일 찾기 어렵습니다.

화면 첫 줄과 셋째 줄 앞에는 점(•) 이 붙고, 가운데 줄만 점 없이 보입니다.

세 줄 다 같은 자리에서 시작하고, 가운데 줄 위아래만 조금 더 벌어져 있습니다

```
정상 항목입니다
저는 p 입니다
또 정상 항목입니다
```

그럼 목록 항목 안에 문단을 넣고 싶을 때는 어떻게 할까요? li 안에 넣으면 됩니다. 규칙은 “ul 의 자식은 li 뿐” 이지, “li 안에는 아무것도 못 넣는다” 가 아닙니다. 섹션 3 에서 바로 해 봅니다.

▣ 직접 해보기 2

위 상자의 <p>저는 p 입니다</p> 를 저는 p 입니다 로 바꿔 보세요. 점이 생깁니다.

li 안에는 여러 가지를 넣을 수 있습니다

섹션 3

li 안은 자유롭습니다. 02단원에서 배운 태그를 그대로 쓸 수 있습니다.

li 를 “한 줄짜리 글자” 라고 생각하기 쉬운데 그렇지 않습니다. li 는 상자입니다. 그 안에 02단원에서 배운 것들을 넣을 수 있습니다.

```
<li><strong>사과</strong> 한 개 1,200원</li>
  ↳ 항목 안의 한 부분만 굵게

<li>바나나<br />두 줄로 쓸 수도 있습니다</li>
  ↳ 항목 안에서 줄바꿈
```

이렇게 넣어도 점은 항목마다 하나씩만 붙습니다. 점은 li 하나에 하나씩 붙는 것이지, 줄마다 붙는 것이 아니기 때문입니다. 아래 결과에서 “바나나” 항목이 두 줄인데 점은 하나인 것을 확인하세요.

화면 첫 줄은 “사과” 만 굵게 보입니다.

둘째 항목은 두 줄로 나뉘어 보이는데 점은 첫 줄에만 하나 붙습니다. 셋째 줄은 “제철입니다” 만 기울어져 보입니다

사과 한 개 1,200원
바나나두 줄로 쓸 수도 있습니다
포도 제철입니다

▣ 직접 해보기 3

위 목록의 “포도” 항목에서 `<em>` 을 `<strong>` 으로 바꿔 보세요. 기울어진 글씨가 굵은 글씨로 바뀝니다.

목록 안에 목록 넣기

섹션 4

큰 항목 아래에 작은 항목을 달 수 있습니다. 중요한 것은 안쪽 `ul` 이 들어가는 자리입니다.

“과일 아래에 사과와 포도를 달고 싶다” 고 해 봅시다. 01단원에서 배운 중첩(태그 안에 태그 넣기) 을 목록에도 그대로 씁니다.

여기서 딱 한 가지만 기억하면 됩니다.

안쪽 `ul` 은 그 항목의 `li` 안에 들어갑니다.

왜 그럴까요? “사과와 포도” 는 “과일” 에 딸린 내용이기 때문입니다. 딸린 내용은 딸릴 대상 안에 들어가야 합니다. 그리고 섹션 2 에서 배웠듯이 `ul` 의 자식 자리에는 `li` 만 올 수 있으니, 안쪽 `ul` 이 갈 수 있는 자리는 `li` 안밖에 없습니다.

```
<ul>
  <li>과일      ← 여기서 li 를 아직 닫지 않습니다
    <ul>        ← 안쪽 목록이 li 안으로 들어갑니다
      <li>사과</li>
      <li>포도</li>
    </ul>
  </li>        ← 안쪽 ul 이 끝난 뒤에 li 를 닫습니다
  <li>채소</li>
</ul>
```

“과일” 을 쓴 다음 바로 `</li>` 를 닫아 버리면 안쪽 `ul` 이 `li` 밖으로 나갑니다. 가장 많이 나는 실수라서 섹션 5 에서 실제 결과까지 같이 봅니다.

결과를 보면 점 모양이 바뀌어 있습니다. 이것도 브라우저가 알아서 하는 일입니다.

- 1단계 • 까만 점
- 2단계 ◦ 속이 빈 동그라미
- 3단계 ▪ 네모

깊이가 눈에 보이라고 브라우저가 단계마다 다른 표시를 씁니다. 여러분이 아무 설정도 하지 않았는데 그렇게 됩니다.

화면	"과일" 앞에는 까만 점(•), 그 아래 "사과" "포도" 앞에는 속이 빈 동그라미(◦)가
----	--

붙고 한 칸 더 안쪽으로 들어가 있습니다. "채소"는 다시 까만 점입니다

과일
사과
포도
채소

화면	세 줄이 계단처럼 한 칸씩 들어가고, 표시가 차례로
----	------------------------------

까만 점(•) → 속이 빈 동그라미(◦) → 작은 네모(■)로 바뀝니다

음식
과일
사과

▶ 직접 해보기 4

위 "2단계 목록"의 채소 아래에도 안쪽 목록을 만들어 "당근"과 "양파"를 넣어 보세요. 를 어디에 둘지 잘 보세요.

자주 하는 실수

섹션 5

다시 강조합니다. 아래 실수는 전부 에러가 나지 않습니다. 어떤 것은 화면까지 똑같이 나옵니다. 그래서 더 위험합니다.

이런 실수를 합니다

안쪽 ul을 li 밖에 둡니다

이런 실수를 합니다

실수 1 — 안쪽 ul 을 li 밖에 둠

```
<ul>
<li>과일</li>      ← 여기서 li 를 닫아 버렸습니다
<ul>               ← 그래서 이 ul 이 li 밖, ul 바로 안에 들어갑니다
  <li>사과</li>
</ul>
<li>채소</li>
</ul>
```

과일 사과 포도 채소 화면이 올바른 코드와 똑같이 나옵니다. 아래 올바른 예와 나란히 놓고 봐도 구분이 안 됩니다. 브라우저가 알아서 비슷하게 그려 주기 때문입니다. 하지만 "사과"가 "과일"에 딸려 있다는 정보가 사라집니다. 08단원에서 CSS 선택자를 배우면 "과일 아래 목록만 꾸미기"가 안 되고, 화면 낭독기도 딸린 관계를 읽어 주지 못합니다. 화면 낭독기는 01단원 개념03에서 이야기한, 화면의 글자를 소리로 읽어 주는 프로그램입니다. 02단원에서도 몇 번 나왔습니다.

이렇게 쓰세요

```
<ul>
<li>과일
  <ul> <li>사과</li> </ul>
</li>      ← 안쪽 ul 이 끝난 뒤에 닫습니다
<li>채소</li>
</ul>
```

이런 실수를 합니다

ul 안에 글자를 직접 씀

이런 실수를 합니다

실수 2 — ul 안에 글자를 직접 씀

```
<ul>
<li>사과</li>
바나나      ← li 로 감싸는 것을 잊었습니다
</ul>
```

사과 바나나 글자는 그대로 보이는데 그 줄만 점이 없습니다. 섹션 2의 p와 같은 일이 일어난 것입니다. "왜 한 줄만 점이 안 붙지?" 하면 li로 안 감싼 줄을 찾으세요.

이런 실수를 합니다

li를 ul 없이 혼자 씀

이런 실수를 합니다

실수 3 — li를 ul 없이 혼자 씀

```
<li>혼자 있는 항목</li>
```

혼자 있는 항목 점이 그대로 붙어서 나옵니다. 그래서 “됐네” 하고 넘어가기 쉽습니다. 하지만 감싸는 상자가 없으니 목록이 아닙니다. 항목만 덩그러니 있는 셈입니다. li는 반드시 ul이나 ol 안에 두세요.

이런 실수를 합니다

줄바꿈 하려고 목록을 씀

이런 실수를 합니다

실수 4 — 줄을 나누려고 목록을 씀

```
<ul>
<li>안녕하세요. 저희 가게에 오신 것을 환영합니다.</li>
</ul>
```

항목이 하나뿐인 목록은 목록이 아닙니다. 그냥 문장입니다. 화면에는 원치 않는 점이 하나 붙고 왼쪽 여백이 생깁니다. 이럴 때는 <p>를 쓰세요. 태그는 모양이 아니라 뜻을 보고 고릅니다.

이런 실수를 합니다

li를 안 닫음

이런 실수를 합니다

실수 5 — li를 안 닫음

```
<ul>
<li>사과
<li>바나나
</ul>
```

화면은 멀쩡히 두 줄로 나옵니다. ol단원의 p와 같습니다. 브라우저가 “다음 li가 시작됐으니 앞의 것은 끝났겠지” 하고 빠진 닫는 태그를 채워 넣습니다. 운이 좋아서 맞은 것이니 항상 닫으세요. 섹션 4처럼 안쪽 목록이 들어가는 순간부터는 이 요행이 통하지 않습니다.

정리

정답

1) 섹션 1의 목록 안, “포도” 아래에 이렇게 넣습니다.

```
<li>딸기</li>
```

→ 점이 붙은 줄이 세 줄에서 네 줄로 늘어납니다.

점 모양과 왼쪽 여백은 앞의 항목들과 똑같습니다.

2) 섹션 2의 가운데 줄을 이렇게 바꿉니다.

```
<li>저는 p 입니다</li>
```

→ 없던 점(•)이 가운데 줄에도 생기고, 세 줄이 완전히 똑같은 모양이 됩니다.

위아래로 조금 더 벌어져 있던 간격도 다른 항목과 같아집니다.

3) 섹션 3의 마지막 항목을 이렇게 바꿉니다.

```
<li>포도 <strong>제철입니다</strong></li>
```

→ “제철입니다”가 기울어진 글씨에서 굵은 글씨로 바뀝니다.

em은 강조해서 읽는 부분, strong은 중요한 부분이라는 뜻입니다.

4) 섹션 4의 “2단계 목록”을 이렇게 고칩니다.

```
<li>채소
  <ul> <li>당근</li> <li>양파</li> </ul>
</li>
```

→ “채소” 아래에 속이 빈 동그라미(•) 항목 두 줄이 한 칸 안으로 들어가 붙습니다.

원래 있던 를 지우고 안쪽 ul 뒤로 옮긴 것이 전부입니다.

순서있는 목록

1. 물 550mL 를 끓입니다 2. 면과 스프를 넣습니다 3. 4분 30초 더 끓입니다
• 연필 • 지우개 • 공책
1. 약관에 동의합니다 2. 아이디와 비밀번호를 정합니다 3. 가입 완료 화면을 확인합니다
A. 첫 번째 항목 B. 두 번째 항목

실행하면 나오는 화면 — 순서있는 목록

03단원 · 개념 02 — 순서 있는 목록

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

앞 파일에서 ui 로 목록을 만들었습니다. 항목 앞에 점(•) 이 붙었습니다. 그런데 이런 목록은 점으로는 부족합니다.

라면 끓이는 법
물 550mL 를 끓입니다
면과 스프를 넣습니다
4분 30초 더 끓입니다

이 셋은 순서를 바꾸면 안 됩니다. 순서 자체가 뜻입니다. 면을 먼저 넣고 물을 끓일 수는 없으니까요.

이럴 때 쓰는 태그가 이 입니다. ordered list, 순서 있는 목록입니다.

```
<ol> <li>물 550mL 를 끓입니다</li> <li>면과 스프를 넣습니다</li>  
</ol>
```

바뀐 것은 바깥 태그 이름 하나뿐입니다. ul 이 ol 로 바뀌었습니다. 안쪽 항목은 그대로 li 를 씁니다. 새로 외울 태그는 없습니다.

ul → •	점이 붙습니다	순서가 뜻이 없을 때
ol → 1.	번호가 붙습니다	순서가 뜻일 때

여기서 제일 중요한 것은 다음 한 가지입니다.

번호는 여러분이 쓰는 것이 아니라 브라우저가 세어 줍니다.

손으로 "1." 이라고 쓰지 않습니다. li 만 나열하면 브라우저가 위에서부터 셉니다. 그래서 중간에 항목을 하나 끼워 넣어도 아래 번호가 저절로 다시 매겨집니다. 손으로 썼다면 그 아래 번호를 전부 고쳐야 했을 것입니다.

이 파일에서는 번호가 앞에 붙는 목록을 만듭니다. 태그는 ol 하나만 새로 배우고, 번호 모양을 바꾸는 속성 type, start, reversed, value 를 봅니다. 그리고 언제 ul 이고 언제 ol 인지 고르는 기준을 익힙니다.

ol로 순서 있는 목록 만들기

섹션 1

ul 을 ol 로 바꾸기만 하면 됩니다. 안쪽은 그대로 li 입니다.

아래 상자 안에 이렇게 씁니다.

```
<ol id="recipe-list"> <li>물 550mL 를 끓입니다</li> <li>면과 스프를 넣습니다  
</li> <li>4분 30초 더 끓입니다</li>  
</ol>
```

코드 어디에도 1, 2, 3 이라는 글자가 없습니다. 그런데 화면에는 번호가 붙습니다. 브라우저가 li 를 위에서부터 세어 준 것입니다.

번호는 왼쪽 여백 안쪽에 오른쪽을 맞춰서 놓입니다. 그래서 항목이 9개를 넘어 10, 11, 처럼 두 자리가 되어도 글자 시작 위치가 흐트러지지 않고 나란히 유지됩니다.

앞 파일과 마찬가지로 이의 자식 자리에도 li 만 씁니다. 점이 번호로 바뀌었을 뿐, 규칙은 똑같습니다.

화면 "1." "2." "3." 이 앞에 붙은 세 줄이 보입니다.

번호 뒤 글자 시작 위치는 세 줄 모두 같습니다

물 550mL 를 끓입니다
면과 스프를 넣습니다
4분 30초 더 끓입니다

이 은 ordered list 의 줄임말입니다. ul 의 u 는 unordered(순서 없는), li 의 o 는 ordered(순서 있는)입니다. 글자 하나 차이로 헷갈리기 쉬우니 o 는 순서(order) 로 외워 두세요.

▹ 직접 해보기

1

위 목록의 맨 앞에 냄비를 꺼냅니다 를 끼워 넣어 보세요. 아래 항목들의 번호가 어떻게 되는지 확인하는 것이 목적입니다.

언제 ul 이고 언제 li 인가

섹션 2

판단 기준은 하나입니다. 순서를 바꿔도 뜻이 그대로면 ul, 뜻이 망가지면 li 입니다.

고민될 때는 항목의 순서를 마음속으로 뒤집어 보세요.

장바구니: 사과 · 바나나 · 포도
뒤집으면: 포도 · 바나나 · 사과 → 뜻이 똑같습니다. 그러니 ul.

라면: 물 끓이기 · 면 넣기 · 4분 30초 끓이기
뒤집으면: 4분 30초 끓이기 · 면 넣기 · 물 끓이기 → 말이 안 됩니다. 그러니 li.

“번호가 보기 좋아서” 나 “점이 촌스러워서” 로 고르면 안 됩니다. 01단원에서 배웠듯 HTML 은 뜻을 붙이는 일이고, 겉모양은 CSS 가 담당합니다. 번호가 붙은 모양이 마음에 들면, 07단원부터 배우는 CSS 로 ul 을 그렇게 꾸미면 됩니다.

판단이 쉬워지는 예를 몇 개 적어 둡니다.

ol 이 맞는 것 : 요리 순서, 가입 절차, 달리기 순위, 회의 안건 번호, 법 조항
ul 이 맞는 것 : 준비물, 메뉴판, 취미, 회원 명단, 태그 목록

회원 명단이 ul 인 이유는 “누가 먼저 적혔는지” 가 뜻을 갖지 않기 때문입니다. 만약 가입한 순서가 중요한 명단이라면 그때는 li 이 맞습니다. 태그 이름은 내용의 성격을 보고 고르는 것이지 정답이 하나로 정해져 있지 않습니다.

화면 점(•) 이 붙은 세 줄이 보입니다

연필
지우개
공책

화면 "1." "2." "3." 이 붙은 세 줄이 보입니다

약관에 동의합니다
아이디와 비밀번호를 정합니다
가입 완료 화면을 확인합니다

▣ 직접 해보기 2

"우리 반 달리기 1등 · 2등 · 3등" 목록을 만든다면 `ul` 과 `li` 중 무엇을 써야 할까요? 직접 목록을 만들어 위 상자 아래에 넣어 보세요.

type 으로 번호 모양 바꾸기

섹션 3

`li` 에 `type` 속성을 주면 번호가 알파벳이나 로마자로 바뀝니다.

01단원에서 속성을 배웠습니다. 여는 태그 안에 "이름=값" 형태로 넣는 추가 정보였습니다. `li` 에 쓸 수 있는 속성 중 첫 번째가 `type` 입니다.

```
<ol type="A">  
  L 속성 이름   L 값 (큰따옴표로 감쌉니다)
```

값은 다섯 가지입니다.

<code>type="1"</code>	1. 2. 3.	숫자 (아무것도 안 쓰면 이것입니다)
<code>type="A"</code>	A. B. C.	대문자 알파벳
<code>type="a"</code>	a. b. c.	소문자 알파벳
<code>type="I"</code>	I. II. III.	대문자 로마 숫자
<code>type="i"</code>	i. ii. iii.	소문자 로마 숫자

대문자 A 와 소문자 a 는 서로 다른 값입니다. 속성 값은 대소문자를 가립니다. 01단원에서 태그 이름은 대소문자를 안 가린다고 했는데, 값은 다릅니다. 헛갈리지 마세요.

겉모양을 바꾸는 일은 원래 CSS 몫입니다. 그런데도 `type` 이 HTML 에 남아 있는 이유가 있습니다. 계약서의 "제2조 나항" 처럼 번호 모양 자체가 내용의 일부인 글이 있기 때문입니다. 그런 경우가 아니라 단지 보기 좋으라고 바꾸는 것이라면, 07단원 이후 CSS 로 하는 편이 맞습니다.

화면 "A." "B." 가 앞에 붙은 두 줄이 보입니다

첫 번째 항목
두 번째 항목

화면 "a." "b." 가 앞에 붙은 두 줄이 보입니다

첫 번째 항목
두 번째 항목

화면 "I." "II." "III." "IV." 가 앞에 붙은 네 줄이 보입니다

첫 번째 항목
두 번째 항목
세 번째 항목
네 번째 항목

화면 "i." "ii." "iii." "iv." 가 앞에 붙은 네 줄이 보입니다

첫 번째 항목
두 번째 항목
세 번째 항목
네 번째 항목

▫ 직접 해보기

3

위 type="A" 목록의 값을 type="i" 로 바꿔 보세요. 그다음 type="Z" 처럼 없는 값도 한 번 넣어 보고 어떻게 되는지 확인하세요.

start 와 reversed

섹션 4

번호는 기본이 1 부터 오름차순입니다. 두 속성으로 시작점과 방향을 바꿀 수 있습니다.

start 는 첫 항목의 번호를 정합니다.

```
<ol start="5"> → 5. 6. 7. ...
```

언제 쓸까요? 긴 목록을 중간에 끊고 설명을 넣은 뒤 다시 이어 쓸 때입니다. 앞에서 4번까지 썼다면 뒤쪽 목록을 start="5" 로 시작해야 번호가 이어집니다. 그냥 새 이 을 만들면 다시 1. 부터 시작해 버립니다.

start 의 값은 type 이 무엇이든 항상 숫자로 씁니다. type="A" 이면서 start="3" 이면 C. 부터 시작합니다. start="C" 라고 쓰지 않습니다.

reversed 는 번호를 거꾸로 씁니다.

<ol reversed> → 항목이 3개면 3. 2. 1.

이 속성은 생김새가 다릅니다. 등호도 값도 없이 이름만 씁니다. 이런 속성을 불리언 속성이라고 부릅니다. “있으면 켜짐, 없으면 꺼짐” 두 가지뿐이라 값이 필요 없습니다. 05단원에서 배울 required 도 같은 종류입니다.

여기서 초보자가 크게 놀라는 일이 하나 있습니다.

<ol reversed="false"> ← 꺼질 것 같지만 그대로 켜집니다

false 라고 썼는데도 거꾸로 나옵니다. 불리언 속성은 값을 아예 보지 않고 “이름이 적혀 있는가” 만 보기 때문입니다. 끄고 싶으면 속성을 통째로 지워야 합니다. 섹션 6 에서 실제 결과를 나란히 놓고 봅니다.

순위를 3위부터 1위 순으로 발표할 때 reversed 가 유용합니다.

화면 "5." "6." 이 앞에 붙은 두 줄이 보입니다

다섯 번째 순서입니다
여섯 번째 순서입니다

화면 "3. 동메달" "2. 은메달" "1. 금메달" 순서로 세 줄이 보입니다.

줄 순서는 코드에 쓴 그대로이고 번호만 거꾸로 붙습니다

동메달
은메달
금메달

reversed 는 번호만 거꾸로 붙입니다. 줄의 순서를 뒤집어 주지는 않습니다. 위 상자에서 “동메달” 이 여전히 첫 줄인 것을 보세요.

▫ 직접 해보기 4

위 start="5" 목록에 type="A" 를 같이 넣어 보세요. (<ol type="A" start="5">) 어떤 글자부터 시작하는지 확인하세요.

value 로 번호 하나만 바꾸기

섹션 5

value 는 li 에 붙이는 속성입니다. 그 항목의 번호를 직접 정합니다.

여기까지 나온 속성 세 개는 전부 li 에 붙였습니다. value 만 li 에 붙입니다. 붙이는 자리가 다르니 주의하세요.


```
<ol>
  <li>하나</li>
  <li value="10">열</li>      ← 이 항목만 번호를 10 으로 지정
  <li>열하나</li>            ← 그다음은 11 로 이어집니다
</ol>
```

중요한 것은 value 가 그 항목 하나만 바꾸는 게 아니라는 점입니다. 브라우저는 그 항목부터 다시 세기 시작합니다. 그래서 아래 항목들의 번호도 따라 바뀝니다.

이 속성은 자주 쓰지 않습니다. 목록 중간에 몇 개를 생략하고 "1번, 그리고 10번부터" 처럼 건너뛰어 보여 줘야 할 때 정도입니다.

요약해 두면 이렇습니다.

ol 에 붙는 것	: type, start, reversed
li 에 붙는 것	: value
화면	"1. 하나" "10. 열" "11. 열하나" 순서로 세 줄이 보입니다.

두 자리 번호가 생기면서 번호가 왼쪽으로 조금 더 나와 붙습니다

```
하나
  열
  열하나
```

▸ 직접 해보기 5

위 목록의 value="10" 을 value="100" 으로 바꿔 보세요. 마지막 줄 번호가 몇으로 바뀌는지 확인하세요.

자주 하는 실수

섹션 6

아래 실수들도 에러가 나지 않습니다. 번호가 이상하게 보이거나, 아무 일도 안 일어나거나 둘 중 하나입니다.

이런 실수를 합니다

번호를 손으로 씀

이런 실수를 합니다

실수 1 — 번호를 손으로 씀

```
<ol>
<li>1. 물을 끓입니다</li>
<li>2. 면을 넣습니다</li>
</ol>
```

1. 물을 끓입니다 2. 면을 넣습니다 번호가 두 번 나옵니다. 브라우저가 붙인 번호와 손으로 쓴 번호가 겹치기 때문입니다. 게다가 중간에 항목을 끼워 넣으면 손으로 쓴 번호는 자동으로 안 고쳐져서 앞뒤가 어긋납니다. 번호는 쓰지 마세요.

이런 실수를 합니다

reversed 를 값으로 끄려고 함

이런 실수를 합니다

실수 2 — reversed 를 "false" 로 끄려고 함

```
<ol reversed="false">
```

첫 줄 둘째 줄 셋째 줄 false 라고 썼는데도 그대로 거꾸로 나옵니다. 불리언 속성은 값을 보지 않고 이름이 있는지만 보기 때문입니다. 끄려면 reversed 를 통째로 지우세요.

이런 실수를 합니다

없는 type 값을 씀

이런 실수를 합니다

실수 3 — 없는 type 값을 씀

```
<ol type="가">
```

첫 줄 둘째 줄 아무 일도 일어나지 않고 그냥 숫자로 나옵니다. 경고도 없습니다. 브라우저는 모르는 값을 만나면 조용히 무시하고 기본값을 씁니다. "가. 나. 다." 같은 한글 번호는 type 으로는 만들 수 없습니다.

이런 실수를 합니다

value 를 이 에 붙임

이런 실수를 합니다

실수 4 — value 를 li 에 붙임

```
<ol value="5">
<li>첫 줄</li>
</ol>
```

첫 줄 둘째 줄 아무 일도 일어나지 않습니다. 1 부터 그대로 셉니다. value 는 li 에 붙는 속성입니다. ol 에서 시작 번호를 정하는 것은 start 입니다. 붙이는 자리를 바꿔 쓰면 조용히 무시되니 화면을 꼭 확인하세요.

이런 실수를 합니다

모양 때문에 li 을 고름

이런 실수를 합니다

실수 5 — 번호가 예뻐서 li 을 고름

```
<ol>
<li>아메리카노</li>
<li>카페라떼</li>
</ol>
```

메뉴판에는 순서가 없습니다. 화면은 멀쩡히 나오지만 뜻이 틀립니다. 화면 낭독기는 “1번 항목, 2번 항목” 이라고 읽어 줍니다. 듣는 사람은 순위나 순서가 있는 줄 알게 됩니다. 모양이 마음에 든다면 ul 로 두고 07단원부터 CSS 로 꾸미세요.

이런 실수를 합니다

안쪽 목록의 번호가 이어질 거라고 생각함

이런 실수를 합니다

실수 6 — 안쪽 목록 번호가 이어질 거라 생각함

준비하기 냄비 꺼내기 물 받기 끓이기 안쪽 목록은 1 부터 다시 셉니다. 바깥의 1 을 이어받아 2, 3 이 되지 않습니다. “1.1”, “1.2” 같은 번호를 원한다면 그것은 CSS 로 만드는 것이라 지금은 할 수 없습니다. 바깥 목록의 다음 항목은 안쪽 목록과 상관없이 2 로 이어집니다.

정리

==

▫ 직접 해보기 6

정답

1) 섹션 1 의 이 안 맨 위에 이렇게 넣습니다.

```
<li>냄비를 꺼냅니다</li>
```

→ 새 항목이 1. 이 되고, 원래 1·2·3 이던 줄이 2·3·4 로 저절로 밀립니다.

여러분이 고친 번호는 하나도 없습니다. 이것이 ol 을 쓰는 이유입니다.

2) 이 을 씁니다. 등수는 순서가 곧 뜻이기 때문입니다.

```
<ol> <li>김민준</li> <li>이서연</li> <li>박지훈</li>
</ol>
```

→ "1. 김민준" "2. 이서연" "3. 박지훈" 으로 보입니다.

ul 로 쓰면 점만 붙어서 누가 1등인지 화면 낭독기가 알 수 없게 됩니다.

3) 섹션 3 의 첫 목록을 이렇게 바꿉니다.

```
<ol id="type-upper" type="i">
```

→ "A." "B." 였던 번호가 "i." "ii." 로 바뀝니다.

이어서 type="Z" 로 바꾸면 없는 값이라 무시되고 "1." "2." 로 돌아옵니다.
확인했으면 type="A" 로 되돌려 두세요.

4) 섹션 4 의 start 목록을 이렇게 바꿉니다.

```
<ol id="start-list" type="A" start="5">
```

→ "E." "F." 로 시작합니다. 알파벳 다섯 번째가 E 이기 때문입니다.

start 값은 type 이 무엇이든 항상 숫자로 씁니다.

5) 섹션 5 의 가운데 항목을 이렇게 바꿉니다.

```
<li value="100">열</li>
```

→ "1. 하나" "100. 열" "101. 열하나" 가 됩니다.

마지막 줄이 101 이 되는 이유는 value 가 그 자리부터 다시 세기 때문입니다.
번호가 세 자리로 길어져도 글자 시작 위치는 그대로입니다.
섹션 1 에서 말한 대로 번호는 오른쪽을 맞춰 놓이기 때문에,
길어진 번호가 왼쪽으로 더 나갈 뿐입니다.

설명 목록

HTML	웹페이지의 뼈대를 만드는 언어입니다.
CSS	뼈대를 꾸미는 언어입니다.
배	물 위를 떠다니는 탈것입니다. 배나무의 열매입니다. 사람 몸의 앞쪽 가운데 부분입니다.
냉장고	냉장 보관고 음식을 차게 보관하는 기계입니다.
화면 크기	15.6인치
무게	

실행하면 나오는 화면 — 설명 목록

03단원 · 개념 03 — 설명 목록

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

지금까지 배운 목록은 항목이 한 종류였습니다.

ul	사과 · 바나나 · 포도	항목만 나열
ol	물 끓이기 · 면 넣기	항목만 나열

그런데 세상에는 항목이 두 종류인 목록이 있습니다.

HTML 웹페이지의 뼈대를 만드는 언어입니다
CSS 뼈대를 꾸미는 언어입니다
↳ 용어 ↳ 그 용어에 대한 설명

왼쪽은 이름이고 오른쪽은 설명입니다. 둘은 짝입니다. 이런 짝 목록을 만드는 태그가 이번 파일의 주제입니다. 태그는 세 개입니다.

dl	짝 목록 전체를 감싸는 상자	(description list, 설명 목록)
dt	왼쪽의 용어	(description term, 설명할 대상)
dd	오른쪽의 설명	(description details, 자세한 내용)

d 는 셋 다 description(설명) 의 d 입니다. t 는 term(용어), d 는 details(내용) 라고 기억하면 헷갈리지 않습니다.

```
<dl> <dt>HTML</dt> <dd>웹페이지의 뼈대를 만드는 언어입니다</dd>  
</dl>
```

ul 이 li 를 담았던 것처럼, dl 은 dt 와 dd 를 담습니다. 다른 점은 담기는 태그가 두 종류라는 것뿐입니다.

이 파일에서는 용어와 설명이 짝을 이루는 목록을 만듭니다. dl, dt, dd 세 태그를 씁니다. 점도 번호도 붙지 않아서 처음에는 “이게 목록인가?” 싶습니다. 대신 설명 쪽만 오른쪽으로 들어가는 모양으로 짝이 눈에 보입니다.

dl, dt, dd 로 짝 만들기

섹션 1

dl 로 감싸고, 그 안에 dt(용어) 와 dd(설명) 를 번갈아 씁니다.

아래 상자 안에 이렇게 썼습니다.

```
<dl id="term-list"> <dt>HTML</dt> <dd>웹페이지의 뼈대를 만드는 언어입니다.  
</dd> <dt>CSS</dt> <dd>뼈대를 꾸미는 언어입니다.</dd>  
</dl>
```

결과에서 확인할 것이 두 가지 있습니다.

1) dd 만 왼쪽에서 안으로 들어갑니다. 재 보면 40px 입니다.

dt 는 들어가지 않고 왼쪽 끝에 그대로 있습니다.
이 어긋남 하나로 "위가 용어, 아래가 설명" 이라는 관계가 눈에 보입니다.

- 2) 점도 번호도 붙지 않습니다.
ul 과 ol 은 표시가 붙었지만 dl 은 붙지 않습니다.
그래서 "이건 목록이 아니라 그냥 글 아닌가?" 하고 착각하기 쉽습니다.
목록이 맞습니다. 짝이라는 뜻이 코드에 들어 있습니다.

여기서 초보자가 가장 많이 오해하는 것을 짚고 갑니다.

dt 는 굵게 나오지 않습니다.

책이나 블로그의 용어집에서 용어가 굵게 인쇄된 것을 본 기억 때문에 "dt 를 쓰면 굵어지겠지" 하고 기대하게 됩니다. 실제로 재 보면 굵기는 400, 즉 본문과 똑같은 보통 굵기입니다. 굵게 만들고 싶으면 09단원에서 CSS 로 하거나, 지금 당장 하고 싶다면 02단원에서 배운 strong 으로 감싸면 됩니다.

화면 "HTML" 은 왼쪽 끝에, 그 아래 설명 줄은 한 칸 안으로 들어가 있습니다.

"CSS" 도 같은 모양으로 이어집니다. 점도 번호도 없고 글자 굵기는 전부 같습니다

HTML
웹페이지의 뼈대를 만드는 언어입니다.
CSS
뼈대를 꾸미는 언어입니다.

dt 와 dd 는 짝의 개수가 같을 필요가 없습니다. 섹션 2 에서 dt 하나에 dd 를 여러 개 다는 경우를 봅니다.

▹ 직접 해보기 1

위 목록에 <dt>브라우저</dt> 와 그 설명 <dd>웹페이지를 열어 보는 프로그램입니다.</dd> 를 한 짝 더 넣어 보세요.

dt 하나에 dd 여러 개

섹션 2

용어 하나에 설명이 여럿일 수 있습니다. dd 를 이어서 쓰면 됩니다.

한 단어에 뜻이 여러 개인 경우를 생각해 보세요.

```
<dl id="multi-dd"> <dt>배</dt> <dd>물 위를 떠다니는 탈것입니다.</dd> <dd>배나무의 열매입니다.</dd> <dd>사람 몸의 앞쪽 가운데 부분입니다.</dd>
</dl>
```

dt 를 다시 쓰지 않고 dd 만 이어 붙입니다. 브라우저는 "다음 dt 가 나오기 전까지의 dd 는 전부 앞 dt 의 설명" 으로 봅니다.

반대로 dt 를 여러 개 쓰고 dd 를 하나만 쓸 수도 있습니다. 뜻이 같은 여러 이름을 한 번에 설명할 때입니다.

```
<dt>냉장고</dt>
<dt>냉장 보관고</dt>
<dd>음식을 차게 보관하는 기계입니다.</dd>
```

규칙은 하나입니다. dl 안에서 dt 와 dd 만 쓰고, 순서는 dt 가 먼저입니다. 몇 개씩 쓸지는 내용에 맞춰 정하면 됩니다.

화면 "배" 가 왼쪽 끝에 한 줄, 그 아래 설명 세 줄이 모두 같은 만큼 안으로 들어가

나란히 붙어 있습니다. 설명끼리는 위아래 간격이 없습니다

배
물 위를 떠다니는 탈것입니다.
배나무의 열매입니다.
사람 몸의 앞쪽 가운데 부분입니다.

화면 왼쪽 끝에 두 줄이 붙어 나오고, 그 아래 설명 한 줄이 안으로 들어가 있습니다

냉장고
냉장 보관고
음식을 차게 보관하는 기계입니다.

▣ 직접 해보기 2

"눈" 이라는 dt 를 만들고 뜻이 다른 dd 두 개(하늘에서 내리는 것 · 몸의 기관) 를 달아 보세요.

언제 쓰나

섹션 3

"왼쪽 이름 · 오른쪽 값" 으로 정리되는 내용이면 dl 자리입니다.

실제로 많이 쓰는 곳은 세 가지입니다.

1) 용어집

섹션 1 에서 본 모양 그대로입니다.

2) 상품 스펙

쇼핑몰에서 "화면 크기 : 15.6인치" 처럼 항목과 값이 짝지어진 표입니다.
dt 가 항목 이름, dd 가 값입니다.

3) FAQ (자주 묻는 질문)

dt 에 질문, dd 에 답을 넣습니다.
질문 하나에 답이 여러 문단이면 섹션 2 처럼 dd 를 여러 개 씁니다.

여기서 한 가지 헛갈리는 점이 생깁니다. 상품 스펙은 다음 파일에서 배울 table 로도 만들 수 있습니다. 무엇을 써야 할까요? 기준은 값이 몇 줄기인가입니다.

```

항목마다 값이 하나 → dl
  화면 크기 : 15.6인치
  무게      : 1.4kg

항목마다 값이 여럿 → table
      1분기 2분기 3분기
매출   100   120   150
    
```

위쪽은 이름 하나에 값 하나라 짝 목록입니다. 아래쪽은 가로와 세로 두 방향으로 읽어야 하니 표입니다. “가로줄과 세로줄이 만나는 자리를 읽어야 하면 표” 라고 기억해 두세요.

화면 항목 이름이 왼쪽 끝에, 값이 한 칸 안으로 들어간 줄로 번갈아 여섯 줄 보입니다

화면 크기
15.6인치
무게
1.4kg
색상
은색

화면 질문이 왼쪽 끝에, 답이 한 칸 안으로 들어가 보입니다.

첫 질문 아래에는 답이 두 줄 연달아 붙어 있습니다

배송은 얼마나 걸리나요?
주문 다음 날 출발해서 보통 이틀 안에 도착합니다.
주말과 공휴일은 하루씩 더 걸립니다.
교환은 어떻게 하나요?
받으신 날부터 7일 안에 신청하시면 됩니다.

dt 에 을 씌워 질문을 굵게 만들 수 있습니다. dt 자체는 굵어지지 않기 때문입니다. 다만 이런 꾸미기는 09단원 CSS 로 하는 편이 더 깔끔합니다.

▹ 직접 해보기

3

위 상품 스펙 목록에 “가격 · 89만원” 짝을 하나 더 넣어 보세요. dt 와 dd 중 어느 쪽이 들여쓰기되는지 확인하세요.

ul·li 과 무엇이 다른가

섹션 4

같은 내용을 세 가지 목록으로 써 보면 차이가 분명해집니다.

“화면 크기는 15.6인치, 무게는 1.4kg” 이라는 내용을 세 가지로 써 보겠습니다.

ul 로 쓰면

- 화면 크기 : 15.6인치
- 무게 : 1.4kg

점이 붙습니다. 항목 안에서 이름과 값을 콜론으로 나눈 것은 사람이 눈으로 읽으라고 한 것이지, 코드에는 짝이라는 정보가 없습니다.

ol 로 쓰면

1. 화면 크기 : 15.6인치
2. 무게 : 1.4kg

순서가 없는 내용인데 번호가 붙어 "1번이 먼저" 라는 없던 뜻이 생깁니다. 틀렸습니다.

dl 로 쓰면

화면 크기

15.6인치

무게

1.4kg

들어쓰기로 짝이 보이고, 코드에도 "이건 이름, 이건 값" 이 들어갑니다.

세 가지 다 화면에는 무언가가 나옵니다. 에러는 어느 쪽도 나지 않습니다. 다른 것은 뜻입니다. 뜻이 맞아야 화면 낭독기가 제대로 읽어 주고, 08단원에서 CSS 선택자로 원하는 부분만 골라 꾸밀 수도 있습니다.

화면

점(•) 이 붙은 두 줄이 보입니다. 이름과 값이 한 줄 안에 붙어 있습니다

화면 크기 : 15.6인치

무게 : 1.4kg

화면

네 줄이 보이고, 값이 적힌 두 줄만 한 칸 안으로 들어가 있습니다

화면 크기

15.6인치

무게

1.4kg

직접 해보기

4

여러분의 휴대전화 정보(기종 · 색상 · 산 곳) 를 dl 로 만들어 위 상자 아래에 넣어 보세요.

자주 하는 실수

섹션 5

dl 은 점도 번호도 없어서 잘못 써도 티가 잘 안 납니다. 그만큼 코드를 눈으로 확인하는 습관이 필요합니다.

이런 실수를 합니다

dt 가 굵게 나올 거라고 기대함

이런 실수를 합니다

실수 1 — dt 가 굵게 나올 거라고 기대함

```
<dt>HTML</dt>
```

HTML 굵어지지 않았습니다. 용어가 보통 굵기 그대로 나옵니다. 재 보면 굵기 값이 400 으로 본문과 같습니다. dt 는 “여기가 용어다” 라는 뜻만 붙이고 겉모양은 건드리지 않습니다. 굵게 하려면 으로 감싸거나 09단원에서 CSS 로 하세요.

이런 실수를 합니다

dl 안에 다른 태그를 바로 넣음

이런 실수를 합니다

실수 2 — dl 안에 p 를 바로 넣음

```
<dl>
<p>아래는 용어 설명입니다</p>
<dt>HTML</dt>
</dl>
```

아래는 용어 설명입니다 HTML 웹페이지의 뼈대입니다. ul 의 규칙과 같습니다. dl 의 자식 자리에는 dt 와 dd 만 옵니다. 화면에는 그냥 한 줄이 더 늘어날 뿐이라 알아채기 어렵습니다. 안내 문장이 필요하면 dl 바깥, 그 위에 p 로 쓰세요.

이런 실수를 합니다

dt 없이 dd 만 씀

이런 실수를 합니다

실수 3 — dt 없이 dd 만 씀

```
<dl>
<dd>설명만 있습니다</dd>
</dl>
```

설명만 있습니다 들여쓰기만 된 줄 하나가 보입니다. 에러는 없습니다. 그래서 “글자를 안으로 밀고 싶을 때” dd 를 쓰는 사람이 있는데, 이것은 뜻을 속이는 일입니다. 무엇에 대한 설명인지가 사라집니다. 들여쓰기는 10단원에서 CSS 로 하세요.

이런 실수를 합니다

dt 를 dl 밖에서 씀

이런 실수를 합니다

실수 4 — dt 를 dl 밖에서 씀

```
<dt>혼자 있는 용어</dt>
```

혼자 있는 용어 평범한 한 줄로 보일 뿐입니다. li 는 혼자 있어도 점이라도 붙어서 티가 났지만, dt 는 원래 아무 표시가 없어서 완전히 똑같아 보입니다. p 를 쓴 것과 화면상 구분이 안 됩니다. 반드시 dl 안에 두세요.

이런 실수를 합니다

dd 를 먼저 쓰고 dt 를 나중에 씀

이런 실수를 합니다

실수 5 — dd 를 먼저 쓰고 dt 를 나중에 씀

```
<dl>
<dd>웹페이지의 뼈대입니다</dd>
<dt>HTML</dt>
</dl>
```

화면에는 설명이 위, 용어가 아래로 나옵니다. 보기에다 어색하지만 더 큰 문제는 어느 용어에 딸린 설명인지 알 수 없게 된다는 점입니다. 브라우저는 dd 를 바로 위의 dt 에 딸린 것으로 봅니다. 위에 dt 가 없으면 딸릴 곳이 없습니다. dt 를 먼저 쓰세요.

정리

▹ 직접 해보기 5

정답

1) 섹션 1 의 dl 안, 마지막 dd 아래에 이렇게 넣습니다.

```
<dt>브라우저</dt>
<dd>웹페이지를 열어 보는 프로그램입니다.</dd>
```

→ 왼쪽 끝에 “브라우저” 한 줄, 그 아래 한 칸 들어간 설명 한 줄이 늘어납니다.

앞의 두 짝과 똑같은 모양이 됩니다.

2) 섹션 2 의 상자 아래에 이렇게 넣습니다.

```
<dl> <dt>눈</dt> <dd>하늘에서 내리는 하얀 것입니다.</dd> <dd>사물을 보는 몸의
기관입니다.</dd>
</dl>
```

→ “눈” 아래에 들여쓰기된 설명 두 줄이 간격 없이 붙어 나옵니다.

```
dt 를 한 번만 썼는데 설명 두 개가 모두 그 아래 딸립니다.
```

3) 섹션 3 의 상품 스펙 목록 맨 아래에 이렇게 넣습니다.

```
<dt>가격</dt>
<dd>89만원</dd>
```

→ “가격” 은 왼쪽 끝에, “89만원” 은 한 칸 들어간 자리에 나옵니다.

```
들여쓰기되는 쪽은 항상 dd 입니다.
```

4) 섹션 4 의 상자 아래에 이렇게 넣습니다. 내용은 본인 것으로 바꾸세요.

```
<dl> <dt>기종</dt> <dd>스마트폰 A</dd> <dt>색상</dt> <dd>검정</dd> <dt>산 곳
</dt> <dd>동네 대리점</dd>
</dl>
```

→ 여섯 줄이 나오고 값이 적힌 세 줄만 안으로 들어갑니다.

```
ul 로 썼을 때와 달리 점이 하나도 없습니다.
```

표 만들기

김민준	90
이서연	85

김민준 90

이서연 85

이름	국어	영어
김민준	90	85
이서연	75	95

1학기 중간고

사 성적

이름	국어
김민준	90

1학기 성적표

이름	국어	영어
김민준	90	85
이서연	75	95

실행하면 나오는 화면 — 표 만들기

03단원 · 개념 04 — 표 만들기

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

앞 파일에서 “가로줄과 세로줄이 만나는 자리를 읽어야 하면 표” 라고 했습니다. 이런 내용입니다.

	국어	영어
김민준	90	85
이서연	75	95

“이서연의 영어 점수”를 읽으려면 세로로 한 번, 가로로 한 번 찾아야 합니다. 이렇게 두 방향으로 읽는 자료가 표입니다.

표를 만드는 태그는 네 개가 기본입니다.

table	표 전체를 감싸는 상자
tr	가로 한 줄 (table row, 행)
th	제목 칸 (table header, 머리 칸)
td	보통 칸 (table data, 자료 칸)

여기서 이 단원에서 제일 중요한 사고방식이 나옵니다.

표는 칸 단위가 아니라 행(tr) 단위로 씁니다.

우리는 표를 볼 때 “3행 2열”처럼 격자로 봅니다. 그런데 코드는 격자가 아니라 위에서 아래로 흐르는 글입니다. 그래서 한 줄을 통째로 쓰고, 그다음 줄을 쓰고, 이렇게 이어 갑니다.

```
<table>
  <tr>           ← 첫째 줄 시작
    <td>김민준</td>
    <td>90</td>
  </tr>          ← 첫째 줄 끝
  <tr>           ← 둘째 줄 시작
    <td>이서연</td>
    <td>75</td>
  </tr>
</table>
```

세로줄(열)을 만드는 태그는 없습니다. “각 행에 칸을 몇 개 넣었는가”로 열이 저절로 정해집니다. 행마다 칸 개수를 똑같이 맞춰야 표가 반듯하게 나오는 이유가 여기에 있습니다.

이 파일에서는 표를 만듭니다. table, tr, th, td로 뼈대를 세우고, caption으로 제목을 달고, thead·tbody·tfoot으로 세 부분을 나눕니다. 마지막 섹션의 경고 하나는 꼭 읽어 주세요. 표는 화면을 나누는 도구가 아닙니다.

표는 행 단위로 씁니다

섹션 1

table 안에 tr을 쓰고, tr 안에 td를 넣습니다. 세 겹 중첩입니다.

아래 상자 안에 이렇게 썼습니다.

```
<table border="1" id="first-table"> <tr> <td>김민준</td> <td>90</td> </tr> <tr> <td>이서연</td> <td>85</td> </tr> </table>
```

태그가 세 겹으로 들어갑니다. table 안에 tr, tr 안에 td 입니다. 01단원에서 배운 중첩이 여기서 제대로 쓰입니다. 들여쓰기를 단계마다 두 칸씩 해 두면 어느 tr 에 속한 td 인지 눈으로 바로 보입니다.

border="1" 이 눈에 걸릴 것입니다. 여기서 짚고 갑니다.

표는 원래 테두리가 없습니다.

테두리 없이 그러면 칸이 어디서 끊기는지 보이지 않아 구조를 배울 수가 없습니다. 그래서 이 단원에서만 table 에 border="1" 을 붙입니다. 보기 편하려고 잠깐 쓰는 것이고, 진짜 방법은 10단원 CSS 입니다. 10단원에 가면 border 속성 하나로 두께·색·모양을 마음대로 정하게 됩니다. 지금은 "테두리를 켜는 스위치" 정도로만 생각하세요.

border 를 뺀 표도 아래에 나란히 두었습니다. 글자만 늘어서 있습니다. 표가 아닌 것처럼 보이지만, 코드상으로는 위의 표와 똑같은 표입니다.

화면 2행 2열의 작은 표가 보입니다. 칸마다 얇은 네모 테두리가 있고

표 바깥에도 테두리가 한 겹 더 둘러져 있습니다

김민준
90

이서연
85

화면 테두리가 하나도 없이 "김민준 90" "이서연 85" 두 줄만 보입니다.

칸 사이는 아주 좁은 간격으로만 벌어져 있습니다

김민준
90

이서연
85

table 은 내용만큼 넓어집니다. 위 표가 화면 왼쪽에 작게 붙어 있는 것은 그 때문입니다. 폭을 정하는 것도 10단원 CSS 일입니다.

▣ 직접 해보기

1

위 첫 번째 표에 세 번째 행을 만들어 “박지훈” 과 “70” 을 넣어 보세요. <tr> 부터 통째로 한 벌 더 쓰면 됩니다.

th 와 td 의 차이

섹션 2

칸에는 두 종류가 있습니다. 제목 칸은 th, 자료 칸은 td 입니다.

표의 첫 줄에 있는 “이름 · 국어 · 영어” 는 자료가 아니라 자료의 이름입니다. 이런 칸에는 td 대신 th 를 씁니다. 한 글자만 다릅니다.

```
<tr> <th>이름</th><th>국어</th><th>영어</th>
</tr>
<tr> <td>김민준</td><td>90</td><td>85</td>
</tr>
```

th 로 쓰면 화면에서 두 가지가 저절로 바뀝니다.

1 글자가 굵어집니다. 재 보면 굵기 700 (td 는 400)

2 칸 안에서 가운데 정렬됩니다. td 는 왼쪽 정렬

이것도 여러분이 시킨 적이 없습니다. th 에 “여기는 제목” 이라는 뜻이 들어 있어서 브라우저가 그렇게 그린 것입니다.

보기에는 이 두 가지가 전부인 것 같지만 더 중요한 차이가 있습니다. 화면 낭독기는 자료 칸을 읽을 때 그 칸이 속한 제목 칸을 함께 읽어 줍니다. “90” 이 아니라 “국어, 90” 이라고 읽는 식입니다. td 로만 만든 표에서는 제목이 무엇인지 알 수 없어 그냥 숫자만 읽고 지나갑니다.

제목 칸은 맨 윗줄에만 있는 것이 아닙니다. 아래 표에서 이름이 적힌 세로 첫 칸도 “이 줄이 누구의 줄인가” 를 알려 주는 제목입니다. 그래서 세로 방향 제목에도 th 를 씁니다. 섹션 5 에서 이어서 봅니다.

화면 첫 줄의 “이름 국어 영어” 만 굵은 글씨로 칸 가운데에 놓입니다.

아래 두 줄의 글자는 보통 굵기이고 칸 왼쪽에 붙어 있습니다

이름			
국어			
영어			
김민준			
90			
85			

이서연

75

95

th 가 굵고 가운데 정렬로 나오는 것은 브라우저가 정해 둔 기본 모양일 뿐입니다. 09단원에서 CSS 를 배우면 이 모양은 얼마든지 바꿉니다. 모양이 아니라 뜻을 보고 th 를 고르세요.

▣ 직접 해보기

2

위 표에서 <td>김민준</td> 를 <th>김민준</th> 로 바꿔 보세요. 그 칸만 어떻게 달라지는지 보세요.

caption 으로 표에 제목 달기

섹션 3

표가 무엇에 대한 표인지 알려 주는 제목입니다. table 의 첫 자식으로 씁니다.

표만 덩그러니 있으면 “이게 무슨 표지?” 하게 됩니다. 제목을 h3 같은 것으로 표 위에 따로 쓸 수도 있지만, 그러면 표와 제목이 남남입니다. caption 을 쓰면 제목이 표에 딸린 것이 됩니다.

```
<table border="1">
  <caption>1학기 중간고사 성적</caption> ← table 을 연 바로 다음 줄
  <tr>
    ...
```

caption 은 table 안에 들어가지만 tr 안에 들어가는 것이 아닙니다. table 의 첫 자식 자리에 옵니다. 위치를 헷갈리지 마세요.

화면에서는 표 위쪽 가운데에 놓입니다. 재 보면 정렬 값이 가운데 정렬로 되어 있습니다. 다만 가운데의 기준은 화면 전체가 아니라 표의 폭입니다. 표가 좁으면 제목도 그 좁은 폭 안에서 가운데에 놓이므로 왼쪽에 붙어 보입니다.

여기서 처음 보면 당황하는 일이 하나 생깁니다. caption 의 폭이 표의 폭에 묶여 있기 때문에, 제목 글자가 표보다 길면 그 좁은 폭 안에서 줄이 접힙니다. 아래 상자의 표는 칸이 두 개뿐이라 폭이 96px 밖에 안 됩니다. 그래서 “1학기 중간고사 성적” 이라는 제목이 두 줄로 접혀 나옵니다. 제목이 길어서 잘못된 것이 아니라 표가 좁아서 그런 것입니다. 표가 넓어지거나 제목이 짧아지면 저절로 한 줄이 됩니다.

재미있는 사실이 하나 있습니다. caption 을 표 맨 아래에 써도 화면에는 위에 나옵니다. 브라우저가 자리를 옮겨 그리기 때문입니다. 섹션 7 에서 실제로 보여 드립니다. 그래도 코드에서는 항상 맨 위에 쓰세요. 사람이 읽을 때 헷갈립니다.

화면 표 바로 위에 “1학기 중간고사 성적” 이 보입니다.

표 폭이 좁아서 “1학기 중간고” / “사 성적” 두 줄로 접혀 있고, 테두리 없이 표의 폭 안에서 가운데에 놓여 있습니다

1학기 중간고사 성적

이름
국어

김민준
90

▣ 직접 해보기 3

위 caption 의 글자를 국어 점수 로 짧게 바꿔 보세요. 두 줄이던 제목이 몇 줄이 되는지 확인하세요.

thead / tbody / tfoot

섹션 4

표의 머리 · 몸통 · 바닥을 구분해 주는 태그입니다.

표가 길어지면 어디까지가 제목 줄이고 어디부터 자료인지 코드에서 잘 안 보입니다. tr 을 세 무리로 묶어 주는 태그가 있습니다.

thead	제목 줄들이 있는 머리 부분
tbody	실제 자료 줄들이 있는 몸통
tfoot	합계·평균처럼 마무리하는 바닥 줄

쓰는 자리는 table 안, tr 을 감싸는 자리입니다. 한 겹이 더 생깁니다.

```
<table> <caption>...</caption> <thead> <tr> ... </tr> </thead> <tbody> <tr> ...
</tr> <tr> ... </tr> </tbody> <tfoot> <tr> ... </tr> </tfoot>
</table>
```

화면 모양은 이 태그들을 쓰나 안 쓰나 거의 같습니다. 그래도 쓰는 이유가 있습니다.

- 1 코드가 읽기 쉬워집니다. 제목 줄과 자료 줄이 눈으로 갈립니다.
- 2 08단원에서 CSS 선택자를 배우면 “몸통 줄만 골라 꾸미기” 가 쉬워집니다.
- 3 종이로 인쇄할 때 표가 여러 장에 걸치면 **thead** 가 장마다 다시 인쇄됩니다.

여기 재미있는 성질이 하나 더 있습니다. **tfoot** 을 코드에서 **tbody** 보다 위에 써도 화면에는 항상 맨 아래에 나옵니다. 브라우저가 이름을 보고 자리를 정하기 때문입니다. 섹션 7 에서 실제 결과를 보여 드립니다.

마지막으로 눈에 안 보이는 사실을 하나 알려 드립니다. **tbody** 를 안 써도 브라우저가 몰래 하나 만들어서 **tr** 들을 그 안에 넣습니다. 섹션 1 에서 만든 표에도 지금 **tbody** 가 들어 있습니다. 코드에는 없는데 말입니다. 나중에 CSS 로 표 안을 골라 꾸밀 때, 코드에 안 쓴 **tbody** 가 **table** 과 **tr** 사이에 끼어 있다는 것을 알아 두면 “왜 안 골라지지?” 하고 헤매지 않습니다.

화면 제목 "1학기 성적표" 아래에 4행 3열 표가 보입니다.

첫 줄과 각 줄 맨 왼쪽 칸, 그리고 마지막 "평균" 칸이 굵은 글씨입니다

1학기 성적표

이름
국어
영어

김민준
90
85

이서연
75
95

평균
82.5
90

화면 칸 하나짜리 작은 표가 보입니다.

겉모습은 평범하지만 브라우저 안에서는 tbody 가 생겨 있습니다

코드에는 tbody 가 없습니다

▸ 직접 해보기

4

위 "세 부분으로 나눈 표"의 tbody 안에 "박지훈 · 88 · 76" 행을 하나 더 넣어 보세요. </tbody> 앞에 넣어야 합니다.

scope 맞보기

섹션 5

th 에 붙이는 속성입니다. 그 제목 칸이 가로줄 제목인지 세로줄 제목인지 알려 줍니다.

아래 표를 보세요. 제목 칸이 두 방향에 있습니다.

	국어	영어	← 이 줄은 세로 한 칸씩을 설명합니다
김민준	90	85	← "김민준" 은 이 가로 한 줄을 설명합니다
이서연	75	95	

사람은 보면 바로 압니다. 그런데 화면 낭독기는 알 수 없습니다. th 라는 것만 알지, 어느 방향을 설명하는 제목인지 모릅니다. 그것을 적어 주는 속성이 scope 입니다.

scope="col"	세로 한 줄(열)의 제목입니다	col 은 column(열)
scope="row"	가로 한 줄(행)의 제목입니다	row 는 행
<th scope="col">국어</th>	→ 아래로 이어지는 국어 점수들의 제목	
<th scope="row">김민준</th>	→ 오른쪽으로 이어지는 점수들의 제목	

여기서 미리 말해 둘 것이 있습니다.

scope 를 붙여도 화면은 하나도 안 바뀝니다.

붙이기 전과 후를 나란히 놓고 봐도 똑같습니다. 색도 굵기도 자리도 그대로입니다. “안 써도 되는 거 아닌가?” 싶어지는데, 눈에 안 보일 뿐 하는 일이 있습니다. 눈이 불편한 분이 이 표를 들을 때 “김민준, 영어, 85” 처럼 두 제목을 함께 들려줍니다. scope 가 없으면 그냥 “85” 만 듣게 됩니다.

지금 단계에서는 “제목 칸에는 방향을 적어 준다” 정도만 익혀 두면 충분합니다.

화면	3행 3열 표가 보입니다. 첫 줄 세 칸과 왼쪽 첫 칸들이 굵고 가운데 정렬입니다.
----	--

scope 를 붙이기 전과 화면이 완전히 똑같습니다

2학년 3반 점수		
이름		
국어		
영어		
김민준	90	85
이서연		

▣ 직접 해보기 5

위 표에서 `scope="col"` 을 전부 지워 보세요. 저장하고 새로고침해서 화면이 얼마나 바뀌는지 확인한 뒤 되돌려 놓으세요.

표를 화면 배치에 쓰지 마세요

섹션 6

이 섹션은 규칙이 아니라 경고입니다. 꼭 읽어 주세요.

표를 배우고 나면 누구나 같은 생각을 합니다.

"왼쪽에 메뉴, 오른쪽에 본문을 놓고 싶은데, 표로 칸을 두 개 만들면 되겠네?"

실제로 됩니다. 아래 상자에서 결과를 보세요. 화면이 좌우로 나뉩니다. 1990년대 후반부터 2000년대 초반까지 실제로 많은 웹사이트가 이렇게 만들어졌습니다.

그런데 지금은 절대 이렇게 하지 않습니다. 이유가 있습니다.

1) 뜻이 거짓말이 됩니다.

`table` 은 "이것은 두 방향으로 읽는 자료입니다" 라는 뜻입니다.
메뉴와 본문은 자료가 아닙니다. 화면 낭독기는 이 표를 만나면
"표, 1행 2열" 이라고 알린 뒤 칸을 하나씩 읽습니다. 듣는 사람은 혼란스럽습니다.

2) 휴대전화에서 무너집니다.

표의 칸은 가로로 나란히 놓이는 것이 규칙이라 좁은 화면에서도 안 접힙니다.
글자가 짓눌리거나 화면 밖으로 빠져나갑니다.
13단원에서 배울 반응형(화면 크기에 따라 배치를 바꾸는 것) 이 불가능해집니다.

3) 고치기가 어렵습니다.

"메뉴를 오른쪽으로 옮겨 주세요" 같은 요청이 오면 `tr` 과 `td` 를 전부 다시 짜야 합니다.

그럼 좌우 배치는 무엇으로 할까요? 이 자료의 뒷부분이 그 답입니다.

12단원 `flex` 가로·세로로 줄 세우기

13단원 `grid` 격자로 나누기

이름이 `grid`(격자) 라서 표와 비슷해 보이지만 완전히 다릅니다. `grid` 는 CSS 라서 뜻을 건드리지 않고 모양만 바꿉니다. 화면이 좁아지면 접히게 할 수도 있습니다.

정리하면 이렇습니다.

표로 만들 것 : 성적표, 시간표, 가격표, 통계 자료
 표로 만들 것 아님 : 메뉴 배치, 좌우 2단 화면, 사진 여러 장 나열

화면 한 줄짜리 표가 보이고, 왼쪽 좁은 칸에 "메뉴",

오른쪽 넓은 칸에 본문 문장이 들어가 좌우로 나뉘어 보입니다

메뉴
 본문이 여기에 들어갑니다. 표로 화면을 나누면 이렇게 됩니다.

판단이 헛갈릴 때 스스로에게 물어보세요. "이 내용을 종이에 옮겨 적을 때도 표로 그릴까?" 성적표는 표로 그립니다. 메뉴와 본문은 표로 그리지 않습니다.

▸ 직접 해보기 6

위 배치용 표를 table 대신 02단원에서 배운 <p> 두 개로 바꿔 보세요. 좌우로는 안 놓고 위아래로 놓이는 것을 확인하세요. (좌우 배치는 12단원에서 배웁니다)

자주 하는 실수

섹션 7

표는 태그가 여러 겹이라 어긋나기 쉽습니다. 그런데도 에러는 나지 않고 모양만 이상해집니다.

이런 실수를 합니다
행마다 칸 개수가 다름

이런 실수를 합니다

실수 1 — 행마다 칸 개수가 다름

```
<tr>
<th>이름</th>
<th>국어</th>
<th>영어</th>
</tr>
<tr>
<td>김민준</td>
<td>90</td>      ← 칸이 두 개뿐입니다
</tr>
```

이름 국어 영어 김민준 90 모자란 자리가 테두리도 없는 빈칸으로 남습니다. 표의 오른쪽 아래가 땀 뚫려 보입니다. 개수가 모자란 행을 찾아 <td></td> 를 하나 채워 넣으면 반듯해집니다. 다음 파일에서 배울 colspan 을 쓰는 방법도 있습니다.

이런 실수를 합니다

td 를 **tr** 없이 씀

이런 실수를 합니다

실수 2 — td 를 **table** 밖에서 씀

```
<td>표 밖의 칸</td>
```

표 밖의 칸 글자만 남고 칸은 사라집니다. 위에 “표 밖의 칸”이라는 글자가 평범하게 보일 텐데, 브라우저가 td 태그를 통째로 버리고 안의 글자만 남긴 것입니다. li 는 혼자 있어도 점이라도 붙었지만 td 는 table 밖에서는 아예 존재하지 못합니다.

이런 실수를 합니다

caption 을 아래에 씀

이런 실수를 합니다

실수 3 — caption 을 표 맨 아래에 씀

```
<table border="1">
<tr><td>김민준</td></tr>
<caption>맨 아래에 쓴 제목</caption>
</table>
```

이름 점수 김민준 90 맨 아래에 쓴 제목 코드에서는 맨 아래인데 화면에서는 맨 위에 나옵니다. 브라우저가 caption 을 무조건 표 위로 끌어올리기 때문입니다. 결과가 맞게 나오니 실수를 알아챌 방법이 없습니다. 코드를 읽는 사람이 헛갈리니 항상 table 을 연 다음 줄에 쓰세요.

이런 실수를 합니다

tfoot 을 위에 씀

이런 실수를 합니다

실수 4 — tfoot 을 **tbody** 보다 위에 씀

```
<tfoot> ... </tfoot>
<tbody> ... </tbody>
```

합계 165 김민준 90 이서연 75 화면에서는 “합계” 줄이 그대로 맨 아래로 내려갑니다. 브라우저가 태그 이름을 보고 자리를 정하기 때문입니다. 결과가 원하는 대로 나오니 넘어가기 쉽지만, 코드에서는 순서대로 thead → tbody → tfoot 으로 쓰세요.

이런 실수를 합니다

tr 안에 글자를 직접 씀

이런 실수를 합니다

실수 5 — tr 안에 글자를 직접 씀

```
<tr>
  김민준
<td>90</td>
</tr>
```

김민준 90 글자가 표 위쪽 바깥으로 튀어나옵니다. 칸이 아닌 글자는 표 안에 있을 자리가 없어서 브라우저가 표 앞으로 밀어내기 때문입니다. tr 안에는 td 나 th 만 넣으세요.

이런 실수를 합니다

표를 배치에 씀

이런 실수를 합니다

실수 6 — 좌우 배치를 표로 함

```
<table>
<tr> <td>메뉴</td> <td>본문</td>
</tr>
</table>
```

섹션 6 에서 본 그것입니다. 화면은 원하는 대로 나옵니다. 그래서 더 조심해야 합니다. 잘 나오는데 틀린 코드가 제일 고치기 어렵습니다. 좌우 배치는 12단원 flex, 13단원 grid 로 합니다.

정리

▸ 직접 해보기

7

정답

1) 섹션 1 의 첫 표, 마지막 </tr> 아래에 이렇게 넣습니다.

```
<tr> <td>박지훈</td> <td>70</td>
</tr>
```

→ 표가 3행 2열이 됩니다. 열 개수는 그대로인데 세로로 한 줄 늘어납니다.

td 개수를 앞 행과 똑같이 두 개로 맞춘 것이 중요합니다.

2) 섹션 2 의 표에서 김민준 칸을 이렇게 바꿉니다.

```
<th>김민준</th>
```

→ 그 칸의 글자만 굵어지고 칸 가운데로 옮겨 갑니다.

옆의 "90", "85" 는 그대로 보통 굵기에 왼쪽 정렬입니다.
한 칸만 바뀌도 그 칸만 달라집니다.

3) 섹션 3 의 caption 을 이렇게 바꿉니다.

```
<caption>국어 점수</caption>
```

→ 두 줄로 접혀 있던 제목이 한 줄로 줄어듭니다.

caption 의 폭은 표 폭에 맞춰집니다. 그래서 표보다 긴 제목은 접히고,
표 안에 들어갈 만큼 짧은 제목은 한 줄로 나옵니다.
확인했으면 원래 제목으로 되돌려 놓으세요.

4) 섹션 4 의 표, </tbody> 바로 앞에 이렇게 넣습니다.

```
<tr> <th>박지훈</th> <td>88</td> <td>76</td>  
</tr>
```

→ 새 줄이 "이서연" 줄 아래, "평균" 줄 위에 들어갑니다.

tfoot 은 항상 맨 아래에 있으므로 그 사이로 들어갑니다.

5) 섹션 5 의 thead 안 th 세 개에서 scope="col" 을 지웁니다.

```
<th>이름</th>  
<th>국어</th>  
<th>영어</th>
```

→ 화면이 하나도 안 바뀝니다. 글자 굵기도 위치도 그대로입니다.

눈에 보이지 않는 정보를 지운 것이라 그렇습니다. 다시 붙여 두세요.

6) 섹션 6 의 표를 이렇게 바꿉니다.

```
<p>메뉴</p>  
<p>본문이 여기에 들어갑니다.</p>
```

→ 좌우로 나뉘어 있던 것이 위아래 두 줄이 됩니다.

p 는 한 줄을 통째로 차지하는 태그이기 때문입니다.
좌우로 놓는 방법은 12단원에서 배웁니다.

칸 합치기

교시	월	화
1	국어	영어
2	체육	체육

교시	월	화
1	국어	영어
2	체육	대회

교시	월	화
이번 주는 시험 기간입니다		
1	국어	영어

시간	과목
오전	국어
오전	영어

시간	과목
----	----

실행하면 나오는 화면 — 칸 합치기

03단원 · 개념 05 — 칸 합치기

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

앞 파일에서 만든 표는 칸이 반듯한 격자였습니다. 그런데 실제 시간표를 보면 이런 칸이 있습니다.

교시	월	화	
1	국어	영어	
2	← 체육 대회 →	한 칸이 두 칸 자리를 차지합니다	

가로로 두 칸을 하나로 합친 것입니다. 이런 것을 만드는 속성이 두 개 있습니다.

colspan	가로로 몇 칸을 차지할지	col 은 column(열)
rowspan	세로로 몇 줄을 차지할지	row 는 행

둘 다 td 나 th 에 붙이고, 값은 "몇 칸인지" 를 숫자로 씁니다.

```
<td colspan="2">체육 대회</td>
  ↳ 이 칸 하나가 가로로 두 칸 자리를 먹습니다
```

쓰는 법 자체는 간단합니다. 그런데 이 단원에서 학생들이 가장 많이 헤매는 곳이 여깁니다. 이유는 딱 하나입니다.

합친 만큼 그 행의 td 개수를 줄여야 합니다.

두 칸을 하나로 합쳤으면 그 행에는 칸이 하나 줄어야 합니다. 원래 있던 td 를 안 지우면 칸이 넘쳐서 표 오른쪽으로 빠져나갑니다. 이 규칙 하나만 몸에 익히면 나머지는 어렵지 않습니다.

머릿속에 이런 그림을 두고 세면 편합니다.

한 행의 칸 수 = 보통 td 개수 + (colspan 값들의 합)
여기서 colspan 이 붙은 td 는 1개가 아니라 그 값만큼 셉니다

이 파일에서는 colspan 과 rowspan 으로 칸을 합칩니다. 태그는 하나도 새로 배우지 않습니다. 속성 두 개가 전부입니다. 대신 합친 뒤에 칸 개수를 다시 세는 연습을 많이 합니다. 잘못된 표와 올바른 표를 나란히 놓아 두었으니 두 표의 오른쪽 끝을 비교해 보세요.

colspan 으로 가로로 합치기

섹션 1

colspan="2" 를 붙이면 그 칸이 가로로 두 칸 자리를 차지합니다.

먼저 합치지 않은 표를 보고, 그다음 합친 표를 봅니다.

합치기 전

```
<tr><th>교시</th><th>월</th><th>화</th></tr>
<tr><td>1</td><td>국어</td><td>영어</td></tr>
<tr><td>2</td><td>체육</td><td>체육</td></tr>
  ↳ "체육" 이 두 번 나옵니다. 사실은 한 시간짜리 행사인데요.
```

합친 뒤

```
<tr><td>2</td><td colspan="2">체육 대회</td></tr>
  ↳ 두 칸을 하나로 합쳤습니다
```

합친 뒤의 행을 보세요. td 가 세 개에서 두 개로 줄었습니다. "체육" 을 두 번 쓰던 것을 한 번만 쓰게 되었으니 당연합니다. 이 "줄이는 일" 이 이 파일의 핵심입니다. 섹션 2 에서 자세히 봅니다.

값은 2 말고 다른 숫자도 됩니다. colspan="3" 이면 세 칸을 먹습니다. 열이 세 개인 표에서 <td colspan="3"> 을 쓰면 그 행 전체가 한 칸이 됩니다. 표 중간에 소제목 줄을 넣을 때 이 방법을 씁니다.

화면 3행 3열 표가 보입니다. 맨 아랫줄에 "체육" 이 나란히 두 칸에 들어 있고

두 칸 사이에 세로 테두리가 그려져 있습니다

교시		
월		
화		
1		
국어		
영어		
2		
체육		
체육		

화면 맨 아랫줄의 두 칸 사이 세로 테두리가 사라지고

"체육 대회" 가 넓어진 한 칸 안에 들어가 있습니다. 표 전체 폭은 그대로입니다

교시	
월	
화	
1	
국어	
영어	
2	
체육 대회	

화면 가운데 줄에 세로 테두리가 하나도 없는 긴 칸이 생기고

그 안에 안내 문장이 들어가 있습니다

교시
월
화

이번 주는 시험 기간입니다

1
국어
영어

▫ 직접 해보기 1

위 "합치기 전" 표에서 `<td>체육</td>` 두 개를 직접 `<td colspan="2">체육 대회</td>` 한 개로 고쳐 보세요. td 하나를 지워야 한다는 점을 잊지 마세요.

합치면 칸 개수를 줄여야 합니다

섹션 2

이 섹션이 이 파일에서 제일 중요합니다. 안 줄이면 칸이 표 밖으로 빠져나갑니다.

실제로 학생들이 하는 일은 보통 이렇습니다.

원래 코드

```
<tr><td>2</td><td>체육</td><td>영어</td></tr>
```

"월·화를 합쳐야지" 하고 colspan 만 붙임

```
<tr><td>2</td><td colspan="2">체육 대회</td><td>영어</td></tr>
```

↳ 이걸 안 지웠습니다

칸을 세어 보겠습니다. 이 표는 열이 세 개(교시·월·화) 입니다.

```
<td>2</td>          → 1칸
<td colspan="2">체육 대회 → 2칸
<td>영어</td>       → 1칸
```

4칸 ← 세 칸짜리 표에 네 칸을 넣었습니다

한 칸이 남습니다. 브라우저는 이 칸을 버리지 않습니다. 표를 한 칸 더 넓혀서 오른쪽에 붙여 버립니다. 그래서 그 줄만 다른 줄보다 오른쪽으로 튀어나옵니다.

아래에 잘못된 표와 올바른 표를 나란히 두었습니다. 두 표의 오른쪽 끝을 손가락으로 짚어 비교해 보세요. 폭이 다릅니다.

고치는 방법은 간단합니다. 남은 td 를 지우면 됩니다.

<tr><td>2</td><td colspan="2">체육 대회</td></tr>
└ 1 + 2 = 3칸. 딱 맞습니다.

표를 고칠 때마다 이렇게 하세요.

- 1 이 표는 열이 몇 개인가? (제목 줄의 칸 수를 셉니다)
- 2 이 행의 칸 수는? (colspan 붙은 것은 그 값만큼 셉니다)
- 3 두 숫자가 같은가?

이런 실수를 합니다

잘못된 예 — colspan 만 붙이고 td 를 안 지움

열이 세 개(교시 · 월 · 화) 인 표의 마지막 행입니다.
<tr> <td>2</td> <td colspan="2">체육 대회</td> <td>영어</td> ← 이 칸이 남습니다
</tr>
1 + 2 + 1 = 4칸

교시 월 화 1 국어 영어 2 체육 대회 영어 표 전체가 한 칸만큼 넓어지고, 위 두 줄의 오른쪽 끝에 빈 자리가 생깁니다. 제목 줄은 세 칸인데 아랫줄은 네 칸이라 오른쪽이 어긋납니다. 예러는 나지 않습니다. 그냥 이상하게 그려질 뿐입니다.

올바른 예 — 남은 td 를 지움

열이 세 개(교시 · 월 · 화) 인 표의 마지막 행입니다.
<tr> <td>2</td> <td colspan="2">체육 대회</td>
</tr>
1 + 2 = 3칸

▫ 직접 해보기 2

위 잘못된 표에서 남은 <td>영어</td> 를 지워 올바른 표와 같은 폭이 되게 고쳐 보세요.

rowspan 으로 세로로 합치기

섹션 3

rowspan="2" 를 붙이면 그 칸이 아래 줄까지 내려와 두 줄 자리를 차지합니다.

같은 값이 세로로 반복될 때 씁니다.

합치기 전		
시간	과목	
오전	국어	
오전	영어	← "오전" 이 두 번
합친 뒤		
시간	과목	
오전	국어	
	영어	← 위 칸이 아래로 내려와 두 줄을 덮습니다

코드는 이렇습니다.

```
<tr><td rowspan="2">오전</td><td>국어</td></tr>
<tr><td>영어</td></tr>
  ↳ 이 행에는 td 가 하나뿐입니다
```

여기서 헷갈리는 점이 나옵니다. rowspan 은 아래 행에 영향을 줍니다. colspan 은 자기 행 안에서 끝났는데 rowspan 은 다음 행의 왼쪽 첫 자리를 미리 차지해 버립니다. 그래서 다음 행은 td 를 하나 적게 씁니다. 섹션 4 에서 세는 법을 정리합니다.

rowspan 을 쓸 때 자주 나오는 물음이 있습니다. “합쳐진 칸의 글자는 위에 붙나요, 가운데 오나요?” 기본은 세로 가운데입니다. 아래 결과에서 “오전” 이 두 줄 높이의 한가운데에 놓여 있는 것을 확인하세요. 이 위치를 바꾸는 것도 CSS 일입니다.

화면 3행 2열 표에서 왼쪽 칸에 "오전" 이 두 번 적혀 있고

두 칸 사이에 가로 테두리가 그어져 있습니다

시간	
과목	
오전	
국어	
오전	
영어	

화면 왼쪽 "오전" 칸이 아래 두 줄에 걸친 큰 칸 하나가 되고

그 사이 가로 테두리가 사라집니다. “오전” 글자는 그 큰 칸의 세로 가운데에 있습니다

시간
과목

오전
국어

영어

▫ 직접 해보기

3

위 표에 "오후·수학" 과 "오후·과학" 두 줄을 더 만들고, "오후" 도 `rowspan="2"` 로 합쳐 보세요.

rowspan 다음 행에서 칸 세기

섹션 4

`rowspan` 도 마찬가지입니다. 내려온 칸이 자리를 먹으니 그만큼 `td` 를 줄입니다.

섹션 2 는 가로 방향이었고, 이번에는 세로 방향입니다. 원리는 같습니다.

아래 격자를 보세요. `o` 는 칸이 있는 자리, `↓` 는 위에서 내려온 칸이 차지한 자리입니다.

	1열	2열	
1행	o	o	<code>td</code> 두 개를 씁니다
2행	o	o	위 칸이 <code>rowspan="2"</code> 라면
3행	↓	o	1열은 이미 찼으니 <code>td</code> 는 하나만 씁니다

3행에서 `td` 를 두 개 쓰면 어떻게 될까요? 1열 자리는 이미 위에서 내려온 칸이 차지하고 있으니 갈 곳이 없습니다. 브라우저는 열을 하나 더 만들어 오른쪽에 붙입니다. 섹션 2 와 똑같은 일이 벌어집니다.

아래 두 표를 비교해 보세요.

이런 실수를 합니다

잘못된 예 — 다음 행에서 "오전" 을 또 씀

```
<tr>
<td rowspan="2">오전</td>
<td>국어</td>
</tr>
<tr>
<td>오전</td>      ← 위에서 칸이 내려왔는데 또 썼습니다
<td>영어</td>
</tr>
```

시간 과목 오전 국어 오전 영어 표에 없던 세 번째 열이 생기고 맨 아랫줄만 오른쪽으로 빠져나갑니다. "오전" 이 두 군데에 보이는 것도 이상합니다. 합치려던 목적과 정반대가 되었습니다.

올바른 예 — 다음 행은 td 하나만

```
<tr>
<td rowspan="2">오전</td>
<td>국어</td>
</tr>
<tr>
<td>영어</td>      ← td 가 하나뿐입니다
</tr>
```

▹ 직접 해보기 4

위 잘못된 표에서 남는 <td>오전</td> 을 지워 올바른 표와 같은 모양으로 고쳐 보세요.

둘을 같이 쓰기

섹션 5

제목이 두 줄인 표를 만들 때 둘을 함께 씁니다. 실제 시간표가 이런 모양입니다.

만들 표는 이렇게 생겼습니다.

	오전	
교시	월	화
	1	국어 영어

"오전" 이 가로로 두 칸 → colspan="2"

"교시" 가 세로로 두 줄 → rowspan="2"

코드로 옮기면 이렇습니다. 행마다 칸을 세면서 읽어 보세요.

```

<tr> <th rowspan="2">교시</th>          1칸 (아래 줄까지 내려감)
      <th colspan="2">오전</th>        2칸
</tr>                                     합 3칸 ✓

<tr> <th>월</th>                        1칸 (첫 자리는 "교시" 가 차지)
      <th>화</th>                        1칸
</tr>                                     합 2칸 + 내려온 1칸 = 3칸 ✓

<tr> <td>1</td><td>국어</td><td>영어</td>
</tr>                                     합 3칸 ✓

```

둘째 행에 th 가 두 개뿐인 것이 처음에는 이상해 보입니다. 왼쪽 첫 자리는 위에서 "교시" 가 내려와 이미 차지하고 있기 때문입니다. 표를 그릴 때는 항상 격자를 먼저 그려 놓고, 이미 찬 자리에 X 표를 하면서 세면 실수가 없습니다.

화면 표 위에 "주간 시간표" 가 보입니다.

왼쪽 "교시" 칸이 위 두 줄에 걸쳐 있고, 그 오른쪽 위에 "오전" 이 두 칸 폭으로, 그 아래에 "월" 과 "화" 가 한 칸씩 있습니다. 맨 아랫줄의 "체육 대회" 는 두 칸 폭을 차지합니다

주간 시간표

교시
오전

월
화

1
국어
영어

2
체육 대회

칸이 많아질수록 세기가 어려워집니다. 종이에 격자를 먼저 그리고 코드를 쓰세요. 머릿속으로만 세다가 한 칸씩 어긋나는 일이 아주 흔합니다.

▫ 직접 해보기 5

위 표의 <th rowspan="2">교시</th> 를 rowspan="3" 으로 바꿔 보세요. 표가 어떻게 어긋나는지 확인한 뒤 되돌리세요.

자주 하는 실수

섹션 6

표가 어긋나 보이면 거의 다 칸 개수 문제입니다. 세는 것부터 하세요.

이런 실수를 합니다

합치고 td 를 안 지움

이런 실수를 합니다

실수 1 — 합치고 남는 칸을 안 지움

섹션 2 와 섹션 4 에서 본 그것입니다. 가장 많이 나는 실수라 다시 적습니다. 표 한쪽이 튀어나오거나 오른쪽에 빈 자리가 생기면 이것부터 의심하세요. `colspan="2"` 를 붙였으면 그 행의 td 를 하나 지우고, `rowspan="2"` 를 붙였으면 다음 행의 td 를 하나 지웁니다.

이런 실수를 합니다

`colspan="1"` 을 씀

이런 실수를 합니다

실수 2 — `colspan="1"` 을 씀

```
<td colspan="1">국어</td>
```

국어 영어 아무 일도 일어나지 않습니다. 1칸을 차지하라는 뜻인데, 원래 모든 칸이 1칸을 차지합니다. 쓰나 마나 같습니다. "합치려면 2 이상" 이라고 기억하세요. 값이 1 이면 지우는 편이 낫습니다.

이런 실수를 합니다

표 밖으로 넘치게 합침

이런 실수를 합니다

실수 3 — 열 개수보다 큰 값을 씀

```
<tr><th>월</th><th>화</th></tr>
<tr><td colspan="5">체육 대회</td></tr>
```

월 화 체육 대회 열이 두 개인 표에 5칸짜리 칸을 넣었습니다. 경고는 없고, 브라우저가 남은 만큼을 그냥 무시합니다. 결과는 `colspan="2"` 를 쓴 것과 같아집니다. 그래서 "값을 크게 주면 알아서 되겠지" 하고 아무 숫자나 넣는 습관이 생기는데, 다른 사람이 코드를 읽을 때 몇 칸짜리인지 알 수 없게 됩니다. 정확한 값을 쓰세요.

이런 실수를 합니다

합쳐진 자리에 td 를 또 넣음

이런 실수를 합니다

실수 4 — rowspan 이 내려온 자리에 td 를 또 넣음

섹션 4 에서 본 경우입니다. rowspan 은 다음 행에 영향을 줍니다. colspan 은 자기 행에서 끝나기 때문에 세기 쉬운데, rowspan 은 몇 줄 아래까지 내려가는지 눈에 안 보여서 놓치기 쉽습니다. rowspan="3" 이면 아래 두 행이 각각 한 칸씩 줄어야 합니다.

이런 실수를 합니다

값에 따옴표를 안 씀

이런 실수를 합니다

실수 5 — 값을 따옴표 없이 씀

```
<td colspan=2>체육 대회</td>
```

월 화 체육 대회 이건 실제로 잘 동작합니다. 값에 빈칸이 없으면 따옴표를 빼도 브라우저가 알아듣습니다. 그런데 값에 빈칸이 하나라도 들어가는 순간 뒷부분이 잘려 나갑니다. 예외를 외우는 것보다 항상 큰따옴표로 감싸는 것이 안전합니다.

이런 실수를 합니다

눈으로만 세다가 어긋남

이런 실수를 합니다

실수 6 — 칸을 세지 않고 눈대중으로 씀

표가 커지면 머릿속으로 세는 것이 불가능해집니다. 표가 이상하면 코드를 노려보지 말고 칸부터 세세요. 제목 줄의 칸 수를 적어 두고, 행마다 "보통 칸 개수 + colspan 값들 + 위에서 내려온 칸" 을 더해 비교하면 어느 행이 범인인지 한 번에 나옵니다.

정리

▹ 직접 해보기

6

정답

1) 섹션 1의 “합치기 전” 표, 마지막 행을 이렇게 바꿉니다.

```
<tr> <td>2</td> <td colspan="2">체육 대회</td>
</tr>
```

→ 맨 아랫줄 두 칸 사이의 세로 테두리가 사라지고 한 칸이 됩니다.

td 를 세 개에서 두 개로 줄인 것이 핵심입니다.
지우지 않으면 섹션 2의 잘못된 표처럼 됩니다.

2) 섹션 2의 잘못된 표에서 마지막 행 끝의 <td>영어</td> 를 지웁니다.

```
<tr> <td>2</td> <td colspan="2">체육 대회</td>
</tr>
```

→ 표 폭이 줄어들어 올바른 표와 똑같아집니다.

위 두 줄 오른쪽 끝에 있던 빈 자리도 함께 사라집니다.

3) 섹션 3의 표, 마지막 </tr> 아래에 이렇게 넣습니다.

```
<tr> <td rowspan="2">오후</td> <td>수학</td>
</tr>
<tr> <td>과학</td>
</tr>
```

→ 표가 5행이 되고, 왼쪽에 “오전” 칸과 “오후” 칸이 각각 두 줄씩 차지합니다.

두 번째 행에 td 를 하나만 쓴 것을 다시 확인하세요.

4) 섹션 4의 잘못된 표에서 마지막 행의 <td>오전</td> 을 지웁니다.

```
<tr> <td>영어</td>
</tr>
```

→ 세 번째 열이 사라지고 2열 표로 돌아옵니다.

“오전” 은 위에서 내려온 칸 하나만 남습니다.

5) 섹션 5의 표에서 첫 행을 이렇게 바꿉니다.

```
<th rowspan="3">교시</th>
```

→ “교시” 칸이 한 줄 더 내려와 셋째 행의 왼쪽 첫 자리까지 차지합니다.

그래서 셋째 행의 1 · 국어 · 영어 가 통째로 한 칸씩 오른쪽으로 밀리고,
표에 네 번째 열이 생겨 그 줄만 오른쪽으로 튀어나옵니다.
셋째 행은 위에서 내려온 1칸 + td 3칸 = 4칸이 되어
다른 행의 3칸과 안 맞기 때문입니다.
확인했으면 rowspan="2" 로 되돌리세요.

참고로 colspan="2" 를 colspan="3" 으로 바꾸면 아무 일도 안 일어납니다.
섹션 6 실수 3 에서 본 대로, 열 개수보다 큰 colspan 은 무시되기 때문입니다.

[illegible]

- 1 이 파일을 VS Code 로 열고, 동시에 브라우저로도 엽니다.
- 2 각 문제의 점선 상자 안, 코드를 쓰라고 적힌 자리에 코드를 씁니다.
- 3 저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 합니다.
- 4 화면이 문제에 적힌 모양과 같은지 눈으로 확인합니다.

1 ~ 13 은 기본, 14 는 응용, 15 는 도전, 16 은 확인 문제입니다.

229

표를 만드는 문제에서는 구조가 보이도록 `<table border="1">` 로 시작하세요. 테두리를 진짜로 다루는 방법은 10단원 CSS 에서 배웁니다.

답을 다 쓴 뒤 연습문제_정답.html 과 비교하세요.

==

점선 상자 안에 코드를 쓰고 저장한 뒤 새로고침하세요. 각 문제마다 결과가 어떻게 보여야 하는지가 적혀 있습니다. 화면이 그대로 나오면 맞은 것입니다. 1 ~ 13 은 기본, 14 는 응용, 15 는 도전, 16 은 확인 문제입니다.

문제 1 (개념01 섹션1)

ul 과 li 로 소품 준비물 목록을 만드세요. 항목은 "김밥", "물통", "뚝자리" 세 개입니다.

기대 출력 점(•) 이 앞에 붙은 세 줄이 왼쪽에서 한 칸 들어간 자리에 보입니다.
이렇게 되면 틀린 것: 점이 안 붙으면 li 로 안 감쌌거나 ul 밖에 쓴 것입니다.

문제 1 — 준비물 목록 만들기

ul 과 li 로 "김밥 · 물통 · 뚝자리" 목록을 만드세요.

출력 점(•) 이 붙은 세 줄이 보입니다.

문제 2 (개념01 섹션3)

아래 세 항목을 ul 목록으로 만들되, 물건 이름만 strong 으로 굵게 만드세요. 연필 500원 / 지우개 300원 / 공책 1200원

기대 출력 점이 붙은 세 줄이 보이고, 각 줄에서 물건 이름만 굵게 보입니다.
이렇게 되면 틀린 것: 줄 전체가 굵으면 strong 을 li 바깥에 씌운 것입니다.

문제 2 — 항목 안의 한 부분만 굵게

"연필 500원 · 지우개 300원 · 공책 1200원" 목록을 만들고 물건 이름만 strong 으로 감싸세요.

출력 세 줄 모두 점이 붙고, 각 줄에서 이름 부분만 굵습니다.

문제 3 (개념01 섹션4)

아래 구조를 중첩 목록으로 만드세요. 음료

커피
주스

음식

안쪽 ul 을 어디에 넣어야 하는지가 이 문제의 핵심입니다.

기대 출력	"음료" 와 "음식" 앞에는 까만 점(•), "커피" 와 "주스" 앞에는 속이 빈 동그라미(◦) 가 붙고 한 칸 더 안으로 들어갑니다. 이렇게 되면 틀린 것: 네 줄이 전부 같은 자리에서 시작하면 중첩이 안 된 것입니다.
-------	--

문제 3 — 목록 안에 목록 넣기

“음료” 아래에 “커피” 와 “주스” 를 넣고, 그다음에 “음식” 항목이 오는 중첩 목록을 만드세요. 안쪽 ul 을 li 안에 넣어야 합니다.

출력	안쪽 두 줄만 속이 빈 동그라미로 한 칸 더 들어갑니다.
----	---------------------------------

문제 4 (개념02 섹션1)

라면 끓이는 순서를 li 로 만드세요. 물을 끓입니다 / 면을 넣습니다 / 4분 뒤 불을 끕니다 번호는 직접 쓰지 마세요.

기대 출력	"1." "2." "3." 이 앞에 붙은 세 줄이 보입니다. 이렇게 되면 틀린 것: "1. 1. 물을 끓입니다" 처럼 번호가 두 번 나오면 번호를 손으로 쓴 것입니다.
-------	---

문제 4 — 순서 있는 목록 만들기

“물을 끓입니다 · 면을 넣습니다 · 4분 뒤 불을 끕니다” 를 li 로 만드세요. 번호는 직접 쓰지 마세요.

출력	1. 2. 3. 이 붙은 세 줄이 보입니다.
----	--------------------------

문제 5 (개념02 섹션2)

아래 두 내용을 각각 알맞은 목록 태그로 만드세요. (가) 우리 집 냉장고 속 음식: 우유, 계란, 김치 (나) 라면 봉지에 적힌 조리 순서: 물 붓기, 면 넣기, 스프 넣기

기대 출력 (가) 는 점(•) 이 붙은 세 줄, (나) 는 번호가 붙은 세 줄로 보입니다.
이렇게 되면 틀린 것: 둘 다 점이거나 둘 다 번호면 고르는 기준을 다시 보세요.
기준은 "순서를 바꿔도 뜻이 같은가" 입니다.

문제 5 — ul 과 이 고르기

(가) 냉장고 속 음식 "우유 · 계란 · 김치" 와 (나) 조리 순서 "물 붓기 · 면 넣기 · 스프 넣기" 를 각각 알맞은 태그로 만드세요.

출력 하나는 점이 붙고 하나는 번호가 붙습니다.

문제 6 (개념02 섹션3)

아래 세 항목을 대문자 알파벳 번호가 붙는 목록으로 만드세요. 가입 신청 / 서류 제출 / 결과 확인

기대 출력 "A." "B." "C." 가 앞에 붙은 세 줄이 보입니다.
이렇게 되면 틀린 것: 숫자로 나오면 type 값이 잘못된 것입니다.
type="a" 는 소문자입니다. 대소문자를 가립니다.

문제 6 — 번호를 알파벳으로 바꾸기

"가입 신청 · 서류 제출 · 결과 확인" 을 대문자 알파벳 번호가 붙는 목록으로 만드세요.

출력 A. B. C. 가 붙은 세 줄이 보입니다.

문제 7 (개념02 섹션4)

"일곱 번째 순서입니다", "여덟 번째 순서입니다" 두 항목을 7번부터 번호가 붙는 목록으로 만드세요.

기대 출력

"7." "8." 이 앞에 붙은 두 줄이 보입니다.
이렇게 되면 틀린 것: 1. 2. 로 나오면 start 를 li 에 붙였거나,
ol 에 start 대신 value 라고 쓴 것입니다. start 는 ol 에 붙습니다.
(li 에 value="7" 을 붙이면 7. 8. 이 나오기는 하지만,
목록 전체의 시작 번호를 정하는 것은 start 입니다)

문제 7 — 시작 번호 바꾸기

“일곱 번째 순서입니다 · 여덟 번째 순서입니다” 를 7번부터 번호가 붙게 만드세요.

출력

7. 8. 이 붙은 두 줄이 보입니다.

문제 8 (개념03 섹션1)

아래 두 짝을 설명 목록으로 만드세요. 태그 : 꺾쇠로 감싼 이름표입니다 요소 : 여는 태그부터 닫는 태그까지의 덩어리입니다

기대 출력

"태그" 와 "요소" 는 왼쪽 끝에, 두 설명 줄은 한 칸 안으로 들어가
모두 네 줄로 보입니다. 점도 번호도 없습니다.
이렇게 되면 틀린 것: 네 줄이 모두 왼쪽 끝에서 시작하면 설명 줄에
dd 대신 dt 를 쓴 것입니다.

문제 8 — 용어집 만들기

“태그 · 꺾쇠로 감싼 이름표입니다” 와 “요소 · 여는 태그부터 닫는 태그까지의 덩어리입니다” 를 dl 로 만드세요.

출력

설명 줄 두 개만 한 칸 안으로 들어갑니다.

문제 9 (개념03 섹션2)

“말”이라는 용어 하나에 설명 두 개를 다세요. 타고 다니는 동물입니다 / 입으로 하는 이야기입니다

기대 출력

"말" 한 줄 아래에 들여쓰기된 설명 두 줄이 간격 없이 붙어 나옵니다.
이렇게 되면 틀린 것: "말" 이 두 번 나오면 dt 를 두 번 쓴 것입니다.
dt 는 한 번만 쓰고 dd 를 두 개 씩니다.

문제 9 — 용어 하나에 설명 두 개

“말”이라는 dt 하나에 “타고 다니는 동물입니다”와 “입으로 하는 이야기입니다” 두 dd를 다세요.

출력 “말” 한 줄 아래에 들여쓰기된 설명 두 줄이 붙습니다.

문제 10 (개념04 섹션1)

아래 내용을 2행 2열 표로 만드세요. table에 border="1"을 붙이세요. 사과 1000 바나나 1500

기대 출력 칸마다 테두리가 있는 2행 2열 표가 보입니다.
글자는 모두 보통 굵기이고 칸 왼쪽에 붙어 있습니다.
이렇게 되면 틀린 것: 테두리가 없으면 border="1"을 안 붙인 것입니다.
글자가 굵으면 td 대신 th를 쓴 것입니다.

문제 10 — 표 만들기

“사과 1000”과 “바나나 1500”을 2행 2열 표로 만드세요. <table border="1">로 시작하세요.

출력 테두리가 있는 2행 2열 표가 보입니다. 글자는 전부 보통 굵기입니다.

문제 11 (개념04 섹션2)

문제 10의 표 위에 제목 줄을 붙여 3행 2열 표로 만드세요. 제목 줄의 내용은 “이름”과 “가격”이고, 제목 칸에 맞는 태그를 쓰세요.

기대 출력 맨 윗줄의 “이름”과 “가격”만 굵은 글씨로 칸 가운데에 놓입니다.
아래 두 줄은 보통 굵기에 왼쪽 정렬입니다.
이렇게 되면 틀린 것: 맨 윗줄도 왼쪽 정렬에 보통 굵기면 td를 쓴 것입니다.

문제 11 — 제목 칸 넣기

문제 10의 표 맨 위에 “이름 · 가격” 제목 줄을 붙여 3행 2열 표로 만드세요. 제목 칸에는 td가 아닌 태그를 씁니다.

출력 맨 윗줄만 굵고 가운데 정렬로 보입니다.

문제 12 (개념04 섹션3·섹션4)

문제 11의 표에 caption으로 “과일 가격표” 제목을 달고, 제목 줄은 thead로, 나머지 두 줄은 tbody로 감싸세요.

기대 출력

표 위에 “과일 가격표”가 한 줄 보이고, 표 모양은 문제 11과 같습니다. 이렇게 되면 틀린 것: 제목이 표 아래에 보이는 일은 없습니다. caption은 어디에 써도 위에 나오기 때문입니다. 제목이 아래 안 보이면 caption을 table 밖에 쓴 것입니다.

문제 12 — caption과 thead·tbody

문제 11의 표에 caption으로 “과일 가격표” 제목을 달고, 제목 줄은 thead, 나머지 두 줄은 tbody로 감싸세요.

출력

표 위에 제목 한 줄이 더 생깁니다. 표 자체는 문제 11과 같습니다.

문제 13 (개념05 섹션1·섹션2)

아래 표를 만드세요. 열은 세 개(교시·월·화)입니다. 1교시: 월요일 국어, 화요일 영어 2교시: 월요일과 화요일 모두 “봉사 활동” (두 칸을 하나로 합칩니다)

기대 출력

3행 3열 표에서 맨 아랫줄만 오른쪽 두 칸이 하나로 합쳐져 보입니다. 표의 오른쪽 끝은 세 줄 모두 같은 자리에서 끝납니다. 이렇게 되면 틀린 것: 맨 아랫줄만 오른쪽으로 튀어나오면 td를 안 지운 것입니다. 합친 행의 td 개수를 세어 보세요. 1 + 2 = 3 이어야 합니다.

문제 13 — colspan으로 가로로 합치기

제목 줄이 “교시·월·화”인 표를 만드세요. 1교시는 “국어·영어”, 2교시는 두 칸을 합쳐 “봉사 활동”한 칸으로 만드세요.

출력

맨 아랫줄만 오른쪽 두 칸이 하나로 합쳐지고, 오른쪽 끝은 가지런합니다.

문제 14 [응용] (개념05 섹션3·섹션4)

아래 표를 rowspan 으로 만드세요. 열은 두 개(시간·과목) 입니다. 오전 : 국어, 영어 ("오전" 을 세로로 두 줄 합칩니다) 오후 : 수학

두 번째 행에 td 를 몇 개 써야 하는지 세어 보고 쓰세요.

기대 출력 4행 2열 표에서 왼쪽 "오전" 칸이 아래 두 줄에 걸쳐 있고, 그 아래에 "오후 · 수학" 줄이 있습니다.
이렇게 되면 틀린 것: 표에 세 번째 열이 생기면 "오전" 을 두 번 쓴 것입니다.
rowspan 이 걸린 다음 행은 td 를 하나 줄여야 합니다.

문제 14 — [응용] rowspan 으로 세로로 합치기

제목 줄이 "시간 · 과목" 인 표를 만드세요. "오전" 은 rowspan 으로 두 줄을 합쳐 "국어" 와 "영어" 를 덮고, 그 아래에 "오후 · 수학" 줄을 넣으세요.

출력 왼쪽 "오전" 칸이 두 줄에 걸쳐 있고 표는 2열 그대로입니다. 합친 다음 줄에 칸을 몇 개 쓸지 세어 보세요.

문제 15 [도전] (개념05 섹션5)

아래 모양의 표를 만드세요. colspan 과 rowspan 을 둘 다 씁니다.

반	1학기	
	중간	기말
1반	90	95
2반	85	88

행마다 칸 수를 세면서 쓰세요. 모든 행이 3칸이어야 합니다.

기대 출력 4행짜리 표가 나오고, 왼쪽 "반" 칸이 위 두 줄에 걸쳐 있습니다.
"1학기" 는 오른쪽 위에서 두 칸 폭을 차지하고, 그 아래에 "중간" 과 "기말" 이 한 칸씩 있습니다.
이렇게 되면 틀린 것: 표에 네 번째 열이 생기면 둘째 줄에 칸을 하나 더 쓴 것입니다.
둘째 줄의 왼쪽 첫 자리는 "반" 이 이미 차지하고 있습니다.

문제 15 — [도전] 제목이 두 줄인 표

왼쪽에 "반"(세로 두 줄), 오른쪽 위에 "1학기"(가로 두 칸), 그 아래에 "중간" 과 "기말" 이 오는 제목을 만들고, 자료 줄로 "1반 90 95" 와 "2반 85 88" 을 넣으세요.

출력	모든 행이 3칸입니다. 오른쪽 끝이 네 줄 모두 같은 자리에서 끝나면 맞은 것입니다.
----	---

문제 16 [확인] (개념04 섹션2 · 개념05 섹션2)

아래 표는 일부러 틀리게 써 두었습니다. 에러는 나지 않지만 두 곳이 이상합니다. 무엇이 이상한지 찾아 고치세요.

(1) 제목 줄인데 굵지도 않고 가운데 정렬도 아닙니다. 왜일까요? (2) 맨 아랫줄만 오른쪽으로 튀어나옵니다. 왜일까요?

기대 출력	고치고 나면 3행 3열 표가 되고, 맨 윗줄만 굵고 가운데 정렬이며 오른쪽 끝이 세 줄 모두 같아집니다. 이렇게 되면 틀린 것: 오른쪽 끝이 여전히 어긋나면 칸 수를 다시 세어 보세요. 합친 행은 $1 + 2 = 3$ 칸이어야 합니다.
-------	--

문제 16 — [확인] 이상한 표 고치기

아래 표는 에러가 나지 않는데 두 곳이 이상합니다. (1) 맨 윗줄이 제목인데 굵지도 가운데 정렬도 아닙니다. (2) 맨 아랫줄만 오른쪽으로 튀어나옵니다. 무엇 때문인지 찾아 고치세요.

교시	
월	
화	
1	
국어	
영어	
2	
전교 조회	
수학	

고친 표를 아래 상자에 새로 쓰세요. 위 표는 비교용으로 그대로 두세요.

자주 넘어지는 자리

- 1 `colspan` 을 쓰면 그 행의 `td` 를 **하나 줄여야** 한다는 것

링크와 이미지

OPEN 04_링크와_이미지

개념 02 – 경로 배우러 가기

개념 04 – 이미지 배우러 가기

01단원 개념02 – 태그의 생김새

개념 05 를 새 탭에서 열기

01 링크와 a 태그

02 경로

03 페이지 안 이동

04 이미지 img

05 그림과 설명 figure

– 연습문제

링크와 a 태그

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



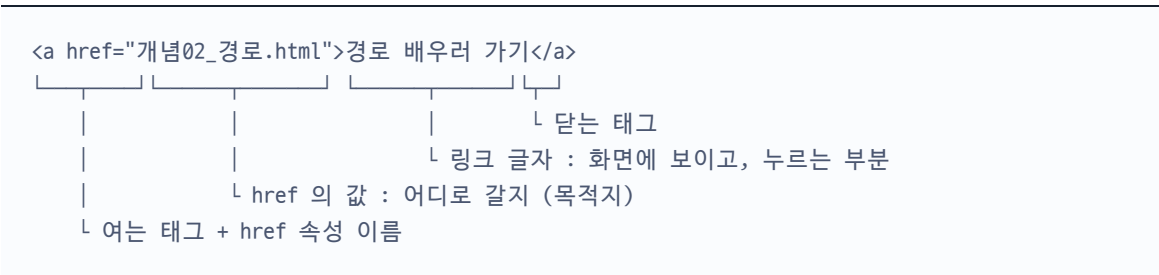
실행하면 나오는 화면 — 링크와 a 태그

지금까지 만든 것은 “한 장짜리 문서” 였습니다. 글자를 쓰고(02단원), 목록과 표로 정리했습니다(03단원). 그런데 웹이 종기와 다른 점은 딱 하나입니다.

글자를 누르면 다른 곳으로 갈 수 있다.

이 “누르면 가는 글자” 를 링크(link) 라고 부릅니다. 웹(web) 이라는 이름 자체가 거미줄이라는 뜻인데, 문서와 문서를 링크로 이어 붙여 그물처럼 만들었기 때문에 붙은 이름입니다.

링크는 a 태그로 만듭니다. a 는 anchor(닻) 의 첫 글자입니다.



여기서 속성(attribute)이라는 말이 다시 나옵니다. 01단원에서 배웠습니다. 속성은 태그에 붙이는 추가 정보이고, 항상 이름=“값” 모양입니다. a 태그에서 href 속성은 없어도 에러가 나지 않지만, 없으면 링크 구실을 전혀 못 합니다. 이 파일 마지막 섹션에서 직접 봅니다.

href 는 hypertext reference(하이퍼텍스트 참조)의 줄임말입니다. “이 글자가 가리키는 곳”이라는 뜻입니다.

이 파일에서는 누르면 다른 문서로 가는 글자를 만듭니다. 아래 상자 안의 파란 밑줄 글자는 전부 진짜로 눌리는 링크입니다. 마음껏 눌러 보고, 브라우저의 뒤로 가기(Alt + ←)로 이 페이지로 돌아오세요.

a 태그와 href

섹션 1

링크는 a 태그로 만들고, 갈 곳은 href 속성에 적습니다.

아래 상자 안에는 이렇게 썼습니다.

```
<p><a href="개념02_경로.html">개념 02 - 경로 배우러 가기</a></p>
<p><a href="개념04_이미지_img.html">개념 04 - 이미지 배우러 가기</a></p>
```

두 파일 다 이 파일과 같은 폴더에 실제로 있습니다. 그래서 파일 이름만 적어도 브라우저가 찾아갑니다. 왜 파일 이름만 적으면 되는지는 다음 파일(개념02)에서 자세히 다룹니다.

a 태그에는 두 부분이 있다는 것을 꼭 구분하세요.

보이는 것	: 여는 태그와 닫는 태그 사이의 글자 (링크 글자)
보이지 않는 것	: href 의 값 (목적지 주소)

학생들이 가장 많이 헷갈리는 것이 이 둘입니다. href 에 적은 파일 이름은 화면에 보이지 않습니다. 화면에 보이는 것은 오직 사이에 낀 글자뿐입니다.

브라우저는 a 요소를 알아서 파란색 + 밑줄로 그립니다. 우리가 색을 지정한 적이 없는데도 그렇습니다. “이건 누를 수 있는 글자다”를 사람이 알아보기 하려고 브라우저가 처음부터 그렇게 정해 둔 것입니다. (07단원에서 이 색을 바꿔 봅니다)

화면 파란 글씨에 밑줄이 그어진 두 줄이 보입니다.

마우스를 올리면 손가락 모양 커서로 바뀝니다

개념 02 - 경로 배우러 가기
개념 04 - 이미지 배우러 가기

링크를 누르면 브라우저 창이 통째로 그 문서로 바뀝니다. 원래 페이지로 돌아오려면 Alt + ← 또는 브라우저 왼쪽 위 뒤로 가기 화살표를 누르세요. 섹션 3에서는 아예 새 탭으로 여는 방법을 배웁니다.

▫ 직접 해보기

1

위 상자 안에 <p>개념 03 으로</p> 를 한 줄 더 넣고, 저장(Ctrl+S) 후 새로고침(F5) 해서 눌러 보세요.

링크 글자에 무엇을 쓸까

섹션 2

“여기를 클릭” 은 쓰지 마세요. 링크 글자만 읽고도 어디로 가는지 알 수 있어야 합니다.

아래 두 링크는 똑같은 파일로 갑니다. 글자만 다릅니다.

```
<a href="../01_HTML_시작하기/개념02_태그의_생김새.html">여기를 클릭</a>  
<a href="../01_HTML_시작하기/개념02_태그의_생김새.html">01단원 개념02 태그의 생김새</a>
```

왜 아래쪽이 좋을까요? 이유가 세 가지 있습니다.

[1] 눈이 안 보이는 사람은 화면 낭독기로 웹을 읽습니다.

01단원 개념03 부터 여러 번 나온, 화면의 글자를 소리로 읽어 주는 프로그램입니다. 영어권 자료나 실무에서는 스크린 리더(screen reader) 라고 부르니 그 이름도 함께 알아 두면 좋습니다. 같은 것입니다. 이 프로그램에는 "페이지의 링크만 쪽 읽어 줘" 라는 기능이 있습니다. 그때 들리는 소리가 이렇습니다.

여기를 클릭, 여기를 클릭, 여기를 클릭 ...

전부 똑같이 들려서 어디로 가는 링크인지 하나도 알 수 없습니다.

[2] 눈이 보이는 사람도 웹페이지를 한 글자씩 읽지 않습니다.

파란 글씨만 눈으로 훑습니다. 그때도 마찬가지입니다.

[3] “클릭” 이라는 말 자체가 마우스를 전제로 합니다.

휴대폰에서는 손가락으로 누르고, 키보드만 쓰는 사람은 Tab 과 Enter 로 갑니다.

이렇게 “누구나 쓸 수 있게 만드는 것” 을 접근성(accessibility) 이라고 부릅니다. 접근성은 특별한 기술이 아니라, 글자를 어떻게 쓰느냐 같은 작은 습관입니다. 이 단원에서 접근성 이야기가 두 번 나옵니다. 여기서 한 번, 개념04 의 alt 에서 한 번입니다.

요령 하나 : 링크 글자만 종이에 옮겨 적어 놓고 읽어 보세요.

그것만 보고 어디로 가는지 모르겠다면 고쳐야 하는 링크입니다.

화면 두 줄 모두 파란 밑줄 링크입니다. 첫 줄은 "여기를 클릭" 네 글자만,

둘째 줄은 "01단원 개념02 — 태그의 생김새" 전체가 파랗습니다

자세한 내용은
여기를 클릭
하세요.

01단원 개념02 - 태그의 생김새

링크 글자는 목적지의 제목을 그대로 쓰는 것이 가장 쉽고 안전합니다. "자세히 보기", "더보기", "바로가기"도 같은 이유로 좋지 않습니다.

◦ 직접 해보기 2

위 상자의 "여기를 클릭" 을 01단원으로 돌아가기 로 바꿔 보세요. 목적지는 그대로 두고 글자만 고치면 됩니다.

새 탭에서 열기

섹션 3

target="_blank" 를 붙이면 새 탭에서 열립니다. 읽던 페이지가 그대로 남습니다.

아래 상자 안에는 이렇게 썼습니다.

```
<a href="개념05_그림과_설명_figure.html" target="_blank" rel="noopener">
  개념 05 를 새 탭에서 열기
</a>
```

target 은 "목표" 라는 뜻이고, 값 _blank 는 "빈 곳" 이라는 뜻입니다. 합치면 "빈 탭에 띄워라" 가 됩니다. 앞의 밑줄(_) 을 빠뜨리면 안 됩니다.

rel="noopener" 는 왜 붙일까요? 새 탭으로 열린 문서는 원래 자기를 열어 준 페이지를 건드릴 수 있습니다. 모르는 사이트로 가는 링크라면 그 사이트가 우리 페이지를 다른 주소로 바꿔치기할 수 있다는 뜻입니다. rel="noopener" 는 그 연결 고리를 끊습니다. 한 줄로 외우면 이렇게 합니다.

target="_blank" 를 쓰면 rel="noopener" 도 같이 쓴다.

언제 새 탭으로 열까요? 기준은 간단합니다.

우리 사이트 안에서 이동 → 그냥 링크 (같은 탭)
바깥 사이트로 나갈 때 → 새 탭
글을 쓰다가 참고 자료를 볼 때 → 새 탭

무조건 새 탭으로 여는 것은 오히려 불편합니다. 탭이 열 개씩 쌓이고, 뒤로 가기 버튼이 먹통이 되기 때문입니다.

마지막 링크는 인터넷 주소(https로 시작하는 주소)를 쓴 예입니다. 이런 주소를 절대경로라고 부르는데, 개념02에서 자세히 봅니다. 인터넷이 연결되어 있지 않으면 열리지 않습니다. 모양만 확인하세요.

화면 파란 밑줄 링크 두 줄입니다. 첫 줄을 누르면 이 탭은 그대로 있고

새 탭이 하나 더 열립니다

개념 05를 새 탭에서 열기

MDN 웹 문서 (인터넷 연결이 필요합니다)

▶ 직접 해보기 3

위 첫 번째 링크에서 `target="_blank"`를 지우고 새로고침한 뒤 눌러 보세요. 이번에는 새 탭이 열리지 않고 이 탭이 바뀝니다. 확인했으면 되돌려 놓으세요.

title로 설명 덧붙이기

섹션 4

title 속성을 붙이면 마우스를 올렸을 때 작은 설명 상자가 뜹니다.

아래 상자 안에는 이렇게 썼습니다.

```
<a href="개념02_경로.html" title="상대경로와 절대경로를 배웁니다">개념 02</a>
```

마우스를 링크 위에 올린 채 1초쯤 기다리면 회색 말풍선이 뜹니다. 이것을 툴팁(tooltip)이라고 부릅니다. 주의할 점이 있습니다.

title은 "덤"입니다. 없어도 되는 정보만 넣으세요.

휴대폰에는 마우스가 없어서 툴팁이 아예 뜨지 않습니다. 키보드로만 이동하는 사람도 보기 어렵습니다. 그래서 꼭 필요한 정보를 title에만 넣으면 그 정보는 사라진 것이나 같습니다. 중요한 말은 title이 아니라 링크 글자에 쓰세요.

head안의 <title> 태그와 이름이 같지만 완전히 다른 것입니다. 헷갈리지 마세요.

head 안의 <title> 태그 → 탭 이름 (문서 전체의 제목)
태그에 붙이는 title 속성 → 마우스를 올렸을 때 뜨는 설명

화면 평소에는 그냥 파란 밑줄 링크입니다. 마우스를 올리고 1 초쯤 기다리면

“상대경로와 절대경로를 배웁니다” 라는 작은 회색 말풍선이 뜹니다

개념 02 - 경로

▫ 직접 해보기 4

위 링크의 title 값을 여러분이 하고 싶은 말로 바꾸고, 마우스를 올려 말풍선이 바뀌는지 확인하세요.

04

download 맛보기

섹션 5

download 를 붙이면 파일이 열리는 대신 내려받아집니다.

아래 상자 안에는 이렇게 썼습니다.

```
<a href="이미지/로고.svg" download>로고 그림 내려받기</a>
<a href="이미지/로고.svg" download="우리로고.svg">이름을 바꿔서 내려받기</a>
```

보통 링크를 누르면 브라우저가 그 파일을 “열어서 보여 줍니다”. download 를 붙이면 대신 “다운로드 폴더에 저장” 합니다.

download 는 값이 없어도 되는 속성입니다. 03단원 개념02 의 reversed 에서 본 불리언 속성이 바로 이런 것입니다. 이름이 있으면 켜진 것이고, 값을 뭐라고 적든 브라우저는 보지 않습니다. 이름만 써 놓으면 “켜짐” 이고, 아예 안 쓰면 “꺼짐” 입니다.

download → 원래 파일 이름 그대로 저장
download="우리로고.svg" → 이 이름으로 바꿔서 저장

주의 : download 는 같은 사이트의 파일에만 확실히 동작합니다.

남의 사이트 파일을 download 로 저장하려고 하면 브라우저가 무시하고 그냥 열어 버립니다. 지금은 “이런 게 있다” 정도만 알면 충분합니다.

아래 링크를 누르면 파일이 화면에 뜨지 않고 브라우저 아래쪽에 내려받기 표시가 나타납니다. 받은 파일은 다운로드 폴더에 들어갑니다.

화면 파란 밑줄 링크 두 줄입니다. 누르면 페이지가 바뀌지 않고

브라우저에 내려받기 표시가 나타납니다

로고 그림 내려받기
이름을 바꿔서 내려받기

▸ 직접 해보기 5

위 첫 번째 링크에서 download 를 지우고 새로고침한 뒤 눌러 보세요. 이번에는 저장되지 않고 그림이 화면에 열립니다. 확인했으면 되돌려 놓으세요.

자주 하는 실수

섹션 6

다시 강조합니다. 아래 실수는 전부 에러가 나지 않습니다. 링크가 조용히 안 될 뿐입니다.

이런 실수를 합니다

href 를 빠뜨림

이런 실수를 합니다

실수 1 — href 를 빠뜨림

```
<a>개념 02 로 가기</a>
```

파란색도 밑줄도 사라지고, 눌러도 아무 일이 없습니다. a 태그는 있지만 갈 곳이 없어서 브라우저가 링크로 취급하지 않습니다. “글자가 검은색 그대로면 href 를 확인하라” 로 기억하세요. 개념 02 로 가기

이렇게 쓰세요

```
<a href="개념02_경로.html">개념 02 로 가기</a>
```

이런 실수를 합니다

파일 이름을 잘못 씀

이런 실수를 합니다

실수 2 — 파일 이름을 잘못 씀

```
<a href="개념2_경로.html">개념 02 로 가기</a>
```

진짜 파일 이름은 개념02_경로.html 인데 개념2 라고 썼습니다. 링크는 파랗게 잘 보입니다. 그런데 누르면 “파일을 찾을 수 없습니다” 라는 하얀 화면이 뜹니다. 글자가 한 자만 달라도 다른 파일입니다. 개념 02 로 가기 (눌러 보세요)

이런 실수를 합니다

따옴표를 안 씀

이런 실수를 합니다

실수 3 — 값을 따옴표로 감싸지 않음

```
<a href=내 파일.html>눌러 보기</a>
```

브라우저는 띄어쓰기가 나오는 순간 값이 끝났다고 판단합니다. 그래서 목적지가 내 가 되어 버리고, 뒤의 파일.html 은 정체 불명의 속성으로 취급됩니다. 값은 항상 따옴표로 감싸세요. 눌러 보기

이런 실수를 합니다

target 값의 밑줄을 빠뜨림

이런 실수를 합니다

실수 4 — target="blank" 라고 씀

```
<a href="개념02_경로.html" target="blank">새 탭에서 열기</a>
```

앞의 밑줄(_) 이 빠졌습니다. 그래도 에러는 나지 않고 새 창이 열립니다. 그런데 이 값은 "blank 라는 이름의 창" 이라는 뜻이라, 다른 링크를 또 누르면 새 탭이 생기지 않고 아까 그 창의 내용만 바뀝니다. 반드시 _blank 로 쓰세요.

이런 실수를 합니다

a 안에 a 를 넣음

이런 실수를 합니다

실수 5 — 링크 안에 링크를 넣음

```
<a href="첫째.html">바깥 <a href="둘째.html">안쪽</a></a>
```

브라우저가 안쪽 a 를 밖으로 끄집어내서 링크 두 개로 쪼개 버립니다. 겉보기에는 멀쩡해 보여서 알아채기 어렵습니다. 링크 안에는 링크를 넣지 마세요. (그림을 링크로 만드는 방법은 개념04 에서 배웁니다)

정리

▫ 직접 해보기

6

정답

1) 섹션 1 의 doc-demo 상자 안, 두 번째 p 아래에 이렇게 넣습니다.

```
<p><a href="개념03_페이지_안_이동.html">개념 03 으로</a></p>
```

→ 상자 안에 파란 밑줄 링크가 한 줄 더 늘어납니다.

누르면 개념03 파일이 열립니다.

2) 섹션 2 의 첫 번째 링크를 이렇게 바꿉니다.

```
<a href="../../01_HTML_시작하기/개념02_태그의_생김새.html">01단원으로 돌아가기</a>
```

→ 목적지는 그대로이고 화면의 글자만 바꿉니다.

링크 글자만 읽어도 어디로 가는지 알 수 있게 되었습니다.

3) 섹션 3 의 첫 링크에서 target="_blank" 를 지우면 이렇게 됩니다.

```
<a href="개념05_그림과_설명_figure.html" rel="noopener">개념 05 를 새 탭에서 열기</a>
```

→ 새 탭이 열리지 않고 이 탭이 개념05 로 바뀝니다.

Alt + ← 로 돌아온 뒤 target="_blank" 를 다시 붙여 두세요.

4) 섹션 4 의 링크에서 title 값만 바꿉니다.

```
<a href="개념02_경로.html" title="오늘 배울 내용입니다">개념 02 - 경로</a>
```

→ 화면의 글자는 그대로이고, 마우스를 올렸을 때 뜨는 말풍선 내용만 바꿉니다.

5) 섹션 5 의 첫 링크에서 download 를 지우면 이렇게 됩니다.

```
<a href="이미지/로고.svg">로고 그림 내려받기</a>
```

→ 누르면 파일이 저장되지 않고, 파란 상자에 HTML 이라고 적힌 로고 그림이

화면 가득 열립니다. 확인했으면 download 를 다시 붙이세요.

경로

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 경로

이 파일이 04단원에서 제일 중요합니다. 그리고 제일 어렵습니다. 처음 배우는 사람이 가장 많이 막히는 곳이 바로 여기입니다.

앞 파일에서 링크를 만들 때 href 에 파일 이름을 적었습니다. 그림을 넣을 때도(개념04) src 에 파일 이름을 적습니다. 그 “적는 자리” 에 들어가는 것이 경로(path) 입니다.

경로 = 파일이 어디에 있는지 알려 주는 길 안내

왜 어려울까요? 사람은 파일을 눈으로 찾지만 브라우저는 그럴 수 없기 때문입니다. 브라우저에게는 딱 하나의 기준밖에 없습니다.

★ 브라우저는 “지금 열려 있는 이 파일이 들어 있는 폴더” 를 기준으로 찾습니다.

이 한 문장이 경로의 전부입니다. 경로가 안 맞을 때는 언제나 “기준이 어디였지?” 를 다시 물으면 됩니다.

지금 우리 자료의 폴더 모양은 이렇습니다.

```
HTMLCSS자료/  
├ 공통.css  
├ 01_HTML_시작하기/  
│   └ 개념02_태그의_생김새.html  
└ 04_링크와_이미지/          ← 지금 이 파일이 있는 폴더  
    ├── 개념01_링크_a태그.html  
    ├── 개념02_경로.html      ← 지금 이 파일  
    ├── 개념03_페이지_안_이동.html  
    ├── 개념04_이미지_img.html  
    ├── 개념05_그림과_설명_figure.html  
    └ 이미지/  
        ├── 사과.svg  
        ├── 고양이.svg  
        ├── 로고.svg  
        ├── 풍경.svg  
        └ 아이콘/  
            └ 별.svg
```

이 그림을 손으로 한 번 옮겨 그려 놓고 이 파일을 읽으세요. 그림 없이 머리로만 하면 반드시 헛갈립니다.

경로는 파일에게 알려 주는 길 안내입니다. 기준은 항상 지금 열려 있는 파일이 들어 있는 폴더 하나뿐입니다.

이 파일 안의 링크와 그림은 전부 진짜로 동작합니다. 눌러 보고 뒤로 가기(Alt + ←) 로 돌아오세요.

폴더 그림부터 보기

섹션 1

경로를 쓰기 전에 폴더 모양을 눈으로 확인하는 습관을 들이세요.

여러분은 사실 첫 단원부터 경로를 쓰고 있었습니다. 이 파일 맨 위 head 안을 보세요.

```
<link rel="stylesheet" href="../../공통.css" />
```

여기 있는 ../../공통.css 가 바로 경로입니다. 이것이 맞기 때문에 지금 이 페이지가 하얀 배경에 예쁘게 보이는 것입니다. 이 경로를 한 글자만 틀리면 글씨체와 색이 전부 사라진 맨 화면이 됩니다.

아래 상자에 우리 자료의 폴더 모양을 그대로 옮겨 놓았습니다. 폴더는 이름 뒤에 빗금(/) 을 붙여 표시했고, 안으로 들어갈수록 오른쪽으로 밀어 썼습니다.

화면 검은 상자 안에 흰 글씨로 폴더 목록이 계단 모양으로 보입니다

```
HTMLCSS자료/  
공통.css  
01_HTML_시작하기/  
개념02_태그의_생김새.html  
04_링크와_이미지/          ← 지금 이 파일이 있는 폴더  
개념01_링크_a태그.html
```

```

개념02_경로.html          ← 지금 이 파일
개념03_페이지_안_이동.html
개념04_이미지_img.html
개념05_그림과_설명_figure.html
이미지/
  사과.svg
  고양이.svg
  로고.svg
  풍경.svg
  아이콘/
    별.svg

```

폴더 안에 폴더가 들어 있을 수 있습니다. 이미지 폴더 안에 아이콘 폴더가 또 들어 있는 것처럼요. 이렇게 몇 겹이든 들어갈 수 있습니다.

▣ 직접 해보기 1

윈도우 탐색기로 04_링크와_이미지 폴더를 열고, 위 그림과 똑같이 생겼는지 눈으로 맞춰 보세요. 이미지 폴더 안까지 들어가 보세요.

같은 폴더에 있는 파일

섹션 2

같은 폴더에 있으면 파일 이름만 쓰면 됩니다. 가장 쉬운 경우입니다.

지금 이 파일은 04_링크와_이미지 폴더 안에 있습니다. 개념01_링크_a태그.html 도 같은 폴더 안에 있습니다. 그래서 이렇게만 쓰면 됩니다.

```
<a href="개념01_링크_a태그.html">개념 01 로 가기</a>
```

폴더 이름을 앞에 붙일 필요가 없습니다. 브라우저는 “지금 이 파일이 있는 폴더” 부터 찾기 시작하기 때문입니다.

생활에 비유하면 이렇습니다. 같은 집에 사는 사람에게 “형 방” 이라고만 말하면 알아듣습니다. “서울시 ○○ 구 ○○동 ○○아파트 101동 502호 형 방” 이라고 말할 필요가 없습니다. 이렇게 “지금 있는 자리 기준으로 말하는 방식” 을 상대경로(relative path) 라고 합니다.

이 단원에서 쓰는 경로는 거의 전부 상대경로입니다.

화면 파란 밑줄 링크 세 줄입니다. 누르면 각각 그 파일로 이동합니다

```

개념 01 - 링크와 a 태그
개념 03 - 페이지 안 이동
04단원 연습문제

```

아래 폴더로 내려가기

사과.svg 는 이미지 폴더 안에 있습니다. 그래서 이렇게 씁니다.

별.svg 는 이미지 폴더 안의 아이콘 폴더 안에 있습니다. 한 겹 더 들어가면 되니까 빗금을 하나 더 씁니다.

3

252

위 폴더로 올라가기

섹션 4

점 두 개와 빗금(../) 은 “한 칸 위 폴더로 나가라” 는 뜻입니다.

지금 이 파일은 04_링크와_이미지 폴더 안에 있습니다. 공통.css 는 그 바깥, 즉 HTMLCSS자료 폴더에 있습니다. 아래로 내려가는 길은 있어도 위로 올라가는 폴더 이름은 없습니다. 그래서 “나간다” 는 뜻의 특별한 표시를 씁니다. 그것이 ../ 입니다.

```
../공통.css
└──┬──┘
  │   └─ 나간 뒤에 찾을 파일
  └─ 04_링크와_이미지 폴더 밖으로 한 칸 나가기 (= HTMLCSS자료 폴더)
```

한 칸 더 나가고 싶으면 ../ 를 하나 더 붙입니다. 개수만큼 나갑니다.

```
../          한 칸 위
../../       두 칸 위
../../       세 칸 위
```

나간 다음에 다시 다른 폴더로 들어갈 수도 있습니다. 아주 자주 쓰는 모양입니다.

```
../01_HTML_시작하기/개념02_태그의_생김새.html
└──┬──┬──┬──┘
  │   │   │   └─ 그 폴더 안의 파일
  │   └─ 다시 들어갈 폴더 이름
  └─ 먼저 04_링크와_이미지 폴더에서 나가기
```

읽는 순서대로 손가락을 움직여 보세요. “밖으로 나간다 → 01단원 폴더로 들어간다 → 그 안의 파일을 연다” 입니다.

아래 링크는 진짜로 01단원 파일을 엽니다. 눌러 보고 Alt + ← 로 돌아오세요.

화면 파란 밑줄 링크 한 줄입니다. 누르면 01단원 파일이 열립니다

01단원 개념02 – 태그의 생김새

이미지/아이콘/별.svg 자리에 있는 파일에서 사과.svg 를 부르려면 ../사과.svg 라고 씁니다. 기준이 바뀌면 경로도 바뀝니다. 경로는 파일마다 다릅니다. 그래서 “이 파일을 부르는 경로” 라는 것은 없고 “어디에서 부르느냐” 를 항상 같이 물어야 합니다.

≡ 직접 해보기 4

이 파일 맨 위 head 안의 href="../공통.css" 에서 ../ 를 지우고 저장한 뒤 새로고침해 보세요. 확인한 다음 반드시 되돌려 놓으세요.

점 하나는 "지금 이 폴더", 맨 앞의 빗금은 "맨 꼭대기부터" 라는 뜻입니다.

[1] ./ 는 "지금 이 폴더" 입니다.

./이미지/사과.svg 와 이미지/사과.svg 는 완전히 같습니다.

./ 는 있으나 없으나 결과가 같습니다. 그래서 보통은 생략합니다.
그래도 배워 두는 이유는, 남의 코드에서 자주 보이기 때문입니다.
"아, 그냥 지금 폴더구나" 하고 읽어 넘길 수 있으면 됩니다.

[2] 맨 앞에 빗금이 오면 뜻이 완전히 달라집니다.

/이미지/사과.svg

이것은 "지금 폴더" 가 아니라 "맨 꼭대기부터" 찾으라는 뜻입니다.
인터넷에 올라간 사이트에서는 그 사이트의 첫 페이지 자리가 꼭대기입니다.
그런데 지금처럼 내 컴퓨터에서 파일을 더블클릭해 열면
꼭대기가 C 드라이브의 맨 위가 됩니다. 그래서 대부분 파일을 못 찾습니다.

내가 쓴 것 : /이미지/사과.svg
브라우저가 찾는 곳 : C 드라이브 맨 위의 이미지 폴더 (그런 폴더는 없습니다)

그래서 이 자료에서는 맨 앞 빗금을 쓰지 않습니다.
수업 중에 "왜 나만 그림이 안 떠요?" 의 절반이 이 실수입니다.

[3] 완전한 인터넷 주소도 경로 자리에 쓸 수 있습니다.

<https://developer.mozilla.org/ko/>

https 로 시작해서 어느 컴퓨터의 어느 파일인지까지 전부 적은 주소입니다.

여기서 두 이름이 나옵니다.

상대경로 : 지금 있는 자리를 기준으로 쓴 경로 (이미지/사과.svg, ../공통.css)
절대경로 : 기준이 필요 없는, 처음부터 끝까지 다 적은 경로
(https://... 또는 맨 앞이 빗금인 /이미지/사과.svg)

무엇을 쓸까요? 기준은 이렇습니다.

내 사이트 안의 파일 → 상대경로
남의 사이트로 나갈 때 → 절대경로 (https 로 시작하는 전체 주소)

아래 상자에서 두 그림은 똑같은 사과입니다. ./ 만 붙었다 없었다 합니다.

화면 똑같이 생긴 작은 사과 그림 두 개가 위아래로 보입니다

화면 파란 밑줄 링크 한 줄입니다. 인터넷이 연결되어 있으면 새 탭에서 열립니다

MDN 웹 문서

▫ 직접 해보기 5

위 첫 상자의 두 번째 그림 경로를 /이미지/사과.svg 로 바꾸고 새로고침해 보세요. 확인했으면 원래대로 이미지/사과.svg 로 되돌려 놓으세요.

이 자리에서 저 파일 부르기

섹션 6

경로는 연습만이 답입니다. 아래 표를 손으로 가리고 하나씩 맞춰 보세요.

푸는 방법은 언제나 같습니다. 세 걸음입니다.

- 1 지금 내가 있는 파일이 어느 폴더 안에 있는지 찾는다
- 2 부르고 싶은 파일까지 폴더 그림 위에서 손가락으로 길을 따라간다
- 3 나갈 때마다 ../ 를, 들어갈 때마다 폴더이름/ 을 쓴다

예를 들어 6번 줄을 풀어 봅시다. 별.svg 는 이미지/아이콘 폴더 안에 있고, 사과.svg 는 이미지 폴더 안에 있습니다.

아이콘 폴더에서 나간다	→ ../
나오면 이미지 폴더 안이다	→ 여기 사과.svg 가 있다
합치면	→ ../사과.svg

7번 줄도 같은 방법입니다.

아이콘 폴더에서 나간다	→ ../
이미지 폴더에서 또 나간다	→ ../../
나오면 04_링크와_이미지 폴더	→ 여기 개념02_경로.html 이 있다
합치면	→ ../../개념02_경로.html

표를 볼 때 “정답을 외우는” 것이 아니라 “손가락으로 따라가는” 연습을 하세요. 폴더 모양이 바뀌면 정답도 통째로 바뀌기 때문에 외우는 것은 소용이 없습니다.

화면 머릿글 세 칸이 굵은 글씨로 있고, 그 아래 일곱 줄짜리 표가 보입니다

지금 있는 파일
부르고 싶은 파일
써야 하는 경로

개념02_경로.html
개념01_링크_a태그.html
개념01_링크_a태그.html

개념02_경로.html
사과.svg
이미지/사과.svg

개념02_경로.html
별.svg
이미지/아이콘/별.svg

개념02_경로.html
공통.css
../공통.css

개념02_경로.html
01단원 개념02
../01_HTML_시작하기/개념02_태그의_생김새.html

아이콘 폴더 안의 파일
사과.svg
../사과.svg

아이콘 폴더 안의 파일
개념02_경로.html
../../개념02_경로.html

표의 마지막 두 줄이 왜 다른지 보세요. 부르는 파일은 같은데 있는 자리가 다르면 경로도 달라집니다. 경로는 파일에 붙어 있는 이름표가 아니라, 두 파일 사이의 길입니다.

▣ 직접 해보기 6

다음 두 줄의 답을 종이에 적어 보세요. (가) 개념04_이미지_img.html 에서 로고.svg 를 부르려면? (나) 아이콘 폴더 안의 파일에서 공통.css 를 부르려면?

자주 하는 실수

섹션 7

경로를 틀려도 에러 메시지는 나오지 않습니다. 그림이 안 나오거나, 눌렀을 때 하얀 화면이 나올 뿐입니다.

이런 실수를 합니다

../ 를 빠뜨림

이런 실수를 합니다

실수 1 — 위로 올라가야 하는데 ../ 를 안 씀

```
<link rel="stylesheet" href="공통.css" />
```

공통.css 는 한 칸 위에 있는데 같은 폴더에서 찾고 있습니다. 브라우저는 못 찾은 채로 그냥 화면을 그립니다. 글씨체와 색, 여백이 전부 사라진 맨 화면이 나옵니다. 에러 메시지는 없습니다. “갑자기 모양이 다 깨졌다” 면 십중팔구 이 실수입니다.

이렇게 쓰세요

```
<link rel="stylesheet" href="../공통.css" />
```

이런 실수를 합니다

대소문자를 다르게 씀

이런 실수를 합니다

실수 2 — 대소문자를 다르게 씀

```

```

진짜 파일 이름은 사과.svg 인데 확장자를 대문자로 썼습니다. 윈도우는 대소문자를 구분하지 않아서 내 컴퓨터에서는 그대로 잘 보입니다. 아래 그림이 멀쩡히 나오는 것이 그 증거입니다. 그런데 인터넷에 올리는 순간 이야기가 달라집니다. 사이트를 올려 두는 컴퓨터는 대부분 대소문자를 그대로 구분해서 그날부터 그림이 깨집니다. “내 컴퓨터에서는 됐는데요” 의 대표 원인입니다. 파일 이름은 탐색기에 보이는 글자 그대로 복사해서 쓰세요.

이런 실수를 합니다

역빗금을 씀

이런 실수를 합니다

실수 3 — 빗금 대신 역빗금(\)

```

```

윈도우 탐색기 주소창이 \ 를 쓰다 보니 그대로 따라 쓰는 실수입니다. 크롬은 이것을 빗금으로 고쳐 읽어 주기 때문에 그림이 그냥 보입니다. 아래 그림이 그렇습니다. 하지만 웹 주소에서 폴더를 나누는 표시는 빗금 (/) 하나뿐이라고 정해져 있습니다. 브라우저가 봐주는 것에 기대지 말고 항상 빗금으로 쓰세요.

이런 실수를 합니다

파일 이름에 띄어쓰기

이런 실수를 합니다

실수 4 — 파일 이름에 띄어쓰기가 있음

```
<a href=이미지/내 사과.svg>사과 보기</a>
```

다음표 없이 쓰면 브라우저는 띄어쓰기에서 값이 끝났다고 봅니다. 목적지가 이미지/내 가 되어 버립니다. 다음표로 감싸면 동작은 하지만, 주소창에서는 띄어쓰기가 %20 이라는 알아보기 어려운 글자로 바뀝니다. 파일 이름에는 띄어쓰기를 넣지 마세요. 대신 밑줄(_) 을 씁니다. 우리 자료의 파일 이름이 전부 개념02_경로.html 처럼 생긴 이유입니다.

이런 실수를 합니다

확장자를 빠뜨림

이런 실수를 합니다

실수 5 — 확장자(.html, .svg) 를 빠뜨림

```
<a href="개념01_링크_a태그">개념 01 로 가기</a>
```

.html 이 빠졌습니다. 링크는 파랑게 잘 보이지만 누르면 파일을 찾을 수 없다는 화면이 뜹니다. 윈도우 탐색기가 확장자를 숨겨 놓아서 이름이 개념01_링크_a태그 인 줄 알고 그대로 옮겨 적는 일이 많습니다. 탐색기의 [보기] 메뉴에서 [파일 확장명] 을 켜 두면 이 실수가 사라집니다.

이런 실수를 합니다

폴더 이름만 쓰고 파일 이름을 안 씀

이런 실수를 합니다

실수 6 — 폴더 이름까지만 씀

```

```

폴더는 그림이 아닙니다. 폴더 안의 어떤 파일인지까지 적어야 합니다. 이미지/사과.svg 처럼요. 이 경우에도 에러는 나지 않고 그림 자리에 alt 에 적은 글자만 남습니다.

정리

▣ 직접 해보기

7

정답

1) 탐색기에서 04_링크와_이미지 폴더를 열면 개념 파일 다섯 개와 연습문제 파일 두 개(연습문제.html, 연습문제_정답.html), 그리고 이미지 폴더가 보입니다. 위 그림에는 지금 이야기하는 파일만 그렸기 때문에 연습문제 두 개가 빠져 있습니다. 이미지 폴더 안에는 svg 파일 네 개와 아이콘 폴더가 있고, 아이콘 폴더 안에는 별.svg 하나가 있습니다.

→ 화면은 바뀌지 않습니다. 폴더 그림과 실제 폴더가 같다는 것을 눈으로

확인하는 것이 목적입니다.

2) 섹션 2 의 doc-demo 상자 안, 마지막 p 아래에 이렇게 넣습니다.

```
<p><a href="개념05_그림과_설명_figure.html">개념 05</a></p>
```

→ 파란 밑줄 링크가 한 줄 더 늘어납니다.

같은 폴더에 있으므로 폴더 이름을 앞에 붙이지 않았습니다.

3) 섹션 3 의 doc-demo 상자 안, 마지막 p 아래에 이렇게 넣습니다.

```
<p></p>
```

→ 사과와 별 아래에 주황색 고양이 얼굴 그림이 한 줄 더 생깁니다.

200 × 200 크기로 보입니다.

4) head 안을 이렇게 바꾸면

```
<link rel="stylesheet" href="공통.css" />
```

→ 화면이 통째로 망가집니다. 노란 안내 상자, 파란 섹션 제목, 검은 코드 상자가

전부 사라지고 글자만 왼쪽에 뽕뽕하게 붙어 나옵니다.
공통.css 를 찾지 못했기 때문입니다. 에러 메시지는 나오지 않습니다.
확인했으면 ../공통.css 로 되돌려 놓으세요.

5) 섹션 5 의 두 번째 그림을 이렇게 바꾸면

```

```

→ 그림이 사라지고 그 자리에 깨진 그림 표시와

"빨간 사과 그림" 이라는 글자만 남습니다.
맨 앞 빗금 때문에 브라우저가 C 드라이브 맨 위에서 찾았기 때문입니다.
확인했으면 이미지/사과.svg 로 되돌려 놓으세요.

6) (가) 개념04_이미지_img.html 은 04_링크와_이미지 폴더 안에 있고

로고.svg 는 그 안의 이미지 폴더에 있습니다. 들어가기만 하면 됩니다.

이미지/로고.svg

(나) 아이콘 폴더에서 나가고(..), 이미지 폴더에서 나가고(..../),

04_링크와_이미지 폴더에서 또 나가면(..../..) HTMLCSS자료 폴더입니다.
거기에 공통.css 가 있습니다.

../../../공통.css

페이지 안 이동

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

창을 작게 만들어 스크롤이 생기게 해 두면 효과를 잘 볼 수 있습니다.



실행하면 나오는 화면 — 페이지 안 이동

지금까지 링크는 항상 “다른 파일” 로 갔습니다. 그런데 링크가 갈 수 있는 곳은 다른 파일만이 아닙니다.

- [1] 같은 문서 안의 다른 자리 ← 이 파일에서 배웁니다
- [2] 메일 쓰기 창
- [3] 전화 걸기 화면

[1] 을 문서 안 이동 또는 앵커 이동이라고 부릅니다. 긴 문서 맨 위에 있는 목차를 눌렀더니 그 부분으로 쓱 내려간 경험이 있을 것입니다. 또 글 맨 아래의 “맨 위로” 버튼도 이것입니다. 새 파일을 여는 것이 아니라 같은 문서 안에서 화면만 움직입니다.

만드는 법은 두 걸음입니다.

- 1 가고 싶은 자리에 이름표를 붙인다 → `id="이름"`
- 2 그 이름 앞에 우물 정(#) 을 붙여 링크한다 → `href="#이름"`

```
<h2 id="menu">메뉴</h2>      ← 도착지에 이름표를 붙이고
<a href="#menu">메뉴 보기</a>  ← 그 이름 앞에 # 을 붙여 부릅니다
```

여기서 id 라는 새 속성이 나옵니다. id 는 identity(신원) 의 줄임말이고, “이 요소의 고유한 이름” 이라는 뜻입니다.

★ id 는 한 문서 안에서 단 하나만 있어야 합니다.

주민등록번호 같은 것이라고 생각하세요. 같은 번호를 가진 사람이 둘이면 안 됩니다. id 는 08단원에서 CSS 로 특정 요소만 꾸밀 때 또 나옵니다. 지금은 “이름표를 붙이는 속성” 으로만 알아 두면 충분합니다.

이 파일의 링크는 다른 파일로 가지 않습니다. 같은 문서 안에서 화면만 움직입니다. 아래 목차를 눌러 보고, 주소창 끝에 # 이 붙는 것도 같이 확인하세요.

id 로 이름표 붙이기

섹션 1

도착하고 싶은 자리에 먼저 id 라는 이름표를 붙입니다.

아래 상자 안에는 이렇게 썼습니다.

```
<p id="noticeBox">여기가 도착지입니다.</p>
```

id 는 어떤 태그에도 붙일 수 있습니다. p 에도, h2 에도, section 에도 붙습니다.

이름 짓는 규칙이 있습니다.

- 1 한 문서 안에서 단 하나여야 합니다
- 2 띄어쓰기를 넣을 수 없습니다
- 3 영어 소문자로 시작하는 것이 안전합니다 (menuList, noticeBox 처럼)
- 4 대소문자를 구분합니다. menuList 와 menulist 는 다른 이름입니다

한글로 id 를 지어도 요즘 브라우저에서는 동작합니다. 하지만 다른 프로그램과 주고받을 때 깨지는 일이 있어서 영어로 씁니다. 우리 자료는 낱말을 이어 쓸 때 두 번째 낱말의 첫 글자를 대문자로 올립니다. 이 방식을 카멜 표기법이라고 부릅니다. 낙타 등처럼 중간이 볼록해서 붙은 이름입니다.

noticeBox menuList pageTop

id 를 붙였다고 화면이 바뀌지는 않습니다. 눈에 보이는 변화는 하나도 없습니다. 아래 상자를 보면 그냥 평범한 문단 두 줄입니다. 이름표는 "나중에 부르기 위한 준비" 일 뿐입니다.

화면 평범한 검은 글씨 두 줄이 보입니다. id 를 붙인 쪽도 겉모습이 같습니다

여기가 도착지입니다. id 는 noticeBox 입니다.
이 줄에는 id 가 없습니다. 두 줄이 똑같이 생겼습니다.

◁ 직접 해보기

1

위 상자의 두 번째 문단에도 id="secondBox" 를 붙여 보세요. 저장하고 새로고침해도 화면이 하나도 안 바뀌는 것을 확인하세요.

으로 그 자리로 점프

섹션 2

href="#이름" 처럼 우물 정(#) 뒤에 id 이름을 적으면 그 자리로 갑니다.

아래 상자 안에는 이렇게 썼습니다.

```
<a href="#recipeStep">만드는 방법 부분으로 가기</a>
```

그리고 이 파일 아래쪽 어딘가에 이런 요소가 있습니다.

```
<h3 id="recipeStep">만드는 방법</h3>
```

누르면 새 페이지가 열리지 않습니다. 화면이 그 자리로 스르륵 움직입니다. 주소창을 보면 파일 이름 뒤에 #recipeStep 이 붙습니다.

```
... /개념03_페이지_안_이동.html#recipeStep
```

이 # 뒤의 부분을 조각(fragment) 이라고 부릅니다. "이 문서의 어느 조각을 보여 달라" 는 뜻입니다.

을 붙이는 자리를 헛갈리지 마세요.

```
도착지 쪽 : id="recipeStep"   ← # 을 붙이지 않습니다
부르는 쪽 : href="#recipeStep" ← # 을 붙입니다
```

이 문서 안 이동은 뒤로 가기도 됩니다. 점프한 다음 Alt + ← 를 누르면 원래 보던 자리로 돌아옵니다.

화면 파란 밑줄 링크 두 줄입니다. 누르면 페이지가 아래쪽으로 움직이고

주소창 끝에 #recipeStep 이 붙습니다

```
만드는 방법 부분으로 가기
잘 만드는 요령 부분으로 가기
```

화면 "만드는 방법" 과 "잘 만드는 요령" 이 굵은 제목으로 보이고

각각 아래에 설명 한 줄씩 있습니다

만드는 방법
재료를 씻고, 썰고, 끓입니다.
잘 만드는 요령
불을 약하게 하고 오래 기다립니다.

지금은 문서가 짧아서 점프해도 화면이 조금밖에 안 움직일 수 있습니다. 브라우저 창의 세로 크기를 줄여 스크롤 막대가 생기게 한 뒤 눌러 보세요.

▣ 직접 해보기 2

섹션 1 에 있는 noticeBox 로 가는 링크를 위 목차 상자에 한 줄 추가해 보세요. <p>도착지로</p>

맨 위로 가기

섹션 3

긴 글 아래쪽에 붙이는 맨 위로 가기 링크도 같은 방법으로 만듭니다.

방법이 두 가지 있습니다.

[1] 문서 맨 위 요소에 id 를 붙이고 그 이름으로 부르기

```
<body id="pageTop"> ← 이 파일이 실제로 이렇게 되어 있습니다
<a href="#pageTop">맨 위로</a>
```

[2] # 하나만 쓰기

```
<a href="#">맨 위로</a>
```

뒤에 아무 이름도 없으면 "이 문서의 맨 위" 라는 약속된 뜻이 됩니다.
짧아서 편하지만, 주소창에 # 만 덜렁 남아서 보기에 좋지 않습니다.
이 자료에서는 [1] 방식을 권합니다.

아래 두 링크는 둘 다 진짜로 동작합니다. 창을 작게 만들어 스크롤이 생긴 상태에서 눌러 보세요.

화면 파란 밑줄 링크 두 줄입니다. 누르면 화면이 문서 맨 위로 올라갑니다

맨 위로 가기
맨 위로 가기 (# 하나만 쓴 방식)

▣ 직접 해보기 3

이 파일 맨 아래 정리 상자 바로 위에 `<p><a href="#pageTop">처음으로</a></p>` 를 넣고 눌러 보세요.

mailto — 메일 보내기

섹션 4

`href="mailto:주소"` 를 쓰면 누를 때 메일 쓰기 창이 열립니다.

아래 상자 안에는 이렇게 썼습니다.

```
<a href="mailto:hong@example.com">도서관에 메일 보내기</a>
```

mailto 뒤에는 반드시 쌍점(:) 이 옵니다. 빗금(/) 은 없습니다. http 처럼 // 를 붙이는 실수를 많이 합니다. mailto 에는 // 가 없습니다.

제목과 내용을 미리 채워 둘 수도 있습니다.

```
mailto:hong@example.com?subject=책 문의&body=안녕하세요
```

```
?subject= 물음표 뒤에 제목
&body=      앰퍼샌드(&) 로 이어 붙여 본문
```

여기서 02단원에서 배운 특수문자가 다시 나옵니다. HTML 안에서 & 는 특별한 뜻을 가진 글자라서 그대로 쓰면 안 됩니다. & 라고 써야 브라우저가 & 하나로 읽습니다.

주의 : 메일 프로그램이 설치되어 있지 않은 컴퓨터에서는 아무 일도 안 일어나거나

"이 링크를 열 앱을 선택하세요" 창이 뜹니다. 여러분 잘못이 아닙니다.
example.com 은 예제용으로 남겨 둔 주소라 실제로 메일이 가지는 않습니다.

화면 파란 밑줄 링크 두 줄입니다. 누르면 컴퓨터에 설치된 메일 프로그램이 열립니다

도서관에 메일 보내기

제목과 내용을 미리 채워서 보내기

메일 주소를 이렇게 페이지에 그대로 적어 두면 광고 메일을 보내는 프로그램이 주소를 긁어 갑니다. 실제 사이트에서는 05단원에서 배울 문의 폼을 쓰는 경우가 더 많습니다. mailto 는 간단한 개인 페이지에서 주로 씁니다.

▣ 직접 해보기 4

위 첫 링크의 메일 주소를 여러분 주소로 바꾸고, 링크 글자도 저에게 메일 보내기 로 바꿔 보세요.

href="tel:번호" 는 휴대폰에서 누르면 바로 전화가 걸리는 링크입니다.

아래 상자 안에는 이렇게 썼습니다.

```
<a href="tel:02-1234-5678">02-1234-5678</a>
```

가게 홈페이지에서 전화번호를 눌렀더니 바로 통화 화면이 뜨는 것이 이것입니다. 휴대폰에서 손가락으로 번호를 다시 옮겨 적지 않아도 되니 아주 편합니다.

번호를 쓸 때 규칙이 있습니다.

숫자와 붙임표(-) 만 씁니다. 괄호나 띄어쓰기는 넣지 않는 것이 안전합니다
tel:021234-5678 처럼 붙여 써도 동작합니다
국제번호까지 쓰려면 tel:+82-2-1234-5678 처럼 씁니다

링크 글자는 전화번호 그대로 쓰는 것이 좋습니다. 컴퓨터로 보는 사람은 눌러도 전화가 안 걸리기 때문에, 번호가 화면에 보여야 손으로 옮겨 적을 수 있습니다.

컴퓨터에서 누르면 대개 아무 일도 안 일어나거나 "이 링크를 열 앱을 선택하세요" 창이 뜹니다. 정상입니다.

화면 "문의 전화: " 뒤에 파란 밑줄로 02-1234-5678 이 보입니다

문의 전화: 02-1234-5678

▹ 직접 해보기

5

위 전화번호를 031-987-6543 으로 바꾸세요. href 안의 번호와 화면에 보이는 글자를 둘 다 고쳐야 합니다.

자주 하는 실수

문서 안 이동은 안 되어도 아무 표시가 없습니다. 눌러도 그냥 가만히 있습니다.

이런 실수를 합니다

#을 빠뜨림

이런 실수를 합니다

실수 1 — href 에서 # 을 빠뜨림

```
<a href="recipeStep">만드는 방법으로</a>
```

이 없으면 브라우저는 이것을 파일 이름으로 읽습니다. 그래서 같은 폴더에서 recipeStep 이라는 파일을 찾다가 실패하고, 파일을 찾을 수 없다는 하얀 화면이 뜹니다. 만드는 방법으로 (눌러 보세요)

이렇게 쓰세요

```
<a href="#recipeStep">만드는 방법으로</a>
```

이런 실수를 합니다

도착지 쪽에도 # 을 붙임

이런 실수를 합니다

실수 2 — id 값에 # 을 붙임

```
<h3 id="#recipeStep">만드는 방법</h3>
```

은 부르는 쪽에만 붙입니다. 이름표 자체에 # 을 넣으면 이름이 #recipeStep 이 되어 버려서 href="#recipeStep" 이 찾는 recipeStep 과 다른 이름이 됩니다. 눌러도 아무 일이 일어나지 않습니다.

이런 실수를 합니다

id 이름이 다름

이런 실수를 합니다

실수 3 — 대소문자가 다름

```
<h3 id="recipeStep"> 인데 <a href="#recipestep"> 이라고 씀
```

S 하나가 소문자로 바뀌었을 뿐인데 완전히 다른 이름입니다. 눌러도 화면이 그대로입니다. 에러도 없고 표시도 없어서 찾기 어렵습니다. 이름표를 만들 때 복사해서 붙여 넣는 습관을 들이면 이 실수가 사라집니다. 만드는 방법으로 (눌러도 안 움직입니다)

이런 실수를 합니다

id 를 두 군데에 씀

이런 실수를 합니다

실수 4 — 같은 id 를 두 군데에 붙임

```
<p id="notice">첫 번째</p>
<p id="notice">두 번째</p>
```

브라우저는 첫 번째 것만 찾아갑니다. 두 번째는 영원히 못 갑니다. 역시 에러는 나지 않습니다. id 는 한 문서에 하나뿐이어야 한다는 규칙을 지키세요. 여러 요소를 한꺼번에 묶고 싶을 때 쓰는 것은 08단원에서 배울 class 입니다.

이런 실수를 합니다

mailto 에 // 를 붙임

이런 실수를 합니다

실수 5 — mailto 뒤에 // 를 붙임

```
<a href="mailto://hong@example.com">메일 보내기</a>
```

https:// 가 눈에 익어서 습관처럼 // 를 붙이는 실수입니다. 메일 창이 열리기는 하는데 받는 사람 칸이 이상하게 채워집니다. mailto: 뒤에는 곧바로 메일 주소만 씁니다.

이런 실수를 합니다

id 에 띄어쓰기를 넣음

이런 실수를 합니다

실수 6 — id 이름에 띄어쓰기를 넣음

```
<h3 id="recipe step">만드는 방법</h3>
```

이름 안에 띄어쓰기가 있으면 href="#recipe step" 으로 부를 수 없습니다. 주소에서 띄어쓰기는 값의 끝으로 취급되기 때문입니다. 눌러도 아무 일이 일어나지 않습니다. 낱말을 이어 붙일 때는 recipeStep 처럼 붙여 쓰세요.

정리

▫ 직접 해보기

6

정답

1) 섹션 1 의 두 번째 문단을 이렇게 바꿉니다.

```
<p id="secondBox">이 줄에는 id 가 없습니다. 두 줄이 똑같이 생겼습니다.</p>
```


→ 화면은 하나도 바뀌지 않습니다. id 는 눈에 보이지 않는 이름표이기 때문입니다.

(글자는 그대로 두고 속성만 붙였습니다)

2) 섹션 2 의 목차 상자 안, 두 번째 p 아래에 이렇게 넣습니다.

```
<p><a href="#noticeBox">도착지로</a></p>
```

→ 파란 밑줄 링크가 한 줄 늘어납니다. 누르면 화면이 위쪽 섹션 1 로 올라가고

주소창 끝에 #noticeBox 가 붙습니다.

3) 정리 상자 바로 위에 이렇게 넣습니다.

```
<p><a href="#pageTop">처음으로</a></p>
```

→ 문서 맨 아래에 파란 밑줄 링크가 생깁니다.

누르면 화면이 문서 맨 위 제목으로 올라갑니다.

4) 섹션 4 의 첫 링크를 이렇게 바꿉니다. (주소는 여러분 것으로)

```
<a href="mailto:mystudent@example.com">저에게 메일 보내기</a>
```

→ 화면의 글자가 “저에게 메일 보내기” 로 바뀝니다.

누르면 받는 사람 칸에 새 주소가 채워진 메일 쓰기 창이 열립니다.

5) 섹션 5 의 링크를 이렇게 바꿉니다.

```
<a id="telLink" href="tel:031-987-6543">031-987-6543</a>
```

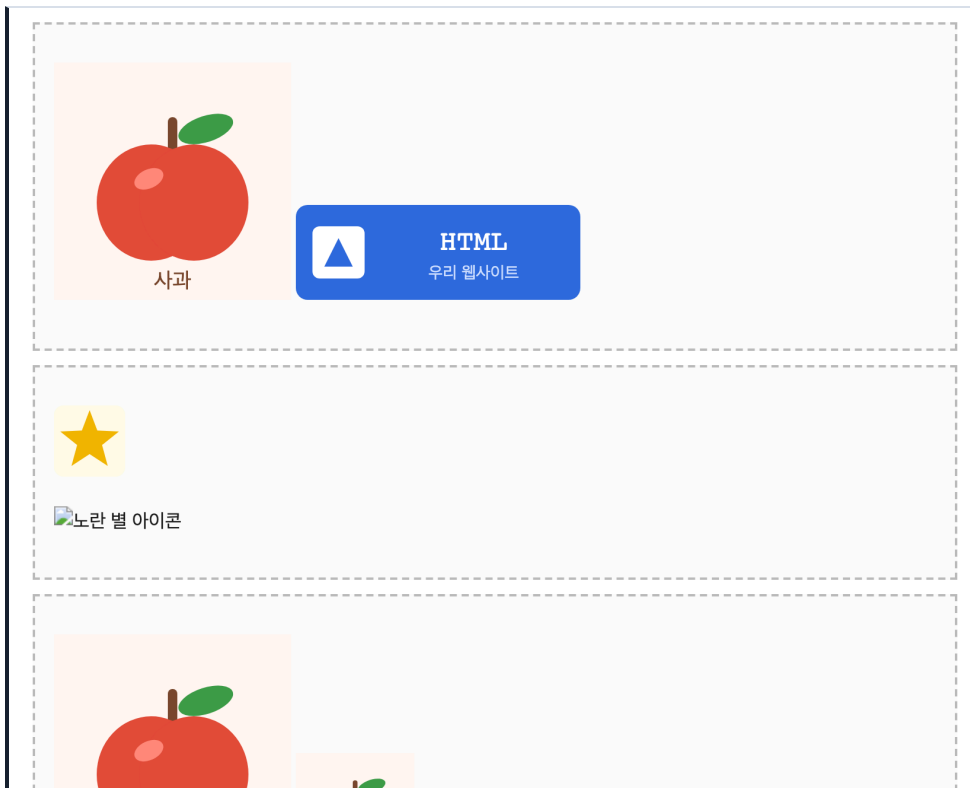
→ 화면에 보이는 번호도 031-987-6543 으로 바꿉니다.

href 만 고치고 글자를 안 고치면, 보이는 번호와 걸리는 번호가 달라집니다.
이것이 실제 가게 홈페이지에서 자주 나는 사고입니다.

이미지 img

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 이미지 img

지금까지는 글자만 다뤘습니다. 이제 그림을 넣습니다.

먼저 아주 중요한 사실 하나를 짚고 갑니다.

★ 그림은 HTML 파일 안에 들어 있지 않습니다.

HTML 파일은 글자로만 된 파일입니다. 그림을 품고 있을 수 없습니다. 대신 “저기 있는 그림을 여기에 걸어 달라”고 부탁만 합니다. 액자와 사진의 관계라고 생각하면 쉽습니다. HTML은 액자이고, 사진은 따로 있는 파일입니다.

```

```

└──┬──────────┬──────────┬──────────┘
| | | |
| | | |└─ 그림을 못 볼 때 대신 읽을 글
| | |└─ 그림 파일이 어디 있는지 (개념02에서 배운 경로)
| |└─ image의 줄임말

그래서 그림이 안 보이는 원인은 거의 항상 둘 중 하나입니다.

1 경로가 틀렸다 (개념02 로 돌아가세요)

2 그림 파일 자체가 그 자리에 없다

img 는 닫는 태그가 없습니다. 01단원 개념02 에서 배운 빈 태그입니다. br 이나 hr 과 같은 종류입니다. "그림을 건다" 는 그 자리에서 일어나는 일이지, 무언가를 감싸는 일이 아니기 때문입니다.

```
<img ... />    ← 이렇게 씁니다
</img>         ← 이런 것은 없습니다
```

이 파일에서는 그림을 페이지에 거는 방법을 배웁니다. 아래에 보이는 그림은 전부 이미지 폴더에 들어 있는 진짜 파일입니다. 인터넷 없이도 보입니다. 그림 하나는 일부러 안 보이게 만들어 두었습니다.

img 태그와 src

섹션 1

그림은 `` 한 줄로 겁니다. 닫는 태그가 없습니다.

아래 상자 안에는 이렇게 썼습니다.

```


```

src 는 source(원본) 의 줄임말입니다. "이 그림의 원본이 어디 있는가" 라는 뜻입니다. src 에 적는 값은 개념02 에서 배운 경로 그대로입니다. 사과.svg 는 이미지 폴더 안에 있으니 이미지/사과.svg 라고 썼습니다.

크기를 정해 주지 않으면 그림은 자기 원래 크기 그대로 나옵니다. 사과.svg 는 원래 200 × 200 이라서 200 × 200 으로 보이고, 로고.svg 는 원래 240 × 80 이라서 240 × 80 으로 보입니다. 크기를 정하는 방법은 섹션 3 에서 배웁니다.

img 는 글자와 나란히 놓입니다. 문단이 새로 시작되지 않습니다. 아래 상자에서 두 그림이 옆으로 나란히 붙어 있는 것이 그 때문입니다. (왜 그런지는 11단원 display 에서 자세히 배웁니다. 지금은 모양만 보세요)

화면 왼쪽에 연분홍 바탕의 빨간 사과 그림, 오른쪽에 파란 바탕에 흰 글씨로

HTML 이라고 적힌 로고가 나란히 보입니다

`<img>`, `<img/>`, `<img />` 는 셋 다 똑같이 동작합니다. 이 자료에서는 빈 태그라는 것이 눈에 잘 띄도록 `<img ... />` 로 통일합니다. 01단원 개념02 의 `<br />` 와 같은 약속입니다.

⇒ 직접 해보기 **1**

위 상자에 풍경 그림을 한 줄 더 넣어 보세요. ``

alt 는 그림을 볼 수 없을 때 대신 읽는 글입니다. 빼먹으면 그림이 아예 없는 것과 같아집니다.

alt 는 alternative text(대체 글) 의 줄임말입니다. "그림 대신 쓸 글" 이라는 뜻입니다. 두 가지 경우에 진짜로 씁니다.

[1] 눈으로 그림을 볼 수 없는 사람이 있습니다.

개념01 에서 이야기한 화면 낭독기가 화면을 소리로 읽어 줍니다.
그림은 소리로 읽을 수 없으니 대신 alt 에 적힌 글을 읽습니다.

alt 를 잘 쓴 경우 → "빨간 사과 그림" 이라고 들립니다
alt 가 없는 경우 → "이미지" 라고만 들리거나 파일 이름이 읽힙니다

사진 열 장이 있는 페이지에서 "이미지, 이미지, 이미지 ..." 만 열 번 들린다고
생각해 보세요. 그 페이지는 그 사람에게 백지나 다름없습니다.

[2] 그림이 안 뜨는 상황이 생각보다 자주 있습니다.

경로를 틀렸을 때, 파일을 지웠을 때, 인터넷이 느릴 때 그렇습니다.
그때 alt 가 있으면 최소한 "여기에 무슨 그림이 있었는지" 는 알 수 있습니다.

아래 두 번째 그림은 일부러 없는 파일을 가리키게 만들었습니다.

```

```

이미지 폴더에 없는그림.svg 라는 파일은 없습니다. 그래서 그림 대신 작은 깨진 그림 표시와 alt 에 적은
글자가 보입니다. 에러 메시지는 어디에도 안 뜹니다. 조용히 글자로 바뀔 뿐입니다.

alt 를 어떻게 쓰면 좋을까요?

좋은 alt : 그림을 못 보는 사람에게 말로 설명하듯이 씁니다
"빨간 사과 그림", "산 두 개와 해가 있는 풍경"
나쁜 alt : "이미지", "사진", "그림1", "사과.svg"
파일 이름을 그대로 옮기는 것은 아무 도움이 안 됩니다

한 가지 예외가 있습니다. 뜻이 없는 장식용 그림입니다. 선을 긋는 무늬나 배경 장식 같은 것은 alt="" 로
비워 둡니다. 비워 두면 화면 낭독기가 그 그림을 아예 건너뜁니다. alt 를 안 쓰는 것과 alt="" 로 비우는 것은
완전히 다릅니다.

alt 를 아예 안 씀 → "여기 뭔가 있는데 뭔지 모르겠다" 가 됩니다
alt="" → "여기는 안 읽어도 되는 장식이다" 가 됩니다

화면 위에는 노란 별 그림이 보입니다. 아래에는 그림 대신 작은 깨진 그림 표시와

“노란 별 아이콘”이라는 검은 글자가 보입니다

아래쪽에 글자가 보인다면 alt 가 제 일을 한 것입니다. 만약 alt 를 안 썼다면 그 자리에는 아주 작은 깨진 그림 표시 하나만 남습니다. 무엇이 있었는지 알 길이 없습니다.

▹ 직접 해보기 2

위 두 번째 그림의 alt 값을 여기에 별 그림이 있어야 합니다 로 바꿔 보세요. 화면의 글자가 그대로 바뀝니다.

width 와 height

섹션 3

04

width 와 height 에 숫자만 적으면 그 픽셀 크기로 보입니다.

아래 상자 안에는 이렇게 썼습니다.

```
           원래 크기 200 x 200
       가로만 지정
  둘 다 지정
```

px 이나 em 같은 단위를 붙이지 않습니다. 숫자만 씁니다. 여기서 숫자의 단위는 픽셀(px) 입니다. 화면을 이루는 아주 작은 점 하나가 1 픽셀입니다.

왜 단위를 못 쓸까요? 브라우저가 이 값을 읽는 방법이 좀 특이하기 때문입니다. 앞쪽 숫자만 읽고 뒤에 붙은 글자는 그냥 버립니다. 그래서 width="100px" 라고 써도 100 으로 읽혀 우연히 잘 동작합니다. 여기에 속아서 "단위를 써도 되는구나" 하고 넘어가면 나중에 크게 당합니다. width="10em" 이라고 쓰면 em 이 버려지고 10 만 남아 10 픽셀짜리 점이 됩니다. 섹션 6 에서 실제 결과를 봅시다.

[1] 하나만 적으면 나머지는 알아서 따라옵니다.

가로를 100 으로 줄이면 세로도 비율에 맞춰 100 이 됩니다.
원래 모양이 그대로 유지됩니다. 이것을 "비율을 지킨다" 고 말합니다.

[2] 둘 다 적으면 그 숫자대로 억지로 늘리거나 눌러 버립니다.

120 x 60 으로 적으면 사과가 옆으로 퍼진 못생긴 모양이 됩니다.
세 번째 그림이 그렇습니다. 일부러 그렇게 만들어 보여 드리는 것입니다.
→ 그래서 보통은 width 하나만 씁니다.

[3] 그런데 처음부터 width 와 height 를 둘 다 적어 두면 좋은 점도 있습니다.

브라우저가 그림을 내려받기 전에 자리를 미리 비워 둘 수 있습니다.
그러면 그림이 늦게 뜨더라도 글이 아래로 밀리며 출렁이지 않습니다.
단, 그때는 원래 그림의 비율과 똑같은 숫자를 적어야 합니다.
(200 × 200 짜리라면 100 × 100, 50 × 50 처럼)

크기를 CSS 로 정하는 방법은 10단원에서 배웁니다. 실제 현장에서는 CSS 쪽을 더 많이 씁니다. 지금은 HTML 만으로 크기를 다루는 방법을 익혀 두세요.

화면 큰 사과, 그보다 작은 사과, 그리고 옆으로 납작하게 눌린 사과가

왼쪽부터 차례로 나란히 보입니다

원래보다 크게 늘리면 흐려집니다. 200 짜리 그림을 800 으로 늘리면 점이 커져서 뭉개져 보입니다. 큰 그림이 필요하면 처음부터 큰 파일을 준비하세요. (지금 쓰는 .svg 는 예외입니다. 이유는 섹션 5 에서 봅니다)

▹ 직접 해보기 3

위 세 번째 그림의 height="60" 을 지우고 새로고침해 보세요. 납작하던 사과가 다시 동그해집니다.

그림을 링크로 만들기

섹션 4

a 태그로 img 를 감싸면 그림 전체가 링크가 됩니다.

아래 상자 안에는 이렇게 썼습니다.

```
<a href="개념02_경로.html">   
</a>
```

01단원에서 배운 중첩입니다. a 가 부모, img 가 자식입니다. 홈페이지 왼쪽 위 로고를 누르면 첫 화면으로 가는 것이 바로 이 방법입니다.

여기서 alt 를 쓰는 요령이 달라집니다. 아주 중요한 부분입니다.

그냥 그림일 때 → alt 에는 "그림이 무엇인지" 를 씁니다 "우리 웹사이트 로고"
링크가 된 그림 → alt 에는 "누르면 어디로 가는지" 를 씁니다 "개념 02 경로 배우러 가기"

왜 그럴까요? 링크 안에 글자가 하나도 없고 그림만 있기 때문입니다. 화면 낭독기는 링크 글자 대신 alt 를 읽습니다. 그러니 alt 가 곧 링크 글자 노릇을 합니다. 여기에 "로고" 라고만 적어 두면, 개념01 에서 배운 "여기를 클릭" 과 똑같은 문제가 됩니다.

옛날 브라우저는 링크가 된 그림에 파란 테두리를 돌려 그렸습니다. 요즘 브라우저는 그리지 않습니다. 그래서 겉보기에는 그냥 그림 같습니다. 마우스를 올려 손가락 커서로 바뀌는지 보면 링크인지 알 수 있습니다.

화면 파란 로고 그림이 하나 보입니다. 테두리는 없고, 마우스를 올리면

손가락 모양 커서로 바뀝니다. 누르면 개념02 파일이 열립니다

▸ 직접 해보기 4

위 링크의 목적지를 개념05_그림과_설명_figure.html 로 바꾸고, alt 도 개념 05 그림과 설명 배우러 가기 로 같이 고치세요.

그림 파일의 종류

섹션 5

img 로 걸 수 있는 그림 파일은 여러 가지입니다. 쓰임이 조금씩 다릅니다.

확장자만 보고도 어떤 그림인지 짐작할 수 있으면 좋습니다. 지금 단계에서는 아래 표를 읽어 보는 정도면 충분합니다.

우리 자료가 전부 .svg 를 쓰는 이유가 여기 있습니다. svg 는 사진이 아니라 “도형을 그리는 방법” 을 글자로 적어 둔 파일입니다. 원을 그리라, 네모를 그리라 하는 지시가 적혀 있고 브라우저가 그때그때 그립니다. 그래서 아무리 크게 늘려도 흐려지지 않습니다. 메모장으로 열어 보면 우리가 쓰는 HTML 과 비슷하게 생긴 글자가 보입니다.

사진에는 svg 를 쓸 수 없습니다. 사진은 점 하나하나의 색을 저장해야 하기 때문에 jpg 나 png 를 씁니다. 도형과 글자로 된 로고나 아이콘에는 svg 가 알맞습니다.

화면 머릿글 세 칸이 짧은 글씨로 있고 그 아래 네 줄짜리 표가 보입니다

확장자
특징
주로 쓰는 곳

.jpg
색이 많은 그림을 작게 저장합니다. 배경을 비울 수 없습니다.
사진

.png
배경을 비워 둘 수 있고 글자가 또렷합니다. 파일이 큼니다.
로고, 화면 갈무리

.gif
움직이는 그림을 만들 수 있습니다. 색을 많이 못 씁니다.
짧은 움직임

.svg

도형 그리는 방법을 글자로 적어 둡니다. 늘려도 안 흐려집니다.
아이콘, 로고

화면 작은 별과 네 배로 큰 별이 나란히 보입니다.

큰 쪽도 가장자리가 매끈합니다

▹ 직접 해보기 5

이미지 폴더의 고양이.svg 를 마우스 오른쪽 버튼으로 눌러 메모장으로 열어 보세요. 그림이 아니라 글자가 들어 있습니다. circle, polygon 같은 낱말을 찾아보세요.

자주 하는 실수

섹션 6

그림이 안 나올 때 브라우저는 아무 말도 해 주지 않습니다. 빈자리만 남을 뿐입니다.

이런 실수를 합니다

alt 를 안 씀

이런 실수를 합니다

실수 1 — alt 를 안 씀

```

```

그림이 잘 보일 때는 아무 문제가 없어 보입니다. 그래서 계속 빠뜨립니다. 문제는 그림이 안 뜨는 날에 드러납니다. 그때 그 자리에는 가로세로 16 정도의 아주 작은 깨진 그림 표시 하나만 남습니다. 눈이 안 보이는 사람에게는 처음부터 아무것도 없는 것과 같습니다. alt 는 선택이 아니라 필수라고 생각하세요.

이렇게 쓰세요

```

```

이런 실수를 합니다

닫는 태그를 씀

이런 실수를 합니다

실수 2 — img 를 닫으려고 함

```
</img>
```

img 는 빈 태그라서 닫는 태그가 없습니다. 그림은 정상으로 보입니다. 브라우저가 그런 것은 없으며 조용히 버리기 때문입니다. 동작은 하지만 규칙에 없는 표기라 쓰지 마세요. 01단원 개념02 의
 를 떠올리면 됩니다.

이런 실수를 합니다

단위를 붙임

이런 실수를 합니다

실수 3 — 크기 값에 단위를 붙임

```

```

HTML 속성으로 크기를 줄 때는 숫자만 씁니다. 브라우저는 앞쪽 숫자만 읽고 뒤에 붙은 글자를 버립니다. 그래서 width="100px" 는 100 으로 읽혀 우연히 잘 동작합니다. 여기에 속아 단위를 써도 되는 줄 알고 10em 이라고 적으면 10 만 남아 10 픽셀짜리 점처럼 작아집니다. 아래가 그 결과입니다. (CSS 에서는 반대로 단위를 꼭 붙여야 합니다. 10단원에서 배웁니다)

이렇게 쓰세요

```

```

이런 실수를 합니다

alt 에 파일 이름을 씀

이런 실수를 합니다

실수 4 — alt 에 파일 이름이나 "이미지" 라고 씀

```

```

화면상으로는 아무 차이가 없습니다. 그래서 실수라는 것도 모릅니다. 하지만 소리로 들으면 "풍경 점 에스브이지" 라고 읽힙니다. alt 는 파일 이름을 옮겨 적는 칸이 아니라, 그림을 못 보는 사람에게 그림을 설명해 주는 칸입니다.

이런 실수를 합니다

그림 파일을 옮기고 경로를 안 고침

이런 실수를 합니다

실수 5 — 그림 파일만 다른 폴더로 옮김

이미지/사과.svg 를 다른 폴더로 옮긴 뒤 HTML 은 그대로 둠

HTML 안에는 그림이 들어 있지 않고 주소만 적혀 있다는 것을 잊었을 때 납니다. 파일을 옮기면 주소가 달라지므로 src 도 같이 고쳐야 합니다. 어제까지 잘 보이던 그림이 오늘 갑자기 안 보인다면 이것부터 확인하세요.

정리

▸ 직접 해보기 6

정답

1) 섹션 1 의 doc-demo 상자 안, 두 그림 뒤에 이렇게 넣습니다.

```

```

→ 사과와 로고 오른쪽에 하늘색 배경의 산 그림이 붙습니다.

세 그림이 $200 + 240 + 320 = 760$ 픽셀이라 상자 안쪽 폭(824) 에 딱 들어가서 한 줄에 나란히 놓입니다. 키가 다른데도 아랫변이 가지런한 것을 보세요. img 는 글자처럼 취급되어 글자 밑줄에 발을 맞추기 때문입니다. 브라우저 창을 좁게 줄이면 자리가 모자라 산 그림이 아랫줄로 내려갑니다.

2) 섹션 2 의 두 번째 img 를 이렇게 바꿉니다.

```

```

→ 깨진 그림 표시 옆의 글자가 “여기에 별 그림이 있어야 합니다” 로 바뀝니다.

그림은 여전히 안 뜹니다. alt 만 바뀌었기 때문입니다.

3) 섹션 3 의 세 번째 img 에서 height="60" 을 지웁니다.

```

```

→ 납작하던 사과가 동그랗게 돌아오고 세로도 120 이 됩니다.

가로만 정하면 세로는 원래 비율대로 따라오기 때문입니다.

4) 섹션 4 의 a 와 img 를 이렇게 바꿉니다.

```
<a href="개념05_그림과_설명_figure.html"> 
</a>
```

→ 화면은 그대로 로고 그림입니다. 누르면 개념05 파일이 열립니다.

alt 도 같이 고쳐야 소리로 듣는 사람도 어디로 가는지 알 수 있습니다.

5) 고양이.svg 를 메모장으로 열면 이런 글자가 보입니다.

```
<circle cx="100" cy="104" r="52" fill="#f0a860" />
<polygon points="56,80 68,30 98,62" fill="#f0a860" />
```

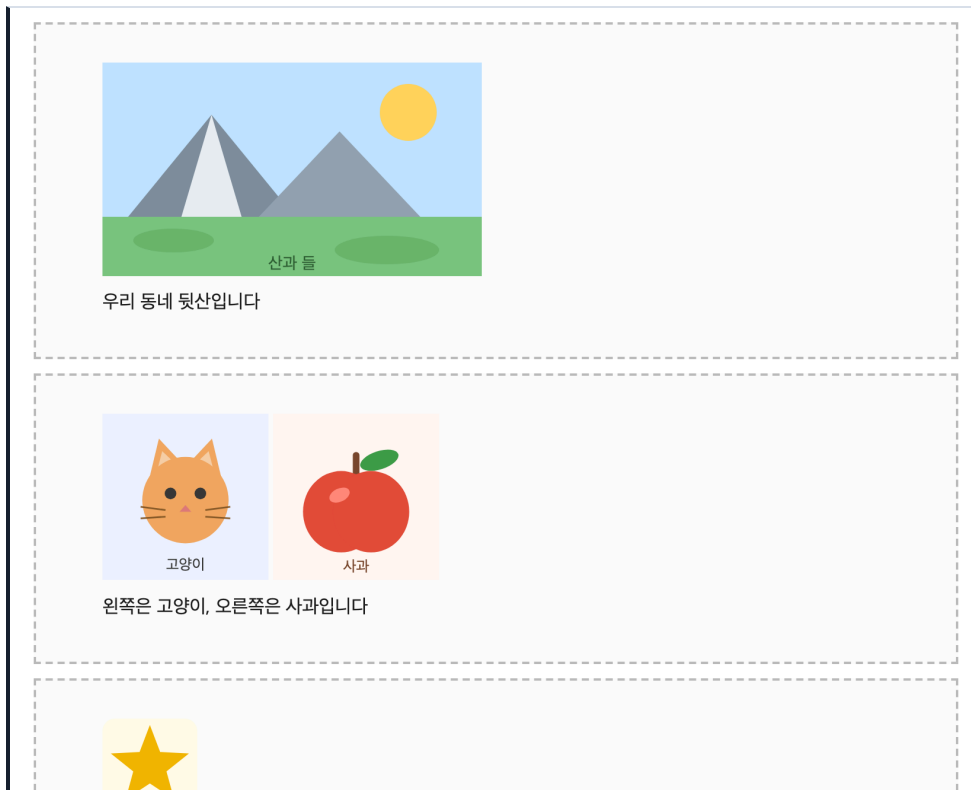
→ 화면은 바뀌지 않습니다. svg 가 "점의 모음" 이 아니라

"그리는 방법을 적어 둔 글" 이라는 것을 눈으로 확인하는 것이 목적입니다.

그림과 설명 figure

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 그림과 설명 figure

앞 파일에서 그림을 거는 방법을 배웠습니다. 그런데 실제 문서에서 그림은 혼자 있는 일이 드뭅니다. 그림 아래에 이런 글이 붙어 있는 것을 많이 보셨을 것입니다.

(그림 3) 2024년 월별 판매량
▲ 지난 봄 학교 앞 벚꽃

이렇게 그림에 붙는 설명글을 캡션(caption) 이라고 부릅니다. 캡션을 그냥 p 로 써도 화면에는 비슷하게 보입니다. 그런데 문제가 있습니다.

```

<p>우리 동네 뒷산입니다</p>
```

이렇게 쓰면 브라우저에게는 “그림 하나” 와 “문단 하나” 가 따로 있는 것으로 보입니다. 둘이 한 짝이라는 사실이 코드 어디에도 적혀 있지 않습니다.

그래서 HTML 은 이 두 가지를 준비해 두었습니다.

```
figure      그림과 그 설명을 한 덩어리로 묶는 상자
figcaption  그 덩어리의 설명글 (figure caption 의 줄임말)

<figure>
  
  <figcaption>우리 동네 뒷산입니다</figcaption>
</figure>
└─ 이 안의 그림과 글이 한 짝이라는 뜻입니다
```

이렇게 “이게 무엇인지” 를 태그 이름으로 알려 주는 것을 의미를 담은 태그라고 부릅니다. 06단원에서 본격적으로 배웁니다. 지금은 “묶어 주면 좋은 점이 있다” 정도로 충분합니다.

이 파일에서는 그림과 설명을 한 짝으로 묶는 방법을 배웁니다. 뒤쪽에서는 소리와 영상, 다른 페이지 끼워 넣기를 생김새만 구경합니다. 실제 소리·영상 파일은 이 폴더에 없으므로 코드만 읽어 보세요.

figure 와 figcaption

섹션 1

figure 로 묶고, 설명은 figcaption 에 씁니다.

아래 상자 안에는 이렇게 썼습니다.

```
<figure>  <figcaption>우리 동네 뒷산입니다</figcaption>
</figure>
```

화면을 보면 두 가지 변화가 눈에 띕니다.

1 그림과 글이 통째로 오른쪽으로 조금 들어갔습니다

2 위아래로 여백이 생겼습니다

브라우저가 figure 에 좌우 40 픽셀씩 여백을 미리 넣어 두었기 때문입니다. 본문에서 살짝 떼어 놓아 “여기는 결들인 자료” 라는 느낌을 줍니다. 마음에 안 들면 10단원에서 CSS 로 없앨 수 있습니다. 지금은 브라우저가 알아서 넣은 것이라는 사실만 알아 두세요.

figcaption 은 figure 안에서만 쓸 수 있습니다. 바깥에 혼자 쓰면 아무 뜻도 없는 글자 덩어리가 됩니다.

그리고 figure 는 그림 전용이 아닙니다. 표, 코드 조각, 인용문처럼 “본문에서 잠깐 떼어 놓고 보는 것” 이면 무엇이든 담습니다. 03단원에서 배운 table 을 figure 로 감싸고 figcaption 에 표 제목을 다는 식입니다.

화면 하늘색 배경에 산이 그려진 그림이 있고, 바로 아래에 검은 글씨로

“우리 동네 뒷산입니다” 가 보입니다. 둘 다 왼쪽에서 조금 안으로 들어가 있습니다

우리 동네 뒷산입니다

figure 를 쓴다고 글씨가 작아지거나 가운데로 모아지는 않습니다. 브라우저가 넣어 주는 것은 좌우 여백뿐입니다. 나머지 꾸밈은 CSS 의 몫입니다.

▫ 직접 해보기 1

위 figcaption 의 글을 2026년 봄에 찍은 사진입니다 로 바꿔 보세요.

figure 안에 여러 개 넣기

섹션 2

그림 여러 장을 한 짝으로 묶고 설명은 하나만 달 수도 있습니다.

아래 상자 안에는 이렇게 썼습니다.

```
<figure>   <figcaption>왼쪽은
고양이, 오른쪽은 사과입니다</figcaption>
</figure>
```

규칙은 두 가지뿐입니다.

1 figcaption 은 figure 하나에 최대 한 개입니다

2 figcaption 은 figure 안에서 맨 앞이나 맨 뒤에 놓습니다

맨 앞에 놓으면 그림 위에, 맨 뒤에 놓으면 그림 아래에 설명이 보입니다. 가운데에 끼워 넣으면 안 됩니다. 브라우저가 그리기는 하지만 규칙에 어긋납니다.

```
<figure>
  <figcaption>설명이 위에 옵니다</figcaption> ← 맨 앞
  <img ... />
</figure>
```

그림이 여러 장일 때 alt 는 어떻게 쓸까요? alt 는 그림 하나하나에 붙는 것이라 각각 따로 씁니다.

figcaption 은 묶음 전체에 하나만 붙습니다. 헛갈리지 마세요.

alt : 그림 한 장마다 하나씩
figcaption : figure 한 덩어리에 하나만

화면 고양이 얼굴과 사과가 나란히 있고, 그 아래에

“왼쪽은 고양이, 오른쪽은 사과입니다” 가 한 줄 보입니다

왼쪽은 고양이, 오른쪽은 사과입니다

▣ 직접 해보기 2

위 figcaption 을 두 그림보다 위쪽으로 옮겨 보세요. 코드에서 줄 순서만 바꾸면 됩니다.

alt 와 figcaption 은 다릅니다

섹션 3

둘 다 설명이지만 읽는 사람이 다릅니다. 같은 말을 두 번 쓰지 마세요.

아주 많이 헛갈리는 부분이라 표로 정리합니다.

alt	화면에 보이지 않습니다. 그림을 볼 수 없을 때만 대신 쓰입니다. "무엇이 그려져 있는가" 를 씁니다.
figcaption	화면에 보입니다. 모두가 읽습니다. "이 그림이 왜 여기 있는가" 를 씁니다.

예를 들어 벚꽃 사진 한 장이 있다고 합시다.

alt="분홍 벚꽃이 가득 핀 가로수길"	← 눈에 보이는 것을 그대로
figcaption="2026년 4월, 학교 앞 벚꽃길"	← 언제 어디인지, 왜 실었는지

둘을 똑같이 쓰면 어떻게 될까요? 화면 낭독기로 듣는 사람은 같은 문장을 연달아 두 번 듣게 됩니다. 틀린 것은 아니지만 듣기에 답답합니다.

그렇다면 figcaption 이 이미 충분히 설명하고 있을 때는 alt 를 어떻게 할까요? 그럴 때는 alt="" 로 비워둡니다. 개념04 에서 배운 그 방법입니다. “설명은 아래 글로 충분하니 이 그림은 건너뛰어도 된다” 는 뜻이 됩니다.

아래 상자에서 두 글이 어떻게 다른지 보세요. 화면에 보이는 것은 figcaption 뿐입니다. alt 는 코드 안에만 있습니다.

화면	노란 별 아이콘 아래에 "즐거찾기를 뜻하는 표시로 씁니다" 가 보입니다.
----	--

alt 에 쓴 “노란 별 다섯 꼭짓점 아이콘” 은 어디에도 보이지 않습니다

즐거찾기를 뜻하는 표시로 씁니다

확인하는 방법이 있습니다. 브라우저에서 그림 위에 마우스를 놓고 오른쪽 버튼 → [검사] 를 누르면 코드가 보입니다. 거기에 alt 값이 적혀 있습니다. 화면에는 없지만 문서에는 분명히 들어 있습니다.

04

위 figure 를 통째로 복사해 하나 더 만들고, 그림은 이미지/로고.svg, alt 는 파란 바탕에 HTML 이라고 적힌 로고, figcaption 은 우리 웹사이트 로고입니다 로 바꿔 보세요. (복사한 쪽의 id 는 지워 주세요. 같은 id 가 두 개면 안 됩니다)

소리와 영상 (생김새만)

섹션 4

소리와 영상도 img 처럼 파일 경로를 적어 겁니다. 이 폴더에는 소리·영상 파일이 없으므로 코드만 봅니다.

먼저 왜 실제 파일을 안 넣었는지 밝혀 둡니다. mp3 나 mp4 는 크기가 커서 수업 자료에 넣으면 내려받기가 힘들어집니다. 문법이 img 와 거의 같기 때문에 코드만 봐도 충분히 익힐 수 있습니다.

— 소리

```
<audio src="소리/새소리.mp3" controls></audio>
```

controls 를 붙이면 재생 단추와 시간 막대가 보입니다.
안 붙이면 아무것도 안 보이고 소리도 안 납니다. 거의 항상 붙입니다.
controls 는 개념01 의 download 처럼 값이 없는 불리언 속성입니다.

— 영상

```
<video src="영상/소개.mp4" controls width="480"></video>
```

width 로 크기를 정하는 것도 img 와 같습니다.
poster="이미지/풍경.svg" 를 붙이면 재생 전에 보여 줄 그림을 정할 수 있습니다.

주의 · audio 와 video 는 빈 태그가 아닙니다. 반드시 닫아야 합니다.

	← img 는 빈 태그, 닫지 않습니다
<video ...></video>	← video 는 닫습니다
<audio ...></audio>	← audio 도 닫습니다

왜 다를까요? 그 안에 넣을 것이 있기 때문입니다.
영상을 못 트는 브라우저를 위한 안내글이나, 자막 파일을 안에 넣습니다.

— 자주 붙이는 속성

controls	재생 단추를 보여 줍니다
autoplay	페이지가 열리자마자 재생합니다 (요즘 브라우저는 대개 막습니다)
loop	끝나면 처음부터 다시 재생합니다
muted	소리를 끈 채로 시작합니다

autoplay 는 쓰지 않는 것이 좋습니다. 갑자기 소리가 나면 놀라고, 회사나 도서관에서는 곤란해집니다. 브라우저들도 그래서 소리 있는 자동 재생을 대부분 막아 두었습니다.

화면 검은 상자 안에 흰 글씨로 코드 한 줄이 보입니다

```
<audio src="소리/새소리.mp3" controls></audio>
```

화면 검은 상자 안에 흰 글씨로 코드 한 줄이 보입니다

```
<video src="영상/소개.mp4" controls width="480"></video>
```

영상은 figure 안에도 넣을 수 있습니다. 그림과 똑같이 figcaption 으로 설명을 붙이면 됩니다.

▸ 직접 해보기

4

이 폴더에 소개.mp4 라는 영상이 있다고 가정하고, 재생 단추가 보이고 가로가 600 인 video 코드를 종이에 한 줄로 적어 보세요.

다른 페이지 끼워 넣기 (생김새만)

섹션 5

iframe 은 페이지 안에 뚫어 놓은 창문입니다. 그 창문으로 다른 페이지가 보입니다.

지도나 유튜브 영상을 남의 블로그에서 본 적이 있을 것입니다. 그것이 iframe 입니다. inline frame(안에 끼운 틀) 의 줄임말입니다.

```
<iframe src="개념01_링크_a태그.html" width="600" height="240"></iframe>
```

src 에 경로나 주소를 적는 것은 img 와 똑같습니다. width, height 로 창문 크기를 정하는 것도 같습니다. iframe 도 빈 태그가 아니라서 반드시 닫아야 합니다.

img 와 결정적으로 다른 점이 하나 있습니다.

img → 그림 한 장이 들어옵니다
iframe → 웹페이지 한 장이 통째로 들어옵니다

그래서 iframe 안에서 스크롤도 되고 링크도 눌립니다. 진짜 페이지이기 때문입니다.

— 주의할 점

1) 아무 사이트나 끼워 넣을 수 있는 것은 아닙니다.

- 많은 사이트가 "우리 페이지는 남의 창문 안에 넣지 마시오" 라고 막아 두었습니다.
그런 주소를 넣으면 빈 상자만 보입니다.
- 2) 지도나 영상처럼 그 사이트가 "이 코드를 복사해 가세요" 하고 알려 준 경우에만 쓰는 것이 안전합니다.
 - 3) 페이지 전체를 iframe 으로 짜는 것은 하지 마세요.
검색이 안 되고, 뒤로 가기가 이상해지고, 주소를 공유할 수 없게 됩니다.

화면 검은 상자 안에 흰 글씨로 코드 한 줄이 보입니다

```
<iframe src="개념01_링크_a태그.html" width="600" height="240"></iframe>
```

iframe 은 이 자료에서 여기 한 번만 나옵니다. 지금은 "페이지 안에 페이지를 넣는 태그가 있다" 는 것만 기억하세요.

▹ 직접 해보기 5

위 코드 상자 아래에 진짜 iframe 을 하나 넣어 보세요. 위 검은 상자에 적힌 코드를 그대로 (< 같은 것 없이) 옮겨 적으면 됩니다.

자주 하는 실수

섹션 6

여기서도 마찬가지입니다. 틀려도 에러가 안 납니다. 의미만 조용히 사라집니다.

이런 실수를 합니다

figure 없이 figcaption 만 씀

이런 실수를 합니다

실수 1 — figure 없이 figcaption 만 씀

```

<figcaption>맛있는 사과입니다</figcaption>
```

글자는 그대로 잘 보입니다. 그래서 잘못된 줄 모릅니다. 하지만 figure 가 없으니 이 글이 어느 그림의 설명인지 아무도 모릅니다. figcaption 은 반드시 figure 안에 넣으세요.

이렇게 쓰세요

```
<figure>

<figcaption>맛있는 사과입니다</figcaption>
</figure>
```

이런 실수를 합니다

figcaption 을 두 개 씀

이런 실수를 합니다

실수 2 — figcaption 을 두 개 씀

```
<figure>
<figcaption>위 설명</figcaption>

<figcaption>아래 설명</figcaption>
</figure>
```

두 줄 다 화면에 보입니다. 겹보기에는 아무 문제가 없습니다. 그래서 이 실수는 끝까지 발견되지 않는 편입니다. figcaption 은 figure 하나에 한 개입니다.

이런 실수를 합니다

alt 와 **figcaption** 에 같은 말을 씀

이런 실수를 합니다

실수 3 — alt 와 **figcaption** 에 똑같은 말을 씀

```

<figcaption>우리 동네 뒷산입니다</figcaption>
```

눈으로 보는 사람에게는 아무 차이가 없습니다. 소리로 듣는 사람은 같은 문장을 연달아 두 번 듣게 됩니다. alt 에는 보이는 것을, figcaption 에는 왜 실었는지를 쓰세요. 설명이 겹칠 때는 alt="" 로 비웁니다.

이런 실수를 합니다

video 를 닫지 않음

이런 실수를 합니다

실수 4 — video 나 **audio** 를 안 닫음

```
<video src="영상/소개.mp4" controls width="480" />
```

img 처럼 생겨서 빈 태그인 줄 알고 닫지 않는 실수입니다. 브라우저는 닫는 태그가 나올 때까지 뒤에 오는 내용을 전부 영상 안으로 집어넣습니다. 그래서 영상 아래에 있어야 할 글이 통째로 사라집니다. “왜 여기부터 아래가 안 보이지?” 하면 닫는 태그를 찾아보세요.

이런 실수를 합니다

figure 를 아무 데나 씀

이런 실수를 합니다

실수 5 — 여백을 만들려고 figure 를 씀

```
<figure><p>그냥 본문입니다</p></figure>
```

figure 를 쓰면 좌우 여백이 생기니까 여백 만드는 도구로 쓰는 경우가 있습니다. 화면상으로는 원하는 대로 되지만, “이건 본문에서 떼어 낸 자료다” 라는 거짓말을 문서에 적어 넣는 셈입니다. 여백은 10단원에서 배울 CSS 로 만드세요.

정리

▹ 직접 해보기 6

정답

1) 섹션 1 의 figcaption 을 이렇게 바꿉니다.

```
<figcaption id="capOne">2026년 봄에 찍은 사진입니다</figcaption>
```

→ 그림 아래 글자만 바꿉니다. 그림과 여백은 그대로입니다.

2) 섹션 2 의 figure 안에서 figcaption 을 맨 위로 올립니다.

```
<figure id="figTwo"> <figcaption id="capTwo">왼쪽은 고양이, 오른쪽은 사과입니다</figcaption>  </figure>
```

→ 설명 글이 두 그림 위로 올라갑니다. 그림 크기와 여백은 그대로입니다.

3) 섹션 3 의 figure 아래에 이렇게 하나 더 넣습니다.

```
<figure>  <figcaption>우리 웹사이트 로고입니다</figcaption></figure>
```

→ 별 그림 아래에 작은 파란 로고와 그 설명이 한 짝 더 생깁니다.

두 덩어리 모두 왼쪽에서 40 픽셀씩 들어가 있습니다.

4) 이렇게 적으면 됩니다.

```
<video src="소개.mp4" controls width="600"></video>
```

→ 같은 폴더에 있으므로 폴더 이름 없이 파일 이름만 적었습니다.

controls 가 있어야 재생 단추가 보이고, video 는 닫아야 합니다.

5) 섹션 5 의 검은 코드 상자 아래에 이렇게 넣습니다.

```
<iframe src="개념01_링크_a태그.html" width="600" height="240"></iframe>
```

→ 회색 테두리를 두른 네모 상자가 생기고, 그 안에 개념01 파일의 맨 위쪽

(제목 "개념 01 - 링크와 a 태그" 와 노란 안내 상자) 이 들어가 보입니다.
상자 안을 따로 스크롤할 수 있습니다.

- 1) 이 파일을 VS Code 로 열고, 동시에 브라우저로도 엽니다.
- 2) 각 상자 안의 "여기에 코드를 쓰세요" 자리에 코드를 씁니다.
- 3) 저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 합니다.
- 4) 주석에 적힌 기대 화면과 같은지 눈으로 맞춰 봅니다.

1~11 은 기본, 12~13 은 응용, 14~15 는 도전, 16 은 확인 문제입니다.

이미지/사과.svg 200 × 200
이미지/고양이.svg 200 × 200
이미지/로고.svg 240 × 80
이미지/풍경.svg 320 × 180
이미지/아이콘/별.svg 60 × 60

[illegible]

290

각 상자 안에 코드를 써 넣고 저장한 뒤 새로고침하세요. 문제 설명과 정답 확인 방법은 VS Code 로 이 파일을 열어 주석에서 읽습니다. 다 풀었으면 연습문제_정답.html 과 맞춰 보세요.

문제 1 (개념01 섹션1)

같은 폴더에 있는 개념01_링크_a태그.html 로 가는 링크를 만드세요. 링크 글자는 "개념 01 — 링크와 a 태그" 로 하세요.

기대 출력 파란 글씨에 밑줄이 그어진 "개념 01 - 링크와 a 태그" 가 한 줄 보입니다. 누르면 개념01 파일이 열립니다. 이렇게 되면 틀린 것: 글씨가 검은색 그대로면 href 를 안 썼거나 a 태그가 아닙니다.

여기에 코드를 쓰세요

문제 2 (개념01 섹션2)

아래 링크는 목적지는 맞지만 링크 글자가 나뻑니다.

```
<a href="개념02_경로.html">여기를 클릭</a>
```

링크 글자만 읽어도 어디로 가는지 알 수 있도록 고쳐 쓰세요. 목적지는 그대로 두고 글자만 바꾸면 됩니다.

기대 출력 파란 밑줄 글자가 "개념 02 - 경로" 처럼 목적지를 알 수 있는 말로 보입니다. 이렇게 되면 틀린 것: "여기를 클릭", "자세히 보기", "더보기" 는 전부 같은 문제입니다.

여기에 코드를 쓰세요

문제 3 (개념01 섹션3)

개념05_그림과_설명_figure.html 을 새 탭에서 여는 링크를 만드세요. 새 탭으로 열 때 같이 붙여야 하는 속성도 빠뜨리지 마세요. 링크 글자는 "개념 05 를 새 탭에서 보기" 로 하세요.

기대 출력 파란 밑줄 링크가 한 줄 보입니다. 누르면 이 탭은 그대로 있고 새 탭이 하나 더 열립니다. 이렇게 되면 틀린 것: 이 탭의 내용이 바뀌면 target="_blank" 가 빠진 것입니다.

여기에 코드를 쓰세요

문제 4 (개념02 섹션3 · 개념04 섹션1)

이미지 폴더 안의 고양이.svg 를 화면에 넣으세요. alt 도 반드시 쓰세요. 파일 이름이 아니라 그림을 설명하는 글로 씁니다.

기대 출력 연한 하늘색 바탕에 주황색 고양이 얼굴이 그려진
200 × 200 크기의 그림이 보입니다.
이렇게 되면 틀린 것: 그림 대신 글자만 보이면 경로가 틀린 것입니다.
이미지 폴더 안에 있다는 것을 잊지 마세요.

여기에 코드를 쓰세요

문제 5 (개념02 섹션3)

이미지 폴더 안의 아이콘 폴더 안에 있는 별.svg 를 화면에 넣으세요. 폴더를 두 번 지나야 합니다.

기대 출력 연한 노란 바탕에 노란 별이 그려진 60 × 60 크기의 작은 그림이 보입니다.
이렇게 되면 틀린 것: 아무것도 안 보이거나 글자만 보이면 폴더를 하나
빠뜨린 것입니다.

여기에 코드를 쓰세요

문제 6 (개념02 섹션4)

01_HTML_시작하기 폴더 안의 개념02_태그의_생김새.html 로 가는 링크를 만드세요. 그 폴더는 이 파일이 있는 폴더 바깥에 있습니다. 먼저 밖으로 나가야 합니다. 링크 글자는 "01단원 개념02 — 태그의 생김새" 로 하세요.

기대 출력 파란 밑줄 링크가 한 줄 보입니다.
누르면 01단원 파일이 열립니다.
이렇게 되면 틀린 것: 눌렀을 때 파일을 찾을 수 없다는 화면이 뜨면 ../ 가
빠졌거나
폴더 이름을 잘못 쓴 것입니다.

여기에 코드를 쓰세요

문제 7 (개념03 섹션1 · 섹션2)

아래 상자 안에 두 가지를 만드세요. (1) id 가 lastPart 인 h3 제목 "마지막 부분" (2) 그 제목으로 점프하는 링크. 링크 글자는 "마지막 부분으로"

순서는 링크를 먼저, 제목을 나중에 쓰세요.

기대 출력

파란 밑줄 링크 한 줄과 그 아래 굵은 제목 "마지막 부분" 이 보입니다.
링크를 누르면 주소창 끝에 #lastPart 가 붙습니다.
이렇게 되면 틀린 것: 눌렀을 때 파일을 찾을 수 없다는 화면이 뜨면 href 에 # 이 빠진 것입니다.

여기에 코드를 쓰세요

문제 8 (개념03 섹션3)

이 문서 맨 위로 올라가는 링크를 만드세요. 이 파일의 body 에는 id="pageTop" 이 이미 붙어 있습니다.
링크 글자는 "맨 위로 가기" 로 하세요.

기대 출력

파란 밑줄로 "맨 위로 가기" 가 보입니다.
누르면 화면이 문서 맨 위 제목으로 올라갑니다.
이렇게 되면 틀린 것: 눌러도 화면이 안 움직이면 id 이름을 잘못 썼거나 # 이 빠진 것입니다.

여기에 코드를 쓰세요

문제 9 (개념03 섹션4)

hong@example.com 으로 메일을 쓰는 링크를 만드세요. 링크 글자는 "도서관에 메일 보내기" 로 하세요.

기대 출력

파란 밑줄로 "도서관에 메일 보내기" 가 보입니다.
누르면 메일 쓰기 창이 열립니다.
(메일 프로그램이 없으면 앱을 고르라는 창이 뜹니다. 정상입니다)
이렇게 되면 틀린 것: 눌렀을 때 파일을 찾을 수 없다는 화면이 뜨면 mailto: 를 빠뜨린 것입니다.

여기에 코드를 쓰세요

문제 10 (개념03 섹션5)

전화번호 031-987-6543 으로 전화를 거는 링크를 만드세요. 링크 글자도 전화번호 그대로 보이게 하세요.

기대 출력

파란 밑줄로 031-987-6543 이 보입니다.
이렇게 되면 틀린 것: 화면에 보이는 번호와 href 안의 번호가 다르면 안 됩니다.
둘 다 031-987-6543 이어야 합니다.

여기에 코드를 쓰세요

문제 11 (개념04 섹션3)

이미지 폴더의 사과.svg 를 가로 120 크기로 넣으세요. 세로는 적지 마세요. alt 도 잊지 마세요.

기대 출력 사과 그림이 120 × 120 으로 보입니다. 동그란 모양이 그대로입니다.
이렇게 되면 틀린 것: 사과가 납작하게 눌렀다면 height 까지 적은 것입니다.
크기가 엉뚱하면 숫자 뒤에 단위를 붙였는지 확인하세요. 숫자만 씁니다.

여기에 코드를 쓰세요

문제 12 [응용] (개념04 섹션4)

이미지 폴더의 로고.svg 를 가로 180 으로 넣고, 그 그림을 누르면 개념02_경로.html 로 가게 만드세요.
링크가 된 그림이므로 alt 에는 “그림이 무엇인지” 가 아니라 “누르면 어디로 가는지” 를 쓰세요.

기대 출력 파란 로고 그림이 180 × 60 으로 보입니다. 테두리는 없습니다.
마우스를 올리면 손가락 커서로 바뀌고, 누르면 개념02 파일이 열립니다.
이렇게 되면 틀린 것: 마우스를 올려도 손가락 커서가 안 나오면 a 로 감싸지 않은 것입니다.
alt 가 "로고" 라면 링크 글자 구실을 못 합니다.

여기에 코드를 쓰세요

문제 13 [응용] (개념05 섹션1 · 섹션3)

이미지 폴더의 풍경.svg 를 figure 로 묶고 설명을 붙이세요. alt : 그림에 무엇이 그려져 있는지 figcaption :
“학교 뒷산에서 내려다본 풍경입니다” 두 글이 서로 다른 말이어야 합니다.

기대 출력 산 그림이 320 × 180 으로 보이고 그 아래에 검은 글씨로
“학교 뒷산에서 내려다본 풍경입니다” 가 보입니다.
그림과 글이 통째로 왼쪽에서 조금 안으로 들어가 있습니다.
이렇게 되면 틀린 것: 안으로 들어가 있지 않으면 figure 로 감싸지 않은
것입니다.

여기에 코드를 쓰세요

문제 14 [도전] (개념05 섹션2 · 섹션3)

figure 하나 안에 그림 두 장을 넣고 설명을 하나만 다세요. 그림 1 : 이미지/고양이.svg 가로 100 그림 2 :
이미지/사과.svg 가로 100 설명 : “고양이와 사과입니다” 단, 설명을 그림들보다 위에 보이게 만드세요. 그림
두 장에는 각각 다른 alt 를 쓰세요.

기대 출력 맨 위에 "고양이와 사과입니다" 가 한 줄 보이고,
그 아래에 고양이와 사과가 100 × 100 크기로 나란히 보입니다.
전체가 왼쪽에서 조금 안으로 들어가 있습니다.
이렇게 되면 틀린 것: 설명이 그림 아래에 있으면 figcaption 을 맨 앞으로 옮기지 않은 것입니다.
figcaption 을 두 개 쓰면 안 됩니다.

여기에 코드를 쓰세요

문제 15 [도전] (개념02~05 전체)

아래 세 가지를 순서대로 모두 만드세요.

(1) id 가 photoZone 인 h3 제목 "사진 모음" (2) 그 제목보다 위쪽에, 그 제목으로 점프하는 링크 "사진 모음으로 가기" (3) 제목 아래에 figure 를 하나 만들고 그 안에

이미지/아이콘/별.svg 를 가로 80 으로 넣되,
그 그림을 누르면 개념05_그림과_설명_figure.html 로 가게 하고,
figcaption 에는 "별을 누르면 개념05 로 갑니다" 를 넣으세요.

기대 출력 파란 밑줄 링크 "사진 모음으로 가기" 가 있고, 그 아래 굵은 제목 "사진 모음",
그 아래에 80 × 80 노란 별 그림과 "별을 누르면 개념05 로 갑니다" 가 보입니다.
별 그림 위에서 마우스가 손가락 모양으로 바뀝니다.
이렇게 되면 틀린 것: 별이 안 보이면 아이콘 폴더를 빠뜨린 것입니다.
별을 눌러도 아무 일이 없으면 a 로 감싸지 않은 것입니다.

여기에 코드를 쓰세요

문제 16 [확인] (개념01 섹션6 · 개념03 섹션6 · 개념04 섹션6)

아래 상자 안에는 이미 코드가 들어 있습니다. 그런데 세 군데가 잘못되어 있습니다. 에러 메시지는 어디에도 안 뜹니다. 화면을 보고 무엇이 이상한지 찾아 고치세요.

(가) "개념 01 로 가기" 줄 : 파랑지도 않고 놀리지도 않습니다. 무엇이 빠졌을까요? (나) "여기가 도착지로 가기" 줄 : 누르면 아래 제목으로 가야 하는데,

파일을 찾을 수 없다는 화면이 나옵니다.

(다) 사과 그림 줄 : width 에 10em 이라고 썼는데 점처럼 아주 작게 나옵니다.

브라우저가 em 을 버리고 10 만 읽었기 때문입니다.
단위를 떼고, 원하던 크기인 100 으로 고치세요.

기대 출력 (고친 뒤)
첫 줄과 둘째 줄이 모두 파란 밑줄 링크가 되고,
둘째 줄을 누르면 "여기가 도착지" 제목으로 화면이 움직이며,
사과 그림이 100 × 100 으로 알맞게 보입니다.
이렇게 되면 틀린 것: 사과가 여전히 점처럼 작으면 단위만 떼고 숫자를 안
고친 것입니다.
10 을 100 으로 바꿔야 원하던 크기가 됩니다.

개념 01 로 가기
여기가 도착지로 가기
여기가 도착지

04단원 마무리 링크와 이미지

자주 넘어지는 자리

1 경로. 폴더를 실제로 열어 놓고 ../ 가 어디로 가는지 손으로 짚어 주세요

폼

OPEN 05_폼

type 을 안 쓴 것

type 에 오타를 낸 것

placeholder 만

`<input type="text">여기 글자</input>`

01 폼이란 무엇인가, 그리고 input

02 label 로 이름 붙이기, fieldset 으로 묶기

03 type 을 바꾸면 칸이 달라집니다

04 select 로 고르기, textarea 로 긴 글 받기

05 버튼, 그리고 값이 어디로 가는가

06 브라우저에게 검사를 시키는 속성들

- 연습문제

폼이란 무엇인가, 그리고 input

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

실행하면 나오는 화면 — 폼이란 무엇인가, 그리고 input

04단원까지 만든 화면에는 공통점이 하나 있었습니다. 제목, 문단, 목록, 표, 링크, 그림 전부 “우리가 미리 써 둔 것을 사람이 읽기만 하는” 화면이었습니다.

이번 단원에서 처음으로 방향이 반대가 됩니다. 사람이 글자를 쳐 넣고, 그 값을 우리가 받습니다. 로그인 화면, 회원가입 화면, 검색창, 배달 주문서, 설문지가 전부 이것입니다.

이렇게 값을 받아 내는 화면 부품 묶음을 폼(form) 이라고 부릅니다. 폼이 하는 일은 두 가지입니다.

- 1 값을 넣을 칸을 화면에 그린다
- 2 사용자가 채운 값을 한 덩어리로 묶어 서버로 보낸다

여기서 서버(server) 라는 말이 처음 나옵니다. 서버는 우리가 보낸 값을 받아서 저장하거나 처리하는 다른 컴퓨터의 프로그램입니다. 회원가입 값을 회원 명단에 적어 두는 쪽이라고 생각하면 됩니다.

★ 이 자료는 1) 까지만 다룹니다. 2) 를 진짜로 하려면 값을 받아 줄 서버 프로그램이 따로 있어야 하는데, 그것을 만드는 일은 HTML 의 몫이 아닙니다. 그래서 이 단원에서는 “값을 넣을 칸을 제대로 그리는 법” 만 배웁니다. 보내는 이야기는 개념05 에서 개념만 짧게 봅니다.

그리고 미리 말해 둡니다. 이 단원에서 만드는 입력칸들은 하나같이 밋밋하고 못생겼습니다. 그게 정상입니다. 브라우저가 기본으로 그려 주는 모양 그대로이기 때문입니다. 꾸미는 것은 09~10단원에서 CSS 로 합니다.

이 파일에서는 값을 넣을 칸을 처음 만들어 봅니다. 화면에 네모난 칸이 나오면 진짜로 마우스로 눌러서 글자를 쳐 보세요. 칸이 작고 밋밋하게 보이는 것은 정상입니다. 꾸미는 것은 09~10단원입니다. 지금은 모양이 아니라 “이 칸이 무엇을 받는 칸인지” 를 적는 연습만 합니다.

폼은 입력칸을 묶는 상자

섹션 1

<form> 은 여러 입력칸을 하나의 묶음으로 만드는 태그입니다. 자기 자신은 화면에 아무것도 그리지 않습니다.

아래 상자 안에는 이렇게 썼습니다.

```
<form> <p>이름 <input type="text" name="userName" /></p> <p>사는 곳
<input type="text" name="userCity" /></p>
</form>
```

form 요소는 눈에 보이는 것이 하나도 없습니다. 테두리도 없고 배경색도 없습니다. 그러면 왜 씁니까?

“이 칸들은 한 세트다” 라고 묶어 두는 것이 form 의 일이기 때문입니다. 나중에 보내기 버튼을 누르면 브라우저는 그 버튼이 들어 있는 form 을 찾고, 그 form 안에 있는 칸들의 값만 모아서 한 덩어리로 보냅니다. form 밖에 있는 칸은 아무리 옆에 붙어 있어도 같이 가지 않습니다.

한 페이지에 form 이 여러 개 있어도 됩니다. 예를 들어 맨 위의 검색창이 form 하나, 아래의 회원가입이 form 하나입니다. 검색 버튼을 눌렀을 때 회원가입 칸의 값까지 딸려 가면 곤란하니까요.

form 은 문단(p) 처럼 한 줄을 통째로 차지하는 블록 요소입니다. (블록 요소는 02단원 개념04 에서 배웠습니다. 한 줄을 통째로 차지하는 성질입니다) 그래서 form 앞뒤로 줄이 바뀝니다.

참고로 아래에서 input 을 p 로 한 번씩 감쌌습니다. 그래야 칸마다 줄이 바뀌기 때문입니다. input 자체는 글자처럼 옆으로 나란히 놓이는 성질이라 그냥 두면 한 줄에 다 붙습니다.

화면 "이름" 과 "사는 곳" 이 각각 한 줄씩 보이고, 글자 오른쪽에

흰 바탕에 가는 회색 테두리인 납작한 네모 칸이 하나씩 있습니다. form 때문에 생긴 테두리나 배경은 전혀 없습니다. 칸을 마우스로 누르면 안에서 커서가 깜빡이고 글자를 칠 수 있습니다

이름
사는 곳

<form>에는 action 과 method 라는 속성이 더 있습니다. “어디로 보낼지” 와 “어떻게 보낼지” 를 정하는 속성인데, 이 자료에서는 실제로 보내지 않으므로 일부러 비워 두었습니다. 개념05 에서 뜻만 짚어 봅니다.

⇒ 직접 해보기 1

위 상자 안 <form> 안에 <p>나이 <input type="text" name="userAge" /></p> 를 한 줄 더 넣어 보세요. 저장하고(Ctrl+S) 새로고침(F5) 하면 칸이 세 줄이 됩니다.

input 태그

섹션 2

<input /> 은 닫는 태그가 없는 빈 태그입니다. type 속성이 이 칸이 어떤 종류인지를 정합니다.

01단원에서 배운 빈 태그를 기억하세요. br 과 hr 처럼 감쌀 내용이 없는 태그입니다. input 도 같습니다. 칸 자체가 하나의 부품이라 안에 넣을 글자가 없습니다.

```
<input type="text" />
```

| | |
 | | | 속성 : 이 칸의 종류를 정합니다
 | | | 태그 이름 : 달는 태그가 없습니다

type 값이 text 이면 "한 줄짜리 글자 칸" 입니다. password, checkbox, date 처럼 다른 값도 많습니다.
그건 개념03 에서 전부 봅니다.

여기서 HTML 의 조용한 성질이 또 나옵니다.

type 을 아예 안 쓰면	→ text 로 그려집니다 (기본값)
type 에 없는 이름을 쓰면	→ 역시 text 로 그려집니다

둘 다 에러가 나지 않습니다. 빨간 글씨도 없습니다. 그래서 type="txt" 처럼 오타를 내면 화면은 멀쩡한데 뜻만 틀린 상태가 됩니다. 예를 들어 type="passwrod" 라고 잘못 쓰면 비밀번호가 그대로 다 보입니다. 아래 세 칸이 똑같이 생긴 것을 직접 확인하세요.

화면 세 줄 모두 설명 글자 오른쪽에 똑같이 생긴 네모 칸이 있습니다.

셋 중 어느 것이 오타인지 화면만 보서는 절대 구별할 수 없습니다.

- 제대로 쓴 것
- type 을 안 쓴 것
- type 에 오타를 낸 것

`<input>`처럼 닫으려고 하지 마세요. 빈 태그라서 닫는 태그가 필요 없습니다. 브라우저는 그 닫는 태그를 조용히 버립니다. 이 자료에서는 빈 태그라는 것이 눈에 띄도록 `<input />`로 씁니다.

▣ 직접 해보기 2

위 상자의 type="text" 를 type="password" 로 고쳐 보세요. 새로고침한 뒤 그 칸에 글자를 쳐 보면 글자 대신 점이 찍힙니다. type 하나가 칸의 성격을 바꾼다는 것을 확인하는 것이 목적입니다.

name

섹션 3

name 이 없는 칸의 값은 아무 데도 가지 않습니다. 그런데 화면은 멀쩡합니다. 그래서 가장 알아채기 어려운 실수입니다.

택배 상자를 열 개 보낸다고 생각해 보세요. 상자에 이름표를 안 붙이면, 받는 쪽은 상자 안의 물건이 무엇인지는 알아도 "이게 누구의 무엇인지" 를 모릅니다.

폼도 똑같습니다. 사용자가 "김민준" 이라고 쳤을 때 받는 쪽에 도착하는 것은 글자 하나가 아니라 이런 한 쌍입니다.

```
userName=김민준
```

여기서 왼쪽의 userName 이 우리가 코드에 적어 준 name 값입니다. 오른쪽이 사용자가 친 값입니다. 칸이 여러 개면 이 쌍이 여러 개 만들어져 이렇게 이어 붙습니다.

```
userName=김민준&userCity=부산&userAge=20
```

이제 왜 name 이 필요한지 분명해집니다. 이름표가 없으면 받는 쪽은 그 값이 이름인지 도시인지 나이인지 알 방법이 없습니다. 그래서 브라우저는 아예 처음부터 name 없는 칸의 값을 보내지 않습니다. 빼먹은 것이 아니라 규칙대로 버리는 것입니다.

이름은 영어 소문자로, 뜻이 드러나게 붙이세요. a, b, x1 처럼 지으면 나중에 받는 쪽에서 아무도 못 알아봅니다.

아래 두 칸을 보세요. 하나는 name 이 있고 하나는 없습니다. 화면에서 둘을 구별할 수 있습니까? 없습니다. 그것이 이 실수의 무서운 점입니다.

화면 두 칸이 완전히 똑같이 생겼습니다. 크기도 색도 같습니다.

둘 다 글자를 칠 수 있습니다. 차이는 코드에만 있습니다

```
이름표 있음
이름표 없음
```

name 은 보낼 때 쓰는 이름표입니다. 개념02 에서 id 라는 비슷하게 생긴 속성이 나오는데, 그것은 페이지 안에서 이 칸 하나를 가리키는 이름표라 쓰임새가 다릅니다. 두 개를 같이 쓰는 일이 아주 많으니 지금 구분해 두세요.

▣ 직접 해보기 3

위 상자의 “이름표 없음” 칸에 name="userNick" 을 넣어 보세요. 새로고침해도 화면은 하나도 안 바뀝니다. 화면이 안 바뀌는 것을 눈으로 확인하는 것이 이 문제의 목적입니다.

value

섹션 4

value 는 페이지가 열리자마자 칸에 이미 들어가 있는 값입니다. 사용자가 지우거나 고칠 수 있습니다.

회원 정보 수정 화면을 떠올려 보세요. 내 이름 칸이 처음부터 비어 있으면 곤란합니다. 지금 저장된 이름이 보여야 합니다. 그때 쓰는 것이 value 입니다.

```
<input type="text" name="userName" value="김민준" />
```

이렇게 쓰면 칸 안에 김민준 이 검은 글씨로 들어간 채로 화면이 열립니다. 그 상태에서 그냥 보내면 김민준 이 그대로 보내집니다. 사용자가 지우고 다른 이름을 치면 그 값이 대신 보내집니다.

중요한 것은 value 가 “진짜 값” 이라는 점입니다. 사용자가 아무것도 안 해도 이 값은 이미 칸에 들어 있는 값입니다.

아래 상자에서 두 번째 칸의 글자를 지워 보세요. 지워집니다. 그것이 value 가 안내문이 아니라 값이라는 증거입니다.

화면 위 칸은 비어 있고, 아래 칸에는 검은 글씨로 "김민준" 이 들어가 있습니다.

아래 칸을 눌러 백스페이스로 지우면 글자가 지워집니다

빈 칸
미리 채운 칸

▣ 직접 해보기 4

위 상자의 value="김민준" 을 여러분 이름으로 바꿔 보세요. 새로고침하면 칸 안의 글자가 바뀝니다.

placeholder 와 value

섹션 5

placeholder 는 값이 아니라 안내문입니다. 회색으로 보였다가 글자를 치는 순간 사라집니다.

입력칸에 회색 글씨로 “이메일을 입력하세요” 같은 안내가 떠 있는 것을 본 적이 있을 겁니다. 그것이 placeholder 입니다. 자리(place) 를 잡아 두는(holder) 임시 글자라는 뜻입니다.

```
<input type="text" placeholder="예) hong@mail.com" />
```

value 와 나란히 놓고 비교하면 이렇습니다.

value	진짜 값입니다. 검은 글씨. 지워야 없어집니다. 아무것도 안 해도 이 값이 보내집니다.
placeholder	안내문일 뿐입니다. 회색 글씨. 글자를 치면 저절로 숨습니다. 이 글자는 절대 보내지지 않습니다.

여기서 초보자가 크게 데이는 지점이 하나 있습니다. placeholder 만 써 놓고 “칸에 값이 들어 있네” 라고 생각하는 것입니다. 그 칸은 사실 비어 있습니다. 그냥 보내면 빈 값이 갑니다.

그리고 placeholder 를 이름표 대신 쓰면 안 됩니다. 글자를 치기 시작하면 안내문이 사라져 버려서 “여기가 무슨 칸이었지?” 하게 됩니다. 그래서 설명은 개념02 의 label 로 따로 합니다.

아래 세 번째 칸처럼 value 와 placeholder 를 같이 쓰면 value 가 이미 값을 채우고 있으므로 placeholder 는 아예 보이지 않습니다.

화면	첫째 칸에는 검은 글씨 “김민준”, 둘째 칸에는 옅은 회색 글씨로
----	--------------------------------------

“이름을 입력하세요”, 셋째 칸에는 검은 글씨 “김민준” 만 보입니다. 셋째 칸의 안내문은 값에 가려 보이지 않습니다. 둘째 칸을 눌러 아무 글자나 치면 회색 글씨가 즉시 사라집니다

value 만
placeholder 만
둘 다

▸ 직접 해보기 5

위 상자의 셋째 칸에서 value=“김민준” 만 지우고 새로고침해 보세요. 가려져 있던 회색 안내문이 나타납니다.

자주 하는 실수

섹션 6

폼의 실수는 화면이 멀쩡해서 더 위험합니다. 아래 다섯 가지는 전부 에러가 나지 않고, 눈으로도 거의 티가 나지 않습니다.

이런 실수를 합니다

name 을 빠뜨림

이런 실수를 합니다

실수 1 — name 을 빠뜨림

```
<input type="text" value="김민준" />
```

칸은 잘 그려지고 글자도 잘 쳐집니다. 그런데 이 칸의 값만 쏙 빠진 채 나머지 값들이 보내집니다. 받는 쪽에서는 이름이 안 왔다고 나옵니다. “다 채웠는데 왜 안 들어갔지?” 의 1등 원인입니다. 이름표 없는 칸

이렇게 쓰세요

```
<input type="text" name="userName" value="김민준" />
```

이런 실수를 합니다

placeholder 를 값으로 착각

이런 실수를 합니다

실수 2 — **placeholder** 를 값이라고 생각함

```
<input type="text" name="city" placeholder="부산" />
```

회색으로 “부산” 이 보이니 값이 들어 있는 것처럼 착각합니다. 그 칸은 비어 있습니다. 사용자가 그냥 넘어가면 도시 값이 빈 채로 갑니다. 기본값을 진짜로 넣고 싶으면 `value="부산"` 을 써야 합니다.

이런 실수를 합니다

input 을 닫으려고 함

이런 실수를 합니다

실수 3 — **input** 을 닫는 태그로 닫음

```
<input type="text">여기 글자</input>
```

`input` 은 빈 태그라 안에 글자를 넣을 수 없습니다. “여기 글자” 는 칸 안이 아니라 칸 바깥에 그냥 떠 있는 글자가 되고, `</input>` 은 조용히 버려집니다. 에러는 없습니다. 여기 글자

이런 실수를 합니다

여러 칸에 같은 name

이런 실수를 합니다

실수 4 — 서로 다른 칸에 같은 **name** 을 붙임

```
<input name="user" />
<input name="user" />
```

이름표가 똑같은 상자 두 개를 보내는 셈입니다. 두 값이 다 가기는 하는데 받는 쪽에서 어느 것이 이름이고 어느 것이 도시인지 구분할 수 없습니다. 칸마다 다른 이름을 붙이세요. (다만 개념03 의 라디오 버튼만은 일부러 같은 `name` 을 씁니다. 그건 이유가 따로 있습니다)

이런 실수를 합니다

따옴표를 빠뜨림

이런 실수를 합니다

실수 5 — 속성 값의 따옴표를 빠뜨림

```
<input type="text" placeholder=이름을 입력하세요 />
```

띄어쓰기가 있는 값에서 따옴표를 빼면, 브라우저는 첫 칸까지만 값으로 읽습니다. 안내문이 “이름을” 까지만 나오고 뒤의 “입력하세요” 는 입력하세요 라는 이름의 속성으로 잘못 읽힙니다. 역시 에러는 없습니다.

정리

▣ 직접 해보기

6

정답

1) 섹션 1 의 form 안, 두 번째 p 아래에 이렇게 넣습니다.

```
<p>나이 <input type="text" name="userAge" /></p>
```

→ 상자 안이 두 줄에서 세 줄로 늘어나고, 새 칸에도 글자를 칠 수 있습니다.

2) 섹션 2 의 셋째 칸을 이렇게 고칩니다.

```
<p>type 에 오타를 낸 것 <input id="badType" type="password" /></p>
```

→ 그 칸에 글자를 치면 글자 대신 까만 점이 하나씩 찍힙니다.

앞의 두 칸은 그대로 글자가 보입니다.

3) 섹션 3 의 둘째 칸을 이렇게 고칩니다.

```
<p>이름표 없음 <input id="noName" type="text" name="userNick" /></p>
```

→ 화면은 조금도 바뀌지 않습니다. 크기도 색도 위치도 그대로입니다.

name 은 화면이 아니라 "보낼 때" 만 쓰이는 속성이기 때문입니다.

4) 섹션 4 의 둘째 칸에서 value 값을 바꿉니다. 예를 들어

```
<input id="valueBox" type="text" name="filled" value="이서연" />
```

→ 칸 안의 검은 글씨가 “김민준” 에서 “이서연” 으로 바뀝니다.

5) 섹션 5 의 셋째 칸에서 value 속성을 통째로 지웁니다.

```
<input id="bothBox" type="text" placeholder="이름을 입력하세요" />
```

→ 검은 글씨 “김민준” 이 사라지고, 그 자리에 옅은 회색 글씨로

"이름을 입력하세요" 가 나타납니다. 셋째 칸이 둘째 칸과 똑같아집니다.

label 로 이름 붙이기, fieldset 으로 묶기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

이 파일은 특히 “직접 눌러 보는” 것이 중요합니다.

실행하면 나오는 화면 — label 로 이름 붙이기, fieldset 으로 묶기

개념01 에서 이렇게 썼습니다.

```
<p>이름 <input type="text" name="userName" /></p>
```

화면에는 “이름”이라는 글자와 네모 칸이 나란히 보입니다. 사람 눈에는 둘이 한 쌍처럼 보입니다.

그런데 브라우저 입장에서는 그렇지 않습니다. 브라우저가 아는 것은 이것뿐입니다.

“이름”이라는 글자가 있다. 그 옆에 칸이 하나 있다.
둘 사이에 아무 관계도 적혀 있지 않다.

사람이 눈으로 만들어 낸 관계일 뿐, 코드에는 그 관계가 없습니다. 그래서 이런 일이 생깁니다.

- "이름" 글자를 눌러도 칸에 커서가 안 들어갑니다
- 화면을 못 보는 사람이 쓰는 프로그램이 이 칸을 "칸" 이라고만 읽어 줍니다
- 체크박스처럼 누를 곳이 작은 부품은 손가락으로 누르기가 아주 어렵습니다

이 관계를 코드에 진짜로 적어 주는 태그가 label 입니다. label 은 "이 글자는 저 칸의 이름이다" 를 브라우저에게 알려 줍니다.

label 을 붙이면 눈에 보이는 변화가 딱 하나 생깁니다.

글자를 눌러도 칸이 커집니다.

이 한 줄이 이 파일의 전부입니다. 반드시 직접 눌러서 확인하세요.

이 파일은 읽기만 해서는 절반도 이해가 안 됩니다. 아래 상자마다 글자를 마우스로 눌러 보세요. 어떤 글자는 눌러도 아무 일이 없고, 어떤 글자는 누르는 순간 칸이 커집니다. 그 차이를 만드는 것이 오늘 배우는 label 입니다.

label 이란

섹션 1

먼저 연결이 없는 쪽을 보고, 무엇이 아쉬운지 느껴 봅시다.

아래 상자에는 이렇게 썼습니다. label 이 없습니다.

```
<p>이름 <input type="text" /></p>
```

화면은 아주 멀쩡합니다. 글자도 칸도 잘 보입니다. 그런데 "이름" 이라는 글자를 마우스로 눌러 보세요. 아무 일도 안 일어납니다. 칸을 직접 정확히 눌러야만 커서가 들어갑니다.

이것이 "코드에 관계가 없다" 의 실제 모습입니다. 브라우저는 그 글자와 그 칸이 한 쌍이라는 것을 모릅니다.

여기서 용어를 하나 정리하고 갑니다.

label(레이블) = 이름표, 라벨.
입력칸이 무엇을 받는 칸인지 알려 주는 글자입니다.

개념01 의 name 도 "이름표" 라고 불렀는데 쓰임새가 다릅니다.

name	→ 값을 보낼 때 쓰는 이름표. 사람 눈에 안 보입니다.
label	→ 사람에게 보여 주는 이름표. 화면에 글자로 보입니다.

화면 "이름" 글자와 빈 칸이 한 줄에 보입니다.

"이름" 글자를 눌러도 커서가 생기지 않고 아무 일도 일어나지 않습니다

이름

☞ 직접 해보기 1

위 상자의 “이름” 글자를 다섯 번쯤 눌러 보세요. 칸에 커서가 안 들어가는 것을 확인하고, 이번에는 칸을 직접 눌러 보세요. 두 동작의 차이를 기억해 두면 다음 섹션이 훨씬 잘 보입니다.

for 와 id

섹션 2

칸에 id 를 붙이고, label 의 for 에 똑같은 값을 적으면 둘이 연결됩니다.

id 라는 속성이 여기서 처음 나옵니다.

id = 이 페이지 안에서 이 요소 하나를 가리키는 이름.
같은 id 를 두 곳에 쓰면 안 됩니다. 주민등록번호 같은 것입니다.

연결은 이렇게 씁니다.

```
<label for="userName">이름</label>
<input type="text" id="userName" name="userName" />
|
for 값과 id 값이 글자 하나까지 똑같아야 합니다
```

for 는 “누구를 위한 이름표인가” 를 적는 칸입니다. for=“userName” 은 “나는 id 가 userName 인 칸의 이름표다” 라는 뜻입니다.

이제 여기서 세 가지 이름이 한꺼번에 나왔으니 정리합시다.

id	페이지 안에서 이 칸을 찾는 이름.	label 이 이걸 보고 짝을 찾습니다.
name	값을 보낼 때 붙는 이름표.	받는 쪽이 이걸 보고 값을 구분합니다.
for	label 이 가리키는 칸의 id.	

id 와 name 을 같은 값으로 적는 경우가 많습니다. 위 예도 둘 다 userName 입니다. 헷갈리기 쉬운데, 값이 같을 뿐 하는 일은 완전히 다릅니다. id 를 지우면 label 연결이 끊기고, name 을 지우면 값이 안 보내집니다.

아래 상자에서 “이름” 글자를 눌러 보세요. 이번에는 칸에 커서가 들어갑니다.

화면 겉모습은 섹션 1 과 똑같습니다.

그런데 “이름” 글자를 누르면 오른쪽 칸 안에서 커서가 깜빡이기 시작합니다

이름

for 값과 id 값은 대소문자까지 똑같아야 합니다. for=“userName” 과 id=“username” 은 다른 값이라 연결되지 않습니다. 그런데 에러는 나지 않습니다. 그냥 조용히 안 놀립니다.

▣ 직접 해보기 2

위 상자의 for="userName" 을 for="userNameee" 로 (끝에 e 하나 더) 바꿔 보세요. 새로고침하고 글자를 누르면 다시 아무 일도 안 일어납니다. 확인했으면 되돌려 놓으세요.

감싸는 방식

섹션 3

label 안에 칸을 넣어 버리면 for 도 id 도 없이 연결됩니다.

방법이 하나 더 있습니다. label 로 글자와 칸을 통째로 감싸는 것입니다.

```
<label>사는 곳 <input type="text" name="userCity" /></label>
```

input 이 label 의 자식이 되었습니다. 브라우저는 "내 안에 있는 칸이 내 짝이구나" 하고 알아서 연결합니다. for 도 id 도 필요 없습니다.

두 방법을 언제 골라 쓸까요?

for 와 id	글자와 칸이 떨어져 있어도 됩니다. 표 안에서 왼쪽 칸에 이름, 오른쪽 칸에 입력칸을 둘 때 이 방법뿐입니다. 대신 id 를 겹치지 않게 관리해야 합니다.
감싸기	짧고 실수가 없습니다. id 를 안 지어도 됩니다. 대신 글자와 칸이 반드시 붙어 있어야 합니다.

둘 다 효과는 완전히 같습니다. 이 자료에서는 상황에 따라 둘 다 씁니다.

아래 상자에서 "사는 곳" 글자를 눌러 보세요. 역시 칸에 커서가 들어갑니다.

화면 "사는 곳" 글자와 칸이 한 줄에 보입니다.

글자를 누르면 칸 안에서 커서가 깜빡입니다. 섹션 2 와 결과가 똑같습니다

사는 곳

label 은 글자처럼 옆으로 흐르는 인라인 요소입니다. 그래서 감싸도 줄이 새로 바뀌지 않습니다. 인라인은 02단원 개념04 에서 배운 그 성질입니다. 한 줄 안에서 옆으로 붙고 줄을 끊지 않습니다. 이 성질을 CSS 로 바꾸는 방법은 11단원에서 배웁니다.

▣ 직접 해보기 3

위 상자 아래에 감싸기 방식으로 "이메일" 칸을 하나 더 만들어 보세요. <label>이메일 <input type="text" name="userMail" /></label> 를 <p> 로 감싸서 넣으면 새 줄에 나옵니다.

체크박스에서 확인

섹션 4

체크박스는 누를 곳이 13픽셀짜리 작은 네모뿐입니다. 연결해 두면 옆의 긴 글자 전체가 누를 곳이 됩니다.

체크박스(type="checkbox") 는 개념03 에서 제대로 배웁니다. 여기서는 "켜고 끄는 작은 네모" 라는 것만 알면 됩니다. label 의 효과를 보여 주기에 이만한 예가 없어서 먼저 씁니다.

아래 두 줄을 직접 눌러 보세요. 반드시 글자 쪽을 누르세요.

연결 안 됨 글자를 눌러도 체크 표시가 생기지 않습니다.
작은 네모를 정확히 조준해서 눌러야 합니다.

연결 됨 "광고 메일을 받겠습니다" 라는 글자 아무 데나 눌러도 체크됩니다.
한 번 더 누르면 풀립니다.

차이가 느껴집니까? 마우스로도 귀찮은데 휴대폰에서 손가락으로 누르면 연결이 없는 체크박스는 거의 못 누릅니다. 그래서 실무에서는 체크박스과 라디오 버튼에 label 을 안 붙이는 경우가 없습니다.

연결이 되면 누를 수 있는 넓이가 이렇게 달라집니다.

연결 없음 [x] 광고 메일을 받겠습니다
└─
└─ 여기 13픽셀만 눌립니다

연결 있음 [x] 광고 메일을 받겠습니다
└──────────┘
└─ 이 줄 전체가 눌립니다

화면 작은 빈 네모와 그 오른쪽에 글자가 있습니다.

글자를 눌러도 네모는 빈 채로 그대로입니다. 네모 자체를 누르면 그때만 파란 바탕에 흰 체크 표시가 생깁니다

광고 메일을 받겠습니다

화면 겉모습은 위와 똑같습니다.

그런데 글자 아무 데나 누르면 네모가 파란 바탕에 흰 체크 표시로 바뀝니다. 한 번 더 누르면 다시 빈 네모가 됩니다

광고 메일을 받겠습니다

▣ 직접 해보기 4

두 상자를 번갈아 눌러 보세요. 위 상자는 글자를 눌러도 안 켜지고, 아래 상자는 켜집니다. 그다음 위 상자의 코드를 <label> 로 감싸서 글자를 눌러도 켜지게 고쳐 보세요.

fieldset 은 관련 있는 칸들을 한 덩어리로 묶는 테두리이고, legend 는 그 덩어리의 제목입니다.

회원가입 화면을 떠올려 보세요. 칸이 열 개쯤 주르륵 있으면 눈이 아픕니다. 보통은 이렇게 나눠 놓습니다.

기본 정보 · 이름, 생년월일

연락처 · 이메일, 전화번호

이 덩어리를 만드는 태그가 fieldset 이고, 대괄호 안의 제목이 legend 입니다.

```
<fieldset> <legend>기본 정보</legend> <p>...</p>
</fieldset>
```

규칙이 두 가지 있습니다.

첫째, legend 는 fieldset 의 첫 자식으로 씁니다. 크롬은 순서가 뒤바뀌어도 첫 번째 legend 를 알아서 테두리 위로 끌어올려 그려 줍니다. 하지만 다른 프로그램까지 그렇게 봐 준다는 보장이 없으니 규칙대로 맨 위에 쓰세요.

둘째, 한 fieldset 안에 legend 는 하나만 씁니다. 두 개를 쓰면 첫 번째만 테두리에 걸친 제목이 되고, 두 번째는 상자 안에 놓인 평범한 글자 줄이 되어 버립니다. 물론 에러는 안 납니다.

fieldset 은 05단원에서 쓸 수 있는 거의 유일한 “묶는 상자” 입니다. div 로 묶는 방법은 06단원에서 배웁니다. 지금 여러 칸을 묶어야 하면 fieldset 이나 p 를 쓰세요.

화면 **열은 회색 테두리로 둘러싸인 상자가 두 개 세로로 놓입니다.**

각 상자의 왼쪽 위 테두리가 끊기고 그 자리에 “기본 정보”, “연락처” 라는 글자가 걸쳐 있습니다. 상자 안에는 이름표와 칸이 줄줄이 들어 있습니다

기본 정보

이름

사는 곳

연락처

이메일

◀ 직접 해보기 5

위 “연락처” 상자 안에 전화번호 칸을 한 줄 더 넣어 보세요. 이름표도 for 로 연결하세요. 상자의 세로 크기가 늘어나고 테두리가 같이 커집니다.

왜 이렇게까지 하나

섹션 6

연결은 눈으로 보이는 편리함과 눈에 안 보이는 사람을 위한 정보를 동시에 줍니다.

04단원 개념01 에서 접근성(accessibility) 이라는 말을 봤습니다. "누구나 쓸 수 있게 만드는 것" 이라는 뜻이었습니다. 폼은 그 말이 가장 크게 걸리는 자리입니다.

화면을 못 보는 사람은 04단원에서 본 화면 낭독기로 웹을 씁니다. 이 프로그램은 화면을 위에서 아래로 읽어 줍니다.

label 이 연결되지 않은 칸에 커서가 가면 이렇게 읽습니다.

"편집창"

무엇을 넣는 칸인지 알 수 없습니다. label 이 연결되어 있으면 이렇게 읽습니다.

"이름, 편집창"

딱 한 단어 차이지만, 이 한 단어가 있어야 폼을 채울 수 있습니다. for 와 id 를 맞추는 일이 귀찮게 느껴질 때 이 장면을 떠올리세요.

눈에 보이는 이득도 큼니다.

- 글자를 눌러도 칸이 커지니 마우스질이 편합니다
- 체크박스처럼 작은 부품을 조준하지 않아도 됩니다
- 휴대폰에서 손가락으로 누를 수 있는 넓이가 몇 배로 커집니다

그리고 placeholder 로 이름표를 대신하면 안 되는 이유도 여기 있습니다. placeholder 는 글자를 치기 시작하면 사라집니다. 화면 낭독기가 안 읽어 주는 경우도 많습니다. 이름표는 label 로, 예시는 placeholder 로. 역할을 나누세요.

화면 "이메일" 이라는 검은 글자 오른쪽에 칸이 있고,

칸 안에는 옅은 회색으로 "예) hong@mail.com" 이 보입니다. "이메일" 글자를 누르면 칸에 커서가 들어가고 회색 글씨는 그대로 남아 있습니다

이메일

▹ 직접 해보기 6

위 칸에 글자를 몇 자 쳐 보세요. 회색 예시는 사라지지만 왼쪽의 "이메일" 이름표는 그대로 남습니다. placeholder 만 있었다면 지금 이 칸이 무슨 칸인지 알 수 없었을 것입니다.

label 관련 실수는 전부 “눌러 봐야 아는” 실수입니다. 화면만 보서는 정상과 구별되지 않습니다.

이런 실수를 합니다

for 와 id 가 다름

이런 실수를 합니다

실수 1 — for 값과 id 값이 다름

```
<label for="userAge">나이</label>
<input id="userage" type="text" />
```

userAge 와 userage 는 다른 값입니다. A 하나 차이입니다. 글자를 눌러도 칸이 안 켜집니다. 에러도 경고도 없습니다. 아래에서 “나이”를 눌러 보세요. 아무 일도 안 일어납니다. 나이

이렇게 쓰세요

```
<label for="userAge">나이</label>
<input id="userAge" type="text" />
```

이런 실수를 합니다

id 를 두 번 씀

이런 실수를 합니다

실수 2 — 같은 id 를 두 칸에 붙임

```
<input id="pick" type="checkbox" />
<input id="pick" type="checkbox" />
```

id 는 페이지 안에서 하나뿐이어야 합니다. 두 개면 label 이 위쪽 하나만 찾아냅니다. 아래 칸은 이름표가 있는 줄 알았는데 사실 없는 상태가 됩니다. 칸을 복사해서 붙여 넣을 때 id 를 안 고쳐서 자주 생깁니다.

이런 실수를 합니다

label 대신 그냥 글자

이런 실수를 합니다

실수 3 — label 없이 글자만 옆에 씀

```
<p>전화번호 <input type="text" /></p>
```

제일 흔합니다. 화면은 완벽해 보입니다. 그런데 글자를 눌러도 반응이 없고, 화면 낭독기는 “편집창”이라고만 읽습니다. 이름표가 될 글자는 <label> 로 감싸는 습관을 들이세요.

이런 실수를 합니다

legend 를 fieldset 밖에 씀

이런 실수를 합니다

실수 4 — legend 를 fieldset 밖에 씀

```
<legend>기본 정보</legend>
<fieldset> <p>이름 ...</p>
</fieldset>
```

제목은 먼저 쓰고 상자를 여는 순서로 쓰면 이렇게 됩니다. legend 가 fieldset 안에 없으니 테두리와 아무 관계 없는 그냥 글자가 되어 상자 위에 덩그러니 놓입니다. 상자의 테두리는 끊긴 곳 없이 사각형으로 이어집니다. 기본 정보 이름

이런 실수를 합니다

label 로 감싸고 for 까지 씀

이런 실수를 합니다

실수 5 — 감싸 놓고 for 에 엉뚱한 값을 씀

```
<label for="abc">약관 동의
<input id="agree" type="checkbox" />
</label>
```

감싸기만 했으면 잘 됐을 텐데, for="abc" 를 덧붙이는 바람에 label 이 abc 라는 없는 칸을 찾으려 갑니다. 그래서 안 놀립니다. 감쌌으면 for 를 쓰지 말고, for 를 쓸 거면 감싸지 마세요.

정리

▹ 직접 해보기 7

정답

1) 눈으로 확인하는 문제입니다. 코드는 고치지 않습니다.

→ 섹션 1의 “이름” 글자는 아무리 눌러도 칸에 커서가 안 들어갑니다.

칸 자체를 누르면 그때만 커서가 깜빡입니다.

2) 섹션 2의 label을 이렇게 바꿉니다.

```
<label id="nameLabel" for="userName">이름</label>
```

→ “이름”을 눌러도 칸이 안 커집니다. 화면 생김새는 하나도 안 바뀝니다.

확인했으면 for="userName"으로 되돌리세요.

3) 섹션 3의 상자 안, 기존 p 아래에 이렇게 넣습니다.

```
<p> <label>이메일 <input type="text" name="userMail" /></label>
</p>
```

→ 새 줄에 “이메일”과 칸이 생기고, 글자를 누르면 칸이 커집니다.

4) 섹션 4의 위쪽 상자를 이렇게 고칩니다.

```
<p id="badCheckRow"> <label> <input id="badCheck" type="checkbox" name="ad1" /> 광고
메일을 받겠습니다
</label>
</p>
```

→ 이제 위 상자도 글자를 누르면 네모에 체크 표시가 생겼다 사라졌다 합니다.

두 상자가 똑같이 동작하게 됩니다.

5) 섹션 5의 “연락처” fieldset 안, 이메일 줄 아래에 이렇게 넣습니다.

```
<p><label for="joinTel">전화번호
</label> <input id="joinTel" type="text" name="joinTel" /></p>
```

→ 연락처 상자가 한 줄만큼 세로로 길어지고 테두리도 같이 늘어납니다.

“전화번호” 글자를 누르면 새 칸에 커서가 들어갑니다.

6) 눈으로 확인하는 문제입니다. 코드는 고치지 않습니다.

→ 칸에 글자를 치면 회색 “예) hong@mail.com”이 사라지고,

왼쪽의 검은 글자 “이메일”은 그대로 남습니다.
지우면 회색 글씨가 다시 나타납니다.

type 을 바꾸면 칸이 달라집니다

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

칸이 나올 때마다 실제로 눌러 보고 글자를 쳐 보세요.

비밀번호

이메일

검색어

전화번호

홈페이지

인원수

방문 날짜

만족도

좋아하는 색

추가 재료 (여러 개 고를 수 있습니다)

☒ 치즈

☐ 베이컨

05

실행하면 나오는 화면 — type 을 바꾸면 칸이 달라집니다

개념01 에서 input 의 type 이 “칸의 종류를 정한다” 고 했습니다. 이 파일에서 그 종류를 전부 봅니다. 열네 가지입니다.

글자를 받는 칸	text password email search tel url
숫자·날짜	number date
고르는 칸	checkbox radio
특별한 칸	file range color
안 보이는 칸	hidden

겉먹을 필요 없습니다. 태그는 여전히 input 하나뿐이고, 바뀌는 것은 type 값 한 글자뿐입니다.

<code><input type="text" /></code>	글자 칸
<code><input type="password" /></code>	글자가 점으로 가려지는 칸
<code><input type="date" /></code>	날짜 고르는 칸

type 을 바꾸면 세 가지가 한꺼번에 바뀝니다.

- 1 생김새 점으로 가려지거나, 달력이 뜨거나, 동그라미가 되거나
- 2 넣는 방법 자판으로 치거나, 마우스로 고르거나, 끌어서 옮기거나
- 3 뜻 이 값이 이메일인지 전화번호인지를 브라우저가 알게 됩니다

3) 이 왜 중요한지는 개념06 에서 확실해집니다. 브라우저가 값의 뜻을 알면 "이건 이메일 모양이 아닌데요" 하고 대신 검사해 줍니다.

★ 이 파일에서 가장 중요한 한 줄을 미리 말해 둡니다. 라디오 버튼은 name 이 같아야 한 무리가 됩니다. 초보자가 폼에서 가장 많이 걸려 넘어지는 곳이 여기입니다. 섹션 4 를 꼭 보세요.

이 파일에는 그중 열세 가지가 살아 있는 채로 들어 있습니다. (text 는 개념01 에서 이미 써 봤으니 여기서는 뺐습니다) 하나하나 직접 눌러 보고 값을 넣어 보세요. 칸이 작고 밋밋한 것은 정상입니다. 꾸미는 것은 09~10단원입니다. 섹션 4 의 라디오 버튼은 특히 천천히 보세요.

글자를 받는 칸들

섹션 1

이 다섯은 겉모습이 text 와 거의 같습니다. 다른 것은 "가려 주느냐" 와 "이 값이 무슨 뜻이냐" 입니다.

하나씩 봅시다.

password	친 글자가 점(●) 으로 가려집니다. 옆 사람이 못 보게 가려 주는 것뿐입니다. 값 자체는 그대로 있고, 보낼 때도 글자 그대로 갑니다. "가려지니까 안전하다" 는 뜻이 절대 아닙니다. 진짜로 안전하게 보내는 일은 서버 쪽 이야기입니다.
email	겉보기는 text 와 똑같습니다. 휴대폰에서 열면 자판에 @ 가 미리 나와 있습니다. 그리고 개념06 에서 배울 검사에서 "골뱅이가 없다" 를 잡아 줍니다.
search	검색창용입니다. 글자를 치면 오른쪽 끝에 지우는 x 표시가 생깁니다.
tel	전화번호용입니다. 휴대폰에서 숫자 자판이 뜹니다. 컴퓨터에서는 text 와 완전히 같아 보입니다.
url	주소용입니다. 역시 겉보기는 같습니다.

정리하면 이렇습니다.

보기에는 거의 같지만 뜻이 다릅니다.
뜻을 제대로 적어 두면 휴대폰 자판과 브라우저 검사가 저절로 따라옵니다.
그러니 "전부 text 로 쓰면 되지" 하지 마세요.

화면 다섯 칸이 모두 똑같은 크기의 네모로 보입니다.

비밀번호 칸에만 글자 대신 까만 점이 하나씩 찍힙니다. 검색어 칸에 글자를 치면 오른쪽 끝에 작은 x 표시가 생기고, 그 x 를 누르면 글자가 한 번에 지워집니다. 나머지 세 칸은 친 글자가 그대로 보입니다

비밀번호

이메일

검색어

전화번호

홈페이지

type="password" 는 어깨 너머로 훑쳐보는 것만 막습니다. 네트워크로 오갈 때 안전하게 만드는 것은 완전히 다른 이야기이고, HTML 이 하는 일이 아닙니다. 이 자료의 범위 밖이니 지금은 그런 게 있다는 것만 아세요.

직접 해보기

1

비밀번호 칸의 type="password" 를 type="text" 로 바꿔 보세요. 새로고침한 뒤 글자를 치면 점 대신 글자가 그대로 보입니다. 확인했으면 되돌려 놓으세요.

숫자·날짜·범위·색

섹션 2

여기서부터는 생김새가 확실히 달라집니다. 화살표, 달력, 미끄럼 막대, 색 상자가 나옵니다.

number 숫자만 받는 칸입니다.

칸 위에 마우스를 올리면 오른쪽 끝에 작은 위아래 화살표가 나타납니다.
그 화살표로 1씩 올리고 내릴 수 있습니다.
글자를 쳐 넣으려고 하면 잘 안 들어갑니다.
얼마부터 얼마까지 받을지는 개념06 의 min, max 로 정합니다.

date 날짜 칸입니다. 처음에는 "연도-월-일" 이라는 회색 안내가 보입니다.

오른쪽 끝의 작은 달력 그림을 누르면 달력이 펼쳐집니다.
숫자를 직접 쳐 넣을 수도 있습니다.
value 로 미리 넣을 때는 반드시 2026-03-15 모양으로 씁니다.
2026/03/15 나 3월 15일 이라고 쓰면 조용히 무시되고 빈 칸이 됩니다.

range 미끄럼 막대입니다. 손잡이를 좌우로 끌어서 값을 정합니다.

따로 정하지 않으면 0 부터 100 까지이고 처음 값은 50 입니다.
그래서 손잡이가 정확히 가운데에 있습니다.
단점이 하나 있습니다. 지금 값이 얼마인지 화면에 안 보입니다.
숫자를 같이 보여 주려면 자바스크립트가 필요해서 이 자료에서는 안 합니다.

color 색을 고르는 칸입니다. 색이 칠해진 작은 네모로 보입니다.

누르면 운영체제의 색 고르기 창이 열립니다.
따로 정하지 않으면 검정(#000000) 입니다.
value 는 #ff0000 처럼 우물 정 자와 여섯 자리로 씁니다.
우물 정 자를 빠뜨리고 ff0000 이라고만 쓰면 브라우저가 못 알아듣고
조용히 검정이 됩니다. 에러는 나지 않습니다.
못 알아듣는 값은 무엇이든 검정으로 바뀐다고 기억하세요.

화면 인원수 칸은 평범한 네모로 보이다가, 마우스를 올리면 오른쪽 끝에

위아래 화살표가 나타납니다. 방문 날짜 칸에는 “연도-월-일” 이 회색으로 있고 오른쪽에 달력 그림이
있습니다. 만족도는 회색 막대에 파란 부분과 동그란 손잡이가 있고 손잡이가 가운데입니다. 좋아하는 색은
까맣게 칠해진 작은 네모 버튼으로 보입니다

인원수
방문 날짜
만족도
좋아하는 색

type="number" 는 숫자처럼 생긴 것을 받는 칸입니다. 전화번호나 우편번호에 쓰면 안 됩니다. 앞자리 0
이 사라지고, 화살표로 전화번호를 1씩 올리는 이상한 일이 생깁니다. 그런 값은 type="tel" 이나
type="text" 를 쓰세요.

▣ 직접 해보기 2

좋아하는 색 칸에 value="#ff0000" 을 넣어 보세요. 새로고침하면 까맣던 네모가 빨갳게 바뀝니다.
그다음 우물 정 자만 빼서 value="ff0000" 으로 바꿔 보면 다시 까맣게 돌아갑니다.

체크박스

섹션 3

네모를 눌러 켜고 끕니다. 몇 개를 켜든 상관없습니다. 각각이 따로 노는 스위치입니다.

체크박스는 “예 / 아니오” 스위치 하나입니다. 개수 제한이 없습니다. 약관 동의, 관심 분야 고르기, 옵션
추가에 씁니다.

```
<input type="checkbox" name="topping" value="cheese" /> 치즈 추가
```

여기서 value 가 지금까지와 다르게 쓰입니다.

글자 칸의 value → 칸에 미리 들어가는 값 (사용자가 고칠 수 있음)
 체크박스의 value → 켜졌을 때 보내지는 값 (사용자가 못 고침)

그러니까 위 코드는 이런 뜻입니다. "이 네모가 켜져 있으면 topping=cheese 를 보내라." 꺼져 있으면 아무것도 안 보냅니다. topping= 이라는 빈 값도 안 갑니다.

value 를 안 쓰면 어떻게 될까요? 켜졌을 때 on 이라는 값이 갑니다.

```
topping=on
```

무엇을 골랐다는 것인지 알 수 없습니다. 그래서 체크박스에는 value 를 꼭 적으세요.

checked 는 "처음부터 켜 둔다" 는 뜻입니다. 값이 필요 없는 속성이라 이름만 씁니다.

```
<input type="checkbox" name="agree" value="yes" checked /> 동의합니다
```

checked="true" 라고 쓸 필요가 없습니다. checked="false" 라고 써도 켜집니다. 이름이 적혀 있다는 사실 자체가 "켄" 이라서 그렇습니다. 끄고 싶으면 checked 를 통째로 지우세요.

화면 회색 테두리 상자의 왼쪽 위에 "추가 재료 (여러 개 고를 수 있습니다)" 가 걸쳐 있고,

안에 작은 네모 세 개가 세로로 있습니다. 맨 위 치즈만 처음부터 파란 바탕에 흰 체크 표시로 켜져 있습니다. 글자를 눌러 세 개를 전부 켤 수도 있고 전부 끌 수도 있습니다

추가 재료 (여러 개 고를 수 있습니다)
 치즈
 베이컨
 올리브

위에서 세 칸의 name 이 모두 topping 으로 같습니다. 체크박스는 이름이 같아도 서로 간섭하지 않습니다. 켜진 것들의 값이 각각 따로 보내질 뿐입니다. (라디오 버튼은 정반대입니다. 바로 다음 섹션입니다)

▸ 직접 해보기 3

베이컨 칸에 checked 를 붙여 보세요. 새로고침하면 치즈와 베이컨 두 개가 켜진 채로 화면이 열립니다. 두 개가 동시에 켜지는 것이 체크박스의 성질입니다.

라디오

섹션 4

라디오 버튼은 name 이 같은 것끼리 한 무리가 됩니다. 한 무리 안에서는 하나만 켜집니다.

옛날 라디오의 채널 버튼에서 온 이름입니다. 하나를 누르면 눌러 있던 다른 버튼이 푹 하고 올라오는 그 버튼입니다.

성별, 결제 수단, 배송 방법처럼 “여러 개 중 딱 하나” 를 고를 때 씁니다.

여기서 초보자가 반드시 걸려 넘어지는 지점이 나옵니다.

브라우저는 무엇을 보고 “이 버튼들이 한 무리” 라고 판단할까요?

옆에 붙어 있어서? 아닙니다. 같은 fieldset 안에 있어서? 아닙니다. 같은 p 안에 있어서? 아닙니다.

오직 name 값이 같은지만 봅니다.

그래서 name 을 하나씩 다르게 지어 주면 버튼들이 서로 남남이 되어 전부 따로 켜집니다. 동그라미가 세 개 다 채워지는 이상한 화면이 나옵니다.

모양은 라디오인데 동작은 체크박스가 되어 버리는 것입니다. 예러는 당연히 안 납니다.

아래에서 잘못된 예와 올바른 예를 직접 눌러서 비교하세요. 반드시 세 개를 차례로 눌러 보세요. 눌러 봐야만 차이가 보입니다.

참고로 value 와 checked 는 체크박스과 똑같이 씁니다.

```
<input type="radio" name="pay" value="card" checked /> 카드  
<input type="radio" name="pay" value="cash" /> 현금
```

한 무리에 checked 를 두 개 이상 붙이면 마지막 하나만 켜집니다.

이런 실수를 합니다

잘못된 예 — name 을 하나씩 다르게 지음

```
<input type="radio" name="size1" value="s" /> 작은 것  
<input type="radio" name="size2" value="m" /> 중간 것  
<input type="radio" name="size3" value="l" /> 큰 것
```

이름이 셋 다 다르니 서로 남남입니다. 세 개를 차례로 눌러 보세요. 셋 다 켜지고 한 번 켜지면 꺼지지도 않습니다. “하나만 고르게 하려고 라디오를 썼는데 다 골라진다” 가 바로 이 상황입니다. 크기를 골라 주세요
작은 것 중간 것 큰 것

올바른 예 — name 을 전부 같게

```
<input type="radio" name="size" value="s" /> 작은 것  
<input type="radio" name="size" value="m" /> 중간 것  
<input type="radio" name="size" value="l" /> 큰 것
```

name 이 같은 라디오는 화면에서 멀리 떨어져 있어도 한 무리입니다. 반대로 나란히 붙어 있어도 name 이 다르면 남남입니다. 판단 기준은 위치가 아니라 name 하나뿐이라는 것을 기억하세요.

▹ 직접 해보기 4

잘못된 예의 name="size1", name="size2", name="size3" 을 전부 name="sizeFix" 로 고쳐 보세요. 새로고침한 뒤 눌러 보면 이제 하나만 켜집니다. 고치기 전과 후를 꼭 둘 다 눌러 보세요.

file 과 hidden

섹션 5

file 은 컴퓨터의 파일을 고르는 칸이고, hidden 은 화면에 아예 안 보이는 칸입니다.

file

```
<input type="file" name="photo" />
```

버튼 하나와 "선택된 파일 없음" 이라는 글자로 보입니다.
버튼을 누르면 컴퓨터의 파일 고르기 창이 열립니다.
파일을 고르면 그 자리에 파일 이름이 나옵니다.

이 칸에는 value 를 쓸 수 없습니다.
"이 파일을 미리 골라 뒤라" 를 코드로 시키면 큰일 나기 때문입니다.
남의 컴퓨터 파일을 마음대로 가져가는 셈이 되니까요.

여러 개를 고르게 하려면 multiple 을 붙입니다.

hidden

```
<input type="hidden" name="pageId" value="notice-12" />
```

화면에 아무것도 안 그려집니다. 자리도 차지하지 않습니다.
그런데 값은 다른 칸들과 똑같이 보내집니다.

무엇에 쓸까요? 사용자가 건드릴 필요는 없지만
받는 쪽이 알아야 하는 값을 슬쩍 같이 보낼 때 씁니다.
예를 들어 "지금 몇 번 게시글에서 온 댓글인지" 같은 값입니다.

★ 안 보인다고 비밀이 아닙니다.
브라우저의 소스 보기로 누구나 볼 수 있습니다.
비밀번호처럼 남이 보면 안 되는 값을 hidden 에 넣으면 안 됩니다.

화면 "사진 올리기" 글자 오른쪽에 [파일 선택] 이라는 회색 버튼이 있고

그 옆에 "선택된 파일 없음" 이라는 글자가 있습니다. 그 아래 두 문장은 보통 문단 두 개가 이어진 것과 똑같은 간격으로 놓입니다. 사이에 hidden 칸이 끼어 있는데도 간격이 조금도 벌어지지 않았습니다

사진 올리기

바로 위 줄과 아래 줄 사이에 hidden 칸이 하나 들어 있습니다.

보이십니까? 안 보입니다. 자리도 전혀 차지하지 않습니다.

▹ 직접 해보기 5

id="hiddenField" 칸의 type="hidden" 을 type="text" 로 바꿔 보세요. 새로고침하면 숨어 있던 칸이 나타나고 안에 notice-12 가 들어 있습니다. 값은 원래부터 거기 있었다는 것을 확인하는 것이 목적입니다.

자주 하는 실수

섹션 6

여기 있는 다섯 가지는 전부 화면이 멀쩡하게 나옵니다. 눌러 보거나 값을 보내 봐야 알 수 있습니다.

이런 실수를 합니다

라디오의 name 을 다르게

이런 실수를 합니다

실수 1 — 라디오 버튼의 name 을 하나씩 다르게 지음

```
<input type="radio" name="pay1" value="card" /> 카드  
<input type="radio" name="pay2" value="cash" /> 현금
```

이 단원 최고의 함정입니다. 동그라미 두 개가 동시에 켜집니다. 한 번 켜지면 꺼지지도 않아서 사용자가 고칠 방법이 없습니다. 라디오를 쓸 때는 name 을 먼저 정하고 그 이름을 복사해 붙이세요.

이렇게 쓰세요

```
<input type="radio" name="pay" value="card" /> 카드  
<input type="radio" name="pay" value="cash" /> 현금
```

이런 실수를 합니다

체크박스에 value 를 안 씀

이런 실수를 합니다

실수 2 — 체크박스에 value 를 안 씀

```
<input type="checkbox" name="topping" /> 치즈  
<input type="checkbox" name="topping" /> 베이컨
```

화면은 완벽합니다. 켜고 끄기도 잘 됩니다. 그런데 켜서 보내면 받는 쪽에 topping=on 만 두 번 도착합니다. 치즈인지 베이컨인지 알 수 없습니다. 옆의 글자는 값이 아닙니다.

이런 실수를 합니다

checked 를 값으로 씀

이런 실수를 합니다

실수 3 — checked="false" 로 끄려고 함

```
<input type="checkbox" name="agree" checked="false" /> 동의
```

켜집니다. checked 는 값이 무엇이든 상관없이 “적혀 있으면 켜” 이기 때문입니다. 끄고 싶으면 checked 를 통째로 지우세요. 아래에서 직접 확인하세요. 왼쪽 네모가 처음부터 켜져 있습니다. 동의

이런 실수를 합니다

date 의 value 모양이 틀림

이런 실수를 합니다

실수 4 — 날짜 칸의 value 를 우리 식으로 씀

```
<input type="date" value="2026/03/15" />
```

빈 칸이 됩니다. 날짜 칸이 알아듣는 모양은 2026-03-15 하나뿐입니다. 빗금이나 한글이 섞이면 조용히 버려집니다. 아래 두 칸을 비교해 보세요. 왼쪽은 비어 있고 오른쪽에는 날짜가 들어 있습니다. 잘못된 값 올바른 값

이런 실수를 합니다

type 오타

이런 실수를 합니다

실수 5 — type 이름에 오타를 낸

```
<input type="chekbox" name="agree" /> 동의
```

모르는 type 은 전부 text 로 그려집니다. 네모 체크박스가 나와야 할 자리에 글자 칸이 덩그러니 나옵니다.
“왜 체크박스가 안 나오지?” 싶으면 type 철자부터 확인하세요. 동의

정리

▢ 직접 해보기

6

정답

1) 섹션 1 의 첫 칸을 이렇게 바꿉니다.

```
<input id="pwField" type="text" name="pw" />
```

→ 글자를 치면 점 대신 친 글자가 그대로 보입니다.

칸의 크기와 위치는 하나도 안 바뀝니다.
확인했으면 type="password" 로 되돌리세요.

2) 섹션 2 의 색 칸을 이렇게 바꿉니다.

```
<input id="colorField" type="color" name="favColor" value="#ff0000" />
```

→ 까망던 네모가 빨강게 칠해집니다.

우물 정 자를 뺀 value="#ff0000" 으로 바꾸면 다시 까망게 돌아갑니다.
못 알아듣는 글자는 조용히 검정으로 바뀌기 때문입니다.

3) 섹션 3 의 베이컨 줄에 checked 를 붙입니다.

```
<input id="checkBacon" type="checkbox" name="topping" value="bacon" checked /> 베이컨
```

→ 페이지를 열자마자 치즈와 베이컨 두 개가 켜져 있습니다.

체크박스는 이름이 같아도 서로 끄지 않기 때문입니다.

4) 섹션 4 잘못된 예의 세 줄에서 name 을 전부 sizeFix 로 바꿉니다.

```
<input id="badRadioS" type="radio" name="sizeFix" value="s" />  
<input id="badRadioM" type="radio" name="sizeFix" value="m" />  
<input id="badRadioL" type="radio" name="sizeFix" value="l" />
```

→ 이제 하나를 누르면 다른 것이 저절로 꺼집니다.

동그라미 세 개 중 항상 하나만 채워집니다.

5) 섹션 5 의 hidden 칸을 이렇게 바꿉니다.

```
<input id="hiddenField" type="text" name="pageId" value="notice-12" />
```

→ 두 문장 사이에 칸이 하나 나타나고 안에 notice-12 가 들어 있습니다.

문장 사이 간격도 그만큼 벌어집니다.

값은 숨어 있을 때도 그대로 있었다는 뜻입니다.

select 로 고르기, textarea 로 긴 글 받기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

목록을 펼쳐 보고, 긴 글 칸에 여러 줄을 쳐 보세요.

실행하면 나오는 화면 — select 로 고르기, textarea 로 긴 글 받기

지금까지 만든 칸은 전부 input 이었습니다. 그런데 input 으로는 잘 안 되는 일이 두 가지 있습니다.

1) 고를 것이 스무 개쯤 될 때

라디오 버튼 스무 개를 늘어놓으면 화면이 감당이 안 됩니다.

접었다 펼치는 목록이 필요합니다. → select

2) 여러 줄짜리 긴 글을 받을 때

input 은 한 줄짜리라 엔터를 쳐도 줄이 안 바뀝니다.

자기소개나 배송 요청 사항을 받을 칸이 필요합니다. → textarea

이 둘은 input 과 태그 이름부터 다릅니다. 그리고 둘 다 빈 태그가 아니라서 닫는 태그가 있습니다.

```
<input type="text" />
<select> ... </select>
<textarea> ... </textarea>
```

빈 태그. 닫지 않습니다.
안에 option 들이 들어갑니다.
안에 처음 값이 들어갑니다.

여기서 textarea 의 유명한 함정이 나옵니다. “안에 들어가는 것이 처음 값”이라는 말은 여는 태그와 닫는 태그 사이의 공백과 줄바꿈까지 전부 값이 된다는 뜻입니다. 섹션 5 에서 눈으로 확인합니다.

이 파일에는 접히는 목록과 여러 줄 입력칸이 살아 있는 채로 들어 있습니다. 목록을 펼쳐 보고, 긴 글 칸에 엔터를 쳐 가며 여러 줄을 써 보세요. 섹션 5 의 “공백이 값이 된다” 는 이 단원에서 두 번째로 큰 함정입니다.

select 와 option

섹션 1

<select> 가 목록 상자이고, 그 안의 <option> 하나하나가 고를 수 있는 항목입니다.

생김새는 이렇습니다.

```
<select name="city"> <option>서울</option> <option>부산</option> <option>대구</option>
</select>
```

select 는 상자이고 option 은 그 안에 들어가는 항목입니다. 03단원의 ul 과 li 관계를 떠올리면 쉽습니다. 바깥 태그 하나에 안쪽 태그 여러 개입니다.

name 은 select 에 붙입니다. option 에 붙이지 않습니다. 어차피 고른 것은 하나뿐이니 이름표도 하나면 됩니다.

화면에는 접힌 상태로 나옵니다. 아무것도 안 골랐을 때 보이는 것은 맨 위 option 입니다. 즉 select 는 “비어 있는 상태” 가 없습니다. 항상 무언가 골라져 있습니다.

이 성질 때문에 실무에서는 맨 위에 안내용 항목을 하나 넣는 일이 많습니다.

```
<option value="">고르세요</option>
```

value 를 빈 값으로 두면 “아직 안 골랐다” 를 표현할 수 있습니다. 개념06 의 required 와 같이 쓰면 “고르세요” 인 채로는 제출이 막힙니다.

화면 "사는 곳" 글자 오른쪽에 "서울" 과 아래쪽 화살표가 든 작은 상자가 있습니다.

상자를 누르면 서울·부산·대구 세 줄이 아래로 펼쳐지고, 하나를 고르면 다시 접히면서 상자 안 글자가 고른 것으로 바뀝니다

사는 곳

서울

부산

대구

상자의 가로 크기는 가장 긴 항목 글자에 맞춰 저절로 정해집니다. 그래서 항목 이름이 짧으면 상자도 아주 작아집니다. 크기를 직접 정하는 방법은 10단원에서 배웁니다.

▹ 직접 해보기 1

위 목록 맨 위에 `<option value="">고르세요</option>` 를 한 줄 넣어 보세요. 새로고침하면 처음 보이는 글자가 "서울" 에서 "고르세요" 로 바뀝니다.

value 와 selected

섹션 2

option 의 value 가 실제로 보내지는 값이고, 태그 사이의 글자는 화면에 보여 주는 글자입니다.

```
<option value="seoul">서울</option>
```

	↳ 화면에 보이는 글자
	↳ 보내지는 값

이렇게 나눠 놓으면 좋은 점이 두 가지입니다.

· 받는 쪽은 짧은 영어 값만 다루면 됩니다. · 나중에 화면 글자를 "서울" 에서 "서울특별시" 로 바뀌도 보내는 값은 seoul 그대로라 받는 쪽을 안 고쳐도 됩니다.

value 를 안 쓰면 어떻게 될까요? 화면에 보이는 글자가 그대로 값이 됩니다. 섹션 1 의 목록이 그런 경우였습니다. 동작은 합니다. 다만 나중에 글자를 바꾸는 순간 값도 같이 바뀌어 버립니다.

selected 는 "처음부터 이걸 골라 뒤라" 입니다. 개념03 의 checked 와 똑같이 이름만 쓰면 됩니다.

```
<option value="busan" selected>부산</option>
```

한 select 안에 selected 를 두 개 이상 쓰면 마지막 것 하나만 골라집니다. 조용히 그렇게 됩니다. 에러는 없습니다.

화면 상자 안에 처음부터 "부산" 이 보입니다.

펼치면 부산 줄이 파란 바탕으로 칠해져 있어 지금 골라진 것임을 알 수 있습니다. 화면 어디에도 seoul, busan 같은 영어는 보이지 않습니다

배송 지역

서울
부산
대구

▹ 직접 해보기 2

위 목록에서 selected 를 "대구" 줄로 옮겨 보세요. 새로고침하면 처음 보이는 글자가 "대구" 가 됩니다. value 는 그대로 두세요. 화면 글자만 바뀌는 것을 확인하는 것이 목적입니다.

optgroup, multiple, size

섹션 3

항목이 많아지면 optgroup 으로 묶고, 여러 개를 고르게 하려면 multiple 을 붙입니다.

optgroup

항목이 스무 개쯤 되면 그냥 늘어놓는 것으로는 못 찾습니다.
비슷한 것끼리 묶고 제목을 겁니다.

```
<select> <optgroup label="수도권"> <option>서울</option> <option>인천</option> </optgroup> <optgroup label="영남"> <option>부산</option> </optgroup> </select>
```

label 속성에 쓴 글자가 묶음 제목으로 나옵니다.
제목은 회색 굵은 글씨로 보이고 고를 수 없습니다. 눌러도 반응이 없습니다.
제목 아래 항목들은 한 칸 들여쓰여 보입니다.

multiple

여러 개를 고를 수 있게 만듭니다.
붙이는 순간 접히는 상자가 아니라 펼쳐진 목록판으로 바뀝니다.

컴퓨터에서 여러 개 고르기 : Ctrl 키를 누른 채 클릭
연달아 여러 개 고르기 : 첫 항목 클릭 후 Shift 키를 누른 채 마지막 항목 클릭

솔직히 말하면 쓰기가 불편해서 실무에서는 잘 안 씁니다.
여러 개를 고르게 하려면 보통 체크박스를 씁니다.
그래도 이런 것이 있다는 것은 알아 두세요.

size

몇 줄을 펼쳐 놓을지 정합니다. size="3" 이면 세 줄이 보이는 목록판이 됩니다.
multiple 없이 size 만 써도 접히지 않은 목록판이 됩니다.
이때도 고를 수 있는 것은 하나뿐입니다.

화면 첫째는 접힌 상자입니다. 펼치면 "수도권" 과 "영남" 이 회색 굵은 글씨로 보이고

그 아래 지역 이름들이 한 칸 안으로 들여쓰여 있습니다. 둘째와 셋째는 접히지 않고 테두리 안에 항목이 세로로 나열된 목록판입니다. 둘째는 Ctrl 을 누른 채 누르면 여러 줄이 파랗게 칠해집니다. 셋째는 아무리 눌러도 한 줄만 파랗게 칠해집니다

묶은 목록

서울
인천

부산
대구

여러 개 고르기

영화
음악
게임
독서

펼쳐 둔 목록

서울
부산
대구

➤ 직접 해보기 3

“여러 개 고르기” 목록에서 multiple 만 지워 보세요. 새로고침하면 목록판 모양은 그대로인데 이제 한 줄만 골라집니다. size 는 모양을, multiple 은 개수를 정한다는 뜻입니다.

textarea

섹션 4

rows 는 보이는 줄 수, cols 는 보이는 가로 글자 수입니다.

```
<textarea name="memo" rows="4" cols="40"></textarea>
```


rows="4" 네 줄이 보이는 높이로 만듭니다. cols="40" 영어 소문자 기준 마흔 글자쯤 들어가는 너비로 만듭니다.

둘 다 안 쓰면 두 줄, 스무 글자짜리 작은 칸이 나옵니다.

여기서 오해하기 쉬운 점이 두 가지 있습니다.

- 1 rows 는 "최대 줄 수" 가 아닙니다. 보이는 높이일 뿐입니다. 네 줄을 넘겨 쓰면 칸 안에 세로 스크롤 막대가 생기고 계속 쓸 수 있습니다.
- 2 cols 는 정확한 픽셀 크기가 아닙니다. 한글은 영어보다 넓어서 마흔 글자가 안 들어갑니다. 크기를 정확히 정하려면 10단원의 width, height 를 써야 합니다.

오른쪽 아래 모서리에 빗금 친 작은 삼각형이 보일 겁니다. 그걸 끌면 사용자가 칸 크기를 직접 늘릴 수 있습니다. 브라우저가 그냥 주는 기능입니다.

그리고 input 과 크게 다른 점이 하나 더 있습니다. textarea 에는 value 속성이 없습니다. 처음 값은 여는 태그와 닫는 태그 사이에 씁니다. 그래서 섹션 5 의 함정이 생깁니다.

화면 위쪽에 가로로 긴 빈 네모 칸이 있고 네 줄 높이입니다.

아래쪽 칸은 눈에 띄게 작아서 두 줄 높이이고, 가로는 위 칸의 절반쯤입니다. 재 보면 위 칸이 301px, 아래 칸이 161px 입니다. 두 칸 모두 오른쪽 아래 모서리에 작은 빗금 삼각형이 있고, 그것을 마우스로 끌면 칸이 커집니다. 네 줄을 넘겨 쓰면 칸 오른쪽에 세로 스크롤 막대가 생깁니다

배송 요청 사항

크기를 안 정한 칸

▹ 직접 해보기

4

위 칸의 rows="4" 를 rows="10" 으로 바꿔 보세요. 새로고침하면 칸이 세로로 길어집니다. cols 도 60 으로 바꿔 가로 길이를 늘려 보세요.

공백 함정

섹션 5

textarea 는 여는 태그와 닫는 태그 사이의 모든 글자를 처음 값으로 삼습니다. 눈에 안 보이는 띄어쓰기와 줄바꿈까지 포함해서입니다.

HTML 을 쓰다 보면 안쪽 태그를 들여쓰는 것이 습관이 됩니다. 01단원에서 그렇게 배웠고, 그래야 읽기 좋으니까요.

그런데 textarea 에서만은 그 습관이 사고를 냅니다. 이렇게 쓴다고 해 봅시다.

```
<textarea name="memo" rows="3" cols="30">
</textarea>
```

보기에는 빈 칸입니다. 실제로는 빈 칸이 아닙니다. 줄바꿈과 들여쓰기 공백이 값으로 들어가 있습니다.

규칙은 딱 하나입니다.

여는 태그 바로 뒤에 오는 줄바꿈 하나만 버려집니다.
그 뒤의 공백과 줄바꿈은 전부 값이 됩니다.

그림으로 보면 이렇습니다.

<textarea>안녕하세요</textarea>

| | |

| | | 이 줄바꿈과 공백도 값입니다

| | | 이 공백 네 칸이 값의 앞에 붙습니다

| | | 이 줄바꿈 하나만 버려집니다

결과가 어떻게 될까요? 칸 안의 글자가 오른쪽으로 밀려서 보입니다. 그리고 사용자가 아무것도 안 쳐도 그 공백들이 그대로 보내집니다.

더 나쁜 것은 “글자를 안 썼는지” 를 검사할 때입니다. 개념06 의 required 는 “비었는지” 를 봅니다.
공백이라도 들어 있으면 비어 있지 않으므로 검사를 그냥 통과해 버립니다.

해결은 아주 간단합니다.

<textarea name="memo" rows="3" cols="30"></textarea>

여는 태그와 닫는 태그를 꼭 붙입니다.

보기에는 답답해도 이렇게 써야 합니다. 아래 두 칸을 비교해 보세요. 글자가 시작하는 자리가 다릅니다.

이런 실수를 합니다

잘못된 예 — 보기 좋게 들여쓰

```
<textarea rows="3" cols="30">
안녕하세요
</textarea>
```

글자가 오른쪽으로 밀려서 시작합니다. 앞에 공백 네 칸이 값으로 들어가 있기 때문입니다. 칸 안을 눌러 커서를 맨 앞으로 옮긴 뒤 백스페이스를 눌러 보면 공백이 하나씩 지워집니다. 안녕하세요

올바른 예 — 딱 붙여서 씸

```
<textarea rows="3" cols="30">안녕하세요</textarea>
```

진짜로 빈 칸을 만들고 싶으면 <textarea></textarea> 처럼 사이에 아무것도 넣지 마세요. 엔터 한 번, 스페이스 한 칸도 안 됩니다.

➤ 직접 해보기 5

잘못된 예의 <textarea> 안에서 줄바꿈과 공백을 전부 지워 <textarea id="trapArea" rows="3" cols="30">안녕하세요</textarea> 로 만들어 보세요. 새로고침하면 글자가 왼쪽 벽에 붙습니다.

datalist

섹션 6

datalist 는 글자 칸에 추천 목록을 붙이는 태그입니다. 목록에 없는 값을 직접 쳐 넣어도 됩니다.

select 와 헛갈리기 쉬우니 차이부터 봅시다.

select	목록 안에서만 고를 수 있습니다. 직접 쳐 넣을 수 없습니다.
datalist	추천만 해 줍니다. 목록에 없는 값을 쳐 넣어도 됩니다.

쓰는 방법은 두 부분으로 나뉩니다.

```
<input type="text" name="city" list="cityList" />
<datalist id="cityList"> <option value="서울"></option> <option value="부산">
</option>
</datalist>
```

input 의 list 값과 datalist 의 id 값을 같게 맞추는 겁니다. label 의 for 와 id 를 맞추던 것과 똑같은 방식입니다.

datalist 자체는 화면에 안 보입니다. 자리도 차지하지 않습니다. 칸을 누르거나 글자를 치기 시작할 때만 아래로 추천이 펼쳐집니다.

여기서 option 의 생김새가 select 때와 다릅니다.

select 안	<code><option value="seoul">서울</option></code>	태그 사이에 보이는 글자
datalist 안	<code><option value="서울"></option></code>	value 만 씁니다

datalist 는 "칸에 넣을 값" 을 추천하는 것이라 value 만 있으면 됩니다.

브라우저마다 생김새가 조금씩 다르고, 오래된 브라우저에서는 추천이 안 뜹니다. 그래도 안 뜰 뿐 칸은 멀쩡히 동작합니다. 그래서 부담 없이 쓸 수 있습니다. 이 자료에서는 맛만 봅니다.

화면 평범한 글자 칸 하나만 보입니다. `datalist` 는 어디에도 안 보입니다.

칸을 누르면 아래로 서울·부산·대구·광주 네 줄이 펼쳐집니다. “서”를 치면 “서울” 한 줄만 남습니다. 목록에 없는 “제주”를 쳐 넣어도 그대로 받아들여집니다.

사는 곳

위 datalist 안에 `<option value="대전"></option>` 을 한 줄 넣어 보세요. 새로그침하고 칸을 누르면 추천이 다섯 줄로 늘어납니다.

자주 하는 실수

섹션 7

textarea 관련 실수는 값이 조용히 오염되는 종류입니다. 화면만 슬쩍 보서는 잘 모릅니다.

이런 실수를 합니다

textarea 안 공백

이런 실수를 합니다

실수 1 — textarea 안을 보기 좋게 들여쓰

```
<textarea name="memo" rows="3" cols="30">
</textarea>
```

빈 칸을 만들려던 것인데 줄바꿈과 공백이 값으로 들어갑니다. 사용자가 아무것도 안 써도 빈 값이 아니게 되고, 개념06 의 required 검사도 그냥 통과해 버립니다. 아래 칸을 누른 뒤 Ctrl+A 로 전체 선택을 해 보세요. 숨어 있던 공백이 드러납니다.

이렇게 쓰세요

```
<textarea name="memo" rows="3" cols="30"></textarea>
```

이런 실수를 합니다

textarea 를 빈 태그처럼 씀

이런 실수를 합니다

실수 2 — textarea 를 input 처럼 빗금으로 닫음

```
<textarea rows="2" cols="30" />그다음에 쓴 글자
```

textarea 는 빈 태그가 아닙니다. 끝의 빗금은 무시되고 태그가 열린 채로 남습니다. 그래서 뒤에 오는 글자가 전부 칸 안으로 빨려 들어갑니다. 닫는 태그를 만날 때까지 계속 먹습니다. 페이지 절반이 칸 안에 들어가기도 합니다. 아래 살아 있는 예에는 `</textarea>` 를 한 문장 뒤에 일부러 붙여 두었습니다. 안 붙이면 이 페이지의 나머지가 전부 칸 안으로 빨려 들어가 아래 내용을 하나도 못 보게 되기 때문입니다. 그래서 여기서는 한 문장만 삼켜진 모습을 보여 줍니다. 그다음에 쓴 글자

이런 실수를 합니다

textarea 에 value

이런 실수를 합니다

실수 3 — textarea 에 value 를 씀

```
<textarea rows="2" cols="30" value="안녕하세요">
</textarea>
```

input 버릇대로 쓴 것입니다. 아무 일도 안 일어납니다. 칸은 빈 채로 나오고 경고도 없습니다. textarea 에는 value 속성이 아예 없습니다. 처음 값은 태그 사이에 씁니다.

이런 실수를 합니다

select 안에 option 을 안 씀

이런 실수를 합니다

실수 4 — select 안에 option 없이 글자만 씀

```
<select name="city">서울 부산 대구</select>
```

아무 항목도 없는 아주 작은 빈 상자가 나옵니다. 눌러도 펼쳐지지 않습니다. 글자는 화면에 나오지도 않습니다. select 는 option 만 항목으로 인정하기 때문입니다. 서울 부산 대구

이런 실수를 합니다

selected 를 여러 개

이런 실수를 합니다

실수 5 — 한 select 안에 selected 를 여러 개 씀

```
<option value="a" selected>가</option>
<option value="b" selected>나</option>
```

둘 다 골라지지 않습니다. 마지막 것 하나만 골라집니다. 여러 개를 고르게 하려면 multiple 을 붙여야 하고, 그때는 목록판 모양으로 바뀝니다. 가 나

정리

▫ 직접 해보기 7

정답

1) 섹션 1 의 select 안 맨 위에 이렇게 넣습니다.

```
<option value="">고르세요</option>
```

→ 접힌 상자에 처음 보이는 글자가 "고르세요" 로 바뀝니다.

펼치면 항목이 네 줄이 되고 맨 위가 "고르세요" 입니다.

2) 섹션 2 에서 selected 를 옮깁니다.

```
<option value="seoul">서울</option>
<option value="busan">부산</option>
<option value="daegu" selected>대구</option>
```

→ 처음 보이는 글자가 "부산" 에서 "대구" 로 바뀝니다.

value 는 그대로이므로 보내지는 값만 daegu 로 달라집니다.

3) 섹션 3 의 두 번째 select 에서 multiple 을 지웁니다.

```
<select id="multiSelect" name="hobby" size="4">
```

→ 네 줄이 보이는 목록판 모양은 그대로입니다.

그런데 이제 Ctrl 을 눌러도 한 줄만 파랗게 칠해집니다.

4) 섹션 4 의 첫 칸을 이렇게 바꿉니다.

```
<textarea id="memoArea" name="memo" rows="10" cols="60"></textarea>
```

→ 칸이 세로로 두 배 넘게 길어지고 가로로도 넓어집니다.

한글로 예순 글자가 다 안 들어가는 것도 같이 확인해 보세요.

5) 섹션 5 잘못된 예의 textarea 를 한 줄로 붙여 씁니다.

```
<p><textarea id="trapArea" rows="3" cols="30">안녕하세요</textarea></p>
```

→ 밀려 있던 "안녕하세요" 가 칸의 왼쪽 위 모서리에 딱 붙습니다.

아래 올바른 예와 똑같은 모양이 됩니다.

6) 섹션 6 의 datalist 안에 이렇게 넣습니다.

```
<option value="대전"></option>
```

→ 칸을 눌렀을 때 펼쳐지는 추천이 네 줄에서 다섯 줄로 늘어납니다.

"대" 를 치면 대구와 대전 두 줄만 남습니다.

버튼, 그리고 값이 어디로 가는가

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

누르라고 적힌 버튼만 누르세요. 누르지 말라고 적힌 버튼도 있습니다.

실행하면 나오는 화면 — 버튼, 그리고 값이 어디로 가는가

지금까지 칸은 실컷 만들었는데 보내는 방법은 안 배웠습니다. 보내는 일을 시키는 것이 버튼입니다.

버튼 태그는 이렇게 생겼습니다.

```
<button>보내기</button>
```

└──┬──┬──┬──┘
 | | | |
 | | └─┬─┘ 닫는 태그
 | └──┬─┘ 버튼에 보일 글자
 └──┬──┘ 여는 태그

input 과 달리 빈 태그가 아닙니다. 태그 사이에 넣은 글자가 버튼에 적힙니다.

그런데 버튼에는 아주 중요한 속성이 하나 있습니다. type 입니다. input 의 type 과 이름은 같지만 뜻이 다릅니다. 버튼의 type 은 “이 버튼을 누르면 무슨 일이 일어나는가” 를 정합니다.

type="submit"	폼을 제출합니다. 페이지가 넘어갑니다.
type="button"	아무 일도 안 일어납니다.
type="reset"	폼 안의 값을 처음 상태로 되돌립니다.

★ 이 파일에서 가장 중요한 한 줄입니다. type 을 안 쓰면 기본이 submit 입니다. 즉 아무 생각 없이 <button>버튼</button> 이라고 쓰면 그 버튼은 누르는 순간 폼을 제출합니다. “그냥 장식으로 놔둔 버튼인데 왜 화면이 새로고침되지?” 의 원인이 이것입니다.

이 파일에는 눌러도 되는 버튼과 지금은 누르지 말아야 하는 버튼이 섞여 있습니다. 상자마다 어느 쪽인지 적어 두었으니 읽고 누르세요. 누르지 말라는 버튼을 눌러도 큰일이 나지는 않습니다. 페이지가 새로고침될 뿐입니다.

button 태그

섹션 1

<button> 은 태그 사이에 넣은 글자가 버튼에 그대로 적히는 태그입니다.

input 은 빈 태그라 안에 아무것도 못 넣었습니다. button 은 다릅니다. 여는 태그와 닫는 태그가 있고, 사이에 글자를 넣습니다.

```
<button type="button">보내기</button>
```

사이에 넣을 수 있는 것이 글자만은 아닙니다. 02단원에서 배운 태그를 넣어도 됩니다.

```
<button type="button"><strong>지금</strong> 신청하기</button>
```

이렇게 하면 버튼 안에서 “지금” 만 굵게 나옵니다. 이것이 뒤에서 볼 input type=“submit” 과의 결정적인 차이입니다.

아래 상자의 버튼은 둘 다 type=“button” 이라 눌러도 아무 일이 없습니다. 마음 놓고 눌러 보세요. 놀리는 느낌만 납니다.

화면	열은 회색 바탕에 가는 테두리가 있는 작은 버튼 두 개가 나란히 있습니다.
----	---

왼쪽에는 “보내기”, 오른쪽에는 “지금 신청하기” 가 적혀 있고 오른쪽 버튼의 “지금” 두 글자만 굵습니다. 누르면 잠깐 눌린 모양이 되었다가 원래대로 돌아옵니다

보내기
지금 신청하기

버튼의 크기는 안에 적힌 글자 길이에 맞춰 저절로 정해집니다. 그래서 글자가 긴 버튼이 더 넓습니다. 크기와 색을 직접 정하는 것은 09~10단원입니다.

위 상자에 버튼을 하나 더 만들어 여러분이 원하는 글자를 넣어 보세요. type="button" 을 꼭 같이 쓰세요. 이유는 바로 다음 섹션에서 나옵니다.

type 을 안 쓰면

섹션 2

type 을 빠뜨린 버튼은 누르는 순간 폼을 제출합니다. 폼이 제출되면 페이지가 넘어가고 쳐 넣은 값이 전부 사라집니다.

이렇게 쓴 버튼이 있다고 합시다.

```
<form> <input type="text" name="userName" /> <button>버튼</button>
</form>
```

type 을 안 썼으니 이 버튼의 type 은 submit 입니다. 브라우저가 그렇게 정합니다. 누르면 무슨 일이 일어날까요? 직접 재 본 결과는 이렇습니다.

1) 주소창 끝에 물음표와 값이 붙습니다.

```
... /개념05_버튼과_전송.html?userName=김민준
```

2) 그 주소로 페이지가 다시 열립니다. 즉 새로고침됩니다.

3) 새로 열린 페이지의 칸은 처음 상태로 돌아갑니다.
방금 쳐 넣은 값이 화면에서 사라집니다.

받아 줄 서버가 없으니 값이 어디에 저장되지도 않습니다. 사용자가 보기에는 "아무 일도 안 하고 화면만 깜빡였다" 입니다.

그래서 초보자가 이런 질문을 하게 됩니다. "그냥 눌러 보라고 만든 버튼인데 왜 화면이 새로고침되죠?" 답은 언제나 같습니다. type 을 안 썼기 때문입니다.

규칙을 이렇게 외우지 말고 이렇게 정하세요.

버튼을 만들 때는 type 을 먼저 씁니다.
제출할 버튼이면 type="submit", 아니면 type="button".

참고로 폼 안의 글자 칸에서 엔터를 쳐도 제출됩니다. 칸이 하나뿐인 폼에서 특히 그렇습니다. 검색창에서 엔터를 치면 검색되는 이유입니다.

화면 "이름" 이름표와 빈 칸이 한 줄, 그 아래에 "보내기" 라고 적힌

회색 버튼이 한 줄로 보입니다. 섹션 1 의 버튼과 생김새가 완전히 똑같습니다. type 을 썼는지 안 썼는지는 화면으로 구별할 수 없습니다

이름

보내기

위 버튼을 눌러도 컴퓨터가 망가지지는 않습니다. 주소창 끝에 ?userName= 이 붙으면서 페이지가 새로고침될 뿐입니다. 원래대로 돌아오려면 주소창의 물음표 뒷부분을 지우고 엔터를 치거나, 파일을 다시 열면 됩니다.

▸ 직접 해보기 2

위 버튼에 type="button" 을 붙여 보세요. 새로고침한 뒤 칸에 글자를 쳐 넣고 버튼을 눌러 보면 이번에는 아무 일도 안 일어나고 쳐 넣은 값도 그대로 남아 있습니다.

type="button"

섹션 3

05

type="button" 은 누르면 정말 아무 일도 안 일어나는 버튼입니다.

"아무 일도 안 하는 버튼이 왜 필요하지?" 싶을 겁니다.

실제 웹페이지에는 제출과 상관없는 버튼이 아주 많습니다.

- 목록을 접었다 펼치는 버튼
- 사진을 다음 장으로 넘기는 버튼
- 항목을 한 줄 더 추가하는 버튼

이런 버튼들이 하는 일은 전부 자바스크립트가 맡습니다. HTML 은 "버튼이 여기 있다" 까지만 만들고, 눌렀을 때 할 일은 따로 붙입니다. 그때 이 버튼이 폼을 제출해 버리면 안 되니까 type="button" 으로 막아 두는 것입니다.

이 자료에서는 자바스크립트를 쓰지 않으므로 type="button" 버튼은 정말로 아무 일도 안 합니다. 눌리는 모양만 잠깐 보였다가 원래대로 돌아옵니다.

아래 상자에서 마음껏 눌러 보세요. 칸에 글자를 쳐 넣고 눌러도 글자가 그대로 남아 있는지 확인하세요.

화면 이름표와 칸이 한 줄, 그 아래에 "눌러도 아무 일 없는 버튼" 이라고 적힌

가로로 긴 회색 버튼이 있습니다. 버튼을 눌러도 화면이 새로고침되지 않고 칸에 쳐 넣은 글자도 그대로 남습니다

여기에 아무 글자나 쳐 보세요

눌러도 아무 일 없는 버튼

▣ 직접 해보기 3

위 칸에 글자를 쳐 넣고 버튼을 다섯 번쯤 눌러 보세요. 글자가 그대로 남아 있는 것을 확인하세요. 섹션 2의 버튼과 정반대입니다.

type="reset"

섹션 4

type="reset"은 빈 칸으로 만드는 것이 아니라 페이지를 처음 열었을 때의 값으로 되돌립니다.

여기서 오해가 하나 자주 생깁니다. reset을 "지우기"라고 생각하는 것입니다. 아닙니다.

지우기	모든 칸을 빈 칸으로 만든다
되돌리기	페이지를 처음 열었을 때의 값으로 돌린다

value로 미리 채워 둔 칸이 있으면 그 값이 다시 나타납니다. checked로 켜 두었던 체크박스도 다시 켜집니다.

아래 상자로 직접 확인하세요. 순서대로 해 보세요.

- 1 이름 칸에는 처음부터 "김민준"이 들어 있습니다.
- 2 그 글자를 지우고 다른 이름을 쳐 넣으세요.
- 3 체크박스를 꺼 보세요. 처음에는 켜져 있습니다.
- 4 "되돌리기" 버튼을 누르세요.
- 5 이름이 "김민준"으로 돌아오고 체크박스도 다시 켜집니다.

빈 칸이 되는 것이 아니라는 점을 눈으로 확인하는 것이 중요합니다.

되돌리기 버튼은 자기가 들어 있는 form 안의 칸만 되돌립니다. 다른 form의 칸은 건드리지 않습니다.

실무에서는 이 버튼을 잘 안 씁니다. 열심히 채운 사용자가 실수로 눌렀다가 다 날아가면 화가 나기 때문입니다. 그래도 type의 세 가지 값 중 하나이니 알아 두세요.

화면 이름 칸에 검은 글씨로 "김민준"이 들어 있고,

아래 네모는 파란 바탕에 흰 체크 표시로 켜져 있습니다. 값을 바꾸고 체크를 끈 다음 "되돌리기"를 누르면 이름이 "김민준"으로 돌아오고 체크도 다시 켜집니다. 이름 칸이 빈 칸이 되지는 않습니다

이름

광고 메일을 받겠습니다

되돌리기

▫ 직접 해보기 4

이름 칸의 value="김민준" 을 지우고 새로고침한 뒤 다시 해 보세요. 이번에는 되돌리기를 누르면 칸이 빈 칸이 됩니다. "처음 상태" 가 무엇이냐에 따라 결과가 달라진다는 뜻입니다.

input type="submit" 과의 차이

섹션 5

05

버튼을 만드는 방법이 하나 더 있습니다. <input type="submit" /> 입니다. 글자를 넣는 자리가 다릅니다. 두 가지를 나란히 놓고 봅시다.

<button type="submit">보내기</button>
└ 태그 사이에 글자를 넣습니다

<input type="submit" value="보내기" />
└ value 속성에 글자를 넣습니다

화면에 나오는 버튼은 거의 똑같이 생겼습니다. 크기까지 같습니다. 그런데 쓸 수 있는 것이 다릅니다.

button	안에 태그를 넣을 수 있습니다. <button>지금 신청</button> 처럼요. 나중에 그림 아이콘을 넣을 수도 있습니다.
input submit	빈 태그라 안에 아무것도 못 넣습니다. 글자 한 줄만 넣을 수 있습니다.

그리고 input type="submit" 에서 value 를 빠뜨리면 브라우저가 정한 기본 글자가 나옵니다. 크롬 한국어에서는 "제출" 입니다. type="reset" 에서 value 를 빠뜨리면 "초기화" 가 나옵니다. 우리가 정한 글자가 아니라서 화면이 갑자기 어색해집니다. value 를 꼭 쓰세요.

어느 쪽을 쓸까요? 요즘은 button 을 씁니다. 안에 무엇이든 넣을 수 있어서 더 자유롭기 때문입니다. 옛날 자료에는 input type="submit" 이 많이 나오니 읽을 줄은 알아야 합니다.

아래 네 개는 전부 form 밖에 있습니다. 그래서 눌러도 제출되지 않습니다. 생김새만 비교하세요.

화면 회색 버튼 네 개가 한 줄에 나란히 있습니다.

앞의 두 개는 "보내기" 로 글자도 크기도 똑같아 구별이 안 됩니다. 셋째에는 "제출", 넷째에는 "초기화" 라고 적혀 있습니다. 우리가 쓴 적 없는 글자가 브라우저에서 저절로 나온 것입니다

보내기

버튼이 form 밖에 있으면 제출 기능이 동작하지 않습니다. 위 네 개가 그렇습니다. 보내야 하는 버튼은 반드시 <form> 안에 넣으세요.

▣ 직접 해보기 5

위 상자의 id="defaultSubmit" 버튼에 value="신청하기" 를 넣어 보세요. 새로고침하면 "제출" 이라고 적혀 있던 버튼의 글자가 "신청하기" 로 바뀌고 버튼도 그만큼 넓어집니다.

action 과 method

섹션 6

action 은 어디로 보낼지, method 는 어떻게 보낼지입니다. 뜻만 알아 두세요.

진짜 웹사이트의 폼은 보통 이렇게 생겼습니다.

```
<form action="/join" method="post">
...
</form>
```

action

값을 받아 줄 주소입니다. 04단원에서 배운 링크 주소와 같은 모양입니다.
안 쓰면 "지금 이 페이지 주소" 로 보냅니다.
그래서 이 자료의 폼들은 눌렀을 때 자기 자신을 다시 여는 것입니다.

method

보내는 방식입니다. 두 가지가 있습니다.

get 값이 주소 뒤에 붙어서 갑니다.

```
/search?q=고양이&page=2
```

주소창에 다 보입니다. 그래서 그 주소를 복사해 남에게 줄 수 있습니다.
검색 결과처럼 "이 화면을 다시 보고 싶다" 는 경우에 맞습니다.
주소에 넣는 것이라 아주 긴 값은 못 보냅니다.

post 값이 주소에 안 붙고 따로 갑니다.

주소창에 안 보입니다. 길이 제한도 사실상 없습니다.
회원가입, 글쓰기, 로그인처럼 "무언가를 새로 만드는" 경우에 씁니다.

안 쓰면 get 입니다.

여기서 오해를 하나 막아 둡시다. post 는 주소창에 안 보일 뿐이지 비밀이 아닙니다. 마음먹고 들여다보면 다 보입니다. 안전하게 만드는 일은 또 다른 이야기입니다.

★ 이 자료에서는 action 을 아예 쓰지 않습니다. 값을 받아 줄 서버가 없기 때문입니다. 받는 쪽을 만드는 것은 HTML 의 일이 아니라서 이 단원의 범위 밖입니다.

아래 폼에는 method="get" 만 써 두었습니다. 화면에는 아무 차이도 안 나타납니다. 속성은 화면이 아니라 동작을 정하기 때문입니다.

화면 "검색어" 이름표와 칸이 한 줄, 그 아래 "검색" 이라고 적힌 회색 버튼이 있습니다.

method="get" 을 썼지만 화면에는 아무 흔적도 나타나지 않습니다. 섹션 2 의 폼과 생김새가 똑같습니다

검색어

검색

▫ 직접 해보기

6

위 폼의 method="get" 을 지우고 새로고침해 보세요. 화면이 하나도 안 바뀝니다. method 를 안 쓰면 원래 get 이기 때문입니다. 확인했으면 되돌려 놓으세요.

자주 하는 실수

섹션 7

버튼의 실수는 누르는 순간에만 드러납니다. 화면을 아무리 들여다봐도 안 보입니다.

이런 실수를 합니다

type 을 안 씀

이런 실수를 합니다

실수 1 — 버튼에 type 을 안 씀

```
<form>
<button>한 줄 더 추가</button>
</form>
```

제출과 아무 상관 없는 버튼인데 누르는 순간 폼이 제출됩니다. 화면이 새로고침되고 사용자가 채운 값이 전부 날아갑니다. 이 단원에서 가장 자주 나는 사고입니다. type 을 안 쓰면 기본이 submit 이라는 것을 잊지 마세요.

이렇게 쓰세요

```
<form>
<button type="button">한 줄 더 추가</button>
</form>
```

이런 실수를 합니다

button 에 **value** 로 글자를 넣음

이런 실수를 합니다

실수 2 — button 의 글자를 **value** 에 씀

```
<button type="button" value="보내기"></button>
```

input 버릇대로 쓴 것입니다. button 의 글자는 태그 사이에 넣어야 합니다. value 에 쓴 글자는 화면에 안 나옵니다. 글자가 하나도 없는 납작한 버튼이 나옵니다. 아래에서 직접 보세요. 처음에는 버튼이 있는지도 잘 안 보입니다.

이렇게 쓰세요

```
<button type="button">보내기</button>
```

이런 실수를 합니다

input submit 안에 글자를 넣음

이런 실수를 합니다

실수 3 — input type="submit" 안에 글자를 넣음

```
<input type="submit">보내기</input>
```

input 은 빈 태그라 안에 글자를 못 넣습니다. 그래서 이렇게 됩니다. 버튼에는 브라우저 기본 글자인 “제출” 이 나오고, 우리가 쓴 “보내기” 는 버튼 밖에 그냥 글자로 남습니다. 글자는 value 에 넣어야 합니다. 보내기

이런 실수를 합니다

reset 을 **지우기로 오해**

이런 실수를 합니다

실수 4 — reset 이 칸을 비워 준다고 생각함

```
<input type="text" name="city" value="부산" />
<button type="reset">지우기</button>
```

버튼에 “지우기” 라고 적어 놓으니 사용자는 빈 칸이 될 것이라고 기대합니다. 그런데 누르면 “부산” 이 다시 나타납니다. reset 은 비우는 것이 아니라 처음 값으로 되돌리는 것이기 때문입니다. 버튼 글자를 “되돌리기” 나 “처음 상태로” 라고 적어야 오해가 없습니다. 지우기

이런 실수를 합니다

버튼을 form 밖에 둬

이런 실수를 합니다

실수 5 — 제출 버튼을 form 밖에 둬

```
<form>
<input type="text" name="userName" />
</form>
<button type="submit">보내기</button>
```

닫는 태그 위치를 잘못 잡아서 버튼이 폼 밖으로 밀려난 경우입니다. 버튼은 멀쩡히 보이는데 눌러도 아무 일이 안 일어납니다. 제출할 폼을 못 찾기 때문입니다. 들여쓰기를 맞춰 두면 이런 실수가 눈에 띄입니다.

정리

◦ 직접 해보기 7

정답

1) 섹션 1 의 상자 안, 두 버튼 뒤에 이렇게 넣습니다.

```
<button type="button">장바구니 담기</button>
```

→ 버튼이 세 개가 되고, 새 버튼은 글자 길이만큼 넓습니다.

눌러도 아무 일이 없습니다.

2) 섹션 2 의 버튼을 이렇게 고칩니다.

```
<p><button id="noTypeBtn" type="button">보내기</button></p>
```

→ 칸에 글자를 쳐 넣고 눌러도 화면이 새로고침되지 않습니다.

주소창도 그대로이고 쳐 넣은 값도 그대로 남아 있습니다.
고치기 전에는 주소창 끝에 ?userName=... 이 붙었습니다.

3) 코드는 고치지 않고 눌러 보는 문제입니다.

→ 몇 번을 눌러도 화면이 깜빡이지 않고 칸의 글자도 그대로입니다.

type="button" 이라 제출을 하지 않기 때문입니다.

4) 섹션 4 의 이름 칸에서 value 속성을 통째로 지웁니다.

```
<input id="resetName" type="text" name="rName" />
```

→ 페이지를 열면 이름 칸이 비어 있습니다.

글자를 쳐 넣고 되돌리기를 누르면 이번에는 빈 칸이 됩니다.
"처음 상태" 가 빈 칸으로 바뀌었기 때문입니다.

5) 섹션 5 의 셋째 버튼을 이렇게 고칩니다.

```
<input id="defaultSubmit" type="submit" value="신청하기" />
```

→ "제출" 이라고 적혀 있던 버튼이 "신청하기" 로 바뀌고,

글자가 길어진 만큼 버튼도 넓어집니다.

6) 섹션 6 의 form 에서 method 를 지웁니다.

```
<form id="methodForm">
```

→ 화면이 하나도 안 바뀝니다. 버튼 위치도 칸 크기도 그대로입니다.

method 는 화면이 아니라 보내는 방식을 정하는 속성이기 때문입니다.
확인했으면 method="get" 을 다시 붙이세요.

브라우저에게 검사를 시키는 속성들

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

빈 칸인 채로 “보내기” 를 눌러 보라는 안내가 나오면 눌러 보세요.

이름 (필수)

이메일 (필수)

별명 (선택)

별명 (최대 5자)

비밀번호 (최소 5자)

인원수 (1명부터 10명까지)

방문 날짜 (2026년 안에서)

만족도 (0부터 10까지 2씩)

실행하면 나오는 화면 — 브라우저에게 검사를 시키는 속성들

지금까지는 칸을 그리는 법만 배웠습니다. 그런데 실제 폼에는 규칙이 있습니다.

- 이름은 반드시 써야 합니다
- 비밀번호는 여덟 자 이상이어야 합니다
- 인원수는 1명부터 10명까지만 됩니다
- 우편번호는 숫자 다섯 자리여야 합니다

이런 규칙을 HTML 속성 몇 개로 적어 두면 브라우저가 대신 검사해 줍니다. 자바스크립트가 전혀 필요 없습니다.

규칙을 어긴 채로 보내기를 누르면 이런 일이 일어납니다.

- 1 폼이 제출되지 않습니다. 페이지가 넘어가지 않습니다.
- 2 문제가 있는 칸으로 커서가 옮겨 갑니다.
- 3 그 칸 아래에 작은 말풍선이 떠서 무엇이 문제인지 알려 줍니다.

말풍선에 나오는 글자는 브라우저가 정한 것입니다. 크롬 한국어에서 빈 칸이면 이렇게 나옵니다.

이 입력란을 작성하세요.

★ 그런데 여기에는 아주 중요한 한계가 있습니다. 이 검사는 사용자를 도와주는 것이지, 값을 지켜 주는 것이 아닙니다. 섹션 7 에서 그 한계를 직접 눈으로 봅니다. 꼭 읽으세요.

이 파일에는 눌러도 페이지가 안 넘어가는 보내기 버튼이 있습니다. 칸을 비운 채로 누르면 검사에 걸려 제출이 막히기 때문입니다. 말풍선이 어떻게 뜨는지 직접 보세요. 반대로 칸을 다 채우고 누르면 그때는 진짜로 제출되어 페이지가 새로고침됩니다.

required

섹션 1

required 는 반드시 채워야 하는 칸이라는 표시입니다. 값이 없으면 제출 자체가 막힙니다.

```
<input type="text" name="userName" required />
```

개념03 의 checked 처럼 값이 없는 속성입니다. 이름만 적으면 됩니다. required="true" 라고 쓸 필요가 없습니다.

빈 칸인 채로 보내기를 누르면 이런 일이 순서대로 일어납니다.

- 1 폼이 제출되지 않습니다. 주소창도 그대로입니다.
- 2 첫 번째로 문제가 있는 칸에 커서가 들어갑니다.
- 3 그 칸 아래에 말풍선이 뜹니다. "이 입력란을 작성하세요."

칸이 여러 개 비어 있으면 위에서부터 하나씩 알려 줍니다. 한꺼번에 다 보여 주지 않습니다.

주의할 점이 두 가지 있습니다.

· 화면에는 아무 표시도 안 납니다. 별표나 빨간 테두리가 저절로 생기지 않습니다. "필수" 라는 것을 사용자에게 알려 주려면 이름표 옆에 직접 적어야 합니다.

· 공백만 쳐 넣으면 통과합니다. 스페이스 한 칸도 값은 값이라서 "비어 있지 않다" 로 봅니다. 개념04 에서 textarea 안에 공백을 넣지 말라고 한 이유가 이것입니다.

아래 상자의 보내기 버튼은 마음 놓고 눌러 보세요. 두 칸이 비어 있으면 제출이 막히므로 페이지가 넘어가지 않습니다.

화면 칸 세 개와 버튼 하나가 세로로 놓입니다.

세 칸은 겹보기에 완전히 똑같습니다. 필수라는 표시가 어디에도 없습니다. 빈 채로 보내기를 누르면 페이지가 넘어가지 않고, 첫째 칸에 커서가 들어가면서 아래에 흰 말풍선이 뜹니다. 말풍선에는 “이 입력란을 작성하세요.” 라고 적혀 있습니다

이름 (필수)

이메일 (필수)

별명 (선택)

보내기

이름 칸을 채우고 다시 누르면 이번에는 이메일 칸에서 막힙니다. 이메일 칸에 abc 라고만 쓰고 누르면 “이메일 주소에 ‘@’를 포함해 주세요. ‘abc’에 ‘@’가 없습니다.” 라는 다른 말풍선이 뜹니다. 뒷문장은 여러분이 친 글자에 따라 달라집니다. type=“email” 이 값의 뜻을 알려 줬기 때문에 가능한 검사입니다.

▫ 직접 해보기

1

세 번째 “별명” 칸에도 required 를 붙여 보세요. 새로고침하고 빈 칸인 채로 보내기를 누르면 세 칸을 차례로 다 채워야 넘어갑니다.

maxlength / minlength

섹션 2

maxlength 는 치는 순간 막고, minlength 는 제출할 때 막습니다.

이름은 비슷하게 생겼는데 동작이 전혀 다릅니다. 여기서 많이 헷갈립니다.

maxlength="5"

여섯 번째 글자부터 아예 안 쳐집니다.
자판을 눌러도 화면에 글자가 안 나옵니다.
말풍선도 안 뜹니다. 그냥 조용히 안 들어갑니다.
그래서 사용자는 "왜 안 써지지?" 하고 당황합니다.
옆에 "최대 5자" 라고 적어 주는 것이 좋습니다.

minlength="5"

네 글자만 쓰고 있어도 아무 일이 없습니다. 잘 쳐집니다.
보내기를 눌렀을 때 그제야 막힙니다.
말풍선에 이렇게 나옵니다.

이 텍스트를 5자 이상으로 늘리세요(현재 4자 사용 중).

정리하면 이렇습니다.

maxlength	넘치지 않게 미리 막는다	→ 치는 순간
minlength	모자라면 나중에 잡는다	→ 보내는 순간

아래 상자에서 직접 해 보세요. 첫 칸에는 여섯 글자를 쳐 보고, 둘째 칸에는 두 글자만 쓰고 버튼을 눌러 보세요.

화면 첫 칸에 글자를 다섯 개까지는 잘 쳐집니다.

여섯 번째부터는 자판을 눌러도 화면에 아무것도 안 나타납니다. 둘째 칸에 두 글자만 치고 보내기를 누르면 제출이 막히고 "이 텍스트를 5자 이상으로 늘리세요(현재 2자 사용 중)." 라는 말풍선이 뜹니다

별명 (최대 5자)

비밀번호 (최소 5자)

보내기

▹ 직접 해보기 2

첫 칸의 maxlength="5" 를 maxlength="2" 로 바꿔 보세요. 새로고침하면 세 번째 글자부터 안 쳐집니다.

min / max / step

섹션 3

min 과 max 는 얼마부터 얼마까지, step 은 몇씩 건너뛰는지입니다.

글자 수가 아니라 값 자체의 범위를 정합니다. number, date, range 에서 씁니다.

```
<input type="number" min="1" max="10" step="2" />
```

min="1" 1보다 작으면 막습니다
max="10" 10보다 크면 막습니다
step="2" 화살표를 누를 때 2씩 움직입니다

날짜에도 똑같이 씁니다. 이때 값은 2026-01-01 모양으로 씁니다.

```
<input type="date" min="2026-01-01" max="2026-12-31" />
```

범위를 벗어난 값을 넣고 보내기를 누르면 이런 말풍선이 뜹니다.

값은 10 이하여야 합니다.
값은 2026-01-01 이후여야 합니다.

range에서는 조금 다르게 동작합니다. 막대는 애초에 범위 밖으로 못 가므로 막을 일이 없습니다. 대신 step 이 손잡이가 멈추는 자리를 정합니다. step="2" 면 0, 2, 4, 6, 8, 10 자리에만 딱딱 멈춥니다.

재미있는 것이 하나 있습니다. step 에 맞지 않는 값을 value 로 넣으면 브라우저가 가까운 자리로 옮겨 버립니다. 예를 들어 step="2" 인 막대에 value="7" 을 주면 실제 값은 8이 됩니다.

min 과 max 를 쓰면 number 칸의 너비도 달라집니다. 브라우저가 "이 정도 자릿수면 되겠다" 하고 칸을 줄이기 때문입니다. 아래에서 앞 섹션의 칸들보다 눈에 띄게 짧은 것을 확인하세요.

화면 인원수 칸은 앞 섹션의 글자 칸보다 눈에 띄게 짧습니다.

마우스를 올리면 나오는 위아래 화살표를 누르면 1, 3, 5, 7, 9 로 2씩 바뀝니다. 날짜 칸도 보통 날짜 칸보다 조금 짧습니다. 만족도 막대의 손잡이는 value 로 7을 줬는데도 8 자리에 가 있습니다. 끌어 보면 여섯 군데에서만 딱딱 멈춥니다

인원수 (1명부터 10명까지)

방문 날짜 (2026년 안에서)

만족도 (0부터 10까지 2씩)

보내기

step 은 화살표로 움직일 때의 간격입니다. 자판으로 직접 4를 쳐 넣는 것은 막지 않습니다. 다만 보내기를 누르면 min="1" 에 step="2" 라서 1, 3, 5, 7, 9 만 맞는 값이므로 그때 말풍선이 뜹니다.

만족도 막대의 step="2" 를 step="5" 로 바꿔 보세요. 새로고침하고 손잡이를 끌면 0, 5, 10 세 자리에서만 멈춥니다.

pattern

섹션 4

pattern 은 값이 어떤 모양이어야 하는지를 적는 속성입니다.

우편번호는 숫자 다섯 자리여야 합니다. "12345" 는 되고 "abc" 나 "12" 는 안 됩니다. 이런 규칙을 적는 것이 pattern 입니다.

```
<input type="text" name="zip" pattern="[0-9]{5}" />
```

여기서 솔직하게 말해 둡니다. pattern 안에 쓰는 [0-9]{5} 같은 표기는 정규식이라는 별도의 표기법입니다. HTML 이 아니라 그 자체로 하나의 공부거리이고, 이 자료의 범위 밖입니다.

지금은 자주 쓰는 것 몇 개를 그대로 베껴 쓰는 정도로 충분합니다.

0-9 · {5} 숫자 다섯 자리 12345

0-9 · {3}-[0-9]{4}-[0-9]{4} 전화번호 모양 010-1234-5678

가-힣 · {2,5} 한글 두 자에서 다섯 자 김민준

읽는 법만 살짝 알려 드리면 이렇습니다. 대괄호는 "이 중 한 글자", 중괄호는 "몇 번 반복" 입니다.

규칙을 어기고 보내기를 누르면 이런 말풍선이 뜹니다.

요청한 형식과 일치시키세요.

이 말만 봐서는 사용자가 무엇을 어떻게 고쳐야 할지 모릅니다. 그래서 title 속성에 설명을 같이 적어 둡니다.

```
<input pattern="[0-9]{5}" title="숫자 다섯 자리로 써 주세요" />
```

이렇게 해 두면 말풍선이 두 줄이 됩니다. 직접 재 본 결과는 이렇습니다.

요청한 형식과 일치시키세요.	← 브라우저가 정한 글자
숫자 다섯 자리로 써 주세요	← 우리가 title 에 적은 글자

title 은 칸에 마우스를 올리고 잠깐 기다려도 뜹니다. 같은 글자가 두 군데에서 쓰이는 셈입니다. pattern 을 쓸 때는 title 을 같이 쓰는 것이 예의라고 생각하세요.

한 가지 더. pattern 은 값이 비어 있을 때는 검사하지 않습니다. "비어도 안 된다" 까지 시키려면 required 를 같이 써야 합니다.

화면 회색 안내문 "예) 06236" 이 든 평범한 칸과 보내기 버튼이 있습니다.

칸에 마우스를 올리고 잠깐 기다리면 "숫자 다섯 자리로 써 주세요" 라는 작은 설명이 뜹니다. 12 만 쓰고 보내기를 누르면 제출이 막히고 두 줄짜리 말풍선이 뜹니다. 윗줄은 굵은 글씨로 "요청한 형식과 일치시키세요.", 아랫줄은 작은 글씨로 "숫자 다섯 자리로 써 주세요" 입니다. 아랫줄이 우리가 title 에 적어 둔 글자입니다. 12345 를 쓰고 누르면 통과하면서 페이지가 새로고침됩니다

우편번호 (숫자 다섯 자리)

보내기

▸ 직접 해보기 4

위 칸의 `pattern="[0-9]{5}"` 를 `pattern="[0-9]{3}"` 으로 바꿔 보세요. 새로고침하면 이번에는 세 자리를 써야 통과합니다.

05

readonly 와 disabled

섹션 5

둘 다 "못 고치는 칸" 인데, 값이 보내지느냐가 결정적으로 다릅니다.

표로 정리하면 이렇습니다.

	readonly	disabled
고칠 수 있나	못 고칩니다	못 고칩니다
생김새	보통 칸과 똑같습니다	회색으로 흐려집니다
커서가 들어가나	들어갑니다	안 들어갑니다
글자 복사	됩니다	안 됩니다
값이 보내지나	보내집니다	보내지지 않습니다

마지막 줄이 핵심입니다.

readonly	"보여는 주되 못 고치게 한다" → 값은 그대로 간다
disabled	"이 칸은 지금 없는 셈 친다" → 값이 아예 안 간다

언제 무엇을 쓸까요?

readonly	주문서에 자동으로 채워진 주문번호처럼 사용자가 고치면 안 되지만 같이 보내야 하는 값
disabled	"다른 주소로 배송" 을 체크하기 전까지 잠가 두는 주소 칸처럼 지금은 쓰지 않는 칸

여기서 아주 흔한 사고가 하나 납니다. “고치지 못하게 하자” 는 생각으로 disabled 를 붙였는데, 보내 보니 그 값이 통째로 빠져 있는 것입니다. 받는 쪽에서는 “주문번호가 안 왔다” 고 나옵니다. 화면은 멀쩡했으니 원인을 찾기가 아주 어렵습니다.

아래 두 칸의 색을 비교하세요. 색만 봐도 구별할 수 있습니다.

화면 위 칸은 보통 칸과 똑같이 흰 바탕에 검은 글씨입니다.

눌러 보면 커서가 들어가고 글자를 마우스로 긁어 복사할 수 있습니다. 다만 자판을 눌러도 글자가 안 바뀝니다. 아래 칸은 글자가 회색으로 흐려져 있고 바탕도 약간 회색입니다. 아무리 눌러도 커서가 안 들어가고 글자를 긁을 수도 없습니다. 맨 아래 버튼도 글자가 흐려져 있고 눌러도 놀리는 모양조차 안 납니다

주문번호 (readonly)

주문번호 (disabled)

못 누르는 버튼

readonly 는 글자 칸에만 쓸 수 있습니다. 체크박스나 라디오, select 에는 효과가 없습니다. 그런 칸을 잠그려면 disabled 를 써야 하는데, 그러면 값도 같이 안 갑니다. 이럴 때 개념03 의 hidden 칸에 같은 값을 넣어 함께 보내는 방법을 씁니다.

▣ 직접 해보기 5

두 번째 칸의 disabled 를 readonly 로 바꿔 보세요. 새로고침하면 회색이던 칸이 위 칸과 똑같은 흰 바탕 검은 글씨가 됩니다.

autofocus 와 autocomplete

섹션 6

autofocus 는 열자마자 커서를 넣고, autocomplete 는 예전에 친 값 추천을 켜고 끕니다.

autofocus

```
<input type="text" name="q" autofocus />
```

페이지를 열자마자 그 칸에 커서가 들어갑니다.

검색 사이트를 열면 바로 검색어를 칠 수 있는 것이 이것입니다.

규칙이 두 가지 있습니다.

- 한 페이지에 하나만 씁니다. 여러 개면 첫 번째만 동작합니다.
- 화면 아래쪽 칸에 붙이면 페이지가 그 칸까지 저절로 내려갑니다.

두 번째 때문에 이 파일에서는 일부러 살아 있는 예를 넣지 않았습니다.

넣었다면 이 파일을 열자마자 화면이 여기까지 뚝 떨어져서

여러분이 맨 위 설명을 못 봤을 겁니다.

대신 아래 직접 해보기에서 여러분이 직접 붙여 보세요.

autocomplete

```
<input type="text" name="userName" autocomplete="off" />
```

브라우저는 예전에 폼에 넣었던 값을 기억했다가 추천해 줍니다.

칸을 눌렀을 때 아래로 목록이 뜨는 그것입니다.

autocomplete="off" 는 그 추천을 끕니다.

기본값은 on 이라 따로 안 쓰면 켜져 있습니다.

공용 컴퓨터에서 쓰는 칸이나, 매번 다른 값을 넣는 칸에 씁니다.

★ 다만 요즘 브라우저는 이 속성을 무시하기도 합니다.

특히 비밀번호 칸에서는 사용자가 정한 설정을 더 우선합니다.

"꺼 달라고 부탁하는 속성" 정도로 생각하세요.

추천 목록이 뜨려면 그 브라우저에 예전 기록이 있어야 합니다.

방금 만든 파일에서는 기록이 없어 아무것도 안 뜹니다.

그래서 아래 두 칸은 화면상 완전히 똑같이 보입니다.

화면 두 칸이 완전히 똑같이 생겼습니다. 크기도 색도 같습니다.

autocomplete 는 화면을 바꾸는 속성이 아니라서 그렇습니다. 차이는 이 브라우저에 예전 입력 기록이 쌓인 뒤에야 드러납니다

추천 컴 (기본값)

추천 끄

위 첫 칸에 autofocus 를 붙이고 저장한 뒤 브라우저에서 F5 를 눌러 보세요. 페이지가 열리자마자 화면이 이 섹션까지 내려오고 첫 칸에 커서가 깜빡입니다. 확인했으면 반드시 지우세요. 안 지우면 이 파일을 열 때마다 화면이 여기로 떨어집니다.

검사의 한계

섹션 7

브라우저 검사는 사용자를 도와주는 장치이지 값을 지켜 주는 장치가 아닙니다.

여기까지 배운 속성들은 참 편리합니다. 그런데 이것만 믿으면 안 되는 이유가 있습니다.

이유 1 · 검사는 브라우저 안에서만 일어납니다.

브라우저를 거치지 않고 값을 직접 보내는 방법이 있습니다.
그렇게 보내면 required 도 pattern 도 아무 일을 못 합니다.
브라우저가 문 앞에서 검사하는 것이지, 값 자체에 자물쇠가 걸린 것이 아닙니다.

이유 2 · 검사 규칙은 화면 소스에 그대로 적혀 있습니다.

누구나 소스를 열어 볼 수 있고, 개발자 도구로 속성을 지울 수도 있습니다.
maxlength="5" 를 지우면 그 순간부터 백 글자도 쳐집니다.

이유 3 · 우리가 만든 값에는 아예 적용되지 않습니다.

사용자가 친 값만 검사합니다.
아래 상자가 그 증거입니다. maxlength 가 5인데
value 로 넣어 둔 열두 글자가 그대로 들어 있습니다.
그 상태로 보내기를 눌러도 막히지 않습니다.

그래서 결론은 늘 같습니다.

브라우저 검사 사용자가 실수를 덜 하게 도와주는 것. 꼭 하세요.
서버 검사 값이 진짜로 규칙에 맞는지 확인하는 것. 반드시 또 하세요.

둘 중 하나만 고르는 것이 아닙니다. 둘 다 합니다. 브라우저 검사는 사용자를 위해, 서버 검사는 데이터를 위해 하는 것입니다. 서버 검사를 만드는 방법은 HTML 의 일이 아니라 이 자료의 범위 밖입니다.

화면 칸 안에 "열두글자가넘는긴값입니다" 열두 글자가 잘리지 않고 전부 보입니다.

다섯 글자까지만 들어간다고 적어 두었는데 열두 글자가 그대로 있습니다. 칸의 크기나 색은 보통 칸과 똑같아서 규칙이 깨졌다는 표시가 전혀 없습니다

별명 (최대 5자라고 적어 둠)

이 버튼은 아무 일도 안 합니다

위 칸에서 글자를 지우고 새로 쳐 보면 그때는 다섯 자까지만 들어갑니다. 사용자가 치는 값만 검사한다는 것이 눈으로 확인됩니다.

◁ 직접 해보기 7

위 칸의 글자를 전부 지우고 여섯 글자를 쳐 보세요. 다섯 글자에서 멈춥니다. 같은 칸인데 결과가 다른 이유를 한 문장으로 적어 보세요.

자주 하는 실수

섹션 8

규칙 속성의 실수는 규칙이 있는 줄 알았는데 없는 종류입니다. 평소에는 잘 돌아가서 더 늦게 발견됩니다.

이런 실수를 합니다

disabled 로 잠그고 값이 안 오는 줄 모름

이런 실수를 합니다

실수 1 — 고치지 못하게 하려고 disabled 를 씀

```
<input type="text" name="orderNo"
value="A-2026-001" disabled />
```

화면에서는 목적을 이뤘습니다. 사용자가 못 고칩니다. 그런데 보내 보면 orderNo 라는 값이 아예 없습니다. disabled 인 칸은 없는 셈 치고 보내지 않기 때문입니다. 고치지 못하게 하면서 값도 보내려면 readonly 를 쓰세요.

이렇게 쓰세요

```
<input type="text" name="orderNo"
value="A-2026-001" readonly />
```

이런 실수를 합니다

required 만 믿음

이런 실수를 합니다

실수 2 — required 를 붙였으니 빈 값은 절대 안 온다고 믿음

```
<input type="text" name="userName" required />
```

스페이스 한 칸만 쳐도 통과합니다. 브라우저는 “비어 있지 않다” 로 봅니다. 게다가 브라우저를 거치지 않고 보내면 검사 자체가 일어나지 않습니다. 서버에서 한 번 더 검사해야 하는 이유입니다.

이런 실수를 합니다

필수 표시를 화면에 안 적음

이런 실수를 합니다

실수 3 — 필수 표시를 화면에 안 적음

```
<label for="a">이름</label>
<input id="a" type="text" required />
```

required 를 붙여도 화면에는 아무 표시도 안 납니다. 별표도, 빨간 테두리도 생기지 않습니다. 사용자는 다 채웠다고 생각하고 버튼을 눌렀다가 말풍선을 보고 당황합니다. 이름표에 “(필수)” 같은 글자를 직접 적어 주세요.

이런 실수를 합니다

maxlength 를 숫자 칸에 씀

이런 실수를 합니다

실수 4 — number 칸에 maxlength 를 씀

```
<input type="number" name="age" maxlength="3" />
```

아무 일도 안 일어납니다. 네 자리든 열 자리든 다 쳐집니다. maxlength 는 글자 수를 세는 속성이라 숫자 칸에는 효과가 없습니다. 숫자 칸의 범위는 min 과 max 로 정합니다.

이런 실수를 합니다

pattern 만 쓰고 required 를 안 씀

이런 실수를 합니다

실수 5 — pattern 만 쓰고 required 를 안 씀

```
<input type="text" name="zip" pattern="[0-9]{5}" />
```

우편번호를 아무것도 안 쓰고 보내면 그냥 통과합니다. pattern 은 값이 있을 때만 모양을 봅니다. 빈 값은 검사하지 않습니다. “비어도 안 된다” 까지 시키려면 required 를 같이 쓰세요.

정리

▢ 직접 해보기

8

정답

1) 섹션 1 의 셋째 칸에 required 를 붙입니다.

```
<input id="reqNick" type="text" name="userNick" required />
```

→ 세 칸을 전부 채워야 제출이 됩니다.

하나라도 비어 있으면 그 칸에 커서가 가고 말풍선이 뜹니다.
화면 생김새는 붙이기 전과 똑같습니다.

2) 섹션 2 의 첫 칸을 이렇게 바꿉니다.

```
<input id="maxField" type="text" name="nick" maxlength="2" />
```

→ 두 글자까지만 쳐지고 세 번째 글자부터는 자판을 눌러도 안 나옵니다.

말풍선도 안 뜨고 아무 알림도 없습니다.

3) 섹션 3 의 막대를 이렇게 바꿉니다.

```
<input id="rangeField" type="range" name="score" min="0" max="10" step="5" value="7" />
```

→ 손잡이가 왼쪽 끝, 가운데, 오른쪽 끝 세 자리에서만 멈춥니다.

value 로 준 7도 가까운 5 자리로 옮겨집니다.

4) 섹션 4 의 칸을 이렇게 바꿉니다.

```
<input id="patField" type="text" name="zip" pattern="[0-9]{3}" title="숫자 다섯 자리로 써 주세요" placeholder="예) 06236" />
```

→ 123 을 쓰면 통과하고, 12345 를 쓰면 오히려 막힙니다.

규칙이 세 자리로 바뀌었기 때문입니다.
title 과 placeholder 는 그대로라 안내문이 실제 규칙과 어긋납니다.
규칙을 바꾸면 안내문도 같이 고쳐야 한다는 것을 확인하세요.

5) 섹션 5 의 둘째 칸에서 disabled 를 readonly 로 바꿉니다.

```
<input id="diField" type="text" name="orderNo2" value="A-2026-001" readonly />
```

→ 회색으로 흐려져 있던 칸이 위 칸과 똑같이 흰 바탕에 검은 글씨가 됩니다.

눌러 보면 커서도 들어가고 글자도 굵을 수 있게 됩니다.

6) 섹션 6 의 첫 칸에 autofocus 를 붙입니다.

```
<input id="acOnField" type="text" name="mail1" autofocus />
```

→ F5 를 누르면 화면이 맨 위가 아니라 섹션 6 위치에서 열립니다.

첫 칸 안에서 커서가 깜빡입니다.
확인한 뒤에는 반드시 autofocus 를 지우세요.

7) 섹션 7 의 칸에서 글자를 전부 지우고 여섯 글자를 쳐 봅니다.

→ 다섯 글자에서 더 안 들어갑니다.

이유 한 문장 예시:
maxlength 는 사용자가 치는 값만 검사하고,
value 로 미리 넣어 둔 값은 검사하지 않기 때문입니다.

폼

푸는 방법

- 1) 이 파일을 VS Code 로 열고, 동시에 브라우저로도 엽니다.
- 2) 각 문제의 상자 안 “여기에 코드를 쓰세요” 자리에 코드를 씁니다.
- 3) 저장(Ctrl+S)하고 브라우저에서 새로고침(F5)해서 결과를 봅니다.
- 4) 소스의 주석에 적힌 기대 결과와 같은지 눈으로 대조합니다.

문제 1~12 는 기본, 13~14 는 응용, 15 는 도전, 16 은 확인 문제입니다.

이 단원의 결과는 대부분 “놀러 봐야” 알 수 있습니다.

화면만 보지 말고 글자를 눌러 보고, 칸에 값을 쳐 넣어 보세요.

★ 답을 다 쓴 뒤 연습문제_정답.html 과 나란히 열어 비교하세요.

실행하면 나오는 화면 — 폼

각 상자 안에 코드를 쓰세요. 문제 설명과 기대 결과는 VS Code 에서 이 파일의 소스에 주석으로 적혀 있습니다. 브라우저에는 여러분이 만든 결과만 보입니다. 칸이 빔빔하게 보이는 것은 정상입니다. 꾸미는 것은 09~10단원입니다.

문제 1 (개념01 섹션1·섹션3)

form 안에 “이름”이라는 글자와 한 줄 입력칸을 하나 만드세요. 입력칸에는 name=“userName” 이름표를 꼭 붙이세요. 글자와 칸은 p로 감싸세요.

기대 출력 “이름” 글자 오른쪽에 흰 바탕에 회색 테두리인 빈 네모 칸이 하나 보입니다. 칸을 누르면 안에서 커서가 깜빡입니다. 이렇게 되면 틀린 것: 칸이 아예 안 보이면 input 철자나 꺾쇠가 빠진 것입니다.

여기에 코드를 쓰세요

문제 2 (개념01 섹션5)

아래 칸에 속성을 하나 더 붙여서, 비어 있을 때 “이메일을 입력하세요”라는 옅은 회색 안내문이 보이게 하세요.

기대 출력 빈 칸 안에 옅은 회색 글씨로 “이메일을 입력하세요”가 보입니다. 칸에 아무 글자나 치면 그 회색 글씨가 즉시 사라집니다. 이렇게 되면 틀린 것: 글씨가 검은색이고 백스페이스로 지워지면 value를 쓴 것입니다.

이메일

문제 3 (개념01 섹션4)

“사는 곳” 칸을 만들고, 페이지를 열자마자 칸 안에 “부산”이 들어 있게 하세요. 이름표는 name=“userCity”로 하세요.

기대 출력 칸 안에 검은 글씨로 “부산”이 들어 있습니다. 백스페이스로 지울 수 있고, 다른 글자로 바꿀 수도 있습니다. 이렇게 되면 틀린 것: 글씨가 회색이고 지워지지 않으면 placeholder를 쓴 것입니다.

여기에 코드를 쓰세요

문제 4 (개념02 섹션2)

아래 두 줄은 이름표와 칸이 연결되어 있지 않습니다. for 와 id 를 써서 연결하세요. id 값은 userAge 로 하세요.

기대 출력 "나이" 라는 글자를 마우스로 누르면 오른쪽 칸 안에서 커서가 깜빡입니다. 이렇게 되면 틀린 것: 글자를 눌러도 아무 일이 없으면 for 와 id 의 글자가 다릅니다. 대소문자까지 똑같아야 합니다.

나이

문제 5 (개념02 섹션3·섹션4)

label 로 감싸는 방법을 써서 체크박스를 하나 만드세요. 옆에 붙는 글자는 "약관에 동의합니다" 이고, 이름표는 name="agree" 입니다. for 와 id 는 쓰지 마세요.

기대 출력 작은 빈 네모와 "약관에 동의합니다" 라는 글자가 한 줄에 보입니다. 글자 아무 데나 누르면 네모가 파란 바탕에 흰 체크 표시로 바뀝니다. 한 번 더 누르면 다시 빈 네모가 됩니다. 이렇게 되면 틀린 것: 네모를 정확히 눌러야만 커지면 label 로 감싸지 않은 것입니다.

여기에 코드를 쓰세요

문제 6 (개념02 섹션5)

"배송 정보" 라는 제목이 달린 묶음 상자를 만들고, 그 안에 "받는 사람" 칸과 "주소" 칸을 한 줄씩 넣으세요. 두 칸 모두 이름표를 for 와 id 로 연결하세요.

기대 출력 열은 회색 테두리 상자가 하나 보입니다.
 상자의 왼쪽 위 테두리가 굵기고 그 자리에 "배송 정보" 글자가 걸쳐 있습니다.
 상자 안에는 이름표와 칸이 두 줄로 들어 있습니다.
 이렇게 되면 틀린 것: 제목이 테두리에 걸치지 않고 상자 위에 따로 놓이면 legend 를 fieldset 밖에 쓴 것입니다.

여기에 코드를 쓰세요

문제 7 (개념03 섹션1·섹션2)

아래 네 가지 칸을 한 줄에 하나씩 만드세요. 이름표는 for 와 id 로 연결하세요.

비밀번호	type 은 password, name 은 pw
이메일	type 은 email, name 은 mail
나이	type 은 number, name 은 age
생일	type 은 date, name 은 birth

기대 출력 비밀번호 칸에 글자를 치면 글자 대신 까만 점이 하나씩 찍힙니다.
 이메일 칸은 글자가 그대로 보입니다.
 나이 칸에 마우스를 올리면 오른쪽 끝에 위아래 화살표가 나타납니다.
 생일 칸에는 회색으로 "연도-월-일" 이 있고 오른쪽에 달력 그림이 있습니다.
 이렇게 되면 틀린 것: 네 칸이 전부 똑같이 생겼다면 type 철자를 확인하세요.
 모르는 type 은 조용히 text 로 그려집시다.

여기에 코드를 쓰세요

문제 8 (개념03 섹션3)

취미를 고르는 체크박스 세 개를 만드세요. 전부 label 로 감싸세요.

영화	name 은 hobby, value 는 movie
음악	name 은 hobby, value 는 music
독서	name 은 hobby, value 는 book

“음악” 만 페이지를 열자마자 켜져 있게 하세요.

기대 출력 네모 세 개가 세로로 놓이고 가운데 “음악” 만 파란 바탕에 흰 체크 표시입니다.
세 개를 전부 켤 수도 있고 전부 끌 수도 있습니다.
이렇게 되면 틀린 것: 하나를 켜면 다른 것이 꺼진다면 type 을 radio 로 쓴 것입니다.

여기에 코드를 쓰세요

문제 9 (개념03 섹션4)

결제 수단을 고르는 라디오 버튼 세 개를 만드세요. 전부 label 로 감싸세요. 세 개 중 반드시 하나만 골라져야 합니다.

카드	value 는 card
현금	value 는 cash
계좌이체	value 는 bank

“카드” 가 페이지를 열자마자 골라져 있게 하세요.

기대 출력 동그라미 세 개가 세로로 놓이고 처음에는 “카드” 에만 파란 점이 있습니다.
다른 것을 누르면 앞의 점이 사라지고 누른 쪽으로 옮겨 갑니다.
항상 하나만 채워져 있습니다.
이렇게 되면 틀린 것: 두 개 이상이 동시에 채워지면 name 을 서로 다르게 쓴 것입니다.
라디오는 name 이 같아야 한 무리가 됩니다.

여기에 코드를 쓰세요

문제 10 (개념04 섹션1·섹션2)

배송 방법을 고르는 목록을 만드세요. 이름표는 name=“ship” 입니다.

일반 배송	보내는 값은 normal
빠른 배송	보내는 값은 quick
새벽 배송	보내는 값은 dawn

“빠른 배송” 이 페이지를 열자마자 골라져 있게 하세요.

기대 출력 접힌 상자에 “빠른 배송” 이 보이고 오른쪽에 아래쪽 화살표가 있습니다. 상자를 누르면 세 줄이 펼쳐지고 “빠른 배송” 줄이 파랗게 칠해져 있습니다. 화면 어디에도 normal, quick, dawn 같은 영어는 보이지 않습니다. 이렇게 되면 틀린 것: 상자에 영어가 보이면 value 자리와 글자 자리를 바꿔 쓴 것입니다.

여기에 코드를 쓰세요

문제 11 (개념04 섹션4·섹션5)

“배송 요청 사항” 을 받는 여러 줄 입력칸을 만드세요. 다섯 줄 높이, 가로 마흔 글자로 하고 이름표는 name=“memo” 입니다. 페이지를 열었을 때 칸이 완전히 비어 있어야 합니다.

기대 출력 가로로 긴 빈 네모 칸이 다섯 줄 높이로 보입니다. 오른쪽 아래 모서리에 작은 빗금 삼각형이 있습니다. 칸을 누르고 Ctrl+A 를 눌러도 파란 선택 표시가 전혀 생기지 않습니다. 이렇게 되면 틀린 것: Ctrl+A 를 눌렀을 때 선택 표시가 생기면 여는 태그와 닫는 태그 사이에 줄바꿈이나 공백이 들어간 것입니다.

여기에 코드를 쓰세요

문제 12 (개념05 섹션2·섹션3)

form 안에 글자 칸 하나와 버튼 하나를 만드세요. 버튼에 적힐 글자는 “미리 보기” 이고, 눌러도 아무 일이 일어나지 않아야 합니다.

기대 출력 칸 아래에 "미리 보기" 라고 적힌 회색 버튼이 있습니다.
칸에 글자를 쳐 넣고 버튼을 눌러도 화면이 새로고침되지 않고
쳐 넣은 글자도 그대로 남아 있습니다.
이렇게 되면 틀린 것: 누를 때 화면이 깜빡이고 주소창 끝에 물음표가 붙으면
type 을 빠뜨린 것입니다. type 을 안 쓰면 기본이 submit 입니다.

여기에 코드를 쓰세요

문제 13 [응용] (개념04 섹션3 + 개념02 섹션3)

지역을 고르는 목록을 만드세요. 항목을 두 묶음으로 나누고 제목을 답니다.

수도권 서울(seoul), 인천(incheon)
영남 부산(busan), 대구(daegu)

목록 전체를 label 로 감싸서 "지역" 이라는 글자와 목록을 연결하세요. 이름표는 name="region" 입니다.

기대 출력 "지역" 글자 오른쪽에 접힌 상자가 있고 "서울" 이 보입니다.
상자를 누르면 목록이 펼쳐지고, "수도권" 과 "영남" 이 회색 굵은 글씨로
있으며
그 아래 지역 이름들이 한 칸 안으로 들여쓰여 있습니다.
회색 제목은 눌러도 골라지지 않습니다.
"지역" 글자를 누르면 목록이 펼쳐지지는 않지만 목록 상자로 포커스가 옮겨
갑니다.
그 상태에서 키보드 아래 화살표를 누르면 고른 항목이 "인천" 으로 바뀝니다.
이것으로 label 과 목록이 연결되었는지 확인할 수 있습니다.
이렇게 되면 틀린 것: 제목이 다른 항목처럼 골라지면 optgroup 이 아니라
option 으로 쓴 것입니다.

여기에 코드를 쓰세요

문제 14 [응용] (개념06 섹션1·섹션2·섹션5)

form 안에 아래 세 가지를 만드세요.

이름 칸 반드시 채워야 하고, 최대 다섯 글자까지만 쳐줍니다. name 은 userName
 주문번호 칸 값은 A-2026-001 이고 사용자가 고칠 수 없습니다.
 단, 이 값도 함께 보내져야 합니다. name 은 orderNo
 보내기 버튼 누르면 폼을 제출하는 버튼

기대 출력 이름 칸에 여섯 번째 글자부터는 자판을 눌러도 안 들어갑니다.
 이름 칸을 비운 채로 보내기를 누르면 페이지가 넘어가지 않고
 그 칸에 "이 입력란을 작성하세요." 라는 말풍선이 뜹니다.
 주문번호 칸은 흰 바탕에 검은 글씨로 보통 칸처럼 보이는데
 자판을 눌러도 글자가 안 바뀝니다.
 이렇게 되면 틀린 것: 주문번호 칸의 글씨가 회색으로 흐려지고 커서도 안
 들어가면
 disabled 를 쓴 것입니다. 그러면 값이 보내지지 않습니다.

여기에 코드를 쓰세요

문제 15 [도전] (개념01 ~ 개념06 전체)

회원가입 폼을 통째로 만드세요. form 하나 안에 아래 순서대로 넣습니다.

묶음 1 제목은 "기본 정보"

이름 글자 칸, 반드시 채워야 함, name 은 userName
 이메일 이메일 칸, 반드시 채워야 함, name 은 userMail
 비밀번호 비밀번호 칸, 최소 여덟 자, name 은 pw

묶음 2 제목은 "추가 정보"

성별 라디오 두 개(남자 male, 여자 female), 하나만 골라짐, name 은 gender
 관심분야 체크박스 두 개(운동 sport, 요리 cook), name 은 interest
 자기소개 여러 줄 칸, 네 줄 높이 가로 마흔 글자, 비어 있어야 함, name 은 intro

묶음 밖 "가입하기" 버튼(제출하는 버튼)과 "되돌리기" 버튼

모든 이름표는 label 로 연결하세요. for 와 id 를 쓰든 감싸든 상관없습니다.

기대 출력 테두리 상자 두 개가 세로로 놓이고, 각각 왼쪽 위에 제목이 걸쳐 있습니다.
상자 아래에 버튼 두 개가 나란히 있습니다.
성별 동그라미는 항상 하나만 채워지고, 관심분야 네모는 둘 다 켤 수 있습니다.
값을 몇 개 바꾼 뒤 "되돌리기" 를 누르면 처음 상태로 돌아갑니다.
이렇게 되면 틀린 것: "되돌리기" 를 눌렀는데 페이지가 새로고침되면 type="reset" 을 안 쓴 것입니다.

여기에 코드를 쓰세요

문제 16 [확인] (개념02 섹션2 + 개념03 섹션4 + 개념05 섹션2)

05

아래 코드는 에러가 하나도 안 납니다. 화면도 멀쩡해 보입니다. 그런데 세 군데가 잘못되어 있습니다. 눌러 보면 알 수 있습니다.

직접 눌러 보고 증상을 확인한 뒤, 무엇이 문제인지 찾아 고치세요.

증상 1 "이름" 글자를 눌러도 오른쪽 칸에 커서가 안 들어갑니다.
증상 2 동그라미 두 개가 동시에 채워집니다. 한 번 켜지면 꺼지지도 않습니다.
증상 3 "미리 보기" 버튼을 누르면 페이지가 새로고침되면서
쳐 넣은 값이 전부 사라집니다.
(새로고침이 일어나도 괜찮습니다. 주소창의 물음표 뒷부분을 지우거나
파일을 다시 열면 원래대로 돌아옵니다)

고친 뒤 기대 화면: "이름" 글자를 누르면 칸에 커서가 들어갑니다.

동그라미는 항상 하나만 채워집니다.
"미리 보기" 를 눌러도 화면이 새로고침되지 않습니다.

힌트: 세 가지 모두 이 단원에서 배운 내용입니다.

첫째는 두 속성의 값이 같은지, 둘째는 무엇이 라디오를 한 무리로 묶는지,
셋째는 버튼에서 무엇을 빠뜨리면 기본값이 submit 이 되는지 생각해 보세요.

이렇게 되면 틀린 것: "이름" 을 눌렀는데 커서가 안 들어가면 for 와 id 가 아직 다릅니다.

동그라미가 여전히 둘 다 켜지면 두 라디오의 name 이 아직 다릅니다.
버튼을 눌렀을 때 아직 새로고침되면 type 을 안 붙인 것입니다.

이름

결제 수단

카드

현금

미리 보기

05단원 마무리 품

자주 넘어지는 자리

- 1 `label` 의 글자를 눌러 **체크박스가 켜지는 것**. `for<=>id` 가 연결됐다는 증거입니다
- 2 `radio` 는 `name` 이 **같아야** 하나만 골라진다는 것

부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

00단원 연습문제 정답 개발환경과 Git

★ 명령만 보지 말고 아래 설명을 읽으세요.

이 단원의 문제는 답을 아는 것보다 **왜 그런지**가 중요합니다.

★★ 그리고 정답을 본 뒤에 반드시 **한 번 더 직접 쳐 보세요**.

읽어서 아는 것과 손이 아는 것은 다릅니다.

00단원 · 개발환경과 Git

명령만 보고 넘어가지 마세요. 아래 설명이 본문입니다.

아래 결과는 실제로 쳐 본 것입니다. 커밋 번호만 여러분과 다릅니다.

문제 1 — 깔린 것 확인하기

```
$ git --version
git version 2.55.0.windows.3

$ node --version
v24.18.0
```

★ --version 은 거의 모든 명령줄 프로그램이 알아듣습니다. 앞으로 뭘 깔든 이걸로 확인하세요.
못 찾는다고 나오면 터미널을 꺾다 켜는 것이 먼저입니다. (개념01 섹션2)

문제 2 — 이름과 메일 확인하기

```
$ git config user.name
홍길동

$ git config user.email
hong@example.com
```

실행하면 나오는 화면 — 개발환경과 Git

명령만 보고 넘어가지 마세요. 아래 설명이 본문입니다. 아래 결과는 실제로 쳐 본 것입니다. 커밋 번호만 여러분과 다릅니다.

문제 1 — 깔린 것 확인하기

```
$ git --version
git version 2.55.0.windows.3

$ node --version
v24.18.0
```

★ --version 은 거의 모든 명령줄 프로그램이 알아듣습니다. 앞으로 뭘 깔든 이걸로 확인하세요. 못 찾는다고 나오면 터미널을 껐다 켜는 것이 먼저입니다. (개념01 섹션2)

문제 2 — 이름과 메일 확인하기

```
$ git config user.name
홍길동

$ git config user.email
hong@example.com
```

값 없이 이름만 주면 “지금 값이 뭐냐” 는 뜻이 됩니다. 값을 같이 주면 설정하는 것이고요. 같은 명령이 둘 다 합니다. ★ 전부 보려면 git config --global --list 입니다. (긴 목록이 나오면 q 로 빠져나옵니다)

문제 3 — 저장소 만들고 첫 커밋

```
$ git init
Initialized empty Git repository in ../.git/

(index.html 을 만들고 <h1>안녕하세요</h1> 를 씁니다)

$ git add .

$ git commit -m "첫 페이지를 만들었습니다"
[main (root-commit) ae4c0c2] 첫 페이지를 만들었습니다
1 file changed, 1 insertion(+)
create mode 100644 index.html

$ git log --oneline
ae4c0c2 첫 페이지를 만들었습니다
```

git add . 의 . 은 “지금 폴더 아래 전부” 입니다. 파일 이름을 하나씩 적어도 됩니다. ★ root-commit 은 첫 커밋에만 붙습니다. 두 번째부터는 안 나옵니다.

여기서 막혔다면 — Author identity unknown 이 나왔을 것입니다. 문제 2의 설정을 먼저 하세요. (개념01 섹션4)

문제 4 — 기호 읽기

- ① M index.html ← 첫째 칸이 공백 = 아직 안 담김
- ② M index.html ← 둘째 칸이 공백 = 담겼고, 담은 뒤로는 안 고침
- ③ (아무것도 없음) ← 커밋했으니 달라진 게 없음

왼쪽 두 칸이 각각 다른 자리를 가리킵니다. 첫째 칸은 담는 곳, 둘째 칸은 작업 폴더입니다. (개념02 섹션4)

★ 이걸 못 읽으면 “커밋했는데 왜 안 들어갔지?” 를 평생 겪습니다. 담기지 않은 것은 커밋되지 않습니다.

★★ git add 뒤에 또 고치면 MM 이 됩니다. “담아 둔 것도 있고, 담은 뒤에 또 고친 것도 있다” 는 뜻입니다. 해 보세요.

문제 5 — 무엇이 바뀌었나

```
$ git diff
diff --git a/index.html b/index.html
index 6944b17..87d06c9 100644
--- a/index.html
+++ b/index.html
@@ -1,2 @@
<h1>안녕하세요</h1>
+<p>시험</p>
```

+ 는 새로 생긴 줄입니다. 앞에 공백만 있는 줄은 그대로인 줄입니다. ★ git add 를 한 뒤에 치면 아무것도 안 나옵니다. 고장이 아닙니다. git diff 는 안 담은 것만 보여 줍니다. 담긴 것은 git diff --staged 입니다. (개념02 섹션5)

문제 6 — 되돌리기

```
$ git restore index.html

$ git status --short
(아무것도 안 나옵니다)
```

★★ 되돌린 내용은 되찾을 수 없습니다. 휴지통에 안 갑니다. 커밋을 안 했으니 Git 도 모르는 내용이기 때문입니다. (개념03 섹션1) 그래서 애매하면 일단 커밋해 두는 것이 낫습니다. 커밋은 얼마든지 되돌릴 수 있습니다.

문제 7 — 담은 것 무르기

```
$ git add .

$ git status --short
M index.html

$ git restore --staged index.html

$ git status --short
M index.html
```

M 이 왼쪽에서 오른쪽으로 옮겨 갔습니다. 담은 곳에서 빠지고 작업 폴더에만 남은 것입니다. 고친 내용은 그대로 있습니다. ★ 완전히 없애려면 --staged 로 뺀 뒤 --staged 없이 한 번 더 (문제 6). 두 단계인 이유가 자리가 둘이기 때문입니다.

문제 8 — .gitignore

```
(.gitignore 파일을 만들고 한 줄만 씁니다)
.env

$ git status --short
?? .gitignore          ← .env 는 사라졌습니다

$ git check-ignore -v .env
.gitignore:1:.env .env
```

check-ignore -v 는 몇 번째 줄의 어느 규칙에 걸렸는지 알려 줍니다. 규칙을 여러 개 쓰다 보면 어느 것이 먹었는지 헛갈리는데, 이걸로 확인합니다. ★ .gitignore 자신은 같이 올려야 합니다. 팀원 모두가 같은 것을 무시해야 하니까요.

★★ 그리고 이미 커밋한 파일에는 안 먹습니다. 문제 13에서 다룹니다.

문제 9 — 가지 만들고 합치기

```
$ git switch -c 인사말
Switched to a new branch '인사말'

(파일을 고칩니다)

$ git commit -am "인사말을 고쳤습니다"
```

```
[인사말 73859b6] 인사말을 고쳤습니다
1 file changed, 1 insertion(+)
```

```
$ git switch main
Switched to branch 'main'
```

```
$ git merge 인사말
Updating 6d15fca..73859b6
Fast-forward
 index.html | 1 +
1 file changed, 1 insertion(+)
```

Fast-forward 는 “표시만 앞으로 옮기면 된다” 는 뜻입니다. main 이 그동안 가만히 있었기 때문입니다. 새 커밋이 안 생깁니다. ★ 이 말이 나오면 충돌이 날 수가 없습니다.

-am 이 안 먹었다면 — 새로 만든 파일이라서입니다. -a 는 이미 알고 있는 파일만 담습니다. (개념05 섹션1)

문제 10 — 충돌 내고 풀기

```
$ git switch -c 가
(<h1> 안의 글자를 '가나다' 로 고칩니다)
$ git commit -am "가나다로"

$ git switch main
(같은 줄을 '라마바' 로 고칩니다)
$ git commit -am "라마바로"

$ git merge 가
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

파일을 열면 이렇게 되어 있습니다.

```
<<<<<< HEAD
<h1>라마바</h1>
=====
<h1>가나다</h1>
>>>>>> 가
```

기호 세 줄과 안 쓸 쪽을 지웁니다. 그리고 담고 커밋합니다.


```
$ git add index.html
$ git commit -m "인사말 충돌을 풀었습니다"
[main 7c6d6e7] 인사말 충돌을 풀었습니다
```

HEAD 쪽이 지금 서 있는 가지(main), 아래쪽이 합치려는 가지입니다. ★ 커밋 전에 파일에서 <<<< 를 검색해 보세요. 하나도 없어야 합니다. 남기고 커밋하는 것이 제일 흔한 실수입니다.

★★ 이 커밋에는 1 file changed 줄이 안 나옵니다. 합치는 커밋은 원래 그렇습니다. 빠뜨린 게 아닙니다.

문제 11 — [손으로] 못 찾다고 합니다

(1) 터미널을 껐다 다시 켜게 합니다.

(2) 프로그램을 켤면 “어디에 있는지” 목록(PATH)이 바뀝니다. 그런데 이미 켜져 있던 터미널은 옛 목록을 들고 있습니다. 새로 켜야 새 목록을 읽습니다. ★ 다시 껐다고 해결이 안 되는 이유가 이것입니다. 터미널이 문제지 설치가 문제가 아닙니다.

(3) 그래도 안 되면 — 컴퓨터를 다시 켵니다 (VS Code 안의 터미널이면 VS Code 를 껐다 켜야 합니다) · 어느 셸에서 켜는지 봅니다. PowerShell 인지 Git Bash 인지. · 설치할 때 “PATH 에 추가” 항목을 안 골랐는지 봅니다. 다시 켤면서 확인합니다.

★★ 이걸 Git 만의 이야기가 아닙니다. Node, Python, 뭘 깔든 같습니다. “깔았는데 못 찾는다” = “터미널을 껐다 켜라” 로 외워 두세요.

문제 12 — [손으로] 커밋했는데 안 들어갔습니다

(1) “커밋할 것이 담겨 있지 않다. 대신 Git 이 모르는 파일은 있다” 는 뜻입니다.

(2) git add 를 빠뜨렸습니다. 고친 것과 담은 것은 다릅니다. 담지 않은 것은 커밋되지 않습니다. ★ 자리가 셋이라는 것(개념02 섹션2)을 모르면 계속 겪습니다.

(3) ?? 파일이름 이 보입니다. ?? 는 “양쪽 다 모르는 새 파일” 입니다. ★ 만약 M 이 보였다면 메시지가 조금 달랐을 것입니다. 메시지를 끝까지 읽으면 어느 경우인지 알 수 있습니다. untracked files present 라고 적혀 있으니 새 파일인 것입니다.

문제 13 — [손으로] .env 를 올려 버렸습니다 ★

(1) 이미 추적 중인 파일이라서입니다.

.gitignore 는 “아직 모르는 파일” 에만 먹습니다. 한 번 커밋해서 Git 이 알고 있는 파일은 계속 따라다닙니다. ★ git check-ignore -v .env 를 쳤을 때 아무것도 안 나오는 것이 신호입니다. 규칙이 틀린 게 아니라 검사 대상이 아닌 것입니다. 규칙이 맞는지 보려면 --no-index 를 붙입니다. (개념03 섹션6)

(2) 추적에서 뺍니다.

```
$ git rm --cached .env
rm '.env'

$ git commit -m ".env 를 추적에서 뺐습니다"
```

--cached 는 "저장소에서만 빼고 내 파일은 두라" 입니다. ★★ --cached 를 빼면 진짜 파일이 지워집니다. 조심하세요.

(3) ★ 안 끝납니다.

옛 커밋에는 그대로 남아 있습니다. 누구든 옛 기록을 꺼내 보면 그 키가 보입니다. GitHub 에도 남아 있습니다. 기록에서 완전히 지우는 방법이 있기는 하지만 어렵고, 이미 받아 간 사람에게서는 못 지웁니다. 한번 나간 비밀은 나간 것입니다.

(4) 순서가 정해져 있습니다.

1. ★ 그 키를 먼저 폐기합니다 (발급한 곳에서 못 쓰게 만듭니다)
2. 새 키를 발급받습니다
3. .gitignore 에 넣고 git rm --cached 로 뺍니다
4. 커밋하고 push 합니다
5. 사용량·청구서를 확인합니다 (누가 썼는지)

★★★ 1번이 제일 먼저입니다. 코드를 먼저 고치고 키를 나중에 폐기하면, 그 사이에도 옛 키는 살아 있습니다. "지우면 되지 않나요?" — 안 됩니다. 폐기 말고 방법이 없습니다.

문제 14 — [손으로] push 가 거부됐습니다

(1) 누군가(또는 다른 컴퓨터의 내가) 먼저 올렸습니다.

내가 모르는 커밋이 GitHub 에 있습니다. 그대로 밀면 그 커밋이 사라지므로 Git 이 막은 것입니다. 막아 준 것이 고마운 일입니다. ★ hint: 줄에 답이 적혀 있습니다. use 'git pull' before pushing again.

(2) 먼저 받아 옵니다.

```
$ git pull
$ git push
```

★ git push --force 라는 것이 있습니다. 쓰지 마세요. 저쪽 커밋을 지우고 덮어씁니다. 남의 하루가 사라질 수 있습니다.

(3) 충돌이 났다면 개념05 섹션5 대로 풁니다.

<<<<<< HEAD 쪽이 내 것, >>>>>> 쪽이 받아 온 것입니다. 기호를 지우고 원하는 모양으로 만든 뒤 add → commit → push. 무서우면 git merge --abort 로 언제든지 빠져나올 수 있습니다.

(4) ★ 습관 두 가지

- 작업을 시작하기 전에 `git pull`
- 조금씩 자주 커밋하고 자주 `push`

하루치를 모아서 한 번에 올리면 충돌이 크게 납니다. 조금씩 자주 주고받으면 부딪히는 범위가 작아서 풀기도 쉽습니다. ★ 그리고 팀에서 같은 파일을 동시에 고치지 않게 나눠 맡는 것이 제일 좋은 예방입니다.

01단원 연습문제 정답 HTML 시작하기

이 파일은 연습문제.html 과 같은 뼈대에 답을 채운 것입니다.

문제 번호와 문장은 그대로입니다.

보는 방법

문제마다 상자가 두 개씩 있습니다.

위 점선 상자 답을 실제로 그린 화면

아래 검은 상자 그 화면을 만든 코드

그리고 그 아래 회색 글씨로 왜 그렇게 되는지 설명이 붙어 있습니다.

정답을 눈으로만 읽으면 남는 것이 없습니다.

반드시 연습문제.html 로 돌아가 손으로 한 번 더 써 보세요.

우리 동네 빵집
매일 아침 7시에 문을 엽니다. 월요일은 쉽니다.
서울시 마포구 행복로 12 1층 우리 동네 빵집
오늘의 빵 내일의 빵
휴무일은 월요일입니다.

실행하면 나오는 화면 — HTML 시작하기

문제마다 실제로 그린 화면과 그 코드를 나란히 놓았습니다. 내가 쓴 코드와 다르더라도 화면이 같으면 맞은 것입니다. 다만 조건에 문단 하나 라고 적혀 있으면 문단 개수까지 같아야 합니다.

문제 1 (개념02 섹션1)

p 태그로 우리 동네 빵집이라는 문단을 하나 만드세요.

기대 출력 상자 안에 "우리 동네 빵집" 이 한 줄로 보입니다.

문제 1 — 문단 하나 만들기

p 태그로 우리 동네 빵집이라는 문단을 하나 만드세요. (개념02 섹션 1)

화면 "우리 동네 빵집" 이 한 줄로 보입니다

우리 동네 빵집

<p>우리 동네 빵집</p>

여는 태그와 닫는 태그 사이에 넣은 글자가 화면에 나옵니다. 닫는 태그의 빗금을 빠뜨리면 빈 문단이 하나 더 생기니 짚을 꼭 확인하세요.

문제 2 (개념02 섹션1)

두 문장을 각각 문단으로 만드세요.

기대 출력 두 줄이 보이고, 두 줄 사이가 약간 벌어져 있습니다.

문제 2 — 문단 두 개 만들기

매일 아침 7시에 문을 엽니다. 와 월요일은 쉽니다. 를 각각 문단으로 만드세요. (개념02 섹션 1)

화면 두 줄이 보이고, 두 줄 사이가 약간 벌어져 있습니다

매일 아침 7시에 문을 엽니다.월요일은 쉽니다.

<p>매일 아침 7시에 문을 엽니다.</p>

<p>월요일은 쉽니다.</p>

줄 사이를 벌리라고 시킨 적이 없는데도 벌어집니다. p 라는 이름표에 "이건 하나의 문단"이라는 뜻이 들어 있어서 브라우저가 문단답게 위아래를 띄워 그리기 때문입니다.

문제 3 (개념02 섹션3)

문단 하나 안에서 주소를 두 줄로 나누세요.

기대 출력 두 줄로 나뉘고, 문제 2 의 두 문단보다 줄 사이가 좁습니다.

문제 3 — 문단 하나 안에서 줄 바꾸기

p 하나 안에서 `<br />` 로 주소를 두 줄로 나누세요. 첫 줄은 서울시 마포구 행복로 12, 둘째 줄은 1층 우리 동네 빵집 입니다. (개념02 섹션 3)

화면 두 줄로 나뉘고, 문제 2 상자보다 두 줄 사이가 좁습니다

서울시 마포구 행복로 121층 우리 동네 빵집

```
<p>서울시 마포구 행복로 12<br />1층 우리 동네 빵집</p>
```

문제 2 와 나란히 보세요. 문단 두 개는 문단 사이 여백까지 생기지만, `<br />` 는 줄만 바꿉니다. 그래서 줄 사이가 더 좁습니다. 주소처럼 한 덩어리인데 줄만 나눠야 하는 글에 쓰는 이유입니다.

문제 4 (개념02 섹션3)

문단 두 개 사이에 가로줄을 하나 그으세요.

기대 출력 글자 한 줄, 회색 가로줄, 글자 한 줄 순서로 보입니다.

문제 4 — 가로줄 긋기

오늘의 빵 문단과 내일의 빵 문단 사이에 `<hr />` 로 가로줄을 하나 그으세요. (개념02 섹션 3)

화면 글자 한 줄, 회색 가로줄 하나, 글자 한 줄 순서로 보입니다

오늘의 빵내일의 빵

```
<p>오늘의 빵</p>
```

```
<hr />
```

```
<p>내일의 빵</p>
```

hr 은 감쌀 내용이 없는 빈 태그라 닫는 태그가 없습니다. `</hr>` 를 쓰면 브라우저가 그냥 버립니다. 에러는 안 납니다.

문제 5 (개념02 섹션4)

휴무일은 월요일입니다. 문단에서 월요일 세 글자만 굵게 만드세요.

기대 출력 한 줄인데 월요일 세 글자만 굵게 보입니다.

문제 5 — 한 단어만 굵게 (중첩)

휴무일은 월요일입니다. 라는 문단을 만들고, strong 으로 월요일 세 글자만 굵게 만드세요. (개념02 섹션 4)

화면 한 줄인데 "월요일" 세 글자만 굵게 보입니다

휴무일은 월요일입니다.

```
<p>휴무일은 <strong>월요일</strong>입니다.</p>
```

strong 이 p 안에 들어갔습니다. 이것이 중첩입니다. p 가 부모, strong 이 자식입니다. 굵게 만들 글자만 정확히 감싸는 것이 요령입니다.

문제 6 (개념05 섹션1)

문단 하나를 만들고 그 위에 주석으로 메모를 남기세요.

기대 출력 "소금빵 2500원" 한 줄만 보입니다. 메모는 안 보입니다.

문제 6 — 주석으로 메모 남기기

소금빵 2500원 문단을 만들고, 그 위 줄에 주석으로 가격은 매달 확인할 것 이라고 메모를 남기세요. (개념05 섹션 1)

가격은 매달 확인할 것

화면 "소금빵 2500원" 한 줄만 보입니다. 메모는 어디에도 없습니다

소금빵 2500원

```
<!-- 가격은 매달 확인할 것 -->
```

```
<p>소금빵 2500원</p>
```

위 상자 안에는 정말로 주석이 들어 있습니다. Ctrl+U 로 소스를 열어 보면 보입니다. 화면에 없다 와 파일에 없다 는 완전히 다른 말입니다.

문제 7 (개념04 섹션1)

단팔빵 2000원 문단에 title 속성을 붙이세요.

기대 출력 글자 모양은 그대로이고, 마우스를 올리면 작은 상자가 뜹니다.

문제 7 — 속성 붙이기

단팔빵 2000원 문단에 title 속성으로 오늘 구운 빵입니다 를 붙이세요. (개념04 섹션 1)

화면 "단팔빵 2000원" 한 줄이 보입니다. 글자 모양은 안 달라졌습니다

단팔빵 2000원

```
<p title="오늘 구운 빵입니다">단팔빵 2000원</p>
```

위 문장에 마우스를 올린 채 1초쯤 기다리면 작은 상자에 오늘 구운 빵입니다 가 뜹니다. 속성은 이렇게 글자 모양을 안 바꾸면서 정보만 덧붙입니다.

문제 8 (개념04 섹션5)

우리 동네 빵집 문단에 id 속성으로 shopName 을 붙이세요.

기대 출력 id 를 붙이기 전과 화면이 완전히 똑같습니다.

문제 8 — id 붙이기

우리 동네 빵집 문단에 id 속성으로 shopName 을 붙이세요. (개념04 섹션 5)

화면 문제 1 상자와 완전히 똑같습니다. id 는 화면에 흔적을 안 남깁니다

우리 동네 빵집

```
<p id="shopName">우리 동네 빵집</p>
```

문제 1 의 상자과 나란히 놓고 보세요. 화면이 완전히 같습니다. id 는 그냥 이름표라서 그 자체로는 아무 일도 하지 않습니다. 08단원에서 CSS 로 이 이름을 부르게 되면 그때부터 쓸모가 생깁니다.

문제 9 (개념05 섹션3)

단팔빵 과 2000원 사이에 스페이스를 열 칸 넣으세요.

기대 출력 사이가 한 칸만 벌어져 보입니다.

문제 9 — 공백은 몇 칸이 될까

단팔빵 과 2000원 사이에 스페이스를 열 칸 넣은 문단을 만들고 화면을 확인하세요. (개념05 섹션 3)

화면 "단팔빵 2000원" 으로, 사이가 한 칸만 벌어져 보입니다

단팔빵 2000원

<p>단팔빵 2000원</p>

코드에는 스페이스가 열 칸 들어 있는데 화면에는 한 칸만 나옵니다. 연달아 있는 공백은 몇 개든 한 칸으로 합쳐지기 때문입니다. 스페이스로 글자 위치를 맞추려는 시도는 언제나 실패합니다.

문제 10 (개념05 섹션4)

문단 하나 안에 세 문장을 엔터로 나눠서 쓰세요.

기대 출력 세 문장이 한 줄로 죽 이어져서 보입니다.

문제 10 — 엔터를 쳐도 줄이 안 나뉜다

문단 하나 안에 아침 7시에 엽니다. 오후 8시에 닫습니다. 월요일은 쉽니다. 를 엔터로 줄을 나눠서 쓰세요. 줄바꿈 태그는 쓰지 않습니다. (개념05 섹션 4)

화면 세 문장이 한 줄로 죽 이어져서 보입니다

아침 7시에 엽니다.
오후 8시에 닫습니다.
월요일은 쉽니다.

<p>아침 7시에 엽니다.
오후 8시에 닫습니다.
월요일은 쉽니다.</p>

코드에서는 세 줄인데 화면에서는 한 줄입니다. 브라우저는 코드의 줄바꿈을 공백 한 칸으로만 취급합니다. 문장 사이가 한 칸씩 벌어져 있는 것이 그 흔적입니다.

문제 11 [응용] (개념05 섹션5)

문제 10의 코드를 세 줄이 되게 고치세요. 문단은 하나여야 합니다.

기대 출력 세 문장이 각각 한 줄씩, 세 줄로 나뉘어 보입니다.

문제 11 — [응용] 진짜로 세 줄 만들기

문제 10의 코드를 가져와 `<br />`로 세 줄이 되게 고치세요. 문단은 여전히 하나여야 합니다. (개념05 섹션 5)

화면 세 문장이 각각 한 줄씩, 세 줄로 나뉘어 보입니다

아침 7시에 엽니다.오후 8시에 닫습니다.월요일은 쉽니다.

```
<p>아침 7시에 엽니다.<br />오후 8시에 닫습니다.<br />월요일은 쉽니다.</p>
```

문제 10과 글자는 완전히 같고 `<br />` 두 개만 늘었습니다. 화면을 나누는 것은 코드의 생김새가 아니라 태그입니다. 문단을 세 개로 나눴다면 줄 사이가 더 넓어져서 이 화면과 달라집니다.

문제 12 [응용] (개념04 섹션3)

문단 하나에 id와 title을 함께 붙이세요.

기대 출력 마우스를 올리면 영업 시간 안내가 통째로 뜹니다.

문제 12 — [응용] 속성 두 개 붙이기

오전 7시부터 오후 8시까지 문단에 `id="openHours"`와 `title="영업 시간 안내"`를 함께 붙이세요. (개념04 섹션 3)

화면 한 줄이 보입니다. 속성을 두 개 붙였지만 모양은 그대로입니다

오전 7시부터 오후 8시까지

```
<p id="openHours" title="영업 시간 안내">오전 7시부터 오후 8시까지</p>
```

속성 사이는 공백 한 칸으로 띄웁니다. 쉼표를 찍지 않습니다. 값에 공백이 있는데 따옴표를 빼면 영업 까지만 값이 되고 시간 부터는 엉뚱한 속성이 되어 버립니다.

문제 13 [응용] (개념02 섹션4 · 개념05 섹션6)

div 하나 안에 문단 두 개를 넣고 들여쓰기를 맞추세요.

기대 출력 두 줄. 첫 줄은 전체가 굵고, 둘째 줄은 2500원 만 굵습니다.

문제 13 — [응용] 두 겹으로 감싸기

div 하나 안에 문단 두 개를 넣으세요. 첫째 문단은 오늘의 추천 전체를 굵게, 둘째 문단은 소금빵 2500원 중 2500원 만 굵게 합니다. 들여쓰기를 두 칸씩 맞추세요. (개념02 섹션 4 · 개념05 섹션 6)

화면 두 줄이 보입니다. 첫 줄은 전체가 굵고 둘째 줄은 "2500원" 만 굵습니다

오늘의 추천
소금빵 2500원

```
<div>
<p><strong>오늘의 추천</strong></p>
<p>소금빵 <strong>2500원</strong></p>
</div>
```

태그가 세 겹으로 겹쳐 있습니다. div 안에 p, p 안에 strong 입니다. 들여쓰기를 두 칸씩 맞춰 두면 어느 것이 어느 것 안에 있는지 한눈에 보입니다. 들여쓰기를 안 해도 화면은 똑같지만, 코드를 읽는 사람이 힘들어집니다.

문제 14 [도전] (개념03 섹션 1~6)

bakery.html 이라는 새 파일에 뼈대를 완성하세요.

기대 출력 탭에 우리 동네 빵집 이 적히고, 본문에 문단 두 개가 두 줄로 보입니다.

문제 14 — [도전] 새 파일에 뼈대 세우기

VS Code 로 bakery.html 을 만들고 빈칸을 채워 완성하세요. 브라우저 탭 이름은 우리 동네 빵집 이어야 하고, body 안에는 문단을 두 개 넣습니다. (개념03 섹션 1~6)

```
<!DOCTYPE html>
<html lang="ko"> <head> <meta charset="UTF-8" /> <title>우리 동네 빵집
</title> </head> <body> <p>매일 아침 7시에 문을 엽니다.</p> <p>월요일은 쉽니다.
</p> </body>
</html>
```

화면 두 줄이 보이고, 두 줄 사이가 약간 벌어져 있습니다

매일 아침 7시에 문을 엽니다. 월요일은 쉽니다.

빈칸에 들어가는 말은 여섯 종류이고, 위에서부터 차례로 <!DOCTYPE html>, lang, head, charset, title, body 입니다. head, title, body 는 여는 자리와 닫는 자리에 한 번씩 들어가므로 같은 말을 두 번 씁니다. charset 을 빼먹으면 한글이 깨질 수 있고, title 을 빼먹으면 탭에 파일 이름이 그대로 뜹니다.

문제 15 [도전] (개념02 · 개념04 · 개념05 종합)

다섯 조건을 모두 만족하는 가게 안내문을 만드세요.

기대 출력 글자 네 줄과 가로줄 하나. 메모는 안 보입니다.

문제 15 — [도전] 가게 안내문 만들기

지금까지 배운 것을 한 번에 씁니다. 조건은 다섯 가지입니다. 주석 메모 한 줄, 우리 동네 빵집 만 굵은 첫 문단, 가로줄 하나,
 로 두 줄인 주소 문단 하나, title 속성이 붙은 마지막 문단. (개념02 · 개념04 · 개념05 종합)

안내문 시작

화면 굵은 글자로 시작하는 한 줄, 회색 가로줄, 주소 두 줄,

마지막 한 줄. 글자는 모두 네 줄입니다. 메모는 안 보입니다

```
우리 동네 빵집 입니다.

서울시 마포구 행복로 121층 우리 동네 빵집
전화번호는 안내문 아래를 보세요.

<!-- 안내문 시작 -->
<p><strong>우리 동네 빵집</strong> 입니다.</p>
<hr />
<p>서울시 마포구 행복로 12<br />1층 우리 동네 빵집</p>
<p title="문의는 전화로 부탁드립니다">전화번호는 안내문 아래를 보세요.</p>
```

주소 문단은 문단 하나입니다. 문단을 두 개로 나누면 줄 사이가 넓어져서 위 화면과 달라집니다. 마지막 문단에 마우스를 올리면 작은 상자에 문의는 전화로 부탁드립니다 가 뜹니다. 맨 위 메모는 Ctrl+U 로만 보입니다.

문제 16 [확인] (개념02 · 개념04 · 개념05)

고장난 코드 세 군데를 찾아 고치세요.

기대 출력 네 줄. 둘째와 셋째 줄은 한 문단 안의 두 줄이라 사이가 좁습니다.

문제 16 — [확인] 고장난 코드 고치기

일부러 고장 난 코드의 세 군데를 찾아 고치는 문제였습니다. 세 가지 모두 에러 메시지가 뜨지 않습니다. (개념02 · 개념04 · 개념05)

이런 실수를 합니다

고장난 코드

```
p>우리 동네 빵집</p>
<p>매일 아침 7시에 문을 엽니다.
오후 8시에 닫습니다.</p>
<p title=영업 안내>궁금한 점은 전화 주세요.</p>
```

고친 코드

```
<p>우리 동네 빵집</p>
<p>매일 아침 7시에 문을 엽니다.<br />오후 8시에 닫습니다.</p>
<p title="영업 안내">궁금한 점은 전화 주세요.</p>
```

화면 네 줄이 보입니다. 둘째와 셋째 줄은 한 문단 안의 두 줄이라

사이가 좁고, 첫째와 둘째 줄 사이는 문단 사이라 더 넓습니다

```
우리 동네 빵집
매일 아침 7시에 문을 엽니다.오후 8시에 닫습니다.
궁금한 점은 전화 주세요.
```

고친 곳 1 — 첫 줄의 여는 꺾쇠 < 가 빠져 있었습니다. 태그로 인식되지 않아 코드가 글자로 그대로 보였습니다. “코드가 화면에 글자로 보인다” 면 십중팔구 꺾쇠나 빗금이 빠진 것입니다.

고친 곳 2 — 두 문장을 엔터로만 나눠 봤습니다. 코드의 줄바꿈은 공백 한 칸이 될 뿐이라 한 줄로 이어졌습니다.
 를 넣어 줄을 나누라고 직접 시켜야 합니다.

고친 곳 3 — title 값에 따옴표가 없었습니다. 값이 공백에서 잘려 영업 까지만 들어가고 안내 는 엉뚱한 속성이 되어 버렸습니다. 화면 글자는 멀쩡해서 눈으로는 절대 못 찾습니다. 마우스를 올려 봐야 합니다.

02단원 연습문제 정답 텍스트 태그

이 파일은 연습문제.html 과 같은 뼈대에 답을 채운 것입니다.

문제 번호와 문장은 그대로 두었습니다.

우리 동네 빵집

메뉴

빵

음료

아침 7시에 문을 엽니다.

저녁 9시에 문을 닫습니다.

서울시 강남구 테헤란로 123

2층 201호

실행하면 나오는 화면 — 텍스트 태그

답을 보기 전에 스스로 한 번 풀어 보세요. 답이 여러 개인 문제도 있습니다. 화면 결과가 같으면 맞은 것입니다. 각 답 아래에 왜 그렇게 되는지를 짧게 붙여 두었습니다.

각 상자 안이 정답 코드가 실제로 그려진 결과입니다. VS Code 에서 이 파일을 열어 상자 안의 코드를 확인하세요. 여러분이 쓴 코드와 글자가 달라도 화면 결과가 같으면 맞은 것입니다.

문제 1 (개념01 섹션1)

문제 1

제목 태그 h2 로 우리 동네 빵집 이라는 제목을 만드세요.

화면 "우리 동네 빵집" 이 크고 굵은 글씨로 한 줄 보입니다

우리 동네 빵집

h2 의 기본 글자 크기는 24px 이고 굵기는 700 입니다. 보통 문단이 16px, 굵기 400 이라서 눈에 띄게 커 보입니다. 크기가 커진 것은 덤이고, h2 를 쓴 진짜 이유는 이 줄이 제목이기 때문입니다.

문제 2 (개념01 섹션2)

문제 2

h2 로 메뉴 를 쓰고, 그 아래에 h3 로 빵 과 음료 를 차례로 쓰세요.

화면 세 줄이 보입니다. "메뉴" 가 가장 크고,

"빵" 과 "음료" 는 서로 같은 크기이면서 "메뉴" 보다 작습니다

메뉴
빵
음료

h3 두 개를 쓴 이유는 빵과 음료가 서로 같은 단계이기 때문입니다. 같은 단계면 같은 숫자를 씁니다. h3 다음에 h4 를 쓰면 음료가 빵에 속한다는 뜻이 되어 버립니다.

문제 3 (개념02 섹션1)

문제 3

문단 태그 p 로 두 문단을 만드세요. 내용은 아침 7시에 문을 엽니다. 와 저녁 9시에 문을 닫습니다. 입니다.

화면 두 줄이 보이고 두 줄 사이가 눈에 띄게 벌어져 있습니다

아침 7시에 문을 엽니다.
저녁 9시에 문을 닫습니다.

간격을 벌리라고 시킨 적이 없는데 벌어집니다. p 에 위아래 16px 여백이 기본으로 붙어 있기 때문입니다. 문단이 두 개라는 뜻을 적었더니 간격은 브라우저가 알아서 준 것입니다.

문제 4 (개념02 섹션3)

문제 4

문단 하나 안에서 주소를 두 줄로 나누세요. 내용은 서울시 강남구 테헤란로 123 과 2층 201호 입니다.

화면 두 줄로 보이는데 두 줄 사이가 벌어지지 않고 바로 붙어 있습니다

서울시 강남구 테헤란로 1232층 201호

문제 3 과 화면을 비교해 보세요. 거기는 사이가 벌어졌고 여기는 붙어 있습니다. 문단이 두 개인 것과 한 문단 안에서 줄만 바뀐 것은 다릅니다. 주소는 한 덩어리이므로 문단 하나 안에서 `<br />` 로 나눕니다.

문제 5 (개념02 섹션5)

문제 5

문단 오전 이야기입니다. 와 오후 이야기입니다. 를 쓰고, 그 사이에 가로줄을 하나 그으세요.

화면 두 문단 사이에 상자 가로폭만큼 긴 회색 가로줄이 하나 보입니다

오전 이야기입니다.

오후 이야기입니다.

`<hr />` 는 빈 태그라 닫는 태그를 쓰지 않습니다. 높이는 2px, 색은 회색입니다. 선을 그으라는 뜻이 아니라 여기서 이야기가 바뀐다는 뜻입니다.

문제 6 (개념03 섹션1)

문제 6

모임은 3시입니다. 라는 문단을 만들고, 3시 두 글자만 중요하다고 표시하세요.

화면 한 줄이 보이고 그중 "3시" 두 글자만 굵습니다

모임은 3시입니다.

 를 써도 화면은 똑같이 나옵니다. 하지만 놓치면 안 되는 정보라서 굵게 한 것이므로 strong 이 맞습니다. 굵기 값은 둘 다 700 이라 눈으로는 구분할 수 없습니다.

문제 7 (개념03 섹션2)

문제 7

나는 오늘 간다. 라는 문단을 만들고, 오늘 두 글자를 힘주어 읽으라고 표시하세요.

화면 한 줄이 보이고 그중 "오늘" 두 글자만 오른쪽으로 비스듬히 기울어져 있습니다.

굵어지지 않는 것입니다

나는 오늘 간다.

소리 내어 읽어 보면 "내일이 아니라 오늘" 이라는 뜻이 들립니다. 말의 억양이 바뀌는 자리가 em 입니다. 중요도가 높은 것은 strong 이고 둘은 종류가 다른 강조입니다.

문제 8 (개념03 섹션4)

문제 8

오늘의 메뉴는 김치찌개 입니다 (1인분 기준) 라는 문단을 만들되, 김치찌개 에는 형광펜을, (1인분 기준) 에는 겉다리 말 표시를 하세요.

화면 한 줄이 보입니다. "김치찌개" 네 글자 뒤에만 노란 칠이 되어 있고,

"(1인분 기준)" 은 나머지 글자보다 작습니다

오늘의 메뉴는 김치찌개 입니다
(1인분 기준)

노란 칠이 줄 끝까지 가지 않고 글자 뒤에서 끝나는 것을 보세요. mark 가 인라인 요소라 글자 길이만큼만 차지하기 때문입니다. 블록 요소였다면 줄 끝까지 칠해졌을 것입니다.

문제 9 (개념03 섹션5)

문제 9

가격이 30,000원 에서 19,000원 으로 바뀌었습니다. 지운 값과 새로 넣은 값을 각각 표시해 한 문단으로 만드세요.

화면 한 줄이 보입니다. "30,000원" 글자 가운데를 가로지르는 줄이 그어져 있고,

"19,000원" 아래에는 밑줄이 그어져 있습니다

원래 가격 30,000원
할인가 19,000원

del 은 가운데 줄, ins 는 밑줄이 기본입니다. 줄을 그으려고 쓴 것이 아니라 지웠다는 뜻을 적은 것입니다. 줄만 긋고 싶다면 09단원의 text-decoration 을 씁니다.

문제 10 (개념04 섹션1)

문제 10

가격은 12,000원 입니다. 라는 문단을 만들고, 12,000원 만 span 으로 감싸세요. 나중에 CSS 로 그 부분만 꾸미려고 손잡이를 다는 것입니다.

화면 한 줄이 보입니다. span 이 없을 때와 화면이 완전히 같습니다

가격은 12,000원 입니다.

span 은 아무 뜻도 없고 화면도 바꾸지 않습니다. 07단원에서 CSS 를 배우기 전까지는 손잡이일 뿐입니다. 인라인 요소라서 줄도 끊지 않습니다.

문제 11 (개념05 섹션1, 섹션2)

문제 11

화면에 p 태그는 <p> 라고 씁니다 라는 한 줄이 보이도록 문단을 만드세요. 꺾쇠가 글자로 보여야 합니다.

화면 "p 태그는 <p> 라고 씁니다" 가 한 줄로 보입니다

p 태그는 <p> 라고 씁니다

꺾쇠를 그대로 썼다면 브라우저가 가운데 <p> 를 진짜 태그로 읽어 문단이 두 개로 쪼개졌을 것입니다. <와 >로 바꿔 쓰면 글자로 남습니다.

문제 12 [응용] (개념03 섹션6)

문제 12 [응용]

물은 H₂O 이고, 3제곱수 3² 는 9입니다. 라는 문단을 만들되, H₂O 의 2 는 아래첨자로, 3² 의 뒤쪽 2 는 위첨자로 만드세요. 화면에서 물 분자식과 거듭제곱이 제대로 보이면 됩니다.

화면 한 줄이 보입니다. "H₂O" 의 2 가 아래쪽에 작게 붙어 있고,

"3²" 의 뒤쪽 2 는 3 의 오른쪽 위에 작게 붙어 있습니다

물은 H₂O 이고,
3제곱수 3² 는 9입니다.

sub 와 sup 은 글자 크기를 13.3333px 로 줄이고 세로 위치를 바꿉니다. 첨자가 위아래로 빠져나오기 때문에 이 문단만 줄 높이가 27px 에서 33px 로 커집니다.

문제 13 [응용] (개념05 섹션3, 섹션4)

문제 13 [응용]

페이지 맨 아래에 넣을 저작권 한 줄을 만드세요. 화면에 © 2026 우리 동네 빵집 · 대표 홍길동 이 보이면 됩니다. 홍 과 길동 사이는 줄이 갈라지지 않는 공백으로 쓰세요.

화면 "© 2026 우리 동네 빵집 · 대표 홍길동" 이 한 줄로 보입니다

©; 2026 우리 동네 빵집 · 대표 홍길동

©; 는 저작권 기호, · 는 가운뎃점입니다. 는 보통 공백처럼 보이지만 줄 끝에 닿아도 그 자리에서 갈라지지 않습니다. 이름이 두 줄로 찢어지는 것을 막아 줍니다.

문제 14 [응용] (개념01 섹션2, 개념02 섹션1, 개념03 섹션1)

문제 14 [응용]

간단한 요리법 한 덩어리를 만드세요. 필요한 것은 이렇습니다. h2 로 김치찌개 만들기, 그 아래 h3 로 재료와 순서, 각 h3 아래에 설명 문단 하나씩, 그리고 재료 설명 안의 김치 한 컵 을 중요 표시하세요.

화면 다섯 줄이 보입니다. "김치찌개 만들기" 가 가장 크고,

"재료" 와 "순서" 가 같은 크기로 그보다 작습니다. 각 소제목 아래에 보통 크기 문단이 한 줄씩 있고, 첫 문단 안의 "김치 한 컵" 만 굵습니다

김치찌개 만들기
재료
돼지고기 200g 과 김치 한 컵이 필요합니다.
순서
냄비에 김치를 볶다가 물을 붓고 끓입니다.

h2 다음이 h3 인 것을 확인하세요. 단계를 건너뛰지 않았습니다. 재료와 순서가 서로 같은 단계라서 둘 다 h3 입니다.

문제 15 [도전] (개념01~05 종합)

문제 15 [도전]

상품 소개 한 덩어리를 만드세요. 아래 여섯 가지를 모두 넣어야 합니다.

1) h2 로 상품명 수제 단팥빵 · 2) 설명 문단 하나, 그 안에 당일 생산 을 중요 표시 · 3) 값이 3,000원 에서 2,500원 으로 바뀐 표시 · 4) (3개 이상 구매 시) 를 겹다리 말로 · 5) 가로줄 하나 · 6) 마지막 줄에 © 2026 우리 동네 빵집 을 겹다리 말로

화면 위에서부터 "수제 단팥빵" 이 크고 굵게 한 줄,

설명 문단 한 줄에서 "당일 생산" 만 굵고, 가격 줄에서 "3,000원" 에 가운데 줄, "2,500원" 에 밑줄, "(3개 이상 구매 시)" 가 작게, 회색 가로줄 하나, 맨 아래 "© 2026 우리 동네 빵집" 이 작게 보입니다

수제 단팔빵
국산 팔으로 당일 생산합니다.
3,000원 2,500원
(3개 이상 구매 시)

© 2026 우리 동네 빵집

상품명을 h2 로 쓴 것이 핵심입니다. strong 으로 써도 굵어지지만 제목이라는 뜻이 사라집니다. 가격 줄에서는 del, ins, small 세 인라인 요소가 한 줄에 나란히 이어집니다. 인라인이라 줄을 끊지 않기 때문입니다.

문제 16 [확인] (개념02 섹션6, 개념03 섹션8, 개념05 섹션5)

문제 16 [확인]

아래 상자의 코드는 에러가 하나도 안 납니다. 그런데 만든 사람이 원한 화면과 다르게 나옵니다. 잘못된 곳 세 군데를 찾아 고치세요.

고치기 전 코드는 이랬습니다.

```
<p>모임은 <strng>오후 3시</strng>입니다.</p>
<p>회사 이름은 AT&copy 라는 회사 입니다.</p>
<p>자세한 내용은 <p>공지 게시판</p> 을 보세요.</p>
```

무슨 일이 일어났는지부터 정리합니다.

- 첫째 줄 strng 이라는 태그는 없습니다. 브라우저는 모르는 태그를 없는 셈 치고 안의 글자만 그립니다. 그래서 안 굵어집니다. 에러는 나지 않습니다.
- 둘째 줄 © 가 저작권 기호로 읽혔습니다. 앰퍼샌드 뒤에 우연히 특수문자 이름이 붙어 있었기 때문입니다. 화면에는 "AT@ 라는 회사" 가 나옵니다. copy 라는 글자가 통째로 사라진 것입니다.
- 셋째 줄 p 안에 p 를 넣었습니다. 브라우저는 안쪽 p 를 만나는 순간 바깥 p 를 강제로 닫아 버립니다. 그래서 한 줄이 세 줄로 쪼개집니다.

고친 코드가 아래 상자 안에 들어 있습니다.

화면 세 줄이 보입니다.

첫 줄의 “오후 3시” 만 굵고, 둘째 줄에 “AT© 라는 회사 입니다.” 가 글자 그대로 보이고, 셋째 줄이 “자세한 내용은 공지 게시판 을 보세요.” 한 줄로 이어집니다

모임은 오후 3시입니다.
회사 이름은 AT© 라는 회사 입니다.
자세한 내용은 공지 게시판 을 보세요.

고친 곳은 셋입니다. strng → strong (철자), AT&copy (앰퍼샌드를 특수문자 표기로), 안쪽 <p> → (블록을 인라인으로). 셋 다 고치기 전에도 에러는 나지 않았습니다. 화면을 눈으로 보고 이상한 것을 알아채는 것이 이 문제의 전부입니다.

셋째 줄은 안쪽 태그를 아예 지워도 화면 결과가 같습니다. 하지만 나중에 CSS 로 그 부분만 꾸밀 계획이 있다면 으로 남겨 두는 편이 좋습니다.

03단원 연습문제 정답

목록과 표

• 김밥 • 물통 • 돛자리
• 연필 500원 • 지우개 300원 • 공책 1200원
• 음료 ◦ 커피 ◦ 주스 • 음식
1. 물을 끓입니다 2. 면을 넣습니다 3. 4분 뒤 불을 끕니다

실행하면 나오는 화면 — 정답

03단원 연습문제 정답 — 목록과 표

읽는 법 문제 파일과 번호 · 문장이 똑같습니다. 비어 있던 상자에 답이 채워져 있고, 그 아래에 왜 그렇게 되는지 적어 두었습니다. 답이 하나만 있는 것은 아닙니다. 화면이 같게 나오면 맞은 것입니다.

먼저 스스로 풀어 보고 나서 여세요.

상자 안에 답이 채워져 있습니다. 아래 회색 글씨는 왜 그렇게 되는지에 대한 설명입니다. 화면이 같게 나왔다면 코드가 조금 달라도 맞은 것입니다.

문제 1 (개념01 섹션1)

문제 1 — 준비물 목록 만들기

ul 과 li 로 “김밥 · 물통 · 돛자리” 목록을 만드세요.

화면 점(•) 이 앞에 붙은 세 줄이 왼쪽에서 한 칸 들어간 자리에 보입니다

김밥
물통
돛자리

항목 하나가 li 하나입니다. 점과 왼쪽 여백은 ul 을 썼기 때문에 브라우저가 저절로 붙인 것입니다. 여백은 재 보면 40px 입니다.

문제 2 (개념01 섹션3)

문제 2 — 항목 안의 한 부분만 굵게

“연필 500원 · 지우개 300원 · 공책 1200원” 목록을 만들고 물건 이름만 strong 으로 감싸세요.

화면 점이 붙은 세 줄이 보이고 각 줄에서 물건 이름 부분만 굵게 보입니다

연필 500원
지우개 300원
공책 1200원

strong 을 li 안에 넣은 것이 핵심입니다. ... 처럼 바깥에 씌우면 줄 전체가 굵어지고, ul 의 자식이 li 가 아니게 되어 규칙도 어깁니다.

문제 3 (개념01 섹션4)

문제 3 — 목록 안에 목록 넣기

“음료” 아래에 “커피” 와 “주스” 를 넣고, 그다음에 “음식” 항목이 오는 중첩 목록을 만드세요. 안쪽 ul 을 li 안에 넣어야 합니다.

화면 "음료" 와 "음식" 앞에는 까만 점(•), "커피" 와 "주스" 앞에는

속이 빈 동그라미(◦) 가 붙고 한 칸 더 안으로 들어갑니다

음료

커피
주스

음식

"음료" 뒤에서 를 바로 닫지 않고, 안쪽 ul 이 끝난 뒤에 닫은 것이 핵심입니다. 표시가 • 에서 ◦ 로 바뀌는 것은 브라우저가 깊이에 따라 알아서 하는 일입니다.

문제 4 (개념02 섹션1)

문제 4 — 순서 있는 목록 만들기

"물을 끓입니다 · 면을 넣습니다 · 4분 뒤 불을 끕니다" 를 이 로 만드세요. 번호는 직접 쓰지 마세요.

화면 "1." "2." "3." 이 앞에 붙은 세 줄이 보입니다

물을 끓입니다
면을 넣습니다
4분 뒤 불을 끕니다

코드 어디에도 1, 2, 3 이 없습니다. 브라우저가 li 를 세어 준 것입니다. 중간에 항목을 끼워 넣어도 번호가 저절로 다시 매겨집니다.

문제 5 (개념02 섹션2)

문제 5 — ul 과 li 고르기

(가) 냉장고 속 음식 "우유 · 계란 · 김치" 와 (나) 조리 순서 "물 붓기 · 면 넣기 · 스프 넣기" 를 각각 알맞은 태그로 만드세요.

화면 점(•) 이 붙은 세 줄이 보입니다

우유
계란
김치

화면 "1." "2." "3." 이 붙은 세 줄이 보입니다

물 붓기
면 넣기
스프 넣기

(가)는 “김치·우유·계란”으로 바꿔 써도 뜻이 같으니 ui 입니다. (나)는 스프를 먼저 넣고 물을 부을 수 없으니 이 입니다. 순서를 바꿔 보는 것이 판단 방법입니다.

문제 6 (개념02 섹션3)

문제 6 — 번호를 알파벳으로 바꾸기

“가입 신청·서류 제출·결과 확인”을 대문자 알파벳 번호가 붙는 목록으로 만드세요.

화면 "A." "B." "C." 가 앞에 붙은 세 줄이 보입니다

가입 신청
서류 제출
결과 확인

type의 값은 대소문자를 가립니다. type="a"라고 쓰면 소문자 "a. b. c."가 나옵니다. 태그 이름은 대소문자를 안 가리지만 속성 값은 가립니다.

문제 7 (개념02 섹션4)

문제 7 — 시작 번호 바꾸기

“일곱 번째 순서입니다·여덟 번째 순서입니다”를 7번부터 번호가 붙게 만드세요.

화면 "7." "8." 이 앞에 붙은 두 줄이 보입니다

일곱 번째 순서입니다
여덟 번째 순서입니다

start 는 이 에 붙습니다. 첫 항목에 value="7" 을 붙여도 화면 결과는 같지만, 목록 전체의 시작을 정하는 것은 start 입니다.

문제 8 (개념03 섹션1)

문제 8 — 용어집 만들기

"태그 · 꺾쇠로 감싼 이름표입니다" 와 "요소 · 여는 태그부터 닫는 태그까지의 덩어리입니다" 를 dl 로 만드세요.

화면 "태그" 와 "요소" 는 왼쪽 끝에, 두 설명 줄은 한 칸 안으로 들어가

모두 네 줄로 보입니다. 점도 번호도 없습니다

태그
꺾쇠로 감싼 이름표입니다
요소
여는 태그부터 닫는 태그까지의 덩어리입니다

들여쓰기되는 쪽은 항상 dd 입니다. 재 보면 40px 입니다. dt 는 굵어지지 않습니다. 굵기가 필요하다면 strong 이나 09단원 CSS 를 씁니다.

문제 9 (개념03 섹션2)

문제 9 — 용어 하나에 설명 두 개

"말" 이라는 dt 하나에 "타고 다니는 동물입니다" 와 "입으로 하는 이야기입니다" 두 dd 를 다세요.

화면 "말" 한 줄 아래에 들여쓰기된 설명 두 줄이 간격 없이 붙어 나옵니다

말
타고 다니는 동물입니다
입으로 하는 이야기입니다

브라우저는 다음 dt 가 나오기 전까지의 dd 를 전부 앞 dt 의 설명으로 봅니다. 그래서 dt 를 다시 쓸 필요가 없습니다.

문제 10 (개념04 섹션1)

문제 10 — 표 만들기

“사과 1000” 과 “바나나 1500” 을 2행 2열 표로 만드세요. <table border="1"> 로 시작하세요.

화면 칸마다 테두리가 있는 2행 2열 표가 보입니다.

글자는 모두 보통 굵기이고 칸 왼쪽에 붙어 있습니다

사과
1000

바나나
1500

tr 이 가로 한 줄, 그 안의 td 가 칸입니다. 행 단위로 씁니다. 열을 만드는 태그는 없고, 각 행의 칸 개수가 같아서 2열이 된 것입니다.

문제 11 (개념04 섹션2)

문제 11 — 제목 칸 넣기

문제 10 의 표 맨 위에 “이름 · 가격” 제목 줄을 붙여 3행 2열 표로 만드세요. 제목 칸에는 td 가 아닌 태그를 씁니다.

화면 맨 윗줄의 “이름” 과 “가격” 만 굵은 글씨로 칸 가운데에 놓입니다.

아래 두 줄은 보통 굵기에 왼쪽 정렬입니다

이름
가격

사과
1000

바나나
1500

th 로 쓰면 굵기가 700, 정렬이 가운데가 됩니다. td 는 굵기 400, 왼쪽 정렬입니다. 이 모양은 브라우저 기본값일 뿐이고, th 를 고르는 진짜 이유는 "이 칸은 제목" 이라는 뜻 때문입니다.

문제 12 (개념04 섹션3·섹션4)

문제 12 — caption 과 thead · tbody

문제 11 의 표에 caption 으로 "과일 가격표" 제목을 달고, 제목 줄은 thead, 나머지 두 줄은 tbody 로 감싸세요.

화면 표 위에 "과일 가격표" 가 한 줄 보이고, 표 모양은 문제 11 과 같습니다

과일 가격표

이름
가격

사과
1000

바나나
1500

caption 은 table 의 첫 자식으로 씁니다. thead · tbody 는 화면을 거의 바꾸지 않지만 코드가 읽기 쉬워지고, 08단원에서 CSS 로 몸통 줄만 골라 꾸밀 때 쓰입니다.

문제 13 (개념05 섹션1·섹션2)

문제 13 — colspan 으로 가로로 합치기

제목 줄이 "교시 · 월 · 화" 인 표를 만드세요. 1교시는 "국어 · 영어", 2교시는 두 칸을 합쳐 "봉사 활동" 한 칸으로 만드세요.

화면 3행 3열 표에서 맨 아랫줄만 오른쪽 두 칸이 하나로 합쳐져 보입니다.

표의 오른쪽 끝은 세 줄 모두 같은 자리에서 끝납니다

교시

월

화

1

국어

영어

2

봉사 활동

마지막 행의 td 가 두 개뿐인 것이 핵심입니다. $1 + 2 = 3$ 으로 위 줄들과 칸 수가 맞습니다. `<td>영어</td>` 를 지우지 않으면 4칸이 되어 그 줄만 오른쪽으로 튀어나옵니다.

문제 14 [응용] (개념05 섹션3·섹션4)

문제 14 — [응용] rowspan 으로 세로로 합치기

제목 줄이 "시간 · 과목" 인 표를 만드세요. "오전" 은 rowspan 으로 두 줄을 합쳐 "국어" 와 "영어" 를 덮고, 그 아래에 "오후 · 수학" 줄을 넣으세요.

화면 4행 2열 표에서 왼쪽 "오전" 칸이 아래 두 줄에 걸쳐 있고,

그 아래에 "오후 · 수학" 줄이 있습니다. "오전" 글자는 두 줄 높이의 세로 가운데에 놓입니다

시간

과목

오전
국어

영어

오후
수학

“영어”가 있는 행에 td가 하나뿐인 것이 핵심입니다. 왼쪽 첫 자리는 위에서 내려온 “오전” 칸이 이미 차지하고 있습니다. 여기에 <td>오전</td>을 또 쓰면 없던 세 번째 열이 생깁니다.

문제 15 [도전] (개념05 섹션5)

문제 15 — [도전] 제목이 두 줄인 표

왼쪽에 “반”(세로 두 줄), 오른쪽 위에 “1학기”(가로 두 칸), 그 아래에 “중간”과 “기말”이 오는 제목을 만들고, 자료 줄로 “1반 90 95”와 “2반 85 88”을 넣으세요.

화면 4행짜리 표가 나오고, 왼쪽 “반” 칸이 위 두 줄에 걸쳐 있습니다.

“1학기”는 오른쪽 위에서 두 칸 폭을 차지하고, 그 아래에 “중간”과 “기말”이 한 칸씩 있습니다. 오른쪽 끝은 네 줄 모두 같은 자리에서 끝납니다

반
1학기

중간
기말

1반
90
95

2반
85
88

행마다 칸을 세어 보세요. 첫 줄은 $1(\text{rowspan}) + 2(\text{colspan}) = 3$ 칸, 둘째 줄은 위에서 내려온 1칸 + 직접 쓴 2칸 = 3칸, 셋째 · 넷째 줄은 3칸입니다. 모든 행이 3칸이라 오른쪽이 가지런합니다.

문제 16 [확인] (개념04 섹션2 · 개념05 섹션2)

문제 16 — [확인] 이상한 표 고치기

아래 표는 에러가 나지 않는데 두 곳이 이상합니다. (1) 맨 윗줄이 제목인데 굵지도 가운데 정렬도 아닙니다. (2) 맨 아랫줄만 오른쪽으로 튀어나옵니다. 무엇 때문인지 찾아 고치세요.

화면 표가 4열이 되어 맨 아랫줄만 오른쪽으로 튀어나옵니다.

위 두 줄의 오른쪽 끝에는 테두리 없는 빈 공간이 남습니다. 맨 윗줄 글자도 보통 굵기에 왼쪽 정렬입니다

교시
월
화

1
국어
영어

2
전교 조회
수학

화면 3행 3열 표가 되고 맨 윗줄만 굵고 가운데 정렬입니다.

오른쪽 끝이 세 줄 모두 같은 자리에서 끝납니다

교시
월
화

1
국어
영어

2
전교 조회

고친 곳은 두 군데입니다. (1) 맨 윗줄의 td 세 개를 th 로 바꿨습니다. 제목 칸이라는 뜻이 없으면 굵기도 정렬도 안 바뀝니다. (2) 마지막 행 끝의 <td>수학</td> 을 지웠습니다. colspan="2" 로 두 칸을 합쳤으니

그 행은 $1 + 2 = 3$ 칸이어야 하는데 4칸이 되어 있었습니다.

둘 다 에러가 나지 않는 종류의 문제입니다. 브라우저는 시키는 대로 그려 줄 뿐이라, 이런 것은 사람이 화면을 보고 찾아야 합니다. 표가 어긋나면 노려보지 말고 행마다 칸을 세세요.

04단원 연습문제 정답 링크와 이미지

문제 번호와 문장은 연습문제.html 과 똑같습니다.

각 답 아래에 왜 그렇게 되는지 짧은 해설을 붙였습니다.

답을 보기 전에 반드시 먼저 풀어 보세요.

눈으로 읽으면 다 아는 것 같지만, 손으로 쓰면 반드시 어딘가 막힙니다.

막히는 그 자리가 여러분이 아직 모르는 곳입니다.

답이 하나뿐인 문제는 많지 않습니다.

링크 글자나 alt 내용은 뜻만 맞으면 다르게 써도 정답입니다.

반면 경로, 태그 이름, 속성 이름은 정확히 같아야 합니다.



실행하면 나오는 화면 — 링크와 이미지

각 상자 안이 정답입니다. 상자 아래 회색 글은 왜 그렇게 되는지에 대한 해설입니다. 링크는 실제로 눌러 보고, 그림 크기는 눈으로 확인하세요.

문제 1 (개념01 섹션1)

같은 폴더에 있는 개념01_링크_a태그.html 로 가는 링크를 만드세요. 링크 글자는 "개념 01 — 링크와 a 태그" 로 하세요.

화면 파란 글씨에 밑줄이 그어진 한 줄이 보입니다. 누르면 개념01 파일이 열립니다

개념 01 - 링크와 a 태그

같은 폴더에 있으므로 폴더 이름 없이 파일 이름만 썼습니다. 파란색과 밑줄은 우리가 넣은 것이 아니라 브라우저가 링크에 자동으로 붙여 주는 모양입니다.

문제 2 (개념01 섹션2)

"여기를 클릭" 을 링크 글자만 읽어도 어디로 가는지 알 수 있게 고쳐 쓰세요.

화면 파란 밑줄로 "개념 02 - 경로" 가 보입니다

개념 02 - 경로

href 는 그대로 두고 사이에 낀 글자만 바꿨습니다. "경로 배우러 가기" 처럼 써도 정답입니다. 글자만 보고 목적지를 알 수 있으면 됩니다.

문제 3 (개념01 섹션3)

개념05_그림과_설명_figure.html 을 새 탭에서 여는 링크를 만드세요. 새 탭으로 열 때 같이 붙여야 하는 속성도 빠뜨리지 마세요.

화면 파란 밑줄 링크입니다. 누르면 이 탭은 그대로 있고 새 탭이 하나 더 열립니다

개념 05 를 새 탭에서 보기

target="_blank" 앞의 밑줄을 빠뜨리면 새 탭이 아니라 "blank 라는 이름의 창" 이 됩니다.
rel="noopener" 는 새 탭으로 열린 쪽이 원래 페이지를 건드리지 못하게 막는 안전장치입니다.

문제 4 (개념02 섹션3 · 개념04 섹션1)

이미지 폴더 안의 고양이.svg 를 화면에 넣으세요. alt 도 반드시 쓰세요.

화면 연한 하늘색 바탕에 주황색 고양이 얼굴이 200 × 200 크기로 보입니다

이미지/ 를 앞에 붙여 폴더 안으로 들어갔습니다. 크기를 안 적었으므로 파일이 가진 원래 크기 200 × 200 그대로 나옵니다. alt 는 “고양이.svg” 가 아니라 그림을 설명하는 말로 써야 합니다.

문제 5 (개념02 섹션3)

이미지 폴더 안의 아이콘 폴더 안에 있는 별.svg 를 화면에 넣으세요.

화면 연노랑 바탕에 노란 별이 60 × 60 크기로 보입니다

폴더를 지날 때마다 빗금을 하나씩 씁니다. 두 겹이므로 빗금도 두 개입니다. 별.svg 라고만 쓰면 이 파일이 있는 폴더에서 찾다가 실패합니다.

문제 6 (개념02 섹션4)

01_HTML_시작하기 폴더 안의 개념02_태그의_생김새.html 로 가는 링크를 만드세요.

화면 파란 밑줄 링크입니다. 누르면 01단원 파일이 열립니다

01단원 개념02 - 태그의 생김새

../ 로 04_링크와_이미지 폴더에서 먼저 나간 뒤, 01_HTML_시작하기 폴더로 다시 들어갔습니다. 나가기와 들어가기 순서대로 읽으면 경로가 그대로 보입니다.

문제 7 (개념03 섹션1 · 섹션2)

id 가 lastPart 인 h3 제목 “마지막 부분” 과 그리로 점프하는 링크를 만드세요.

화면 파란 밑줄 링크 한 줄과 그 아래 굵은 제목 “마지막 부분” 이 보입니다.

누르면 주소창 끝에 #lastPart 가 붙습니다

마지막 부분으로
마지막 부분

은 부르는 쪽에만 붙입니다. 도착지의 id 값은 lastPart 이지 #lastPart 가 아닙니다. 지금은 문서가 짧아 화면이 조금만 움직일 수 있습니다.

문제 8 (개념03 섹션3)

이 문서 맨 위로 올라가는 링크를 만드세요. body 에 id=“pageTop” 이 붙어 있습니다.

화면 파란 밑줄로 "맨 위로 가기" 가 보입니다. 누르면 문서 맨 위로 올라갑니다

맨 위로 가기

 라고 써도 맨 위로 갑니다. 다만 주소창에 # 만 남아 보기에 좋지 않아서, 이름을 붙여 부르는 쪽을 권합니다.

문제 9 (개념03 섹션4)

hong@example.com 으로 메일을 쓰는 링크를 만드세요.

화면 파란 밑줄로 "도서관에 메일 보내기" 가 보입니다.

누르면 메일 쓰기 창이 열립니다

도서관에 메일 보내기

mailto: 뒤에는 쌍점만 오고 빗금은 없습니다. mailto:// 라고 쓰면 받는 사람 칸이 이상하게 채워집니다.

문제 10 (개념03 섹션5)

전화번호 031-987-6543 으로 전화를 거는 링크를 만드세요.

화면 파란 밑줄로 031-987-6543 이 보입니다

031-987-6543

href 안의 번호와 화면에 보이는 번호가 같아야 합니다. 컴퓨터로 보는 사람은 눌러도 전화가 안 걸리므로, 번호가 눈에 보여야 옮겨 적을 수 있습니다.

문제 11 (개념04 섹션3)

이미지 폴더의 사과.svg 를 가로 120 크기로 넣으세요. 세로는 적지 마세요.

화면 사과 그림이 120 × 120 으로 보입니다. 동그란 모양 그대로입니다

가로만 정하면 세로는 원래 비율대로 따라옵니다. 원본이 200 × 200 인 정사각형이라 120 × 120 이 되었습니다. 값에는 단위를 붙이지 않습니다. 브라우저가 앞쪽 숫자만 읽고 뒤 글자를 버리기 때문에 120px 은 우연히 120 이 되지만, 10em 이라고 쓰면 10 픽셀이 되어 버립니다.

문제 12 [응용] (개념04 섹션4)

로고.svg 를 가로 180 으로 넣고, 누르면 개념02_경로.html 로 가게 만드세요. alt 에는 "누르면 어디로 가는지" 를 쓰세요.

화면 파란 로고 그림이 180 × 60 으로 보입니다. 테두리는 없고

마우스를 올리면 손가락 커서로 바뀝니다
a 가 부모, img 가 자식입니다. 감싸기만 하면 그림 전체가 눌립니다. 링크 안에 글자가 없으므로 alt 가 링크 글자 노릇을 합니다. 그래서 "우리 로고" 가 아니라 갈 곳을 적었습니다. 가로 180 을 주니 세로는 비율대로 60 이 되었습니다.

문제 13 [응용] (개념05 섹션1 · 섹션3)

풍경.svg 를 figure 로 묶고 figcaption 으로 설명을 붙이세요. alt 와 figcaption 이 서로 다른 말이어야 합니다.

화면 산 그림이 320 × 180 으로 보이고 그 아래에

"학교 뒷산에서 내려다본 풍경입니다" 가 보입니다. 둘 다 왼쪽에서 40 픽셀 안으로 들어가 있습니다

학교 뒷산에서 내려다본 풍경입니다

alt 에는 보이는 것(산 두 개와 해)을, figcaption 에는 어디서 찍은 무엇인지를 썼습니다. 두 글이 같으면 소리로 듣는 사람이 같은 문장을 두 번 듣게 됩니다. 좌우 여백은 브라우저가 figure 에 미리 넣어 둔 것입니다.

문제 14 [도전] (개념05 섹션2 · 섹션3)

figure 하나에 그림 두 장, 설명 하나. 설명을 그림들보다 위에 보이게 하세요.

화면 맨 위에 "고양이와 사과입니다" 가 한 줄 보이고, 그 아래에

고양이와 사과가 100 × 100 크기로 나란히 보입니다

고양이와 사과입니다

figcaption 을 줄 순서만 바꿔 맨 앞으로 올렸습니다. 맨 앞이면 위에, 맨 뒤면 아래에 보입니다. 가운데에 끼우면 안 됩니다. alt 는 그림마다 하나씩, figcaption 은 묶음 전체에 하나뿐입니다.

문제 15 [도전] (개념02~05 전체)

점프 링크 + 도착지 제목 + 링크가 된 그림을 figure 로 묶기.

화면 파란 밑줄 링크, 그 아래 굵은 제목 "사진 모음",

그 아래 80 × 80 노란 별과 "별을 누르면 개념05 로 갑니다" 가 보입니다

사진 모음으로 가기

사진 모음

별을 누르면 개념05 로 갑니다

세 가지가 한꺼번에 들어 있습니다. #photoZone 은 문서 안 이동, 이미지/아이콘/별.svg 는 두 겹 경로, a 로 감싼 img 는 그림 링크입니다. 그림이 링크가 되었으므로 alt 에는 갈 곳을 적었습니다. figure 안에 a 를 넣어도 아무 문제가 없습니다.

문제 16 [확인] (개념01 섹션6 · 개념03 섹션6 · 개념04 섹션6)

잘못된 세 군데를 고친 결과입니다.

(가) 개념 01 로 가기

→ href 가 없어서 링크가 아니었습니다. href 를 붙였습니다.

(나)

→ # 이 없어서 arriveHere 라는 파일을 찾고 있었습니다. # 을 붙였습니다.

(다)

→ 브라우저가 em 을 버리고 10 만 읽어 10 픽셀짜리 점이 되었습니다.
HTML 속성에는 단위를 붙이지 않습니다. 10em 을 100 으로 고쳤습니다.

화면 첫 두 줄이 파란 밑줄 링크가 되었고, 사과 그림이 100 × 100 으로

작아졌습니다. 둘째 줄을 누르면 "여기가 도착지" 로 화면이 움직입니다

개념 01 로 가기
여기가 도착지로 가기
여기가 도착지

세 가지 모두 에러가 나지 않는 잘못이었습니다. (가)는 글자 색이 검은색인 것으로, (나)는 눌러 봤을 때 하얀 오류 화면이 뜨는 것으로, (다)는 그림이 점처럼 작아진 것으로 알아챌 수 있습니다. HTML은 틀려도 조용합니다. 화면을 눈으로 확인하는 수밖에 없습니다.

05단원 연습문제 정답 품

보는 방법

연습문제.html 과 이 파일을 나란히 열고 비교하세요.

답이 여러 가지인 문제가 있습니다. 결과가 같으면 맞은 것입니다.

예를 들어 이름표를 for 와 id 로 연결하든 label 로 감싸든 결과는 같습니다.

각 답 아래에 왜 그렇게 되는지 짧은 설명을 붙여 두었습니다.

답만 베끼지 말고 설명을 읽으세요.

★ 문제 14, 15 의 보내기 버튼은 진짜로 제출합니다.

칸을 다 채우고 누르면 페이지가 새로고침됩니다.

주소창의 물음표 뒷부분을 지우거나 파일을 다시 열면 돌아옵니다.

이름

이메일

사는 곳

나이

☐ 약관에 동의합니다

배송 정보

받는 사람

실행하면 나오는 화면 — 품

상자 안에 있는 것이 답의 실행 결과입니다. 코드는 VS Code 에서 이 파일의 소스를 보세요. 결과가 같으면 코드가 조금 달라도 맞은 것입니다. 특히 이름표를 연결하는 방법은 두 가지 다 정답입니다.

문제 1 (개념01 섹션1.섹션3)

form 안에 "이름" 이라는 글자와 한 줄 입력칸을 하나 만드세요. 입력칸에는 name="userName" 이름표를 꼭 붙이세요.

답:

```
<form> <p>이름 <input type="text" name="userName" /></p>
</form>
```

화면 "이름" 글자 오른쪽에 흰 바탕에 회색 테두리인 빈 네모 칸이 있습니다

이름

form 은 아무것도 안 그립니다. 칸들을 한 묶음으로 만드는 상자일 뿐입니다. name 이 없으면 사용자가 값을 채워도 그 값은 어디에도 보내지지 않습니다. 화면은 똑같아서 알아챌 수 없으니 습관처럼 붙이세요.

문제 2 (개념01 섹션5)

비어 있을 때 "이메일을 입력하세요" 라는 회색 안내문이 보이게 하세요.

답:

```
<p>이메일 <input type="text" name="userMail" placeholder="이메일을 입력하세요" /></p>
```

화면 빈 칸 안에 열린 회색 글씨로 "이메일을 입력하세요" 가 보입니다.

글자를 치면 회색 글씨가 즉시 사라집니다

이메일

placeholder 는 값이 아니라 안내문입니다. 보내지지 않습니다. value 를 썼다면 검은 글씨가 들어가고 사용자가 지워야 없어집니다. 둘을 같이 쓰면 값이 안내문을 가려 안내문이 아예 안 보입니다.

문제 3 (개념01 섹션4)

"사는 곳" 칸에 처음부터 "부산" 이 들어 있게 하세요.

답:

```
<p>사는 곳 <input type="text" name="userCity" value="부산" /></p>
```

화면 칸 안에 검은 글씨로 "부산" 이 들어 있고 백스페이스로 지워집니다

사는 곳

value 는 진짜 값입니다. 사용자가 아무것도 안 해도 이 값이 보내집니다. 회원 정보 수정 화면처럼 지금 저장된 값을 보여 줘야 할 때 씁니다.

문제 4 (개념02 섹션2)

for 와 id 를 써서 이름표와 칸을 연결하세요. id 값은 userAge 로 하세요.

답:

```
<p><label for="userAge">나이</label> <input id="userAge" type="text" name="userAge" /></p>
```

화면 "나이" 글자를 누르면 오른쪽 칸 안에서 커서가 깜빡입니다

나이

for 값과 id 값이 글자 하나까지 같아야 연결됩니다. 대소문자도 구별합니다. 값이 어긋나면 에러 없이 그냥 안 눌립니다. 여기서는 id 와 name 이 둘 다 userAge 인데, 값만 같을 뿐 id 는 연결용, name 은 보낼 때 쓰는 이름표입니다.

문제 5 (개념02 섹션3·섹션4)

label 로 감싸는 방법으로 체크박스를 만드세요.

답:

```
<p> <label><input type="checkbox" name="agree" /> 약관에 동의합니다</label> </p>
```

화면 작은 빈 네모와 글자가 한 줄에 있습니다.

글자를 누르면 네모가 파란 바탕에 흰 체크 표시로 바뀌고, 한 번 더 누르면 다시 빈 네모가 됩니다

약관에 동의합니다

감싸면 for 와 id 를 안 써도 브라우저가 짝을 알아냅니다. 체크박스는 누를 곳이 13픽셀짜리 작은 네모라 연결이 특히 중요합니다. 연결하면 글자까지 누를 곳이 되어 훨씬 편해집니다.

문제 6 (개념02 섹션5)

“배송 정보” 라는 제목이 달린 묶음 상자를 만드세요.

답:

```
<fieldset> <legend>배송 정보</legend> <p><label for="recvName">받는 사람</label> <input id="recvName" type="text" name="recvName" /></p> <p><label for="recvAddr">주소</label> <input id="recvAddr" type="text" name="recvAddr" /></p></fieldset>
```

화면 열린 회색 테두리 상자가 있고, 왼쪽 위 테두리가 굵긴 자리에

“배송 정보” 글자가 걸쳐 있습니다. 상자 안에는 이름표와 칸이 두 줄로 들어 있습니다

배송 정보
받는 사람

주소

legend 는 fieldset 의 첫 자식으로 씁니다. fieldset 밖에 쓰면 테두리와 아무 관계 없는 그냥 글자가 되어 상자 위에 따로 놓입니다. 05단원에서 여러 칸을 묶을 때 쓸 수 있는 상자입니다.

문제 7 (개념03 섹션1·섹션2)

비밀번호, 이메일, 나이(숫자), 생일(날짜) 칸을 만드세요.

답:

```
<p><label for="ansPw">비밀번호</label> <input id="ansPw" type="password" name="pw" /></p> <p><label for="ansMail">이메일</label> <input id="ansMail" type="email" name="mail" /></p> <p><label for="ansAge">나이</label> <input id="ansAge" type="number" name="age" /></p> <p><label for="ansBirth">생일</label> <input id="ansBirth" type="date" name="birth" /></p>
```

화면 비밀번호 칸에 글자를 치면 까만 점이 하나씩 찍힙니다.

이메일 칸은 글자가 그대로 보입니다. 나이 칸에 마우스를 올리면 오른쪽 끝에 위아래 화살표가 나타납니다. 생일 칸에는 회색으로 “연도-월-일” 이 있고 오른쪽에 달력 그림이 있습니다. 앞 세 칸은 가로 길이가 같고 생일 칸만 눈에 띄게 짧습니다

비밀번호

이메일

나이

생일

type 값 하나로 생김새와 넣는 방법이 한꺼번에 바뀝니다. type 에 오타를 내면 예러 없이 조용히 text 로 그려집니다. 네 칸이 전부 똑같이 생겼다면 철자를 의심하세요.

문제 8 (개념03 섹션3)

취미 체크박스 세 개를 만들고 "음악" 만 켜 두세요.

답:

```
<p><label><input type="checkbox" name="hobby" value="movie" /> 영화</label></p>
<p><label><input type="checkbox" name="hobby" value="music" checked /> 음악</label>
</p>
<p><label><input type="checkbox" name="hobby" value="book" /> 독서</label></p>
```

화면 네모 세 개가 세로로 놓이고 가운데 "음악" 만 파란 바탕에 흰 체크 표시입니다.

세 개를 전부 켤 수도 있고 전부 끌 수도 있습니다

영화

음악

독서

체크박스는 name 이 같아도 서로 끄지 않습니다. 각각 따로 노는 스위치입니다. value 를 안 쓰면 켜졌을 때 on 이라는 값이 가서 무엇을 골랐는지 알 수 없게 됩니다. 꼭 적으세요.

문제 9 (개념03 섹션4)

결제 수단 라디오 세 개를 만들고 "카드" 를 골라 두세요.

답:

```
<p><label><input type="radio" name="pay" value="card" checked /> 카드</label></p>
<p><label><input type="radio" name="pay" value="cash" /> 현금</label></p>
<p><label><input type="radio" name="pay" value="bank" /> 계좌이체</label></p>
```

화면 동그라미 세 개가 세로로 놓이고 처음에는 "카드" 에만 파란 점이 있습니다.

다른 것을 누르면 앞의 점이 사라지고 누른 쪽으로 옮겨 갑니다

카드
현금
계좌이체

라디오 버튼을 한 무리로 묶는 것은 위치가 아니라 name 입니다. 세 개의 name 이 모두 pay 라서 하나만 켜집니다. 하나라도 다르게 쓰면 그 버튼은 혼자 노는 스위치가 되어 따로 켜집니다.

문제 10 (개념04 섹션1·섹션2)

배송 방법 목록을 만들고 "빠른 배송" 을 골라 두세요.

답:

```
<select name="ship"> <option value="normal">일반 배송
</option> <option value="quick" selected>빠른 배송</option> <option value="dawn">새벽
배송</option>
</select>
```

화면 접힌 상자에 "빠른 배송" 이 보이고 오른쪽에 아래쪽 화살표가 있습니다.

펼치면 세 줄이 나오고 "빠른 배송" 줄이 파랗게 칠해져 있습니다. 영어 값은 화면에 나오지 않습니다

배송 방법

일반 배송
빠른 배송
새벽 배송

option 의 value 가 보내는 값이고, 태그 사이의 글자가 화면에 보이는 글자입니다. 나눠 두면 나중에 화면 글자를 바꿔도 보내는 값은 그대로라 받는 쪽을 안 고쳐도 됩니다.

문제 11 (개념04 섹션4·섹션5)

다섯 줄 높이, 가로 마흔 글자짜리 여러 줄 칸을 만드세요. 완전히 비어 있어야 합니다.

답:

```
<textarea name="memo" rows="5" cols="40"></textarea>
```

화면 가로로 긴 빈 네모 칸이 다섯 줄 높이로 보입니다.

오른쪽 아래 모서리에 작은 빗금 삼각형이 있고 끌면 칸이 커집니다. 칸을 누르고 Ctrl+A 를 눌러도 파란 선택 표시가 전혀 생기지 않습니다

배송 요청 사항

여는 태그와 닫는 태그를 딱 붙여 써야 진짜 빈 칸이 됩니다. 보기 좋게 줄을 바꾸면 그 줄바꿈과 들여쓰기 공백이 그대로 값이 되어 버립니다. 그러면 사용자가 아무것도 안 써도 빈 값이 아니게 됩니다.

문제 12 (개념05 섹션2·섹션3)

눌러도 아무 일이 일어나지 않는 "미리 보기" 버튼을 만드세요.

답:

```
<form> <p><input type="text" name="title" /></p> <p><button type="button">미리 보기
</button></p>
</form>
```

화면 칸 아래에 "미리 보기" 라고 적힌 회색 버튼이 있습니다.

칸에 글자를 쳐 넣고 버튼을 눌러도 화면이 새로고침되지 않고 쳐 넣은 글자도 그대로 남아 있습니다

제목

미리 보기

type 을 안 쓰면 기본이 submit 이라 누르는 순간 제출됩니다. 제출과 상관없는 버튼에는 반드시 type="button" 을 쓰세요. 버튼을 만들 때 type 부터 쓰는 습관을 들이면 이 사고가 사라집니다.

문제 13 [응용] (개념04 섹션3 + 개념02 섹션3)

optgroup 으로 묶은 지역 목록을 만들고 label 로 감싸세요.

답:

```
<label>지역
  <select name="region"> <optgroup label="수도권"> <option value="seoul">서울
</option> <option value="incheon">인천
</option> </optgroup> <optgroup label="영남"> <option value="busan">부산
</option> <option value="daegu">대구</option> </optgroup> </select>
</label>
```

화면 "지역" 글자 오른쪽에 접힌 상자가 있고 "서울" 이 보입니다.

상자를 누르면 목록이 펼쳐지고 “수도권” 과 “영남” 이 회색 굵은 글씨로 있으며 그 아래 지역 이름들이 한 칸 안으로 들여쓰여 있습니다. 회색 제목은 눌러도 골라지지 않습니다. “지역” 글자를 누르면 목록이 펼쳐지지는 않지만 포커스가 목록 상자로 옮겨 갑니다. 이어서 키보드 아래 화살표를 누르면 고른 항목이 “인천” 으로 바뀝니다

지역

서울

인천

부산

대구

optgroup 의 label 속성에 쓴 글자가 묶음 제목이 됩니다. 제목은 항목이 아니라서 고를 수 없습니다.

label 로 감싸면 select 도 이름표와 연결됩니다. 다만 글자를 눌러도 목록이 저절로 펼쳐지지는 않습니다. 체크박스는 글자를 누르면 바로 켜지지만, 목록 상자는 포커스만 옮겨 갑니다. 크롬은 목록 상자를 직접 눌렀을 때만 목록을 펼치기 때문입니다. 연결이 되었는지 확인하려면 글자를 누른 뒤 키보드 아래 화살표를 눌러 보세요. 고른 항목이 바뀌면 포커스가 넘어간 것입니다. 연결해 두면 화면 낭독기도 이 목록의 이름이 “지역” 이라고 읽어 줍니다.

문제 14 [응용] (개념06 섹션1·섹션2·섹션5)

필수 + 최대 글자 수 + 못 고치지만 보내지는 칸을 만드세요.

답:

```
<form> <p><label for="n14">이름
</label> <input id="n14" type="text" name="userName" required maxlength="5" />
</p> <p><label for="o14">주문번호
</label> <input id="o14" type="text" name="orderNo" value="A-2026-001" readonly />
</p> <p><button type="submit">보내기</button></p>
</form>
```

화면 이름 칸에 여섯 번째 글자부터는 자판을 눌러도 안 들어갑니다.

비운 채로 보내기를 누르면 페이지가 넘어가지 않고 “이 입력란을 작성하세요.” 라는 말풍선이 그 칸 아래에 뜹니다. 주문번호 칸은 흰 바탕에 검은 글씨라 보통 칸과 똑같이 보이는데 자판을 눌러도 글자가 안 바뀝니다. 마우스로 굵어 복사는 됩니다

이름 (필수, 최대 5자)

주문번호

보내기

readonly 는 못 고치게 하면서 값은 그대로 보냅니다. disabled 를 쓰면 글씨가 회색으로 흐려지고 그 칸의 값이 아예 보내지지 않습니다. 여기서 자주 실수합니다. maxlength 는 치는 순간 막고, required 는 보낼 때 막습니다.

문제 15 [도전] (개념01 ~ 개념06 전체)

회원가입 폼을 통째로 만드세요.

답: 아래 상자 안의 구조를 그대로 보세요.

이름표 연결은 for 와 id 를 쓰는 label 로 감싸든 정답입니다.
여기서는 글자 칸에는 for 와 id 를, 라디오와 체크박스에는 감싸기를 썼습니다.
실무에서도 이렇게 섞어 씁니다.

화면 테두리 상자 두 개가 세로로 놓이고 왼쪽 위에 "기본 정보", "추가 정보" 가

각각 걸쳐 있습니다. 성별 동그라미는 처음에 "남자" 가 켜져 있고 항상 하나만 채워집니다. 관심 분야 네모는 둘 다 켤 수 있습니다. 상자 아래에 "가입하기" 와 "되돌리기" 버튼이 나란히 있습니다. 값을 바꾸고 되돌리기를 누르면 성별이 다시 "남자" 로 돌아오고 켜 둔 관심 분야도 꺼지며 글자 칸도 처음 상태가 됩니다

기본 정보
이름 (필수)

이메일 (필수)

비밀번호 (최소 8자)

추가 정보
성별
남자
여자
관심 분야
운동
요리
자기소개

가입하기
되돌리기

fieldset 으로 묶으면 긴 폼이 한눈에 들어옵니다. “성별” 과 “관심 분야” 처럼 여러 칸 전체의 제목은 label 이 아니라 그냥 문단으로 적습니다. label 은 칸 하나에 붙이는 이름표이기 때문입니다. type=“reset” 은 비우는 것이 아니라 처음 상태로 되돌립니다.

문제 16 [확인] (개념02 섹션2 + 개념03 섹션4 + 개념05 섹션2)

세 군데를 고치는 문제였습니다.

고친 곳 1 label 의 for 와 input 의 id 를 맞췄습니다.

for=“userName” 인데 id=“username” 이라 연결이 끊겨 있었습니다.
대소문자까지 같아야 합니다.

고친 곳 2 라디오 두 개의 name 을 같게 맞췄습니다.

name=“pay1” 과 name=“pay2” 는 서로 남남이라 둘 다 켜졌습니다.
name 이 같아야 한 무리가 되어 하나만 켜집니다.

고친 곳 3 버튼에 type=“button” 을 붙였습니다.

type 을 안 쓰면 기본이 submit 이라 누를 때 폼이 제출되었습니다.

화면 "이름" 글자를 누르면 오른쪽 칸에서 커서가 깜빡입니다.

동그라미는 처음에 “카드” 가 켜져 있고 “현금” 을 누르면 카드가 꺼집니다. 항상 하나만 채워집니다. “미리 보기” 를 눌러도 화면이 새로고침되지 않고 쳐 넣은 값이 그대로 남습니다

이름

결제 수단
카드
현금

미리 보기

세 가지 모두 에러가 나지 않는 실수였습니다. 화면만 봐서는 고치기 전과 후가 똑같이 생겼습니다. HTML 은 틀려도 조용하기 때문에, 만든 사람이 직접 눌러 보고 확인해야 합니다.

부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.